

spintent

SPANISH PARSE INTENTS

V0.99 2026-10-07*

©2026 by Pablo González L[†]

CTAN: <https://www.ctan.org/pkg/spintent>

 <https://github.com/pablgonz/spintent>

Resumen

El paquete **spintent** proporciona una serie de utilidades para docentes de educación primaria y secundaria que necesiten crear documentos PDF *accesibles* (*tagged* PDF) en español usando Lua $\text{\LaTeX}$.

Índice

1	Descripción	1	3.3	El comando <code>\spmoney</code>	6
1.1	Carga del paquete	1	3.3.1	Llaves para <code>\spmoney</code>	7
1.2	El paquete <code>babel</code>	2	3.3.2	Monedas y alias para <code>\spmoney</code>	7
1.3	Fuentes tipográficas	2	3.4	El comando <code>\spshort</code>	8
1.4	El paquete <code>hyperref</code>	2	3.4.1	Llaves para <code>\spshort</code>	8
2	El comando <code>\setspintent</code>	3	3.4.2	Argumentos para <code>\spshort</code>	9
3	The big four	3	4	Utilidades generales	10
3.1	El comando <code>\spnum</code>	3	5	Utilidades para símbolos	11
3.1.1	Llaves para <code>\spnum</code>	3	6	Utilidades para números	13
3.2	El comando <code>\spunit</code>	4	7	Utilidades para geometría	19
3.2.1	Llaves para <code>\spunit</code>	4	8	Utilidades para álgebra	34
3.2.2	Argumentos para <code>\spunit</code>	4	9	Referencias	38
3.2.3	El comando <code>\spnewunit</code>	6	10	Change history	39
3.2.4	El comando <code>\spunitalias</code>	6	11	Índice de la Documentación	40
			12	Implementation	43
			13	Índice de la Implementación	187

Motivación y agradecimientos

Desde que desarrollé `scontents`[7] en 2019 y, más tarde, `enumext`[8] en 2024, me he ido interesando cada vez más por la accesibilidad en los documentos PDF. Al principio era un tema prácticamente desconocido para mí, pero a medida que fui aprendiendo sobre él comprendí la importancia que tiene, especialmente en el ámbito educativo.

El equipo del proyecto de $\text{\LaTeX}$ [19] ha realizado un trabajo extraordinario en este campo. Hoy es posible generar documentos etiquetados, validarlos con herramientas como `veraPDF`[20] o inspeccionar su estructura con `ngpdf`[21]. Sin embargo, siempre me quedó una duda: cuando un estudiante utiliza NVDA[11] junto con MathCAT[12], ¿realmente escucha lo que el o la docente quiso expresar?

Basta con generar un documento accesible siguiendo las recomendaciones de *The LaTeX Tagged PDF Project* y escucharlo con NVDA/MathCAT para darse cuenta de que, en muchos casos, la *verbalización* no coincide con lo que uno espera. Esto no resulta extraño: las reglas del español, y en particular el lenguaje matemático, dependen en gran medida del contexto en que se utilizan.

El objetivo de **spintent** nace precisamente de esta necesidad: proporcionar herramientas para que el o la docente pueda indicar explícitamente el contexto de determinadas expresiones matemáticas desde el propio código fuente de $\text{\LaTeX}$, de modo que el documento PDF resultante no solo sea *accesible*, sino que también produzca una *verbalización más natural* y adecuada para el estudiante al utilizar NVDA/MathCAT.

El nuevo estándar PDF 2.0[14, 18, 16], las especificaciones de WTPDF[15] y la guía más reciente de la PDF Association, «Best Practice Guide: Math in PDF»[17], coinciden en que el camino hacia una accesibilidad matemática de calidad pasa por el uso de MathML. En ese contexto, Lua $\text{\LaTeX}$ ofrece una plataforma especialmente adecuada para incorporar esta tecnología.

*Este archivo describe la documentación para la versión v0.99 revisada por última vez el 2026-10-07.

[†]E-mail: «pablgonz@educarchile.cl».

Licencia y requerimientos

El paquete `spintent` carga y requiere el paquete `unicode-math`[1] y que se ejecute bajo Lua $\TeX$ por lo que es necesario contar con una distribución moderna de $\TeX$ como $\TeX$ Live 2026. Ha sido probado con las clases estándar proporcionadas por $\LaTeX$: `book`, `report`, `article` y `letter` en tamaños `10pt`, `11pt` y `12pt`.

Se otorga permiso para copiar, distribuir y/o modificar este paquete bajo los términos de $\LaTeX$ Project Public License (LPPL), versión 1.3 o posterior (<https://www.latex-project.org/lppl.txt>). El paquete tiene la condición de mantenido al estilo «*A Work in Progress*»¹.

- El requerimiento mínimo es $\LaTeX$ release 2026-11-01.

Este paquete solo se puede utilizar solo con Lua $\TeX$ y *tagged* PDF activo. Está presente en $\TeX$ Live y MiK $\TeX$, para instalarlo, utilice el gestor de paquetes de su distribución. Si desea una instalación manual, descargue `spintent.zip` y descomprímalo, luego ejecute `luatex spintent.ins`, mueva todos los archivos a las ubicaciones correspondientes y luego ejecute `mktexlsr`. Para generar la documentación, ejecute `arara spintent.dtx`.

```
spintent.sty  ⇒ TDS:tex/luatex/spintent/
spintent.lua  ⇒ TDS:tex/luatex/spintent/
spintent.pdf  ⇒ TDS:doc/luatex/spintent/
README.md    ⇒ TDS:doc/luatex/spintent/
spintent.dtx ⇒ TDS:source/luatex/spintent/
spintent.ins  ⇒ TDS:source/luatex/spintent/
```

1 Descripción

El nombre `spintent` puede entenderse como «*Spanish Parse Intent*» o también como «*School Parse Intent*». Ambas interpretaciones reflejan el objetivo del paquete y el público para el que fue diseñado: docentes de educación primaria y secundaria.

Está pensado para crear «*apuntes*», «*hojas de ejercicios*», «*clases escritas*» enfocadas en el estudiante. Solo un docente sabe cual es el nivel de aprendizaje del estudiante. El paquete intenta sobrescribir muchas de las reglas dictadas por la heurística de NVDA/MathCAT teniendo esto presente.

- El paquete `spintent` usa y abusa del atributo `:intent` de MathML por medio del comando `\MathMLintent` definido en `latex-lab-mathintent`[30], un encapsulado de `\luamml_attribute:een` proveída por `luamml`[2]. ¡Gracias Marcel!

1.1 Carga del paquete

El paquete `spintent` trabaja solo con Lua $\TeX$ y *tagged* PDF activo usando `tagging=on` dentro del comando `\DocumentMetadata` al inicio del archivo. Las llaves («*keys*») aceptadas por `\DocumentMetadata` y en general para el código de *tagged* PDF están documentadas en los paquetes y `tagpdf`[29] `documentmetadata-support`[30], es deber del usuario revisar la documentación asociada a ellos.

```
\DocumentMetadata
{
  lang=es-CL,
  pdfversion=2.0,
  pdfstandard=ua-2,
  tagging=on,
  tagging-setup={math/setup={mathml-SE}},
}
\documentclass{article}
\usepackage[spanish]{babel}
\usepackage{spintent}
```

1.2 El paquete babel

El paquete `babel-spanish`[10] está marcado como «parcialmente compatible» para *tagged* PDF, pero, es absolutamente necesario para la creación de documentos en español. En caso de encontrar problemas de compatibilidad con otros paquetes se puede desactivar usando:

```
\usepackage[spanish,shorthands=off]{babel}
```

Muchas de las «shorthands» proveídas por `babel-spanish` hoy no son necesarias, más allá de `\sptext` para las abreviaturas y ordinales el resto son solo atajos de teclas para escritura que hoy en día no se ocupan. El paquete `spintent` provee el comando `\spshort` (§3.4) que cubre estos casos, de esta manera podemos cargar `babel`[9] con todas sus funcionalidades sin problemas.

¹A *Work in Progress* es un álbum en vivo de Neil Peart (* 1952–† 2020), baterista y principal letrista de Rush. El nombre no pretende formar parte de la terminología de la LPPL; simplemente describe el estado permanente de este paquete mientras sigo aprendiendo sobre *accesibilidad* en PDF.

1.3 Fuentes tipográficas

Como `spintent` es un paquete Lua $\TeX$ debemos distinguir entre las fuentes tipográficas OpenType/TrueType versus las fuentes de 8bit estándar usadas por $\TeX$ para el *modo texto* y el *modo matemático*. Para evitar problemas con *tagged* PDF preferiremos no combinar fuentes OpenType/TrueType con fuentes 8bit y trabajaremos solo con fuentes OpenType/TrueType que tengan soporte para matemáticas.

La carga de las fuentes tipográficas para Lua $\TeX$ se maneja por medio del paquete `fontspec`[3], si no se especifica ninguna se usarán las fuentes Latin Modern para el *modo texto* y Latin Modern Math para el *modo matemático*.

El tipo de fuente tipográfica escogida es preferencia de cada usuario, en lo personal utilizo las fuente Libertinus para el *modo texto* por medio del paquete `libertinus-otf`[4] y la fuente Erewhon-Math para el modo matemático cargada por el paquete `fourier-otf`[5]. Adicionalmente cargo la fuente Source Code Pro para el texto tipo «máquina de escribir» por medio del paquete `sourcecodepro`[6].

```
% \DocumentMetadata{...}
\documentclass{article}
\usepackage[spanish]{babel}
\usepackage[osf,nomath,mono=false]{libertinus-otf}
\usepackage[osf,semibold]{sourcecodepro}
\usepackage[no-text]{fourier-otf}
\usepackage{spintent}
```

1.4 El paquete hyperref

El paquete `hyperref`[23] es «casi obligatorio» cuando se crean documentos *tagged* PDF, se encarga de las referencias cruzadas `\label` y `\ref`, las notas al pie `\footnote`, los hipervínculos `\href` y muchos otros elementos como marcadores, índices y metadatos.

```
\DocumentMetadata
{
  lang=es-CL,
  pdfversion=2.0,
  pdfstandard=ua-2,
  tagging=on,
  tagging-setup={math/setup={mathml-SE}},
}
\documentclass{article}
\usepackage[spanish]{babel}
\usepackage[osf,nomath,mono=false]{libertinus-otf}
\usepackage[osf,semibold]{sourcecodepro}
\usepackage[no-text]{fourier-otf}
\setmathfont{Erewhon-Math}[range={cal,bfcal},StylisticSet=1]
\usepackage{spintent, hyperref}
\hypersetup
{
  colorlinks      = true,
  pdfstartview    = FitH,
  pdfdisplaydoctitle = true,
  pdftitle        = {Test para el paquete spintent},
  pdfinfo         =
  {
    Title   = {Test},
    Subject = {spintent},
  },
}
```

2 El comando \setspintent

`\setspintent` `\setspintent{comando}{key-uno = valor, key-dos = valor, key-tres = valor, ...}`

El comando `\setspintent` establece las llaves (*keys*) de forma global para los comandos provistos por `spintent`. Puede utilizarse tanto en el preámbulo como en el cuerpo del documento tantas veces como se desee. Las llaves (*keys*) definidas en el *argumento opcional* [*key = val*] de los comandos tienen prioridad sobre las establecidas por este, anulando las configuraciones globales de `\setspintent`.

Muchas de las utilidades que provee el paquete `spintent` implementan el sistema [*key = val*] y están pensadas para ser utilizadas en la generación de «*hojas de ejercicios*» o «*clases escritas*». Se recomienda usar `\setspintent` y establecerlas de manera global para cada documento, para mantener la coherencia en la verbalización de NVDA/MathCAT a lo largo del mismo.

El sistema [*key = val*] utilizado por `spintent` se implementa usando el módulo `l3keys` de `expl3`[24]. Por diseño, las llaves que reciben un *valor* después del signo '=' NO aceptan *valores vacíos* y es obligatorio pasar uno válido; las llaves documentadas como (*valor prohibido*) no reciben *ningun valor*. En caso de omitir un *valor* válido o pasar un *valor vacío* a una llave que acepte *valor* o pasar un *valor* a una llave del tipo (*valor prohibido*) se devolverá un "error" y se detendrá la compilación del documento.

3 The big four

El paquete `spintent` provee cuatro grandes comandos («*the big four*»): `\spnum` para el manejo de números, `\spunit` para el manejo de unidades, `\spmoney` para el manejo de dinero y `\spshort` para el manejo de abreviaturas, acrónimos y números ordinales.

- También se incluyen las utilidades `\spnewunit` y `\spunitalias` asociadas a al comando `\spunit`.

3.1 El comando \spnum

`\spnum` `\spnum{<número>}` `\spnum{<número,número>}` `\spnum{<número.número>}` `\spnum{<número;número>}`
`\spnum[key = val]{<número,número;número>}` `\spnum[key = val]{<número,número;número e exponente>}`
`\spnum[key = val]{<número.número;número>}` `\spnum[key = val]{<número.número;número e exponente>}`

El comando `\spnum` trabaja en *modo matemático* y recibe el *argumento obligatorio* {<número>}, donde la «*parte entera*» puede iniciar con los signos '+' o '-', la «*parte decimal finita*» **debe** iniciar con una *coma* ',' o con un *punto* '.', y la «*parte decimal infinita (periódica)*» **debe** iniciar con un *punto y coma* ';'.

Si no se proporciona el *argumento opcional* [*key = val*] y se ejecuta `\$ \spnum{+23} \$` se imprimirá '+23' en la salida PDF y NVDA/MathCAT verbalizará «*más veintitrés*», si se ejecuta `\$ \spnum{3,14} \$` se imprimirá '3,14' en la salida PDF y NVDA/MathCAT verbalizará «*tres coma catorce*», si se ejecuta `\$ \spnum{3,1;4} \$` se imprimirá '3,14' en la salida PDF y NVDA/MathCAT verbalizará «*tres coma uno con cuatro periódico*».

Además, el *argumento obligatorio* {<número>} admite al final de este un sufijo de «*notación científica*». Este se indica mediante las letras 'e' o 'E', acompañadas de los signos opcionales '+' o '-' seguido por los dígitos del exponente. Por ejemplo `\$ \spnum{3,14e3} \$` imprimirá '3,14 · 10³' en la salida PDF y NVDA/MathCAT verbalizará «*tres coma catorce por diez al cubo*».

- La *coma* ',' o el *punto* '.' son *separadores decimales* y NO de millares, por lo tanto solo pueden aparecer una «única vez» dentro del {<argumento>}. El {<argumento>} puede contener espacios matemáticos válidos, pero, NO puede contener comandos con o sin argumentos como `\textstyle`, `\mathrm{#1}` o `\mathbf{#1}` por ejemplo. Cualquier entrada fuera de esta sintaxis devolverá un "error" deteniendo la compilación del documento.

3.1.1 Llaves para \spnum

`cifras-sep = {true | false}` default: true

Establece el *espaciado* utilizado para agrupar los dígitos en bloques de tres, tanto en la parte entera como en la parte decimal finita. Si se establece en `true` se siguen las normas de la RAE y se utiliza un *espacio fino* '\', entre las cifras; si se establece en `false`, se respetan los espacios matemáticos introducidos en el argumento. Por ejemplo `\$ \spnum{100000} \$` imprimirá '100 000' en la salida PDF y `\$ \spnum[cifras-sep=false]{10\,00\,00} \$` imprimirá '10 00 00' en la salida PDF.

`read-sign = {terse | clear}` default: terse

Establece la *forma* en la que NVDA/MathCAT verbalizará los signos '+' o '-'. Si se establece en `terse`, NVDA/MathCAT verbalizará «*más*» o «*menos*» **antes** de leer el número; si se establece en `clear`, NVDA/MathCAT verbalizará «*positivo*» o «*negativo*» **después** de leer el número. Por ejemplo `\$ \spnum{-23} \$` imprimirá '-23' en la salida PDF y NVDA/MathCAT verbalizará «*menos veintitrés*» y `\$ \spnum[read-sign=clear]{-23} \$` imprimirá '-23' en la salida PDF y NVDA/MathCAT verbalizará «*veintitrés negativo*».

`read-period-sep = {coma | punto}` default: coma

Establece la *forma* en la que NVDA/MathCAT verbalizará e imprimirá el signo de separación ';' de la «*parte decimal infinita (periódica)*» cuando no se tiene la «*parte decimal finita*». Si se establece en `coma`, se imprimirá ',' en la salida PDF y NVDA/MathCAT verbalizará «*coma*»; si se establece en `punto`, se imprimirá '.' en la

²Los «*cuatro grandes*» del Thrash Metal: Metallica, Megadeth, Slayer y Anthrax, que me acompañan mientras desarrollo `spintent`.

salida PDF y NVDA/MathCAT verbalizará «*punto*». Por ejemplo `\spnum{3;4}` imprimirá ‘3,4’ en la salida PDF y NVDA/MathCAT verbalizará «*tres coma cuatro periódico*» y `\spnum[read-period-sep=punto]{3;4}` imprimirá ‘3.4’ en la salida PDF y NVDA/MathCAT verbalizará «*tres punto cuatro periódico*»

- Esta llave es necesaria SOLO cuando se utilizan «*decimales periódicos puros*», es decir, cuando no se ha incluido en el `{(argumento)}` una coma ‘,’ o un punto ‘.’ antes del signo de separación ‘;’ periódica.

`notation-sym = {(times | cdot)}`

default: `cdot`

Establece el *símbolo* que se utilizará para imprimir la «*notación científica*» al usar ‘e’ o ‘E’. Si se establece en `times` imprimirá ‘×’ en la salida PDF y NVDA/MathCAT verbalizará «*por*»; si se establece en `cdot` imprimirá ‘·’ en la salida PDF y NVDA/MathCAT verbalizará «*por*». Por ejemplo `\spnum{3,14e3}` imprimirá ‘3,14 · 10³’ en la salida PDF y NVDA/MathCAT verbalizará «*tres coma catorce por diez al cubo*» y `\spnum[notation-sym=times]{3,14e3}` imprimirá ‘3,14 × 10³’ en la salida PDF y NVDA/MathCAT verbalizará «*tres coma catorce por diez al cubo*».

- Decir «*notación científica*» aquí es un poco exagerado, en realidad es «*multiplicación por potencias de 10*», no se verifica si la entrada cumple o no con la condición de estar escrita en «*notación científica*».

3.2 El comando \spunit

<code>\spunit</code>	<code>\spunit{(unidad)}</code>	<code>\spunit[key = val]{(unidad)}</code>	<code>\spunit[key = val]{(número unidad)}</code>
	<code>\spunit[key = val]{(número unidad*unidad)}</code>	<code>\spunit[key = val]{(número unidad*unidad/unidad)}</code>	
	<code>\spunit[key = val]{(número unidad*unidad/unidad*...)}</code>	<code>\spunit[key = val]{(número e exponente unidad ...)}</code>	

El comando `\spunit` trabaja en *modo matemático* y recibe el *argumento obligatorio* `{(unidad)}`, el cual puede contener una «*parte numérica*» opcional la cual se procesa internamente por el comando `\spnum` al inicio del `{(argumento)}` seguido de una `{(unidad)}` o un `{(alias)}` valido descritos en §3.2.2 o una `{(unidad)}` creada por el comando `\spnewunit` (§3.2.3) o un `{(alias)}` creado por el comando `\spunitalias` (§3.2.4).

Si no se proporciona el *argumento opcional* `[key = val]`, `\spunit{23 m}` imprimirá ‘23 m’ en la salida PDF y NVDA/MathCAT verbalizará «*veintitrés metros*».

La multiplicación entre las `{(unidades)}` se realiza mediante un *asterisco* ‘*’ y las potencias de estas se indican con un *circunflejo seguido del exponente entre llaves* ‘^`{(exponente)}`’. Por ejemplo si se ejecuta `\spunit{23 m*s^{2}}` se imprimirá ‘23 m s²’ en la salida PDF y NVDA/MathCAT verbalizará «*veintitrés metros segundos al cuadrado*».

Se pueden usar *fracciones lineales* con las `{(unidades)}` utilizando una «*única barra*» ‘/’ para la separación entre las `{(unidades)}` del numerador y el denominador, teniendo presente que el denominador NO puede contener números de ningún tipo. Por ejemplo: `\spunit{23 m/s^{2}}` imprimirá ‘23 m/s²’ en la salida PDF y NVDA/MathCAT verbalizará «*veintitrés metros por segundos al cuadrado*».

- Solo imprimiremos *fracciones lineales*, si se requiere otra forma fraccionaria para explicaciones y otros usos se debe usar el comando por separado. El `{(argumento)}` puede contener espacios matemáticos validos, pero, NO puede contener comandos con o sin argumentos como `\textstyle`, `\mathrm{#1}` o `\mathbf{#1}` por ejemplo, NO puede contener más de una «*única barra*» ‘/’ y NO puede contener números en el denominador. Cualquier entrada fuera de esta sintaxis devolverá un “error” deteniendo la compilación del documento.

3.2.1 Llaves para \spunit

El comando `\spunit` posee las *(meta-keys)* `cifras-sep`, `read-period-sep` y `notation-sym` pasadas al comando `\spnum` (§3.1.1).

`print-cdot = {(true | false)}`

default: `false`

Establece si se *imprime* `\cdot` ‘·’ o un *espacio fino* ‘\,’ para la multiplicación de `{(unidades)}`. Si se establece en `true`, se imprimirá ‘·’ entre las `{(unidades)}` en la salida PDF; si se establece en `false` se imprimirá un *espacio fino* ‘\,’ entre las `{(unidades)}` en la salida PDF.

`read-cdot = {(true | false)}`

default: `false`

Establece la *forma* en la que NVDA/MathCAT verbalizará la multiplicación de `{(unidades)}`. Si se establece en `true`, NVDA/MathCAT verbalizará «*por*» entre las `{(unidades)}`; si se establece en `false`, NVDA/MathCAT no verbalizará nada entre las `{(unidades)}`.

`read-slash = {(por | partido-por)}`

default: `por`

Establece la *forma* en la que NVDA/MathCAT verbalizará la «*barra*» ‘/’ de separación entre el numerador y el denominador. Si se establece en `por`, NVDA/MathCAT verbalizará «*por*»; si se establece en `partido-por`, NVDA/MathCAT verbalizará «*partido por*».

3.2.2 Argumentos para \spunit

Resumen de `{(unidades)}` y `{(alias)}` soportados por `\spunit`.

Entrada	Alias soportados	Nombre unidad	Salida	NVDA/MathCAT
m	metro, metros	Metro	m	«metro»
g	gramo, gramos	Gramo	g	«gramo»
s	segundo, segundos	Segundo (Tiempo)	s	«segundo (Tiempo)»
l	litro, l-litro	Litro (minúscula)	l	«litro (minúscula)»
h	hora, horas	Hora	h	«hora»
d	dia, dias	Día	d	«día»
A	amperio, ampere	Amperio	A	«amperio»
V	voltio, volt	Voltio	V	«voltio»
W	vatio, watt	Vatio	W	«vatio»
J	julio, joule	Julio	J	«julio»
N	newton	Newton	N	«newton»
Pa	pascal	Pascal	Pa	«pascal»
Hz	hercio, hertz	Hercio	Hz	«hercio»
Ω	ohmio, ohm, Omega	Ohmio	Ω	«Ohmio»
F	faradio	Faradio	F	«Faradio»
C	culombio	Culombio	C	«Culombio»
K	kelvin, Kelvin	Kelvin	K	«Kelvin»
°C	celsius, degC, °C	Grados Celsius	°C	«Grados Celsius»
°F	fahrenheit, degF, °F	Grados Fahrenheit	°F	«Grados Fahrenheit»
cal	caloria, calorías	Caloría	cal	«Caloría»
b	bit	Bit	b	«Bit»
B	byte, bytes	Byte	B	«Byte»
in	pulgada, pulgadas	Pulgada	in	«Pulgada»
ft	pie, pies	Pie	ft	«Pie»
yd	yarda, yardas	Yarda	yd	«Yarda»
mi	milla, millas	Milla	mi	«Milla»
lb	libra, libras	Libra	lb	«Libra»
oz	onza, onzas	Onza	oz	«Onza»
gal	galon, galones	Galón	gal	«Galón»
Å	angstrom	Angstrom	Å	«Angstrom»
μ	micro	Micro (Prefijo)	μ	«Micro (Prefijo)»
Ω	mho	Mho / Siemens	Ω	«Mho / Siemens»
'	minmin	Minuto (Arco)	'	«Minuto (Arco)»
"	segseg	Segundo (Arco)	"	«Segundo (Arco)»
u	u-masa	Unidad masa atómica	u	«Unidad masa atómica»

Unidades directas (sin alias):

L	-	Litro (Mayúscula)	L	«Litro (Mayúscula)»
ℓ	-	Litro (Cursiva)	ℓ	«Litro (Cursiva)»
w	-	Vatio (Minúscula)	w	«Vatio (Minúscula)»
°K	-	Grado Kelvin	°K	«Grado Kelvin»
Bq	-	Becquerel	Bq	«Becquerel»
Da	-	Dalton	Da	«Dalton»
Gy	-	Gray	Gy	«Gray»
S	-	Siemens	S	«Siemens»
Sv	-	Sievert	Sv	«Sievert»
T	-	Tesla	T	«Tesla»
Wb	-	Weber	Wb	«Weber»
atm	-	Atmósfera	atm	«Atmósfera»
au	-	Unidad astronómica	au	«Unidad astronómica»
bar	-	Bar	bar	«Bar»
cd	-	Candela	cd	«Candela»
dB	-	Decibelio	dB	«Decibelio»
dyn	-	Dina	dyn	«Dina»
eV	-	Electronvoltio	eV	«Electronvoltio»
erg	-	Ergio	erg	«Ergio»
ha	-	Hectárea	ha	«Hectárea»
kat	-	Katal	kat	«Katal»
kg	-	Kilogramo	kg	«Kilogramo»
km	-	Kilómetro	km	«Kilómetro»
lm	-	Lumen	lm	«Lumen»

Continúa en la siguiente página...

Continuación de la tabla 1				
Entrada	Alias soportados	Nombre unidad	Salida	MathCAT
lx	-	Lux	lx	«Lux»
cm	-	Centímetro	cm	«Centímetro»
min	-	Minuto (Tiempo)	min	«Minuto (Tiempo)»
mm	-	Milímetro	mm	«Milímetro»
mol	-	Mol	mol	«Mol»
pc	-	Pársec	pc	«Pársec»
rad	-	Radián	rad	«Radián»
rpm	-	Rev. por minuto	rpm	«Rev. por minuto»
sr	-	Estereorradián	sr	«Estereorradián»
t	-	Tonelada	t	«Tonelada»
a	-	Año / Área (are)	a	«Año / Área (are)»
y	-	Año (Year)	y	«Año (Year)»
°	-	Grado (Arco)	°	«Grado (Arco)»

3.2.3 El comando \spnewunit

\spnewunit

\spnewunit{<nueva unidad>}{<verbalización NVDA>}

El comando `\spnewunit` trabaja en *modo texto* y recibe dos *argumentos obligatorios*: `{<nueva unidad>}` que establece lo que se imprimirá en la salida PDF y `{<verbalización NVDA>}` que establece como NVDA/MathCAT verbalizará la `{<nueva unidad>}`.

La `{<nueva unidad>}` no debe estar registrada como `{<unidad>}` o `{<alias>}` para `\spunit` descritos en §3.2.2. Si la `{<nueva unidad>}` ya está registrada se devolverá un “error” y se detendrá la compilación del documento.

La declaración de la `{<nueva unidad>}` es «global» y se almacena en el módulo `spintent.lua` como una `{<unidad>}` valida para pasar como `{<argumento>}` a `\spunit`. Por ejemplo: `\spnewunit{px}{píxeles}` creará la `{<nueva unidad>}` ‘px’ y cuando se utilice `$\spunit{100 px}$` se imprimirá ‘100 px’ en la salida PDF y NVDA/MathCAT verbalizará «cien píxeles».

3.2.4 El comando \spunitalias

\spunitalias

\spunitalias{<nuevo alias>}{<expresión de unidades>}

El comando `\spunitalias` trabaja en *modo texto* y recibe dos *argumentos obligatorios*: `{<nuevo alias>}` que establece es el nuevo `{<alias>}` de `{<argumento>}` para `\spunit` y `{<expresión de unidades>}` que puede contener multiplicación o división de `{<unidades>}` ya registradas que se imprimirán en la salida PDF.

El `{<nuevo alias>}` no debe estar registrado como `{<unidad>}` o `{<alias>}` para `\spunit` descritos en §3.2.2 o estar definido como `{<unidad>}` por medio del comando `\spnewunit`. Si el `{<nuevo alias>}` ya está registrado se devolverá un “error” y se detendrá la compilación del documento.

La declaración de un `{<nuevo alias>}` es «global» y se almacena en el módulo `spintent.lua` como una `{<alias>}` valido para pasar como `{<argumento>}` a `\spunit`. Por ejemplo: `\spunitalias{caudal}{m^3/s}` creará el `{<alias>}` ‘caudal’ y cuando se utilice `$\spunit{50 caudal}$` se imprimirá ‘50 m³/s’ en la salida PDF y NVDA/MathCAT verbalizará «cincuenta metros al cubo por segundos».

3.3 El comando \spmoney

\spmoney

\spmoney{<cifra>}

\spmoney{<cifra símbolo moneda>}

\spmoney{<cifra alias moneda>}

\spmoney[<key = val>]{<cifra>}

\spmoney[<key = val>]{<cifra símbolo moneda>}

\spmoney[<key = val>]{<cifra alias moneda>}

El comando `\spmoney` trabaja en *modo matemático* y recibe el *argumento obligatorio* `{<cifra>}` que contiene la «cifra numérica» positiva o negativa la cual se procesa internamente por el comando `\spnum` acompañada de «símbolo moneda» o «alias moneda» opcionales descritos en §3.3.2.

Si no se proporciona el *argumento opcional* `[<key = val>]`, `$\spmoney{500}$` imprimirá ‘\$500’ en la salida PDF y NVDA/MathCAT verbalizará «quinientos pesos».

El comportamiento de `\spmoney` depende de como se pase «símbolo moneda» o «alias moneda» en el *argumento obligatorio* donde tienen prioridad por sobre los valores por defecto. Por ejemplo: `$\spmoney{500 dólares}$` imprimirá ‘\$500’ en la salida PDF y NVDA/MathCAT verbalizará «quinientos dólares» aunque la moneda por defecto sean pesos.

Si se pasa un «alias moneda» del tipo ISO (código de divisas) como CLP o USD (mayúsculas) se modifica la verbalización en NVDA/MathCAT agregando el país y se ajusta la salida PDF al estandar ISO. Por ejemplo: `$\spmoney{500 CLP}$` imprimirá ‘CLP500’ en la salida PDF y NVDA/MathCAT verbalizará «quinientos pesos chilenos».

Si se pasa un «*alias moneda*» del tipo ISO (código de divisas) como `clp` o `usd` (minúsculas) se modifica la verbalización en NVDA/MathCAT agregando el país, pero, NO se modifica la salida PDF imprimiendo el «*símbolo de moneda*» correspondiente. Por ejemplo: `\spmoney{500 clp}` imprimirá '\$500' en la salida PDF y NVDA/MathCAT verbalizará «*quinientos pesos chilenos*»

Pasar de manera directa en el *argumento obligatorio* un «*símbolo moneda*» como '\$', '€', '£' o un «*alias moneda*» como peso, euro o libra produce la salida PDF y verbalización estándar esperada para NVDA/MathCAT. Los «*alias moneda*» se pueden pasar con o sin tilde, por ejemplo dolares o dólares.

- Los «*alias moneda*» se deben escribir en minúsculas y los códigos ISO en minúsculas o todo en mayúsculas. Cualquier otra escritura, por ejemplo PESO o Usd, devolverá un “error” y se detendrá la compilación del documento.
- La posición del «*símbolo moneda*» a la izquierda o a la derecha de la «*cifra numérica*» y la verbalización para NVDA/MathCAT se manejan desde el módulo Lua `spintent.lua` y no se proveen llaves para modificar este comportamiento.
- El «*símbolo moneda*» es tomado de la fuente tipográfica definida para el *modo matemático* al momento de ejecutar el comando (o del *modo texto* si se establece `money-math=false`), se debe tener presente en caso de querer usar un «*símbolo moneda*» poco habitual que esté soportada por `spintent`. El `{(argumento)}` puede contener espacios matemáticos válidos, pero, NO puede contener comandos con o sin argumentos como `\textstyle`, `\mathrm{#1}` o `\mathbf{#1}` por ejemplo, con la única excepción de `\$`. La coma ‘,’ o el punto ‘.’ como *separador decimal* solo pueden aparecer una «única vez». Cualquier entrada fuera de esta sintaxis devolverá un “error” deteniendo la compilación del documento.

3.3.1 Llaves para \spmoney

El comando `\spmoney` posee la «*meta-key*» `cifras-sep` pasada al comando `\spnum` (§3.1.1).

`currency = {(moneda | alias)}` default: peso

Establece el *símbolo moneda* por defecto que se imprimirá en la salida PDF y cómo lo verbalizará NVDA/MathCAT cuando SOLO se pasa una «*cifra numérica*» como `{(argumento)}` al comando `\spmoney`. El «*símbolo moneda*» o el «*alias moneda*» pasado a esta llave debe ser alguno de los descritos en §3.3.2.

`sub-units = {(true | false)}` default: false

Establece si se usarán *subunidades* de la moneda en curso para la verbalización en NVDA/MathCAT. La moneda en curso debe aceptar *subunidades*. Por ejemplo: `\spmoney[sub-units=true]{100,5 dolares}` imprimirá '\$100,5' en la salida PDF y NVDA/MathCAT verbalizará «*cient dólares con cincuenta centavos*».

- Si la «*cifra numérica*» no tiene parte decimal, la llave `sub-units` no tiene efecto. Si la moneda en curso no admite *subunidades* se emite una advertencia y la llave tampoco tiene efecto.

`money-math = {(true | false)}` default: true

Establece si el *símbolo moneda* se imprimirá en *modo texto* o en *modo matemático*. Si se establece en `true`, se usará la fuente tipográfica definida para el *modo matemático* en curso para imprimir el *símbolo moneda* en la salida PDF; si se establece en `false`, se usará la fuente tipográfica definida para el *modo texto* para imprimir el *símbolo moneda* en la salida PDF.

3.3.2 Monedas y alias para \spmoney

Resumen de los «*símbolos moneda*» y «*alias moneda*» soportados en el `{(argumento)}` de `\spmoney`.

Entrada	Alias	Nombre	Salida	NVDA/MathCAT
\$	peso, pesos	peso	\$	« <i>peso / pesos</i> »
dolar	dolares, dólar, dólares	dólar	\$	« <i>dólar / dólares</i> »
€	euro, euros	euro	€	« <i>euro / euros</i> »
£	libra, libras	libra	£	« <i>libra / libras</i> »
¥	yen, yenes	yen	¥	« <i>yen / yenes</i> »
yuan	yuanes	yuan	¥	« <i>yuan / yuanes</i> »
₩	won, wons	won	₩	« <i>won / wons</i> »
₪	shekel, shekels, sequel, sequeles	séquel	₪	« <i>séquel / séqueles</i> »
₹	rupia, rupias	rupia	₹	« <i>rupia / rupias</i> »
₱	rublo, rublos	rublo	₱	« <i>rublo / rublos</i> »
₺	lira, liras	lira	₺	« <i>lira / liras</i> »
₴	grivna, grivnas	grivna	₴	« <i>grivna / grivnas</i> »
¢	centavo, centavos	centavo	¢	« <i>centavo / centavos</i> »
Divisas en formato ISO (Mayúsculas)				
CLP	—	peso chileno	CLP	« <i>peso chileno / pesos chilenos</i> »
MXN	—	peso mexicano	MXN	« <i>peso mexicano / pesos mexicanos</i> »
USD	—	dólar	USD	« <i>dólar estadounidense / dólares estadounidenses</i> »

Continúa en la siguiente página...

Cuadro 2 – Continuación de la página anterior

Entrada	Alias	Nombre	Salida	NVDA/MathCAT
EUR	—	euro	EUR	«euro / euros»
GBP	—	libra	GBP	«libra / libras»
JPY	—	yen	JPY	«yen / yenes»
CNY	—	yuan	CNY	«yuan / yuanes»
KRW	—	won	KRW	«won / wons»
ILS	—	séquel	ILS	«séquel / séqueles»
INR	—	rupia	INR	«rupia / rupias»
RUB	—	rublo	RUB	«rublo / rublos»
TRY	—	lira	TRY	«lira / liras»
UAH	—	grivna	UAH	«grivna / grivnas»

Códigos ISO en minúsculas

clp	—	peso chileno	\$	«peso chileno / pesos chilenos»
mxn	—	peso mexicano	\$	«peso mexicano / pesos mexicanos»
usd	—	dólar	\$	«dólar estadounidense / dólares estadounidenses»
eur	—	euro	€	«euro / euros»
gbp	—	libra	£	«libra / libras»
jpy	—	yen	¥	«yen / yenes»
cny	—	yuan	¥	«yuan / yuanes»
krw	—	won	₩	«won / wons»
ils	—	séquel	₪	«séquel / séqueles»
inr	—	rupia	₹	«rupia / rupias»
rub	—	rublo	₽	«rublo / rublos»
try	—	lira	₺	«lira / liras»
uah	—	grivna	₴	«grivna / grivnas»

3.4 El comando \spshort

```
\spshort{<abreviaturas | acrónimos | números ordinales>}
\spshort[<key = val>]{<abreviaturas | acrónimos | números ordinales>}
\spshort[<read-arg={<verbalización NVDA>}]{<comandos LATEX>}
```

El comando `\spshort` trabaja solo en modo texto y recibe un argumento obligatorio del tipo `{<abreviaturas>}`, `{<acrónimos>}` o `{<números ordinales>}` descritos en §3.4.2.

Si no se proporciona el argumento opcional `[<key = val>]` y se ejecuta `\spshort{dr.a}` se imprimirá ‘Dr.^a’ en la salida PDF y NVDA/MathCAT verbalizará «doctora»; si se ejecuta `\spshort{onu}` se imprimirá ‘ONU’ en la salida PDF y NVDA/MathCAT verbalizará «organización de las naciones unidas» y si se ejecuta `\spshort{mineduc}` se imprimirá ‘Mineduc’ en la salida PDF y NVDA/MathCAT verbalizará «ministerio de educación».

Para la escritura de `{<números ordinales>}` se puede ejecutar `\spshort{1.a}` que imprimirá ‘1.^a’ en la salida PDF y NVDA/MathCAT verbalizará «primera»; si se ejecuta `\spshort{1.o}` imprimirá ‘1.^o’ en la salida PDF y NVDA/MathCAT verbalizará «primero».

Para crear un nueva `{<abreviatura>}`, `{<acrónimo>}` o `{<número ordinal>}` que no se encuentre registrada por spintent dentro del módulo Lua spintent.lua descritas en §3.4.2 se debe utilizar la llave `read-arg` y definir esta dentro del argumento obligatorio que puede contener `{<comandos LATEX>}` en modo texto los cuales se imprimirán en salida PDF.

- Para la escritura de `{<números ordinales>}` se debe tener cuidado y no confundir la «letra o voladita» ‘o’ con el símbolo de grado ‘°’. Si la fuente tipográfica del modo texto está utilizando `Numbers=OldStyle` los números pueden quedar muy separados de la «letra voladita», la solución para esto es agregar `\addfontfeatures{Numbers=Lining}` dentro de un grupo junto al comando: `{\addfontfeatures{Numbers=Lining}\spshort{1.o}}`.
- El comando `\spshort` está pensado como un reemplazo para `shorthands` y el comando `\sptext` provistos por el paquete `babel` en español compatible con `tagged` PDF. Es de las pocas utilidades que provee el paquete `spintent` que trabaja solo en modo texto. Internamente utiliza la llave `E` (expansion text) proveída por el paquete `tagpdf`[29].

3.4.1 Llaves para \spshort

```
read-arg = {<verbalización NVDA>} default: no usado

Establece la verbalización del argumento {<comandos LATEX>} de la nueva {<abreviatura>}, {<acrónimo>}
o {<número ordinal>} para NVDA. El valor de esta llave siempre debe pasarse entre ‘{ }’. Por ejemplo:
\spshort[read-arg={octavo}]{8.\textsuperscript{\emph{avo}}} creará una nueva {<abreviatura>}
que imprimirá ‘8.avo’ en la salida PDF y NVDA verbalizará «octavo».
```

```
small-caps = {<true | false>} default: false
```

Establece si se usarán versalitas para imprimir `{<acrónimos>}` y para la «letra volada» en `{<abreviaturas>}` y `{<números ordinales>}` respetando las reglas de la RAE para el uso de estas. Si se establece en `true`, las abreviaturas

con mayúscula como EE. UU. y los acrónimos de cinco o más letras como Unicef no se verán afectados; si se establece en `false`, no se aplicarán *versalitas*. Por ejemplo: `\spshort[small-caps=true]{dr.a}` imprimirá ‘Dr.^A’ en la salida PDF y NVDA verbalizará «*doctora*».

3.4.2 Argumentos para `\spshort`

El cuadro 3 resume las abreviaturas, expresiones latinas y títulos registrados en el módulo Lua `spintent.lua`.

Cuadro 3: Abreviaturas soportadas

Entrada	Alias / Variantes	Salida PDF	Lectura NVDA
Abreviaturas con letras voladas			
dr.a	dr. ^a	Dr. ^a	« <i>doctora</i> »
dr.as	—	Dr. ^{as}	« <i>doctoras</i> »
prof.a	—	Prof. ^a	« <i>profesora</i> »
d.a	d. ^a	D. ^a	« <i>doña</i> »
m.a	—	M. ^a	« <i>maría</i> »
n.o	n. ^o	N. ^o	« <i>número</i> »
n.os	—	N. ^{os}	« <i>números</i> »
c.ia	—	C. ^{ia}	« <i>compañía</i> »
Abreviaturas comunes y expresiones latinas			
pág.	pag.	pág.	« <i>página</i> »
vol.	—	vol.	« <i>volumen</i> »
etc.	—	etc.	« <i>etcétera</i> »
et al.	et. al.	et al.	« <i>y otros</i> »
ibíd.	ibid.	ibíd.	« <i>ibídem</i> »
op. cit.	op.cit.	op. cit.	« <i>obra citada</i> »
loc. cit.	loc.cit.	loc. cit.	« <i>lugar citado</i> »
v. gr.	v.gr.	v. gr.	« <i>verbigracia</i> »
i. e.	i.e.	i. e.	« <i>esto es</i> »
e. g.	e.g.	e. g.	« <i>por ejemplo</i> »
p. ej.	p.ej.	p. ej.	« <i>por ejemplo</i> »
Títulos, profesiones y cargos			
sr.	—	Sr.	« <i>señor</i> »
sra.	—	Sra.	« <i>señora</i> »
srta.	—	Srta.	« <i>señorita</i> »
dr.	—	Dr.	« <i>doctor</i> »
dra.	—	Dra.	« <i>doctora</i> »
dres.	—	Dres.	« <i>doctores</i> »
dras.	—	Dras.	« <i>doctoras</i> »
prof.	—	Prof.	« <i>profesor</i> »
profa.	—	Profa.	« <i>profesora</i> »
profs.	—	Profs.	« <i>profesores</i> »
ing.	—	Ing.	« <i>ingeniero</i> »
ings.	—	Ings.	« <i>ingenieros</i> »
lic.	—	Lic.	« <i>licenciado</i> »
v. b.	v.b.	V. B.	« <i>visto bueno</i> »
Abreviaturas compuestas e institucionales			
s. a.	s.a.	S. A.	« <i>sociedad anónima</i> »
ee. uu.	ee.uu.	EE. UU.	« <i>estados unidos</i> »
dd. hh.	dd.hh.	DD. HH.	« <i>derechos humanos</i> »
jj. oo.	jj.oo.	JJ. OO.	« <i>juegos olímpicos</i> »
ff. aa.	ff.aa.	FF. AA.	« <i>fuerzas armadas</i> »
rr. ee.	rr.ee.	RR. EE.	« <i>relaciones exteriores</i> »
cc. aa.	cc.aa.	CC. AA.	« <i>comunidades autónomas</i> »
p. d.	p.d.	P. D.	« <i>posdata</i> »
a. c.	a.c.	a. C.	« <i>antes de Cristo</i> »
d. c.	d.c.	d. C.	« <i>después de Cristo</i> »
d. n. i.	d.n.i.	D. N. I.	« <i>documento nacional de identidad</i> »
r. u. t.	r.u.t.	R. U. T.	« <i>rol único tributario</i> »
r. u. n.	r.u.n.	R. U. N.	« <i>rol único nacional</i> »

El cuadro 4 detalla las siglas y acrónimos registrados en el módulo Lua `spintent.lua`.

Cuadro 4: Siglas y acrónimos soportados

Entrada	Alias / Variantes	Salida PDF	Lectura NVDA
Siglas (Deletreo o lectura expandida)			
dni	—	DNI	«documento nacional de identidad»
rut	—	RUT	«rol único tributario»
run	—	RUN	«rol único nacional»
onu	—	ONU	«organización de las naciones unidas»
rae	—	RAE	«real academia española»
ong	—	ONG	«organización no gubernamental»
ongs	—	ONGs	«organizaciones no gubernamentales»
urss	—	URSS	«unión de repúblicas socialistas soviéticas»
oea	—	OEA	«organización de los estados americanos»
oms	—	OMS	«organización mundial de la salud»
fmi	—	FMI	«fondo monetario internacional»
bid	—	BID	«banco interamericano de desarrollo»
otan	—	OTAN	«organización del tratado del atlántico norte»
ue	—	UE	«unión europea»
ru	—	RU	«reino unido»
Acrónimos silábicos			
unicef	—	Unicef	«fondo de las naciones unidas para la infancia»
unesco	—	Unesco	«organización de las naciones unidas para la educación, la ciencia y la cultura»
mercosur	—	Mercosur	«mercado común del sur»
cepal	—	Cepal	«comisión económica para américa latina»
mineduc	—	Mineduc	«ministerio de educación»
conicet	—	Conicet	«consejo nacional de investigaciones»

El cuadro 5 muestra ejemplos de números ordinales registrados en el módulo Lua `spintent.lua`.

Cuadro 5: Números ordinales soportados

Entrada	Alias / Variantes	Salida PDF	Lectura NVDA
Sufijos para ordinales (Ejemplos)			
1.a	1. ^a	1. ^a	«primera»
2.o	2. ^o	2. ^o	«segundo»
3.er	—	3. ^{er}	«tercer»
4.os	—	4. ^{os}	«cuartos»
5.as	—	5. ^{as}	«quintas»

Los números ordinales se verbalizan con palabras desde el 1 hasta el 99. Desde el 100 el ordinal no se convierte en palabras y se entrega a NVDA/MathCAT tal como se imprime, por ejemplo 100. o como «100^o».

4 Utilidades generales

Además de los «cuatro grandes» y sus asociados, el paquete `spintent` provee tres utilidades de uso general: `\spsiglo` para la escritura y lectura de siglos, `\sptime` para la escritura y lectura de tiempo y `\spdate` para la escritura y lectura de fechas.

El comando `\spsiglo`

`\spsiglo` `\spsiglo{<siglo>}` `\spsiglo[<key = val>]{<siglo>}`

El comando `\spsiglo` trabaja en *modo texto* y en *modo matemático*, recibe el argumento obligatorio `{<siglo>}`, el cual puede ser un número arábigo ‘21’ o un número romano ‘XXI’ mayor que ‘0’ y menor que ‘4000’.

Si no se proporciona el argumento opcional `[<key = val>]` y se ejecuta `\spsiglo{21}` o `\spsiglo{XXI}` se imprimirá ‘xxi’ en la salida PDF y NVDA/MathCAT verbalizará «veintiuno».

- El argumento obligatorio $\{\langle siglo \rangle\}$ «*siempre*» es convertido a *número romano en versalitas* y es verbalizado por NVDA/MathCAT como *número arábigo*. Esta es la única utilidad que provee el paquete `spintent` que trabaja en *modo texto* y en *modo matemático*. Internamente utiliza la llave `alt` proveída por el paquete `tagpdf` en *modo texto* y usa `\MathMLintent` para el *modo matemático*.

Llaves para `\spsiglo`

print-era = $\{\langle ac \mid dc \mid none \rangle\}$

default: none

Establece si se *imprime* o no la abreviatura después de imprimir el argumento $\{\langle siglo \rangle\}$. Si se establece en `ac`, `\spsiglo[print-era=ac]{XXI}` imprimirá ‘xxi a. C.’ en la salida PDF y NVDA/MathCAT verbalizará «*veintiuno antes de cristo*»; si se establece en `dc`, `\spsiglo[print-era=dc]{XXI}` imprimirá ‘xxi d. C.’ en la salida PDF y NVDA/MathCAT verbalizará «*veintiuno después de cristo*»; si se establece en `none`, `\spsiglo[print-era=none]{XXI}` imprimirá ‘xxi’ en la salida PDF y NVDA/MathCAT verbalizará «*veintiuno*».

El comando `\sptime`

`\sptime`

`\sptime{\langle horas : minutos \rangle}``\sptime{\langle horas : minutos : segundos \rangle}`

El comando `\sptime` trabaja en *modo matemático* y recibe el argumento obligatorio $\{\langle horas : minutos \rangle\}$ o $\{\langle horas : minutos : segundos \rangle\}$. Por ejemplo: `\sptime{23:45}` imprimirá en «*modo texto*» la salida PDF ‘23:45’ y NVDA/MathCAT verbalizará «*las veintitrés y cuarenta y cinco minutos*».

El comando `\spdate`

`\spdate`

`\spdate{\langle DD/MM/YYYY \rangle}``\spdate{\langle YYYY/MM/DD \rangle}``\spdate{\langle YYYY-MM-DD \rangle}``\spdate{\langle DD-MM-YYYY \rangle}`

El comando `\spdate` trabaja en *modo matemático* y recibe el argumento obligatorio $\{\langle DD/MM/YYYY \rangle\}$, $\{\langle DD-MM-YYYY \rangle\}$ o $\{\langle YYYY-MM-DD \rangle\}$. Por ejemplo: `\spdate{1981-01-02}` y `\spdate{02/01/1981}` imprimirán en «*modo texto*» la salida PDF ‘1981-01-02’ y ‘02/01/1981’ respectivamente y en ambos casos, NVDA/MathCAT verbalizará «*dos de enero de mil novecientos ochenta y uno*».

- Si se pasa el argumento $\{\langle YYYY/MM/DD \rangle\}$, internamente se convertirá en $\{\langle DD/MM/YYYY \rangle\}$ y se imprimirá de esa forma por compatibilidad con NVDA/MathCAT.

5 Utilidades para símbolos

El paquete `spintent` provee cuatro comandos `\spnewread`, `\spread`, `\spdsym` y `\spmsym` para trabajar con «*símbolos matemáticos*» descritos en esta sección. Están pensando desde el punto de vista docente, para situaciones donde la lectura de un «*símbolo*» (acorde al contexto) es relevante.

- A diferencia de las otras utilidades implementados por `spintent` (que analizan y dan formato a la salida), estos no procesan los «*símbolos matemáticos*», solo afectan la *forma* (semántica) en que serán verbalizados por NVDA/MathCAT.

El comando `\spnewread`

`\spnewread`

`\spnewread{\langle símbolo \rangle}{\langle verbalización NVDA \rangle}`

El comando `\spnewread` trabaja en *modo texto* y recibe dos argumentos obligatorios: $\{\langle símbolo \rangle\}$ y $\{\langle verbalización NVDA \rangle\}$ los cuales se almacenan de forma interna para ser utilizados como $\{\langle argumento \rangle\}$ en `\spread`. Solo se establece como NVDA/MathCAT verbalizará el $\{\langle símbolo \rangle\}$ al utilizarlo con `\spread` sin modificar la impresión de este.

El argumento $\{\langle símbolo \rangle\}$ no debe estar registrado. Si ya está registrado se devolverá un “error” y se detendrá la compilación del documento. La declaración es «global» y se almacena en el módulo `spintent.lua`. Por ejemplo: `\spnewread{\cong}{es-congruente-con}` registrará ‘`\cong`’ y cuando se utilice `\spread{\cong}` se imprimirá ‘≅’ en la salida PDF y NVDA/MathCAT verbalizará «*es congruente con*».

- El $\{\langle símbolo \rangle\}$ debe ser un único comando, pero puede ser uno definido por el usuario. Por ejemplo, con `\newcommand{\xcuad}{x^2}`, `\spnewread{\xcuad}{x-al-cuadrado}` registrará ‘`\xcuad`’ y cuando se utilice `\spread{\xcuad}` se imprimirá ‘x²’ en la salida PDF y NVDA/MathCAT verbalizará «*x al cuadrado*».
- Los espacios y los guiones bajos de $\{\langle verbalización NVDA \rangle\}$ se convierten en guiones ‘-’, por lo que {es congruente con} equivale a {es-congruente-con}. Los guiones repetidos se reducen a uno y se eliminan los del inicio y del final. Si el resultado queda vacío se devolverá un “error” y se detendrá la compilación del documento.

El comando `\spread`

```
\spread <{<símbolo>} <spread[<key = val>]>{<símbolo>}>
\spread[<read-arg={<verbalización NVDA>}>]{<comandos LATEX>}>
```

El comando `\spread` trabaja en *modo matemático* y recibe el *argumento obligatorio* de la forma `{<símbolo>}` previamente registrado por el comando `\spnewread` o `{<comandos LATEX>}` para ser usado junto a la llave `read-arg`.

Si el argumento es `{<símbolo>}`, `$\spread{<cong>}$` imprimirá ‘ $\cong$ ’ en la salida PDF y el argumento `{<verbalización NVDA>}` establecido al registrar `{<símbolo>}` con `\spnewread` se verbalizará en NVDA/MathCAT.

Si el argumento es `{<comandos LATEX>}` y no se establece la llave `read-arg` se devolverá un “error” y se detendrá la compilación del documento.

Llaves para `\spread`

`read-arg = {<verbalización NVDA>}` default: *no usado*

Establece la *verbalización* del argumento `{<comandos LATEX>}`. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’ y sobrescribe la verbalización del `{<símbolo>}` si está registrado previamente. Por ejemplo: `$\spread[read-arg={es-paralela-a}]{\parallel}$` imprimirá ‘ $\parallel$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*es paralela a*».

`silent = {<true | false>}` default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del argumento `{<símbolo>}` o `{<comandos LATEX>}` pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

- Al utilizar *silent* no es necesario que el argumento `{<símbolo>}` esté registrado o usar la llave `read-arg` con `{<comandos LATEX>}`. Dependiendo de que contenga `{<argumento>}` puede que *silent* no sea suficiente, en ese caso utilice `read-arg={_}`.

El comando `\spusep`

```
\spusep <{\{ | [ | ( | ) | ] | \} }> <spusep[<key = val>]{<{\{ | [ | ( | ) | ] | \} }>}>
```

El comando `\spusep` trabaja en *modo matemático* y recibe el *argumento obligatorio* `{<{\{ | [ | ( | ) | ] | \} }>}`, el cual debe ser «*solo uno*» de los paréntesis soportados: ‘(’, ‘)’, ‘[’, ‘]’, ‘{’, ‘}’ o ‘\’.

Si no se proporciona el *argumento opcional* `[<key = val>]`, `$\spusep{< >}$` imprimirá ‘(’ en la salida PDF y NVDA/MathCAT verbalizará «*abrir paréntesis*» acorde a su propia heurística.

- El motivo de existencia de `\spusep` es que los comandos `\left`, `\right`, `\Uleft` y `\Uright` provistos por Lua^T_E^X crean un *grupo* implícito que dificulta el código para *tagged* PDF cuando se necesita interrumpir algo entre ellos, por ejemplo al construir expresiones complejas con `\spnZ`, `\spmQ` o `\spdiv`. El comando `\spusep` imprime el *delimitador* sin crear ningún grupo.

Llaves para `\spusep`

`size = {<none | big | Big | bigg | Bigg>}` default: *none*

Establece el *tamaño del paréntesis* pasado como `{<argumento>}` que se imprimirá en la salida PDF al ejecutar el comando sin modificar la verbalización de este. Si se establece en *none*, se imprime el `{<argumento>}` en el tamaño por defecto del modo matemático en curso; si se establece en *big*, se usa `\bigl/\bigr` antes del `{<argumento>}`; si se establece en *Big*, se usa `\Bigl/\Bigr` antes del `{<argumento>}`; si se establece en *bigg*, se usa `\biggl/\biggr` antes del `{<argumento>}`; y si se establece en *Bigg*, se usa `\Biggl/\Biggr` antes del `{<argumento>}`.

`silent = {<true | false>}` default: *false*

Establece si se *silencia* en NVDA/MathCAT la verbalización del `{<argumento>}` pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spdsym`

```
\spdsym <spdsym[<key = val>]>
```

El comando `\spdsym` trabaja en *modo matemático* y recibe el *argumento opcional* `[<key = val>]` mediante el cual se establece la «*forma*» en la que se imprimirá el *símbolo de división* y como se verbalizará en NVDA/MathCAT.

Si no se proporciona el *argumento opcional* `[<key = val>]`, `$\spdsym$` imprimirá ‘:’ en la salida PDF y NVDA/MathCAT verbalizará «*dividido*».

Llaves para \spdsym

`prefix = {⟨prefijo⟩}` default: no usado

Establece el *prefijo* que será *verbalizado* por NVDA/MathCAT «antes» de *verbalizar* el {⟨valor⟩} establecido por la llave `read-sym`. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`suffix = {⟨sufijo⟩}` default: no usado

Establece el *sufijo* que será *verbalizado* por NVDA/MathCAT «después» de *verbalizar* el {⟨valor⟩} establecido por la llave `read-sym`. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`set-sym = {⟨old-style | two-dots | slash⟩}` default: two-dots

Establece el *símbolo de división* que se imprimirá en la salida PDF. Si se establece en `old-style`, imprimirá '÷'; si se establece en `two-dots`, imprimirá '⋮' y si se establece `slash`, imprimirá '/'.

`read-sym = {⟨dividido | dividido-por | dividido-entre⟩}` default: dividido

Establece la *forma* con la cual NVDA/MathCAT verbalizará el *símbolo de división*. Si se establece en `dividido`, NVDA/MathCAT verbalizará «dividido»; si se establece en `dividido-por`, NVDA/MathCAT verbalizará «dividido por» y si se establece en `dividido-entre`, NVDA/MathCAT verbalizará «dividido entre».

El comando \spmsym

`\spmsym` `\spmsym` `\spmsym[⟨key = val⟩]`

El comando `\spmsym` trabaja en *modo matemático* y recibe el *argumento opcional* [⟨key = val⟩] mediante el cual se establece la «forma» en la que se imprimirá el *símbolo de multiplicación* y como se verbalizará en NVDA/MathCAT.

Si no se proporciona el *argumento opcional* [⟨key = val⟩], `$\spmsym$` imprimirá '·' en la salida PDF y NVDA/MathCAT verbalizará «por».

Llaves para \spmsym

`prefix = {⟨prefijo⟩}` default: no usado

Establece el *prefijo* que será *verbalizado* por NVDA/MathCAT «antes» de *verbalizar* el {⟨valor⟩} establecido por la llave `read-sym`. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`suffix = {⟨sufijo⟩}` default: no usado

Establece el *sufijo* que será *verbalizado* por NVDA/MathCAT «después» de *verbalizar* el {⟨valor⟩} establecido por la llave `read-sym`. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`set-sym = {⟨cross | dot | asterisk⟩}` default: dot

Establece el *símbolo de multiplicación* que se imprimirá en la salida PDF. Si se establece en `cross`, imprimirá '×'; si se establece en `dot`, imprimirá '⋅' y si se establece en `asterisk`, imprimirá '*'.

`read-sym = {⟨por | multiplicado-por | veces⟩}` default: por

Establece la *forma* con la cual NVDA/MathCAT verbalizará el *símbolo de multiplicación*. Si se establece en `por`, NVDA/MathCAT verbalizará «por»; si se establece en `multiplicado-por`, NVDA/MathCAT verbalizará «multiplicado por» y si se establece en `veces`, NVDA/MathCAT verbalizará «veces».

`not-print = {⟨true | false⟩}` default: false

Deshabilita la *impresión* del *símbolo de multiplicación*. Cuando se establece esta llave, NVDA/MathCAT verbalizará el *símbolo de multiplicación* acorde al valor establecido por la llave `read-sym`, pero, no se imprimirá este en la salida PDF.

6 Utilidades para números

El paquete `spintent` provee utilidades para trabajar con números descritas en esta sección.

El comando \spdiv

`\spdiv` `\spdiv{⟨dividendo⟩}{⟨divisor⟩}` `\spdiv[⟨key = val⟩]{⟨dividendo⟩}{⟨divisor⟩}`

El comando `\spdiv` trabaja en *modo matemático* y recibe dos *argumentos obligatorios* {⟨dividendo⟩} y {⟨divisor⟩}.

Si no se proporciona el *argumento opcional* [⟨key = val⟩], `$\spdiv{20}{26}$` imprimirá '20 : 26' en la salida PDF y NVDA/MathCAT verbalizará «veinte dividido veintiséis»

Llaves para `\spdiv`

El comando `\spdiv` posee las *meta-keys* `cifras-sep`, `read-sign`, `read-period-sep` y `notation-sym` pasadas al comando `\spnum` (§3.1.1) junto a las *meta-keys* `set-sym` y `read-sym` pasadas al comando `\spdsym` (pág. 13).

`wrap-parens` = { *true* | *false* } default: *false*

Establece si se *imprimen* paréntesis redondos ‘(...)’ alrededor de los *argumentos* { *dividendo* } y { *divisor* } pasados a `\spdiv`. Si se establece en *true*, se imprimirán paréntesis redondos alrededor de ambos *argumentos*; si se establece en *false*, no se imprimirán los paréntesis.

`read-parens` = { *true* | *false* } default: *true*

Establece si NVDA/MathCAT *verbalizará* los paréntesis redondos ‘(...)’ alrededor de los *argumentos* { *dividendo* } y { *divisor* } pasados a `\spdiv`. Si se establece en *true*, NVDA/MathCAT verbalizará la lectura de los paréntesis; si se establece en *false*, NVDA/MathCAT no los verbalizará.

El comando `\spmul`

`\spmul` `\spmul{factor}{factor}` `\spmul[key = val]{factor}{factor}`

El comando `\spmul` trabaja en *modo matemático* y recibe dos *argumentos obligatorios* { *factor* } y { *factor* }. Si no se proporciona el *argumento opcional* [*key = val*], `\spmul{20}{26}` imprimirá ‘20 · 26’ en la salida PDF y NVDA/MathCAT verbalizará «veinte por veintiséis».

Llaves para `\spmul`

El comando `\spmul` posee las *meta-keys* `cifras-sep`, `read-sign`, `read-period-sep` y `notation-sym` pasadas al comando `\spnum` (§3.1.1) junto a las *meta-keys* `set-sym` y `read-sym` pasadas al comando `\spmsym` (pág. 13).

`wrap-parens` = { *true* | *false* } default: *false*

Establece si se *imprimen* paréntesis redondos ‘(...)’ alrededor de los *argumentos* { *factor* } y { *factor* } pasados a `\spmul`. Si se establece en *true*, se imprimirán paréntesis redondos alrededor de ambos *argumentos*; si se establece en *false*, no se imprimirán los paréntesis.

`read-parens` = { *true* | *false* } default: *true*

Establece si NVDA/MathCAT *verbalizará* los paréntesis redondos ‘(...)’ alrededor de los *argumentos* { *factor* } y { *factor* } pasados a `\spmul`. Si se establece en *true*, NVDA/MathCAT verbalizará la lectura de los paréntesis; si se establece en *false*, NVDA/MathCAT no los verbalizará.

El comando `\spnD`

`\spnD` `\spnD` `\spnD(número)` `\spnD[key = val](número)` `\spnD(letra)` `\spnD[key = val](letra)`

El comando `\spnD` trabaja en *modo matemático* y recibe el *argumento opcional delimitado por paréntesis* de la forma (*número*) que se procesa internamente por `\spnum` o (*letra*) que se ejecuta de manera directa.

Si no se proporciona el *argumento opcional* [*key = val*] ni el *argumento opcional delimitado por paréntesis*, `\spnD` imprimirá ‘D(*n*)’ en la salida PDF y NVDA/MathCAT verbalizará «divisores de ene».

Si se usa el *argumento* (*número*), `\spnD(6)` imprimirá ‘D(6)’ en la salida PDF y NVDA/MathCAT verbalizará «divisores de seis».

Si se usa el *argumento* (*letra*), `\spnD(m)` imprimirá ‘D(*m*)’ en la salida PDF y NVDA/MathCAT verbalizará «divisores de eme».

🚫 El número 0 no es válido: `\spnD(0)` devolverá un “error” y se detendrá la compilación del documento.

Llaves para `\spnD`

`concept` = { *true* | *false* } default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «divisores de» al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT verbalizará «divisores de» y el (*argumento*); si se establece en *false*, NVDA/MathCAT solo verbalizará el (*argumento*).

`prefix` = { *prefijo* } default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el concepto «divisores de». El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

`result` = { *true* | *false* } default: *false*

Establece si se *imprime* el resultado de calcular los *divisores* del *argumento* (*número*). Si se establece en *true*, `\spnD[result=true](6)` imprimirá ‘D(6) = {1, 2, 3, 6}’ en la salida PDF y NVDA/MathCAT verbalizará «divisores de seis son uno, dos, tres y seis»; si se establece en *false*, no se imprimirá ni verbalizará el resultado de calcular los *divisores*.

`result-num` = { *número natural* } default: 5

Establece la *cantidad* de números que se mostrarán en la salida PDF al establecer `result=true`. Si se establece en **2**, solo se imprimirán dos valores en la salida PDF; si no se establece, se imprimirán como máximo **5** valores y luego se añadirá ‘...’ en la salida PDF.

`silent = {⟨true | false⟩}` default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «divisores de» y del (*argumento*) pasado al comando sin modificar la impresión de este. Si se establece en **true**, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en **false**, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spnM`

`\spnM` `\spnM` `\spnM(⟨número⟩)` `\spnM[⟨key = val⟩](⟨número⟩)` `\spnM(⟨letra⟩)` `\spnM[⟨key = val⟩](⟨letra⟩)`

El comando `\spnM` trabaja en *modo matemático* y recibe el *argumento opcional delimitado por paréntesis* de la forma (*número*) que se procesa internamente por `\spnum` o (*letra*) que se ejecuta de manera directa.

Si no se proporciona el *argumento opcional* [*key = val*] ni el *argumento opcional delimitado por paréntesis*, `\spnM` imprimirá ‘M(*n*)’ en la salida PDF y NVDA/MathCAT verbalizará «múltiplos de ene».

Si se usa el argumento (*número*), `\spnM(6)` imprimirá ‘M(6)’ en la salida PDF y NVDA/MathCAT verbalizará «múltiplos de seis».

Si se usa el argumento (*letra*), `\spnM(m)` imprimirá ‘M(*m*)’ en la salida PDF y NVDA/MathCAT verbalizará «múltiplos de eme».

🚫 El número 0 no es válido: `\spnM(0)` devolverá un “error” y se detendrá la compilación del documento.

Llaves para `\spnM`

`concept = {⟨true | false⟩}` default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «múltiplos de» al ejecutar el comando sin modificar la impresión de este. Si se establece en **true**, NVDA/MathCAT verbalizará «múltiplos de» y el (*argumento*); si se establece en **false**, NVDA/MathCAT solo verbalizará el (*argumento*).

`prefix = {⟨prefijo⟩}` default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el concepto «múltiplos de». El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

`result = {⟨true | false⟩}` default: *false*

Establece si se *imprime* el resultado de calcular los *múltiplos* del argumento (*número*). Si se establece en **true**, `\spnM[result=true](6)` imprimirá ‘M(6) = {6, 12, 18, 24, 30, ...}’ en la salida PDF y NVDA/MathCAT verbalizará «múltiplos de seis son seis, doce, dieciocho, veinticuatro, treinta puntos suspensivos»; si se establece en **false**, no se imprimirá ni verbalizará el resultado de calcular los *múltiplos*.

`result-num = {⟨número natural⟩}` default: *5*

Establece la *cantidad* de números que se mostrarán en la salida PDF al establecer `result=true`. Si se establece en **2**, solo se imprimirán dos valores en la salida PDF; si no se establece, se imprimirán como máximo **5** valores y luego se añadirá ‘...’ en la salida PDF.

`silent = {⟨true | false⟩}` default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «múltiplos de» y del (*argumento*) pasado al comando sin modificar la impresión de este. Si se establece en **true**, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en **false**, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spmcD`

`\spmcD` `\spmcD` `\spmcD(⟨número, número, ...⟩)` `\spmcD(⟨letra, letra, ...⟩)` `\spmcD[⟨key = val⟩](⟨número, número, ...⟩)`

El comando `\spmcD` trabaja en *modo matemático* y recibe el *argumento opcional delimitado por paréntesis* de la forma (*número, número, ...*) que se procesa internamente por `\spnum` o (*letra, letra, ...*) que se ejecuta de manera directa.

Si no se proporciona el *argumento opcional* [*key = val*] ni el *argumento opcional delimitado por paréntesis*, `\spmcD` imprimirá ‘mcd’ en la salida PDF y NVDA/MathCAT verbalizará «máximo común divisor».

Si se usa el argumento (*número, número, ...*), `\spmcD(2, 3, 6)` imprimirá ‘mcd(2, 3, 6)’ en la salida PDF y NVDA/MathCAT verbalizará «máximo común divisor entre dos, tres y seis».

Si se usa el argumento (*letra, letra, ...*), `\spmcD(m, n)` imprimirá ‘mcd(*m, n*)’ en la salida PDF y NVDA/MathCAT verbalizará «máximo común divisor entre eme y ene».

🚫 Los argumentos (*número, número, ...*) o (*letra, letra, ...*) requiere al menos dos elementos.

Llaves para `\spmc`

`concept = {\true | false}` default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto establecido por la llave `read-cmd` al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT verbalizará «*máximo común divisor ...*» o «*m c d ...*» y el (*argumento*); si se establece en *false*, NVDA/MathCAT solo verbalizará el (*argumento*).

`read-cmd = {\terse | clear}` default: *clear*

Establece la *forma* en la que NVDA/MathCAT verbalizará el *concepto* asociado al comando sin modificar la impresión de este. Si se establece en *terse*, NVDA/MathCAT verbalizará «*m c d*»; si se establece en *clear*, NVDA/MathCAT verbalizará «*máximo común divisor*».

`upper-case = {\true | false}` default: *false*

Establece si se usarán *mayúsculas* en la salida PDF al ejecutar el comando sin modificar la verbalización de este. Si se establece en *true*, se imprimirá 'MCD' en la salida PDF; si se establece en *false*, se imprimirá 'mcd' en la salida PDF.

`prefix = {\prefijo}` default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el concepto establecido por la llave `read-cmd`. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`result = {\true | false}` default: *false*

Establece si se *imprime* el resultado de calcular el «*máximo común divisor*» del argumento (*número, número, ...*). Si se establece en *true*: `\spmc[result=true](2, 3, 6)` imprimirá 'mcd(2, 3, 6) = 1' en la salida PDF y NVDA/MathCAT verbalizará «*máximo común divisor entre dos, tres y seis es igual a uno*»; si se establece en *false* no se imprimirá ni verbalizará el resultado de calcular el «*máximo común divisor*».

`silent = {\true | false}` default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «*máximo común divisor ...*» y del (*argumento*) pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spmc`

`\spmc` `\spmc` `\spmc(\número, número, ...)` `\spmc(\letra, letra, ...)` `\spmc[key = val](\número, número, ...)`

El comando `\spmc` trabaja en *modo matemático* y recibe el *argumento opcional delimitado por paréntesis* de la forma (*número, número, ...*) que se procesa internamente por `\spnum` o (*letra, letra, ...*) que se ejecuta de manera directa.

Si no se proporciona el *argumento opcional* [*key = val*] ni el *argumento opcional delimitado por paréntesis*, `\spmc` imprimirá 'mcm' en la salida PDF y NVDA/MathCAT verbalizará «*mínimo común múltiplo*».

Si se usa el argumento (*número, número, ...*), `\spmc(2, 3, 6)` imprimirá 'mcm(2, 3, 6)' en la salida PDF y NVDA/MathCAT verbalizará «*mínimo común múltiplo entre dos, tres y seis*».

Si se usa el argumento (*letra, letra, ...*), `\spmc(m, n)` imprimirá 'mcm(m, n)' en la salida PDF y NVDA/MathCAT verbalizará «*mínimo común múltiplo entre eme y ene*».

🔹 Los argumentos (*número, número, ...*) o (*letra, letra, ...*) requiere al menos dos elementos.

Llaves para `\spmc`

`concept = {\true | false}` default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto establecido por la llave `read-cmd` al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT verbalizará «*mínimo común múltiplo ...*» o «*m c m ...*» y el (*argumento*); si se establece en *false*, NVDA/MathCAT solo verbalizará el (*argumento*).

`read-cmd = {\terse | clear}` default: *clear*

Establece la *forma* en la que NVDA/MathCAT verbalizará el *concepto* asociado al comando sin modificar la impresión de este. Si se establece en *terse*, NVDA/MathCAT verbalizará «*m c m*»; si se establece en *clear*, NVDA/MathCAT verbalizará «*mínimo común múltiplo*».

`upper-case = {\true | false}` default: *false*

Establece si se usarán *mayúsculas* en la salida PDF al ejecutar el comando sin modificar la verbalización de este. Si se establece en *true*, se imprimirá 'MCM' en la salida PDF; si se establece en *false*, se imprimirá 'mcm' en la salida PDF.

`prefix = {\prefijo}` default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el concepto establecido por la llave `read-cmd`. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`result = {\true | false}` default: *false*

Establece si se *imprime* el resultado de calcular el «*mínimo común múltiplo*» del argumento (*⟨número, número, ...⟩*). Si se establece en *true*: $\$ \backslash \text{spmcm}[\text{result}=\text{true}](2, 3, 6) \$$ imprimirá ‘mcm(2, 3, 6) = 6’ en la salida PDF y NVDA/MathCAT verbalizará «*mínimo común múltiplo entre dos, tres y seis es igual a seis*»; si se establece en *false* no se imprimirá ni verbalizará el resultado de calcular el «*mínimo común múltiplo*».

`silent = {⟨true | false⟩}` default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «*mínimo común múltiplo ...*» y del (*⟨argumento⟩*) pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spnZ`

`\spnZ` $\backslash \text{spnZ}\{\langle \text{número entero} \rangle\}$ $\backslash \text{spnZ}[\langle \text{key} = \text{val} \rangle]\{\langle \text{número entero} \rangle\}$

El comando `\spnZ` trabaja en *modo matemático* y recibe el *argumento* obligatorio $\{\langle \text{número entero} \rangle\}$ el cual se procesa internamente bajo el comando `\spnum`. Este comando está dedicado exclusivamente al manejo de «*números enteros*», y añade algunas utilidades prácticas para el demarcado e interacción con NVDA/MathCAT.

Llaves para `\spnZ`

El comando `\spnZ` posee las *⟨meta-keys⟩* *cifras-sep* y *read-sign* pasadas al comando `\spnum` (§3.1.1).

`wrap-parens = {⟨true | false⟩}` default: *false*

Establece si se *imprimen* paréntesis redondos ‘(...)’ alrededor del argumento $\{\langle \text{número entero} \rangle\}$ pasado a `\spnZ`. Si se establece en *true*, se imprimirán paréntesis redondos alrededor del $\{\langle \text{número entero} \rangle\}$; si se establece en *false*, no se imprimirán los paréntesis.

`read-parens = {⟨true | false⟩}` default: *true*

Establece si NVDA/MathCAT *verbalizará* los paréntesis redondos ‘(...)’ alrededor del *argumento* $\{\langle \text{número entero} \rangle\}$ pasado a `\spnZ`. Si se establece en *true*, NVDA/MathCAT verbalizará la lectura del paréntesis ‘(...)’; si se establece en *false*, NVDA/MathCAT no los verbalizará.

El comando `\spmQ`

`\spmQ` $\backslash \text{spmQ}\{\langle \text{entero} \rangle\}\{\langle \text{numerador} \rangle\}\{\langle \text{denominador} \rangle\}$ $\backslash \text{spmQ}[\langle \text{key} = \text{val} \rangle]\{\langle \text{entero} \rangle\}\{\langle \text{numerador} \rangle\}\{\langle \text{denominador} \rangle\}$

El comando `\spmQ` trabaja en *modo matemático* y recibe *tres argumentos obligatorios*: $\{\langle \text{número entero} \rangle\}$, $\{\langle \text{numerador} \rangle\}$ y $\{\langle \text{denominador} \rangle\}$, los procesa internamente y devuelve un «*número mixto*».

Si no se proporciona el *argumento opcional* $[\langle \text{key} = \text{val} \rangle]$, $\$ \backslash \text{spmQ}\{2\}\{5\}\{7\} \$$ imprimirá ‘ $2\frac{5}{7}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*dos y la fracción con numerador cinco y denominador siete*».

- En el caso de las fracciones, y en especial con los «*números mixtos*», no se pueden manejar muchas cosas desde el lado de *spintent*. Por ejemplo, la entrada $\$3\backslash \text{frac}\{5\}\{7\} \$$ con *tagged* PDF se verbaliza por NVDA/MathCAT como «*tres más cinco séptimos*», lo cual no está mal, pero pedagógicamente hablando no es del todo correcto.

Llaves para `\spmQ`

`join-word = {⟨entero | enteros | y⟩}` default: *y*

Establece la *forma* en la que NVDA/MathCAT verbaliza la conexión entre el entero y la fracción que forman el *número mixto*. Si se establece en *entero*, $\$ \backslash \text{spmQ}[\text{join-word}=\text{entero}]\{1\}\{5\}\{7\} \$$ imprimirá ‘ $1\frac{5}{7}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*un entero la fracción con numerador cinco y denominador siete*»; si se establece en *enteros*, $\$ \backslash \text{spmQ}[\text{join-word}=\text{enteros}]\{2\}\{5\}\{7\} \$$ imprimirá ‘ $2\frac{5}{7}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*dos enteros la fracción con numerador cinco y denominador siete*»; si se establece en *y*, $\$ \backslash \text{spmQ}[\text{join-word}=y]\{2\}\{5\}\{7\} \$$ imprimirá ‘ $2\frac{5}{7}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*dos y la fracción con numerador cinco y denominador siete*».

`use-spnz = {⟨true | false⟩}` default: *true*

Establece si el $\{\langle \text{numerador} \rangle\}$ y el $\{\langle \text{denominador} \rangle\}$ que forman la fracción del *número mixto* se procesan internamente bajo el comando `\spnZ`, añadiendo así *semántica de entero* en la estructura *MathML* de la salida PDF. Si se establece en *true*, $\{\langle \text{numerador} \rangle\}$ y $\{\langle \text{denominador} \rangle\}$ se pasarán a `\spnZ` y, por ejemplo, $\$ \backslash \text{spmQ}\{2\}\{5\}\{7\} \$$ se verbalizará «*dos y la fracción con numerador cinco y denominador siete*»; si se establece en *false*, los argumentos se tipografían directamente sin procesamiento semántico adicional y NVDA/MathCAT aplica sus propias reglas a la fracción: $\$ \backslash \text{spmQ}[\text{use-spnz}=\text{false}]\{2\}\{5\}\{7\} \$$ se verbalizará «*dos y cinco séptimos*».

`use-dfrac = {⟨true | false⟩}` default: *false*

Establece si la fracción del *número mixto* se imprime usando `\dfrac` en lugar de `\frac`. Si se establece en *true*, al usar $\$ \backslash \text{spmQ}[\text{use-dfrac}=\text{true}]\{2\}\{5\}\{7\} \$$ imprimirá la fracción en tamaño *display* ‘ $2\frac{5}{7}$ ’ en la salida PDF; si se establece en *false*, la fracción se imprime con `\frac` siguiendo el tamaño matemático del contexto en curso.

`wrap-parens = {⟨true | false⟩}` default: *false*

Establece si se *imprimen* paréntesis redondos ‘(...)’ alrededor del *número mixto* completo devuelto por `\spmQ`. Si se establece en `true`, se imprimirán paréntesis redondos cuyo tamaño se ajusta automáticamente al estilo matemático en curso; si se establece en `false`, no se imprimirán los paréntesis.

`read-parens = {\true | false}` default: `true`

Establece si NVDA/MathCAT *verbalizará* los paréntesis redondos ‘(...)’ alrededor del *número mixto* cuando la llave `wrap-parens` está activa. Si se establece en `true`, NVDA/MathCAT verbalizará la lectura de los paréntesis; si se establece en `false`, NVDA/MathCAT no los verbalizará.

El comando `\sprazon`

`\sprazon` `\sprazon{\langle antecendente \rangle}{\langle consecuente \rangle}` `\sprazon[key = val]{\langle antecendente \rangle}{\langle consecuente \rangle}`

El comando `\sprazon` trabaja en *modo matemático* y recibe *dos argumentos obligatorios*: `{\langle antecendente \rangle}` y `{\langle consecuente \rangle}`, los tipografía como una fracción y devuelve una «razón».

Si no se proporciona el *argumento opcional* `[key = val]`, `\sprazon{3}{4}` imprimirá ‘ $\frac{3}{4}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «tres es a cuatro».

- El `{\langle antecendente \rangle}` y el `{\langle consecuente \rangle}` se tipografían directamente, sin procesamiento semántico adicional.

Llaves para `\sprazon`

`silent = {\true | false}` default: `false`

Establece si se *silencia* en NVDA/MathCAT la verbalización de la *razón* pasada al comando sin modificar la impresión de esta. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, NVDA/MathCAT sí verbalizará la salida del comando.

`use-dfrac = {\true | false}` default: `false`

Establece si la fracción de la *razón* se imprime usando `\dfrac` en lugar de `\frac`. Si se establece en `true`, al usar `\sprazon[use-dfrac=true]{3}{4}` imprimirá la fracción en tamaño *display* ‘ $\frac{3}{4}$ ’ en la salida PDF; si se establece en `false`, la fracción se imprime con `\frac` siguiendo el tamaño matemático del contexto en curso.

`inline = {\true | false}` default: `false`

Establece si la *razón* se imprime en línea con dos puntos ‘:’ en lugar de una fracción. Si se establece en `true`, `\sprazon[inline=true]{3}{4}` imprimirá ‘3 : 4’ en la salida PDF y NVDA/MathCAT verbalizará «tres es a cuatro»; si se establece en `false`, la razón se imprime como una fracción.

- La llave `use-dfrac` no tiene efecto cuando la llave `inline` se establece en `true`, porque en ese caso no se imprime una fracción.

El comando `\spProp`

`\spProp` `\spProp{\langle a \rangle}{\langle b \rangle}{\langle c \rangle}{\langle d \rangle}` `\spProp[key = val]{\langle a \rangle}{\langle b \rangle}{\langle c \rangle}{\langle d \rangle}`

El comando `\spProp` trabaja en *modo matemático* y recibe *cuatro argumentos obligatorios*: `{\langle a \rangle}` y `{\langle b \rangle}` forman la primera *razón*, `{\langle c \rangle}` y `{\langle d \rangle}` forman la segunda, y devuelve una «proporción» con las dos razones unidas por el signo igual.

Si no se proporciona el *argumento opcional* `[key = val]`, `\spProp{3}{4}{6}{8}` imprimirá ‘ $\frac{3}{4} = \frac{6}{8}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «tres es a cuatro como seis es a ocho».

- Los cuatro argumentos se tipografían directamente, sin procesamiento semántico adicional.

Llaves para `\spProp`

`silent = {\true | false}` default: `false`

Establece si se *silencia* en NVDA/MathCAT la verbalización de la *proporción* pasada al comando sin modificar la impresión de esta. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, NVDA/MathCAT sí verbalizará la salida del comando.

`use-dfrac = {\true | false}` default: `false`

Establece si las fracciones de la *proporción* se imprimen usando `\dfrac` en lugar de `\frac`. Si se establece en `true`, al usar `\spProp[use-dfrac=true]{3}{4}{6}{8}` imprimirá las fracciones en tamaño *display* ‘ $\frac{3}{4} = \frac{6}{8}$ ’ en la salida PDF; si se establece en `false`, las fracciones se imprimen con `\frac` siguiendo el tamaño matemático del contexto en curso.

`inline = {\true | false}` default: `false`

Establece si las *razones* de la *proporción* se imprimen en línea con dos puntos ‘:’ en lugar de fracciones. Si se establece en `true`, `\spProp[inline=true]{3}{4}{6}{8}` imprimirá ‘3 : 4 = 6 : 8’ en la salida PDF y NVDA/MathCAT verbalizará «tres es a cuatro como seis es a ocho»; si se establece en `false`, las razones se imprimen como fracciones.

- La llave `use-dfrac` no tiene efecto cuando la llave `inline` se establece en `true`, porque en ese caso no se imprimen fracciones.

7 Utilidades para geometría

El paquete **spintent** provee comandos para trabajar con *geometría* descritos en esta sección. Todos los comandos trabajan en *modo matemático* y reciben como $\{\langle \text{argumento} \rangle\}$ base una *única* letra (mayúscula en el caso de puntos) con un modificador del tipo subíndice/prima, un número o comando.

Algunos comandos reciben $\{\langle \text{argumento} \rangle\}$ único (modo “*single*”) o más de uno (modo “*batch*”) donde opcionalmente se pueden separar por comas ‘,’.

- Las primas se debe pasar de forma directa ‘ $\prime$ ’ y los subíndices siempre deben escribirse usando llaves ‘ $A_{\{...\}}$ ’ y nunca sin estas ‘ A_n ’ que funciona solo por coincidencia. .

El comando $\backslash\text{spcoord}$

$\backslash\text{spcoord}$	$\backslash\text{spcoord}\{\langle \text{abscisa}, \text{ordenada} \rangle\}$	$\backslash\text{spcoord}[\langle \text{key} = \text{val} \rangle]\{\langle \text{abscisa}, \text{ordenada} \rangle\}$
	$\backslash\text{spcoord}\{\langle \text{abscisa}; \text{ordenada} \rangle\}$	$\backslash\text{spcoord}[\langle \text{key} = \text{val} \rangle]\{\langle \text{abscisa}; \text{ordenada} \rangle\}$

El comando $\backslash\text{spcoord}$ trabaja en *modo matemático* y recibe el *argumento obligatorio* $\{\langle \text{abscisa}, \text{ordenada} \rangle\}$ o $\{\langle \text{abscisa}; \text{ordenada} \rangle\}$ el cual se procesa internamente bajo el comando $\backslash\text{spnum}$ cuando es numérico no fraccionario.

Si no se proporciona el *argumento opcional* $[\langle \text{key} = \text{val} \rangle]$, $\backslash\text{spcoord}\{-3, 4\}$ imprimirá ‘ $(-3, 4)$ ’ en la salida PDF y NVDA/MathCAT verbalizará «menos tres coma cuatro»; $\backslash\text{spcoord}\{x_{\{1\}}, y_{\{1\}}\}$ imprimirá ‘ (x_1, y_1) ’ y verbalizará «*axis subíndice uno y subíndice uno*».

- El comando $\backslash\text{spcoord}$ solo acepta un *separador* ‘,’ o ‘;’ para la abscisa y la ordenada cuando son valores numéricos no fraccionarios.

Llaves para $\backslash\text{spcoord}$

$\text{silent} = \{\langle \text{true} \mid \text{false} \rangle\}$ default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del $\{\langle \text{argumento} \rangle\}$ pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

El comando $\backslash\text{spPoint}$

$\backslash\text{spPoint}$	$\backslash\text{spPoint}\{\langle \text{punto} \rangle\}$	$\backslash\text{spPoint}[\langle \text{key} = \text{val} \rangle]\{\langle \text{punto} \rangle\}$
	$\backslash\text{spPoint}\{\langle \text{punto} \rangle\}(\langle \text{abscisa}, \text{ordenada} \rangle)$	$\backslash\text{spPoint}[\langle \text{key} = \text{val} \rangle]\{\langle \text{punto} \rangle\}(\langle \text{abscisa}, \text{ordenada} \rangle)$
	$\backslash\text{spPoint}\{\langle \text{punto} \rangle\}(\langle \text{abscisa}; \text{ordenada} \rangle)$	$\backslash\text{spPoint}[\langle \text{key} = \text{val} \rangle]\{\langle \text{punto} \rangle\}(\langle \text{abscisa}; \text{ordenada} \rangle)$

El comando $\backslash\text{spPoint}$ trabaja en *modo matemático* y recibe el *argumento obligatorio* $\{\langle \text{punto} \rangle\}$ el cual es una *única letra mayúscula* con prima ‘ $\prime$ ’ o subíndice ‘ $_{\{...\}}$ ’ y el *argumento opcional delimitado por paréntesis* $(\langle \text{abscisa}, \text{ordenada} \rangle)$ el cual se procesa internamente bajo el comando $\backslash\text{spcoord}$.

Si no se proporciona el *argumento opcional* $[\langle \text{key} = \text{val} \rangle]$, $\backslash\text{spPoint}\{A\}$ imprimirá ‘ A ’ en la salida PDF y NVDA/MathCAT verbalizará «*punto A*»; $\backslash\text{spPoint}\{A_{\{1\}}\}$ imprimirá ‘ A_1 ’ y verbalizará «*punto A sub uno*».

Si se proporciona el *argumento opcional delimitado por paréntesis*, $\backslash\text{spPoint}\{A\}(-3, 4)$ imprimirá ‘ $A(-3, 4)$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*punto A menos tres coma cuatro*»; $\backslash\text{spPoint}\{A_{\{1\}}\}(-3, 4)$ imprimirá ‘ $A_1(-3, 4)$ ’ y verbalizará «*punto A sub uno menos tres coma cuatro*».

Llaves para $\backslash\text{spPoint}$

$\text{read-cmd} = \{\langle \text{school} \mid \text{mathcat} \rangle\}$ default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará el argumento $\{\langle \text{punto} \rangle\}$. Si se establece en *school*, NVDA/MathCAT verbalizará el argumento $\{\langle \text{punto} \rangle\}$ sin el prefijo «mayúscula»; $\backslash\text{spPoint}\{A\}$ se verbaliza «*punto A*»; si se establece en *mathcat*, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

$\text{prefix} = \{\langle \text{prefijo} \rangle\}$ default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el concepto «*punto*». El valor de esta llave *siempre* debe pasarse entre llaves ‘ $\{ \}$ ’.

$\text{concept} = \{\langle \text{true} \mid \text{false} \rangle\}$ default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «*punto*» al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT verbalizará el concepto y el $\{\langle \text{argumento} \rangle\}$; si se establece en *false*, NVDA/MathCAT solo verbalizará el $\{\langle \text{argumento} \rangle\}$ acorde al valor establecido por la llave *read-cmd*.

$\text{silent} = \{\langle \text{true} \mid \text{false} \rangle\}$ default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «*punto*» y del $\{\langle \text{argumento} \rangle\}$ pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spvec`

<code>\spvec</code>	<code>\spvec{⟨letra⟩}</code>	<code>\spvec{⟨punto, punto⟩}</code>
	<code>\spvec{⟨letra⟩} (⟨abscisa, ordenada⟩)</code>	<code>\spvec{⟨punto, punto⟩} (⟨abscisa, ordenada⟩)</code>
	<code>\spvec[⟨key = val⟩]{⟨letra⟩} (⟨abscisa, ordenada⟩)</code>	<code>\spvec[⟨key = val⟩]{⟨punto, punto⟩} (⟨abscisa, ordenada⟩)</code>

El comando `\spvec` trabaja en *modo matemático* y recibe un *argumento obligatorio* de la forma `{⟨letra⟩}` (minúscula) o `{⟨punto, punto⟩}` junto al *argumento opcional delimitado por paréntesis* `(⟨abscisa, ordenada⟩)` el cual se procesa internamente bajo el comando `\spcoord`.

Si el *argumento* es de la forma `{⟨punto, punto⟩}` con prima(s) ‘`’` o subíndice(s) ‘`_⟨...⟩`’ opcionales se aplica `\overrightarrow` en la impresión; si el *argumento* es de la forma `{⟨letra⟩}` (minúscula) con subíndice ‘`_⟨...⟩`’ opcional se aplica `\vec` en la impresión.

Si no se proporciona el *argumento opcional* `[⟨key = val⟩]`, `\spvec{a}` imprimirá ‘ $\vec{a}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «vector a »; `\spvec{AB}` imprimirá ‘ $\overrightarrow{AB}$ ’ y verbalizará «vector $A B$ ».

Si se proporciona el *argumento opcional delimitado por paréntesis*, `\spvec{a}(-3,4)` imprimirá ‘ $\vec{a} = (-3, 4)$ ’ en la salida PDF y NVDA/MathCAT verbalizará «vector a igual a menos tres coma cuatro»; `\spvec{AB}(-3,4)` imprimirá ‘ $\overrightarrow{AB} = (-3, 4)$ ’ y verbalizará «vector $A B$ igual a menos tres coma cuatro».

Llaves para `\spvec`

`read-cmd = {⟨school | mathcat⟩}` default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará el argumento `{⟨punto, punto⟩}`. Si se establece en *school*, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spvec{AB}` se verbaliza «vector $A B$ »; si se establece en *mathcat*, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix = {⟨prefijo⟩}` default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el concepto «vector». El valor de esta llave *siempre* debe pasarse entre llaves ‘`{ }`’.

`autofit = {⟨true | false⟩}` default: *false*

Establece si se *utiliza* `\vphantom{b}` al ejecutar el comando cuando se utiliza el argumento `{⟨letra⟩}`. Si se establece en *true*, se aplicará `\vphantom{b}` al inicio del argumento `{⟨letra⟩}`; si se establece en *false*, no se aplicará `\vphantom{b}`.

`concept = {⟨true | false⟩}` default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «vector» al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT verbalizará el concepto y el `{⟨argumento⟩}`; si se establece en *false*, NVDA/MathCAT solo verbalizará el `{⟨argumento⟩}` acorde al valor establecido por la llave `read-cmd`.

`silent = {⟨true | false⟩}` default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «vector» y del `{⟨argumento⟩}` pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\sprecta`

<code>\sprecta</code>	<code>\sprecta{⟨punto, punto⟩}</code>	<code>\sprecta{⟨letra⟩}</code>
	<code>\sprecta[⟨key = val⟩]{⟨punto, punto⟩}</code>	<code>\sprecta[⟨key = val⟩]{⟨letra⟩}</code>

El comando `\sprecta` trabaja en *modo matemático* y recibe un *argumento obligatorio* de la forma `{⟨letra⟩}` o `{⟨punto, punto⟩}` con prima(s) ‘`’` o subíndice(s) ‘`_⟨...⟩`’ opcionales. Si el *argumento* es de la forma `{⟨punto, punto⟩}` se aplica `\overleftarrow` en la impresión; si el *argumento* es de la forma `{⟨letra⟩}` (mayúscula o minúscula), se imprimirá solo la `{⟨letra⟩}` sin aplicar `\overleftarrow`.

Si no se proporciona el *argumento opcional* `[⟨key = val⟩]`, `\sprecta{AB}` imprimirá ‘ $\overleftarrow{AB}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «recta $A B$ »; `\sprecta{m}` imprimirá ‘ m ’ en la salida PDF y NVDA/MathCAT verbalizará «recta m ».

Llaves para `\sprecta`

`read-cmd = {⟨school | mathcat⟩}` default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará el argumento `{⟨punto, punto⟩}` y el argumento `{⟨letra⟩}` cuando está en mayúscula. Si se establece en *school*, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\sprecta{AB}` se verbaliza «recta $A B$ »; si se establece en *mathcat*, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix = {⟨prefijo⟩}` default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el concepto «recta». El valor de esta llave *siempre* debe pasarse entre llaves ‘`{ }`’.

`concept = {⟨true | false⟩}`

default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «*recta*» al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT verbalizará el concepto y el {⟨*argumento*⟩}; si se establece en *false*, NVDA/MathCAT solo verbalizará el {⟨*argumento*⟩} acorde al valor establecido por la llave `read-cmd`.

`silent = {⟨true | false⟩}`

default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «*recta*» y del {⟨*argumento*⟩} pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\sprayo`

`\sprayo {⟨punto, punto⟩}`
`\sprayo [⟨key = val⟩] {⟨punto, punto⟩}`

El comando `\sprayo` trabaja en *modo matemático* y recibe el *argumento obligatorio* {⟨*punto, punto*⟩} con prima(s) ‘*’* o subíndice(s) ‘*_ { . . . }*’ opcionales. Representa un *rayo* o *semirrecta* con origen en el primer punto y dirección hacia el segundo, aplicando `\overrightarrow` en la impresión.

Si no se proporciona el *argumento opcional* [⟨*key = val*⟩], `\sprayo{AB}` imprimirá ‘ $\overrightarrow{AB}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*rayo A B*».

Llaves para `\sprayo`

`read-cmd = {⟨school | mathcat⟩}`

default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará el argumento {⟨*punto, punto*⟩}. Si se establece en *school*, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\sprayo{AB}` se verbaliza «*rayo A B*»; si se establece en *mathcat*, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`read-sty = {⟨rayo | semirrecta⟩}`

default: *rayo*

Establece el *concepto* que verbalizará NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. Si se utiliza `\sprayo[read-sty=rayo]{AB}`, NVDA/MathCAT verbalizará «*rayo A B*»; si se utiliza `\sprayo[read-sty=semirrecta]{AB}`, NVDA/MathCAT verbalizará «*semirrecta A B*».

`prefix = {⟨prefijo⟩}`

default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el «*concepto*» establecido por la llave `read-sty`. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

`concept = {⟨true | false⟩}`

default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del «*concepto*» establecido por la llave `read-sty` al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT verbalizará el concepto y el {⟨*argumento*⟩}; si se establece en *false*, NVDA/MathCAT solo verbalizará el {⟨*argumento*⟩} acorde al valor establecido por la llave `read-cmd`.

`silent = {⟨true | false⟩}`

default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del «*concepto*» establecido por la llave `read-sty` y del {⟨*argumento*⟩} pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

- La impresión ‘ $\overrightarrow{AB}$ ’ en la salida PDF es idéntica en ambos casos. La elección depende de la terminología del libro de texto o del currículo nacional: en Chile y España se prefiere «semirrecta»; en México y otros países de América Latina se usa «rayo».

El comando `\spside`

`\spside {⟨punto, punto⟩}`
`\spside [⟨key = val⟩] {⟨punto, punto⟩}`

El comando `\spside` trabaja en *modo matemático* y recibe el *argumento obligatorio* {⟨*punto, punto*⟩} con prima(s) ‘*’* o subíndice(s) ‘*_ { . . . }*’ opcionales. Representa un *lado*, *segmento* o *trazo* determinado por dos puntos, aplicando `\overline` en la impresión.

Si no se proporciona el *argumento opcional* [⟨*key = val*⟩], `\spside{AB}` imprimirá ‘ $\overline{AB}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*lado A B*».

Llaves para `\spside`

`read-cmd` = { `<school | mathcat>` } default: `school`

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento { `<punto, punto>` }. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spside{AB}` se verbaliza «lado A B»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`read-sty` = { `<lado | segmento | trazo>` } default: `lado`

Establece el *concepto* que verbalizará NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. Si se utiliza `\spside[read-sty=lado]{AB}`, NVDA/MathCAT verbalizará «lado A B»; si se utiliza `\spside[read-sty=segmento]{AB}`, NVDA/MathCAT verbalizará «segmento A B» y si se utiliza `\spside[read-sty=trazo]{AB}`, NVDA/MathCAT verbalizará «trazo A B»

`prefix` = { `<prefijo>` } default: `no usado`

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el «concepto» establecido por la llave `read-sty`. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`concept` = { `<true | false>` } default: `true`

Establece si se *silencia* la verbalización en NVDA/MathCAT del «concepto» establecido por la llave `read-sty` al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT verbalizará el concepto y el { `<argumento>` }; si se establece en `false`, NVDA/MathCAT solo verbalizará el { `<argumento>` } acorde al valor establecido por la llave `read-cmd`.

`read-arg` = { `<verbalización NVDA>` } default: `no usado`

Sobrescribe el *concepto* establecido por la llave `read-sty` que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'. Por ejemplo: `\spside[read-arg={hipotenusa}]{AB}` imprimirá $\overline{AB}$ en la salida PDF y NVDA/MathCAT verbalizará «hipotenusa A B».

`silent` = { `<true | false>` } default: `false`

Establece si se *silencia* la verbalización en NVDA/MathCAT del «concepto» establecido por la llave `read-sty` y del { `<argumento>` } pasado al comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spmside`

`\spmside` `\spmside{<punto, punto>}`
 `\spmside[<key = val>]{<punto, punto>}`

El comando `\spmside` trabaja en *modo matemático* y recibe el *argumento obligatorio* { `<punto, punto>` } con prima(s) ' ' o subíndice(s) ' _{...}' opcionales. Representa la *medida* del *lado*, *segmento* o *trazo* determinado por dos puntos. La distinción entre `\spside` y `\spmside` es semántica: `\spside` representa el *objeto geométrico* $\overline{AB}$; `\spmside` representa su *medida* (un número real positivo).

Si no se proporciona el *argumento opcional* [`<key = val>`], `\spmside{AB}` imprimirá $\overline{AB}$ en la salida PDF y NVDA/MathCAT verbalizará «medida del lado A B».

Llaves para `\spmside`

`read-cmd` = { `<school | mathcat>` } default: `school`

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento { `<punto, punto>` }. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spmside{AB}` se verbaliza «medida del lado A B»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`read-sty` = { `<lado | segmento | trazo>` } default: `lado`

Establece el *concepto* que verbalizará NVDA/MathCAT después de verbalizar el término «medida del» al ejecutar el comando sin modificar la impresión de este. Si se utiliza `\spmside[read-sty=lado]{AB}`, NVDA/MathCAT verbalizará «medida del lado A B»; si se utiliza `\spmside[read-sty=segmento]{AB}`, NVDA/MathCAT verbalizará «medida del segmento A B» y si se utiliza `\spmside[read-sty=trazo]{AB}`, NVDA/MathCAT verbalizará «medida del trazo A B»

`prefix` = { `<prefijo>` } default: `no usado`

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «medida del» seguido por el «concepto» establecido por la llave `read-sty`. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`concept` = { `<true | false>` } default: `true`

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «medida del» y del «concepto» establecido por la llave `read-sty` al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT verbalizará el término «medida del», el concepto y el { `<argumento>` }; si se establece en `false`, NVDA/MathCAT solo verbalizará el { `<argumento>` } acorde al valor establecido por la llave `read-cmd`.

`read-arg = {⟨verbalización NVDA⟩}`

default: *no usado*

Sobrescribe el *concepto* establecido por la llave `read-sty` que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’ y se debe usar en conjunto con la llave `concept`.

Por ejemplo: `$\spmside[concept=false,read-arg={medida de la cuerda}]{AB}$` imprimirá ‘AB’ en la salida PDF y NVDA/MathCAT verbalizará «medida de la cuerda A B».

`print-m = {⟨true | false⟩}`

default: *false*

Establece si se *imprime* la notación visual ‘m’ antes aplicar `\overline` al {⟨argumento⟩}. Si se establece en *true*, `$\spmside[print-m=true]{AB}$` imprimirá ‘m $\overline{AB}$ ’ en la salida PDF, si se establece en *false* imprimirá ‘AB’ en la salida PDF. En ambos casos NVDA/MathCAT verbalizará «medida del lado A B».

`silent = {⟨true | false⟩}`

default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «medida del», del «concepto» establecido por la llave `read-sty` y del {⟨argumento⟩} pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spangle`

<code>\spangle</code>	<code>{⟨punto letra griega número⟩}</code>	<code>\spangle{⟨punto, punto, punto⟩}</code>
<code>\spangle</code>	<code>[⟨key = val⟩]{⟨punto letra griega número⟩}</code>	<code>\spangle[⟨key = val⟩]{⟨punto, punto, punto⟩}</code>

El comando `\spangle` trabaja en *modo matemático* y recibe un *argumento obligatorio* de la forma {⟨punto | letra griega | número⟩} o {⟨punto, punto, punto⟩} aplicando `\angle` o `\rightanglesqr` en la impresión. Solo los argumentos de la forma {⟨punto⟩} o {⟨punto, punto, punto⟩} admiten prima(s) ‘’ o subíndice(s) ‘_{}...’ opcionales.

Si no se proporciona el *argumento opcional* [⟨key = val⟩], `$\spangle{A}$` imprimirá ‘∠A’ en la salida PDF y NVDA/MathCAT verbalizará «ángulo en A»; `$\spangle{\alpha}$` imprimirá ‘∠α’ en la salida PDF y NVDA/MathCAT verbalizará «ángulo alfa»; `$\spangle{1}$` imprimirá ‘∠1’ en la salida PDF y NVDA/MathCAT verbalizará «ángulo uno» y `$\spangle{AOB}$` imprimirá ‘∠AOB’ en la salida PDF y NVDA/MathCAT verbalizará «ángulo A O B».

- Si no se tiene instalada la fuente matemática NewCMath-Regular.otf parte del paquete `newcomputermodern` en el sistema o no se provee una fuente matemática con soporte para `\rightanglesqr` se usará `\rightangle` en su reemplazo.

Llaves para `\spangle`

`read-cmd = {⟨school | mathcat⟩}`

default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento {⟨punto⟩} o {⟨punto, punto, punto⟩}. Si se establece en *school*, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `$\spangle{AOB}$` se verbaliza «ángulo A O B»; si se establece en *mathcat*, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix = {⟨prefijo⟩}`

default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el concepto «ángulo». El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

`recto = {⟨true | false⟩}`

default: *false*

Establece si se *usará* `\rightanglesqr` en la impresión añadiendo el término «recto» a la verbalización en NVDA/MathCAT. Si se establece en *true*, `$\spangle[recto=true]{A}$` imprimirá ‘⊥A’ en la salida PDF y NVDA/MathCAT verbalizará «ángulo recto en A», `$\spangle[recto=true]{AOB}$` imprimirá ‘⊥AOB’ en la salida PDF y NVDA/MathCAT verbalizará «ángulo recto A O B»; si se establece en *false*, imprimirá usando `\angle` en la salida PDF y NVDA/MathCAT omitirá el término «recto».

`concept = {⟨true | false⟩}`

default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «ángulo» al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT verbalizará el concepto «ángulo» y el {⟨argumento⟩}; si se establece en *false*, NVDA/MathCAT solo verbalizará el {⟨argumento⟩} acorde al valor establecido por la llave `read-cmd`.

`silent = {⟨true | false⟩}`

default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «ángulo» y del {⟨argumento⟩} pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spmangle`

```
\spmangle <{punto | letra griega | número}> <{punto, punto, punto}>
\spmangle[<key = val>]{<punto | letra griega | número>} <{punto, punto, punto}>
```

El comando `\spmangle` trabaja en *modo matemático* y recibe un *argumento obligatorio* de la forma `{<punto | letra griega | número>}` o `{<punto, punto, punto>}` aplicando `\measuredangle` o `\measuredrightangle` en la impresión. Solo los argumentos de la forma `{<punto>}` o `{<punto, punto, punto>}` admiten prima(s) ‘`’` o subíndice(s) ‘`_`’ opcionales.

Si no se proporciona el *argumento opcional* `[<key = val>]`, `\spmangle{A}` imprimirá ‘ $\angle A$ ’ en la salida PDF y NVDA/MathCAT verbalizará «medida del ángulo en A»; `\spmangle{\alpha}` imprimirá ‘ $\angle \alpha$ ’ en la salida PDF y NVDA/MathCAT verbalizará «medida del ángulo alfa»; `\spmangle{1}` imprimirá ‘ $\angle 1$ ’ en la salida PDF y NVDA/MathCAT verbalizará «medida del ángulo uno» y `\spmangle{AOB}` imprimirá ‘ $\angle AOB$ ’ en la salida PDF y NVDA/MathCAT verbalizará «medida del ángulo A O B».

- La distinción entre `\spangle` y `\spmangle` es semántica: `\spangle` representa el *objeto geométrico* ‘ $\angle AOB$ ’; `\spmangle` representa su *medida* (un valor en grados).

Llaves para `\spmangle`

```
read-cmd = {<school | mathcat>} default: school
```

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento `{<punto>}` o `{<punto, punto, punto>}`. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spmangle{AOB}` se verbaliza «medida del ángulo A O B»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

```
prefix = {<prefijo>} default: no usado
```

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «medida del» seguido por el concepto «ángulo». El valor de esta llave *siempre* debe pasarse entre llaves ‘`{ }`’.

```
recto = {<true | false>} default: false
```

Establece si se *usará* `\measuredrightangle` en la impresión añadiendo el término «recto» a la verbalización en NVDA/MathCAT. Si se establece en `true`, `\spmangle[recto=true]{A}` imprimirá ‘ $\angle A$ ’ en la salida PDF y NVDA/MathCAT verbalizará «medida del ángulo recto en A»; `\spmangle[recto=true]{AOB}` imprimirá ‘ $\angle AOB$ ’ en la salida PDF y NVDA/MathCAT verbalizará «medida del ángulo recto A O B»; si se establece en `false`, imprimirá usando `\measuredangle` en la salida PDF y NVDA/MathCAT omitirá el término «recto».

```
print-m = {<true | false>} default: false
```

Establece si se *imprime* la notación visual ‘m’ antes del argumento pasado al comando sin modificar la verbalización de este. Si se establece en `true`, `\spmangle[print-m=true]{AOB}` imprimirá ‘m $\angle AOB$ ’ en la salida PDF, si se establece en `false` imprimirá ‘ $\angle AOB$ ’ en la salida PDF. En ambos casos NVDA/MathCAT verbalizará «medida del ángulo A O B».

```
concept = {<true | false>} default: true
```

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «medida del» seguido por el concepto «ángulo» al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT verbalizará «medida del ángulo» y el `{<argumento>}`; si se establece en `false`, NVDA/MathCAT solo verbalizará el `{<argumento>}` acorde al valor establecido por la llave `read-cmd`.

```
silent = {<true | false>} default: false
```

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «medida del», del concepto «ángulo» y del `{<argumento>}` pasado al comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spang`

```
\spang <{grados}>
\spang <{grados : minutos}>
\spang <{grados : minutos : segundos}>
```

El comando `\spang` trabaja en *modo matemático* y recibe el *argumento obligatorio* `{<grados>}`, `{<grados : minutos>}` o `{<grados : minutos : segundos>}`. Los procesa internamente bajo los comandos `\spnum` y `\spunit` para imprimir y verbalizar. Por ejemplo, `\spang{45:30:15}` imprimirá ‘45°30’15” en la salida PDF y NVDA/MathCAT verbalizará «cuarenta y cinco grados treinta minutos quince segundos».

El comando `\sparc`

<code>\sparc</code>	<code>\sparc{⟨punto, punto⟩}</code>	<code>\sparc{⟨punto, punto, punto⟩}</code>
	<code>\sparc[⟨key = val⟩]{⟨punto, punto⟩}</code>	<code>\sparc[⟨key = val⟩]{⟨punto, punto, punto⟩}</code>

El comando `\sparc` trabaja en *modo matemático* y recibe un *argumento obligatorio* de la forma `{⟨punto, punto⟩}` o `{⟨punto, punto, punto⟩}` con prima(s) `'` o subíndice(s) `'_{...}'` opcionales aplicando `\widearc` en la impresión.

Si no se proporciona el *argumento opcional* `[⟨key = val⟩]`, `\sparc{AB}` imprimirá $\widehat{AB}$ en la salida PDF y NVDA/MathCAT verbalizará «arco A B».

- El acento `\widearc` es proporcionado por algunos paquetes como `fourier-otf` o `kpfonts-otf`, en caso de no estar disponible se utilizará una versión interna para este.

Llaves para `\sparc`

`read-cmd = {⟨school | mathcat⟩}` default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento `{⟨punto, punto⟩}` o `{⟨punto, punto, punto⟩}`. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\sparc{AB}` se verbaliza «arco A B»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix = {⟨prefijo⟩}` default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el concepto «arco». El valor de esta llave *siempre* debe pasarse entre llaves `{ }`.

`concept = {⟨true | false⟩}` default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «arco» al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT verbalizará «arco» y el `{⟨argumento⟩}`; si se establece en `false`, NVDA/MathCAT solo verbalizará el `{⟨argumento⟩}` acorde al valor establecido por la llave `read-cmd`.

`silent = {⟨true | false⟩}` default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «arco» y del `{⟨argumento⟩}` pasado al comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spmarc`

<code>\spmarc</code>	<code>\spmarc{⟨punto, punto⟩}</code>	<code>\spmarc{⟨punto, punto, punto⟩}</code>
	<code>\spmarc[⟨key = val⟩]{⟨punto, punto⟩}</code>	<code>\spmarc[⟨key = val⟩]{⟨punto, punto, punto⟩}</code>

El comando `\spmarc` trabaja en *modo matemático* y recibe un *argumento obligatorio* de la forma `{⟨punto, punto⟩}` o `{⟨punto, punto, punto⟩}` con prima(s) `'` o subíndice(s) `'_{...}'` opcionales colocando `'m'` antes de aplicar `\widearc` en la impresión.

Si no se proporciona el *argumento opcional* `[⟨key = val⟩]`, `\spmarc{AB}` imprimirá $m\widehat{AB}$ en la salida PDF y NVDA/MathCAT verbalizará «medida del arco A B».

- La distinción entre `\sparc` y `\spmarc` es semántica: `\sparc` representa el *objeto geométrico* $\widehat{AB}$; `\spmarc` representa su *medida* (un valor en grados).

Llaves para `\spmarc`

`read-cmd = {⟨school | mathcat⟩}` default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento `{⟨punto, punto⟩}` o `{⟨punto, punto, punto⟩}`. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spmarc{AB}` se verbaliza «medida del arco A B»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix = {⟨prefijo⟩}` default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «medida del» seguido por el concepto «arco». El valor de esta llave *siempre* debe pasarse entre llaves `{ }`.

`concept = {⟨true | false⟩}` default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «medida del» y el concepto «arco» al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT verbalizará «medida del arco» y el `{⟨argumento⟩}`; si se establece en `false`, NVDA/MathCAT solo verbalizará el `{⟨argumento⟩}` acorde al valor establecido por la llave `read-cmd`.

`silent = {⟨true | false⟩}` default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «medida del», el concepto «arco» y el `{⟨argumento⟩}` pasado al comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spPoly`

```
\spPoly \spPoly{⟨punto, punto, punto, ...⟩}
\spPoly[⟨key = val⟩]{⟨punto, punto, punto, ...⟩}
```

El comando `\spPoly` trabaja en *modo matemático* y recibe el *argumento obligatorio* $\{\langle \text{punto, punto, punto, ...} \rangle\}$, una lista de al menos tres $\{\langle \text{puntos} \rangle\}$ con prima(s) ‘ $\prime$ ’ o subíndice(s) ‘ $_{\dots}$ ’ opcionales. El *símbolo* usado en la impresión se maneja por medio de la llave `print-sym` y la verbalización del concepto asociado se determina acorde al número de puntos en el argumento y la llave `concept`.

Si son tres puntos se aplica `\triangle` y añade el concepto «*triángulo*» a la verbalización; si son cuatro puntos se aplica `\mdlgwhtsquare` y añade el concepto «*cuadrado*» a la verbalización; si son cinco o más puntos no se aplica ningún símbolo y se añade a la verbalización el nombre del polígono: «*pentágono*», «*hexágono*», «*heptágono*» u «*octágono*» de cinco a ocho puntos, y «*polígono*» desde nueve puntos.

Si no se proporciona el *argumento opcional* $[\langle \text{key} = \text{val} \rangle]$, `\spPoly{ABC}` imprimirá ‘ $\triangle ABC$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*triángulo A B C*»; `\spPoly{ABCD}` imprimirá ‘ $\square ABCD$ ’ y verbalizará «*cuadrado A B C D*»; `\spPoly{ABCDE}` imprimirá ‘ $ABCDE$ ’ y verbalizará «*pentágono A B C D E*».

Llaves para `\spPoly`

```
read-cmd = {\school | mathcat} default: school
```

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento $\{\langle \text{punto, punto, punto, ...} \rangle\}$. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spPoly{ABC}` se verbaliza «*triángulo A B C*»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

```
prefix = {\prefijo} default: no usado
```

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el $\{\langle \text{argumento} \rangle\}$ o el concepto «*triángulo*», «*cuadrado*», «*pentágono*», «*hexágono*», «*heptágono*», «*octágono*» o «*polígono*» establecido por la llave `concept`. El valor de esta llave *siempre* debe pasarse entre llaves ‘ $\{ \}$ ’.

```
concept = {\true | false} default: true
```

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto («*triángulo*», «*cuadrado*», «*pentágono*», «*hexágono*», «*heptágono*», «*octágono*» o «*polígono*») acorde al número de puntos en el $\{\langle \text{argumento} \rangle\}$ al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT verbalizará el concepto y el $\{\langle \text{argumento} \rangle\}$; si se establece en `false`, NVDA/MathCAT solo verbalizará el $\{\langle \text{argumento} \rangle\}$ acorde al valor establecido por la llave `read-cmd`.

```
print-sym = {\true | false} default: true
```

Establece si se *usará* `\triangle` o `\mdlgwhtsquare` en la impresión sin modificar la verbalización de este. Si se establece en `true`, `\spPoly[print-sym=true]{ABC}` imprimirá ‘ $\triangle ABC$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*triángulo A B C*»; `\spPoly[print-sym=true]{ABCD}` imprimirá ‘ $\square ABCD$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*cuadrado A B C D*»; si se establece en `false`, solo se imprimirá el $\{\langle \text{argumento} \rangle\}$ en la salida PDF y la verbalización será la misma.

🔊 Esta llave no tiene efecto con cinco o más puntos, porque no hay un símbolo definido para ellos.

```
silent = {\true | false} default: false
```

Establece si se *silencia* la verbalización en NVDA/MathCAT de los conceptos establecido por la llave `concept` y el $\{\langle \text{argumento} \rangle\}$ pasado al comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spAfig`

```
\spAfig \spAfig(⟨punto, punto, punto, ...⟩)
\spAfig[⟨key = val⟩] \spAfig[⟨key = val⟩](⟨punto, punto, punto, ...⟩)
\spAfig(⟨1 | 2 | 3 | ... | n⟩) \spAfig((F | P | B | L | T))
\spAfig[⟨key = val⟩](⟨1 | 2 | 3 | ... | n⟩) \spAfig[⟨key = val⟩]((F | P | B | L | T))
```

El comando `\spAfig` trabaja en *modo matemático* y recibe el *argumento opcional delimitado por paréntesis* de la forma $(\langle 1 | 2 | 3 | \dots | n \rangle)$, $(\langle \text{punto, punto, punto, ...} \rangle)$ o $((F | P | B | L | T))$ aplicando `\mathcal{A}` a la impresión. El *símbolo* usado en la impresión se maneja por medio de la llave `print-sym` y la verbalización del concepto asociado se determina acorde al $(\langle \text{argumento} \rangle)$ y la llave `concept`.

Si se proporciona el *argumento* $(\langle 1 | 2 | 3 | \dots | n \rangle)$, `\spAfig(1)` imprimirá ‘ $\mathcal{A}_1$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*área uno*»; `\spAfig(n)` imprimirá ‘ $\mathcal{A}_n$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*área ene*».

Si se proporciona el *argumento* $(\langle \text{punto, punto, punto, ...} \rangle)$, `\spAfig(ABC)` imprimirá ‘ $\mathcal{A}_{\triangle ABC}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*área del triángulo A B C*»; `\spAfig(ABCD)` imprimirá ‘ $\mathcal{A}_{\square ABCD}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*área del cuadrado A B C D*»; `\spAfig(ABCDE)` imprimirá ‘ $\mathcal{A}_{ABCDE}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*área del pentágono A B C D E*». Con cinco o más puntos no

se imprime ningún símbolo y se añade a la verbalización el nombre del polígono: «pentágono», «hexágono», «heptágono» u «octágono» de cinco a ocho puntos, y «polígono» desde nueve puntos.

Si se proporciona el *argumento* ((F | P | B | L | T)), $\$ \backslash \text{spAfig}(F) \$$ imprimirá ' $\mathcal{A}_F$ ' en la salida PDF y NVDA/MathCAT verbalizará «área de la figura»; $\$ \backslash \text{spAfig}(F_{\{1\}}) \$$ imprimirá ' $\mathcal{A}_{F_1}$ ' en la salida PDF y NVDA/MathCAT verbalizará «área de la figura uno».

Si no se proporciona el *argumento opcional* [(key = val)] ni el *argumento opcional delimitado por paréntesis*, $\$ \backslash \text{spAfig} \$$ imprimirá ' $\mathcal{A}$ ' en la salida PDF y NVDA/MathCAT verbalizará «área».

- El *argumento* ((punto, punto, punto, ...)) acepta prima(s) ' ' o subíndice(s) '_{...}'. El *argumento* ((F | P | B | L | T)) solo acepta subíndices '_{...}' y tiene una impresión y verbalización fija para cada letra: F «figura», P «polígono», B «basal», L «lateral» y T «total».

Llaves para \spAfig

`read-cmd = { (school | mathcat) }` default: school

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: $\$ \backslash \text{spAfig}(ABC) \$$ se verbaliza «área del triángulo A B C»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix = { (prefijo) }` default: no usado

Establece el *prefijo* que será verbalizado por NVDA/MathCAT antes de verbalizar el término «área del» y el concepto «nombre de la figura» acorde al ((argumento)) pasado al comando. El valor de esta llave siempre debe pasarse entre llaves '{ }'.

`concept = { (true | false) }` default: true

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «área del» y el concepto «nombre de la figura» acorde al ((argumento)) pasado al comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT verbalizará el término «área del», el concepto «nombre de la figura» y el ((argumento)); si se establece en `false`, NVDA/MathCAT solo verbalizará el ((argumento)) acorde al valor establecido por la llave `read-cmd`.

- Utilizar `concept=false` sin el ((argumento)) es equivalente a usar `silent=true`.

`read-arg = { (nombre) }` default: no usado

Sobrescribe el término «área del» y el concepto «nombre de la figura» cuando se utiliza el argumento ((punto, punto, punto, ...)) que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave siempre debe pasarse entre llaves '{ }' y se debe usar en conjunto con la llave `concept`. Por ejemplo: $\$ \backslash \text{spAfig}[\text{read-arg}=\{\text{trapezio}\}](ABCD) \$$ imprimirá ' $\mathcal{A}_{\square ABCD}$ ' en la salida PDF y NVDA/MathCAT verbalizará «área del trapecio A B C D».

- Esta llave no se puede utilizar con los argumentos ((F | P | B | L | T)) y ((1 | 2 | 3 | ... | n)).

`concept* (valor prohibido)` default: no usado

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «nombre de la figura» acorde al ((argumento)) pasado al comando sin modificar la impresión de este. Si se establece, NVDA/MathCAT verbalizará el término «área» y el ((argumento)) acorde al valor establecido por la llave `read-cmd`.

`read-sub = { (true | false) }` default: false

Establece si se *antepone* la palabra «sub» al verbalizar los argumentos ((1 | 2 | 3 | ... | n)) y ((F | P | B | L | T)) para NVDA/MathCAT. Si se utiliza ((1 | 2 | 3 | ... | n)) y se establece en `true`, $\$ \backslash \text{spAfig}[\text{read-sub}=true](1) \$$ imprimirá ' $\mathcal{A}_1$ ' en la salida PDF y NVDA/MathCAT verbalizará «área sub uno»; si se establece en `false`, la impresión será la misma y se verbaliza «área uno».

Si se utiliza ((F | P | B | L | T)) y se establece en `true`, $\$ \backslash \text{spAfig}[\text{read-sub}=true](F_{\{1\}}) \$$ imprimirá ' $\mathcal{A}_{F_1}$ ' en la salida PDF y NVDA/MathCAT verbalizará «área de la figura sub uno»; si se establece en `false`, la impresión será la misma y se verbaliza «área de la figura uno».

`print-sym = { (auto | triangle | square | none) }` default: auto

Establece si se *usará* `\triangle` o `\mdlgwhtsquare` en la impresión para los argumentos ((punto, punto, punto, ...)) y ((1 | 2 | 3 | ... | n)) pasado al comando sin modificar la verbalización de este. Si se utiliza ((punto, punto, punto, ...)) se infiere el *símbolo* acorde al número de puntos; si establece en `auto` se imprime y si se establece en `none` se omite: $\$ \backslash \text{spAfig}[\text{print-sym}=auto](ABC) \$$ y $\$ \backslash \text{spAfig}[\text{print-sym}=none](ABC) \$$ imprimirán ' $\mathcal{A}_{\triangle ABC}$ ' y ' $\mathcal{A}_{ABC}$ ' en la salida PDF y en ambos casos NVDA/MathCAT verbalizará «área del triángulo A B C».

Si se utiliza ((1 | 2 | 3 | ... | n)) los valores `auto` y `none` quedan sin efecto; si se establece en `triangle` o `square` anida el argumento bajo el *símbolo*: $\$ \backslash \text{spAfig}[\text{print-sym}=triangle](1) \$$ imprimirá ' $\mathcal{A}_{\triangle 1}$ ' en la salida PDF y NVDA/MathCAT verbalizará «área del triángulo uno»; $\$ \backslash \text{spAfig}[\text{print-sym}=square](1) \$$ imprimirá ' $\mathcal{A}_{\square 1}$ ' en la salida PDF y NVDA/MathCAT verbalizará «área del cuadrado uno».

- Esta llave no se puede utilizar con el argumento ((F | P | B | L | T)).

`silent = { (true | false) }` default: false

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «área del», el concepto «nombre de la figura» y el $\langle \text{argumento} \rangle$ pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spPfig`

<code>\spPfig</code>	<code>\spPfig</code>	<code>\spPfig(\punto, punto, punto, ...)</code>
<code>\spPfig[key = val]</code>	<code>\spPfig[key = val](\punto, punto, punto, ...)</code>	
<code>\spPfig(1 2 3 ... n)</code>	<code>\spPfig(F P B L T)</code>	
<code>\spPfig[key = val](1 2 3 ... n)</code>	<code>\spPfig[key = val](F P B L T)</code>	

El comando `\spPfig` trabaja en *modo matemático* y recibe el *argumento opcional delimitado por paréntesis* de la forma $\langle 1 | 2 | 3 | \dots | n \rangle$, $\langle \text{punto, punto, punto, ...} \rangle$ o $\langle F | P | B | L | T \rangle$ aplicando `\mathcal{P}` a la impresión. El *símbolo* usado en la impresión se maneja por medio de la llave `print-sym` y la verbalización del concepto asociado se determina acorde al $\langle \text{argumento} \rangle$ y la llave `concept`.

Si se proporciona el *argumento* $\langle 1 | 2 | 3 | \dots | n \rangle$, `\spPfig(1)` imprimirá ' $\mathcal{P}_1$ ' en la salida PDF y NVDA/MathCAT verbalizará «perímetro uno»; `\spPfig(n)` imprimirá ' $\mathcal{P}_n$ ' en la salida PDF y NVDA/MathCAT verbalizará «perímetro ene».

Si se proporciona el *argumento* $\langle \text{punto, punto, punto, ...} \rangle$, `\spPfig(ABC)` imprimirá ' $\mathcal{P}_{\triangle ABC}$ ' en la salida PDF y NVDA/MathCAT verbalizará «perímetro del triángulo A B C»; `\spPfig(ABCD)` imprimirá ' $\mathcal{P}_{\square ABCD}$ ' en la salida PDF y NVDA/MathCAT verbalizará «perímetro del cuadrado A B C D»; `\spPfig(ABCDE)` imprimirá ' $\mathcal{P}_{ABCDE}$ ' en la salida PDF y NVDA/MathCAT verbalizará «perímetro del pentágono A B C D E». Con cinco o más puntos no se imprime ningún símbolo y se añade a la verbalización el nombre del polígono: «pentágono», «hexágono», «heptágono» u «octágono» de cinco a ocho puntos, y «polígono» desde nueve puntos.

Si se proporciona el *argumento* $\langle F | P | B | L | T \rangle$, `\spPfig(F)` imprimirá ' $\mathcal{P}_F$ ' en la salida PDF y NVDA/MathCAT verbalizará «perímetro de la figura»; `\spPfig(F_{1})` imprimirá ' $\mathcal{P}_{F_1}$ ' en la salida PDF y NVDA/MathCAT verbalizará «perímetro de la figura uno».

Si no se proporciona el *argumento opcional* $\langle \text{key} = \text{val} \rangle$ ni el *argumento opcional delimitado por paréntesis*, `\spPfig` imprimirá ' $\mathcal{P}$ ' en la salida PDF y NVDA/MathCAT verbalizará «perímetro».

- El *argumento* $\langle \text{punto, punto, punto, ...} \rangle$ acepta prima(s) ' ' o subíndice(s) ' $\dots$ '. El *argumento* $\langle F | P | B | L | T \rangle$ solo acepta subíndices ' $\dots$ ' y tiene una impresión y verbalización fija para cada letra: F «figura», P «polígono», B «basal», L «lateral» y T «total».

Llaves para `\spPfig`

`read-cmd` = $\{ \langle \text{school} | \text{mathcat} \rangle \}$ default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento. Si se establece en *school*, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spPfig(ABC)` se verbaliza «perímetro del triángulo A B C»; si se establece en *mathcat*, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix` = $\{ \langle \text{prefijo} \rangle \}$ default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «perímetro del» y el concepto «nombre de la figura» acorde al $\langle \text{argumento} \rangle$ pasado al comando. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`concept` = $\{ \langle \text{true} | \text{false} \rangle \}$ default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «perímetro del» y el concepto «nombre de la figura» acorde al $\langle \text{argumento} \rangle$ pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT verbalizará el término «perímetro del», el concepto «nombre de la figura» y el $\langle \text{argumento} \rangle$; si se establece en *false*, NVDA/MathCAT solo verbalizará el $\langle \text{argumento} \rangle$ acorde al valor establecido por la llave `read-cmd`.

- Utilizar `concept=false` sin el $\langle \text{argumento} \rangle$ es equivalente a usar `silent=true`.

`read-arg` = $\{ \langle \text{nombre} \rangle \}$ default: *no usado*

Sobrescribe el término «perímetro del» y el concepto «nombre de la figura» cuando se utiliza el argumento $\langle \text{punto, punto, punto, ...} \rangle$ que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves '{ }' y se debe usar en conjunto con la llave `concept`. Por ejemplo: `\spPfig[read-arg={trapecio}](ABCD)` imprimirá ' $\mathcal{P}_{\square ABCD}$ ' en la salida PDF y NVDA/MathCAT verbalizará «perímetro del trapecio A B C D».

- Esta llave no se puede utilizar con los argumentos $\langle F | P | B | L | T \rangle$ y $\langle 1 | 2 | 3 | \dots | n \rangle$.

`concept*` $\langle \text{valor prohibido} \rangle$ default: *no usado*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «nombre de la figura» acorde al $\langle \text{argumento} \rangle$ pasado al comando sin modificar la impresión de este. Si se establece, NVDA/MathCAT verbalizará el término «perímetro» y el $\langle \text{argumento} \rangle$ acorde al valor establecido por la llave `read-cmd`.

`read-sub` = $\{ \langle \text{true} | \text{false} \rangle \}$ default: *false*

Establece si se *antepone* la palabra «sub» al verbalizar los argumentos $(\langle 1 | 2 | 3 | \dots | n \rangle)$ y $(\langle F | P | B | L | T \rangle)$ para NVDA/MathCAT. Si se utiliza $(\langle 1 | 2 | 3 | \dots | n \rangle)$ y se establece en `true`, $\$ \backslash \text{spPfig}[\text{read-sub}=\text{true}](1) \$$ imprimirá ‘ $\mathcal{P}_1$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*perímetro sub uno*»; si se establece en `false`, la impresión será la misma y se verbaliza «*perímetro uno*».

Si se utiliza $(\langle F | P | B | L | T \rangle)$ y se establece en `true`, $\$ \backslash \text{spPfig}[\text{read-sub}=\text{true}](F_{\{1\}}) \$$ imprimirá ‘ $\mathcal{P}_{F_1}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*perímetro de la figura sub uno*»; si se establece en `false`, la impresión será la misma y se verbaliza «*perímetro de la figura uno*».

`print-sym = { \langle auto | triangle | square | none \rangle }`

default: *auto*

Establece si se usará `\triangle` o `\mdlgwhtsquare` en la impresión para los argumentos $(\langle \text{punto}, \text{punto}, \text{punto}, \dots \rangle)$ y $(\langle 1 | 2 | 3 | \dots | n \rangle)$ pasado al comando sin modificar la verbalización de este. Si se utiliza $(\langle \text{punto}, \text{punto}, \text{punto}, \dots \rangle)$ se infiere el *símbolo* acorde al número de puntos; si establece en `auto` se imprime y si se establece en `none` se omite: $\$ \backslash \text{spPfig}[\text{print-sym}=\text{auto}](ABC) \$$ y $\$ \backslash \text{spPfig}[\text{print-sym}=\text{none}](ABC) \$$ imprimirán ‘ $\mathcal{P}_{\triangle ABC}$ ’ y ‘ $\mathcal{P}_{ABC}$ ’ en la salida PDF y en ambos casos NVDA/MathCAT verbalizará «*perímetro del triángulo A B C*».

Si se utiliza $(\langle 1 | 2 | 3 | \dots | n \rangle)$ los valores `auto` y `none` quedan sin efecto; si se establece en `triangle` o `square` anida el argumento bajo el *símbolo*: $\$ \backslash \text{spPfig}[\text{print-sym}=\text{triangle}](1) \$$ imprimirá ‘ $\mathcal{P}_{\triangle_1}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*perímetro del triángulo uno*»; $\$ \backslash \text{spPfig}[\text{print-sym}=\text{square}](1) \$$ imprimirá ‘ $\mathcal{P}_{\square_1}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*perímetro del cuadrado uno*».

🔗 Esta llave no se puede utilizar con el argumento $(\langle F | P | B | L | T \rangle)$.

`silent = { \langle true | false \rangle }`

default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «*perímetro del*», el concepto «*nombre de la figura*» y el $\{ \langle \text{argumento} \rangle \}$ pasado al comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, NVDA/MathCAT sí verbalizará la salida del comando.

Los comandos `\sprad` y `\spdiam`

`\sprad`
`\spdiam`

`\sprad[\langle key = val \rangle]`
`\spdiam[\langle key = val \rangle]`

Los comandos `\sprad` y `\spdiam` trabajan en *modo matemático* y reciben el *argumento opcional* $[\langle key = val \rangle]$ mediante el cual se establece la «*forma*» en la que se imprimirán en la salida PDF las letras ‘*r*’ y ‘*d*’ y como se verbalizarán en NVDA/MathCAT.

Si no se proporciona el *argumento opcional* $[\langle key = val \rangle]$, $\$ \backslash \text{sprad} \$$ imprimirá ‘*r*’ en la salida PDF y NVDA/MathCAT verbalizará «*radio*»; $\$ \backslash \text{spdiam} \$$ imprimirá ‘*d*’ en la salida PDF y NVDA/MathCAT verbalizará «*diámetro*».

Llaves para `\sprad` y `\spdiam`

`concept = { \langle true | false \rangle }`

default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT de los conceptos «*radio*» o «*diámetro*» al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, $\$ \backslash \text{sprad} \$$ y $\$ \backslash \text{spdiam} \$$ se verbalizan como «*radio*» y «*diámetro*» en NVDA/MathCAT; si se establece en `false`, se verbalizarán las letras ‘*r*’, ‘*R*’, ‘*d*’ o ‘*D*’ acorde al valor establecido por la llave `read-cmd`.

`read-cmd = { \langle school | mathcat \rangle }`

default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará las letras ‘*R*’ o ‘*D*’ cuando `concept=false` y `upper-case=true`. Si se establece en `school`, NVDA/MathCAT verbalizará las letras ‘*R*’ o ‘*D*’ sin el prefijo «mayúscula»: si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`upper-case = { \langle true | false \rangle }`

default: *false*

Establece si se *imprime* usando mayúsculas. Si se establece en `true`, $\$ \backslash \text{sprad}[\text{upper-case}=\text{true}] \$$ y $\$ \backslash \text{spdiam}[\text{upper-case}=\text{true}] \$$ imprimirán ‘*R*’ y ‘*D*’ en la salida PDF; si se establece en `false` se imprimirá usando minúsculas.

🔗 La verbalización al establecer esta llave es controlada por la llave `read-cmd`.

`print-sym = { \langle true | false \rangle }`

default: *false*

Establece si se usará `\diameter` antes de la letra ‘*d*’ o ‘*D*’ en la impresión del comando `\spdiam`. Si se establece en `true`, $\$ \backslash \text{spdiam}[\text{print-sym}=\text{true}] \$$ imprimirá ‘ $\emptyset d$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*diámetro*».

🔗 La llave `print-sym` no tiene ningún efecto en `\sprad`.

`sub-index = { \langle subíndice \rangle }`

default: *no usado*

Establece un *subíndice* sobre las letras ‘*r*’, ‘*R*’, ‘*d*’ o ‘*D*’. El valor de esta llave *siempre* debe pasarse entre llaves ‘ $\{ \}$ ’. Por ejemplo: $\$ \backslash \text{sprad}[\text{sub-index}=\{1\}] \$$ imprimirá ‘ r_1 ’ en la salida PDF y NVDA/MathCAT verbalizará «*radio uno*».

`read-sub = {⟨true | false⟩}`

default: *false*

Establece si se *antepone* la palabra «*sub*» al verbalizar el subíndice establecido por la llave `sub-index` al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, `\sprad[sub-index={1}, read-sub=true]` se verbalizará «*radio sub uno*» en NVDA/MathCAT; si se establece en *false*, NVDA/MathCAT verbalizará «*radio uno*».

`read-arg = {⟨verbalización NVDA⟩}`

default: *no usado*

Sobrescribe el *concepto* establecido por la llave `concept` que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves '{ }' y se debe usar en conjunto con la llave `concept`. Por ejemplo: `\sprad[read-arg={medida exacta}]` imprimirá 'r' en la salida PDF y NVDA/MathCAT verbalizará «*radio medida exacta*»; `\sprad[concept=false, read-arg={medida exacta}]` imprimirá 'r' y NVDA/MathCAT verbalizará «*medida exacta*».

- Las llaves `sub-index` y `read-sub` no tienen efecto al usar `read-arg`.

`silent = {⟨true | false⟩}`

default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del *concepto* establecido por la llave `concept` y de las letras 'r', 'R', 'd' o 'D' al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spcirc`

<code>\spcirc</code>	<code>\spcirc</code>	<code>\spcirc[⟨key = val⟩]</code>
<code>\spcirc(⟨centro⟩)</code>	<code>\spcirc(⟨centro⟩)</code>	<code>\spcirc[⟨key = val⟩](⟨centro⟩)</code>
<code>\spcirc(⟨centro, radio⟩)</code>	<code>\spcirc(⟨centro, radio⟩)</code>	<code>\spcirc[⟨key = val⟩](⟨centro, radio⟩)</code>

El comando `\spcirc` trabaja en *modo matemático* y recibe el *argumento opcional delimitado por paréntesis* de la forma `(⟨centro⟩)` o `(⟨centro, radio⟩)`, donde `⟨radio⟩` es opcional y debe pasarse junto al argumento `(⟨centro⟩)` separado por una coma ',' obligatoria.

La forma en la que se procesa el `⟨radio⟩` junto a su impresión y verbalización se establece de manera interna: Si es un `⟨número⟩` entero o decimal se usa `\spnum`; si es una `⟨medida⟩` se usa `\spunit`; si es un segmento `⟨punto punto⟩` se usa `\spside` o `\spmside` acorde al valor establecido por la llave `radio-sty`; y si es una letra se imprime tal cual. Cualquier otra forma devuelve un "error" y se detiene la compilación del documento.

Si no se proporciona el *argumento opcional* `[⟨key = val⟩]` ni el *argumento opcional delimitado por paréntesis*, `\spcirc` imprimirá '⊙' en la salida PDF y NVDA/MathCAT verbalizará «*circunferencia*».

Si se proporciona el *argumento* `(⟨centro⟩)`, `\spcirc(⟨centro⟩)` imprimirá '⊙(O)' en la salida PDF y NVDA/MathCAT verbalizará «*circunferencia de centro en O*».

Si se proporciona el *argumento* `(⟨centro, radio⟩)`, `\spcirc(⟨centro, radio⟩)` imprimirá '⊙(O, 3)' en la salida PDF y NVDA/MathCAT verbalizará «*circunferencia de centro en O y radio tres*».

- El argumento `(⟨centro⟩)` y el argumento `⟨radio⟩` cuando tiene la forma `⟨punto punto⟩` aceptan prima '' o subíndice `_{...}`.

Llaves para `\spcirc`

El comando `\spcirc` posee las *meta-keys* `spside` pasada al comando `\spside` (v. pág. 21) y `spmside` pasada al comando `\spmside` (v. pág. 22).

`read-cmd = {⟨school | mathcat⟩}`

default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará el argumento `(⟨centro⟩)`. Si se establece en *school*, NVDA/MathCAT verbaliza el argumento `(⟨centro⟩)` sin el prefijo «mayúscula»; si se establece en *mathcat*, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

- Esta llave tiene efecto sobre el argumento `⟨radio⟩` cuando tiene la forma `⟨punto punto⟩`.

`prefix = {⟨prefijo⟩}`

default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el «*concepto*» establecido por la llave `read-sty`. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`print-sym = {⟨usual | odot⟩}`

default: *odot*

Establece si se *usará* `\bigodot` o la letra 'C' en la impresión del comando sin modificar la verbalización de este. Si se establece en *usual*, `\spcirc[print-sym=usual](O)` imprimirá 'C(O)' en la salida PDF; si se establece en *odot*, `\spcirc[print-sym=odot](O)` imprimirá '⊙(O)' en la salida PDF.

`read-sty = {⟨circun | circle⟩}`

default: *circun*

Establece el *concepto* que verbalizará NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. Si se establece en *circun*, `\spcirc[read-sty=circun](O)` se verbalizará «*circunferencia de centro en O*» en NVDA/MathCAT; si se establece en *circle*, `\spcirc[read-sty=circle](O)` se verbalizará «*círculo de centro en O*» en NVDA/MathCAT.

`diameter = {⟨true | false⟩}`

default: *false*

Establece *como* se verbaliza el argumento *(radio)* al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, `\spc[radio=true](O, 3)` se verbalizará «circunferencia de centro en *O* y diámetro tres» en NVDA/MathCAT; si se establece en `false`, `\spc[radio=false](O, 3)` se verbalizará «circunferencia de centro en *O* y radio tres» en NVDA/MathCAT.

`radio-sty = { (side | mside) }`

default: *mside*

Establece el *comando* que procesa internamente el argumento *(radio)* cuando tiene la forma *(punto punto)*. Si se establece en *side* se usará `\spside{(punto punto)}`, `\spc[radio-sty=side](O, AB)` imprimirá $\odot(O, \overline{AB})$ en la salida PDF y NVDA/MathCAT verbalizará «circunferencia de centro en *O* y radio lado *A B*»; si se establece en *mside* se usará `\spmside{(punto punto)}`, `\spc[radio-sty=mside](O, AB)` imprimirá $\odot(O, AB)$ en la salida PDF y NVDA/MathCAT verbalizará «circunferencia de centro en *O* y radio medida del lado *A B*».

- Las *(meta-keys)* `spside` y `spmside` permiten modificar la impresión y verbalización del argumento *(radio)* cuando tiene la forma *(punto punto)*.

`concept = { (true | false) }`

default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del «concepto» establecido por la llave `read-sty` al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT verbalizará el «concepto» y el argumento *(centro)* o *(centro, radio)*; si se establece en `false`, NVDA/MathCAT solo verbalizará el argumento *(centro)* o *(centro, radio)* acorde al valor establecido por la llave `read-cmd`.

- Utilizar `concept=false` sin el argumento *(centro)* es equivalente a usar `silent=true`.

`sub-index = { (subíndice) }`

default: *no usado*

Establece un *subíndice* sobre `\bigodot` o la letra 'C' establecida por la llave `print-sym`. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'. Por ejemplo: `\spc[sub-index={1},radio-sty=mside](O, AB)` imprimirá $\odot_1(O, AB)$ en la salida PDF y NVDA/MathCAT verbalizará «circunferencia uno de centro en *O* y radio medida del lado *A B*».

`read-sub = { (true | false) }`

default: *false*

Establece si se *antepone* la palabra «sub» al verbalizar el subíndice establecido por la llave `sub-index` al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, `\spc[sub-index={1},read-sub=true](O)` se verbalizará «circunferencia sub uno de centro en *O*»; si se establece en `false`, NVDA/MathCAT verbalizará «circunferencia uno de centro en *O*».

`read-arg = { (verbalización NVDA) }`

default: *no usado*

Sobrescribe el *concepto* establecido por la llave `read-sty` que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'. Por ejemplo: `\spc[read-arg={disco}](O)` imprimirá $\odot(O)$ en la salida PDF y NVDA/MathCAT verbalizará «disco de centro en *O*».

`silent = { (true | false) }`

default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del «concepto» establecido por la llave `read-sty`, del argumento *(centro)* o *(centro, radio)* pasados al comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spAcirc`

<code>\spAcirc</code>	<code>\spAcirc</code>	<code>\spAcirc[(key = val)]</code>
<code>\spAcirc(centro)</code>	<code>\spAcirc[(key = val)](centro)</code>	
<code>\spAcirc(1 2 3 ... n)</code>	<code>\spAcirc[(key = val)](1 2 3 ... n)</code>	

El comando `\spAcirc` trabaja en *modo matemático* y recibe el *argumento opcional delimitado por paréntesis* de la forma *(centro)* o *(1 | 2 | 3 | ... | n)*, aplicando `\mathcal{A}` a la impresión.

Si no se proporciona el *argumento opcional* `[(key = val)]` ni el *argumento opcional delimitado por paréntesis*, `\spAcirc` imprimirá ' $\mathcal{A}$ ' en la salida PDF y NVDA/MathCAT verbalizará «área de la circunferencia».

Si se proporciona el *argumento* *(centro)*, `\spAcirc(O)` imprimirá ' $\mathcal{A}_{\odot(O)}$ ' en la salida PDF y NVDA/MathCAT verbalizará «área de la circunferencia con centro en *O*».

Si se proporciona el *argumento* *(1 | 2 | 3 | ... | n)*, `\spAcirc(1)` imprimirá ' $\mathcal{A}_{\odot_1}$ ' en la salida PDF y NVDA/MathCAT verbalizará «área de la circunferencia uno».

- El *argumento* *(centro)* acepta prima '' o subíndice '_{...}' y no acepta un *(radio)*. Una coma ',' en el argumento siempre produce un "error".

Llaves para `\spAcirc`

`read-cmd` = { `<school | mathcat>` }

default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará el argumento (`<centro>`). Si se establece en *school*, NVDA/MathCAT verbaliza el argumento (`<centro>`) sin el prefijo «mayúscula»; si se establece en *mathcat*, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix` = { `<prefijo>` }

default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «área de ...» seguido por el «concepto» establecido por la llave `read-sty`. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`read-sty` = { `<circun | circle>` }

default: *circun*

Establece el *concepto* que verbalizará NVDA/MathCAT después de verbalizar el término «área de ...» al ejecutar el comando sin modificar la impresión de este. Si se establece en *circun*, `\spAcirc[read-sty=circun](0)` se verbalizará «área de la circunferencia con centro en O» en NVDA/MathCAT; si se establece en *circle*, `\spAcirc[read-sty=circle](0)` se verbalizará «área del círculo con centro en O» en NVDA/MathCAT.

`concept` = { `<true | false>` }

default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «área de ...» y del «concepto» establecido por la llave `read-sty` al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT verbalizará el término, el concepto y el argumento (`<centro>`) o (`<1 | 2 | 3 | ... | n>`); si se establece en *false*, NVDA/MathCAT solo verbalizará el argumento (`<centro>`) acorde al valor establecido por la llave `read-cmd`.

- Utilizar `concept=false` sin el argumento (`<centro>`) o con el argumento (`<1 | 2 | 3 | ... | n>`) es equivalente a usar `silent=true`.

`read-arg` = { `<verbalización NVDA>` }

default: *no usado*

Sobrescribe el *concepto* establecido por la llave `read-sty` que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves '{ }' y se debe usar en conjunto con la llave `concept`. Por ejemplo: `\spAcirc[read-arg={esfera}](0)` se verbalizará «área de la esfera con centro en O» en NVDA/MathCAT; `\spAcirc[concept=false,read-arg={esfera}](0)` se verbalizará «esfera con centro en O» en NVDA/MathCAT.

- Utilizar esta llave con `concept=false` y el argumento (`<1 | 2 | 3 | ... | n>`) es equivalente a usar `silent=true`.

`print-sym` = { `<usual | odot>` }

default: *odot*

Establece si se *usará* `\bigodot` o la letra 'C' en la impresión del comando sin modificar la verbalización de este. Si se establece en *usual*, `\spAcirc[print-sym=usual](0)` imprimirá ' $\mathcal{A}_{C(O)}$ ' en la salida PDF; si se establece en *odot*, `\spAcirc[print-sym=odot](0)` imprimirá ' $\mathcal{A}_{\odot(O)}$ ' en la salida PDF.

`sub-index` = { `<subíndice>` }

default: *no usado*

Establece un *subíndice* sobre `\bigodot` o la letra 'C' establecida por la llave `print-sym` cuando se utiliza el argumento (`<centro>`). El valor de esta llave *siempre* debe pasarse entre llaves '{ }'. Por ejemplo: `\spAcirc[sub-index={1}](0)` imprimirá ' $\mathcal{A}_{\odot_1(O)}$ ' en la salida PDF y NVDA/MathCAT verbalizará «área de la circunferencia uno con centro en O».

`read-sub` = { `<true | false>` }

default: *false*

Establece si se *antepone* la palabra «sub» al verbalizar el subíndice establecida por la llave `sub-index` o al usar el argumento (`<1 | 2 | 3 | ... | n>`) al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, `\spAcirc[read-sub=true](1)` se verbalizará «área de la circunferencia sub uno» en NVDA/MathCAT; si se establece en *false*, NVDA/MathCAT verbalizará «área de la circunferencia uno».

`silent` = { `<true | false>` }

default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «área de ...», del concepto establecido por la llave `read-sty` y del argumento (`<centro>`) o (`<1 | 2 | 3 | ... | n>`) pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spPcirc`

`\spPcirc` `\spPcirc`

`\spPcirc(<centro>)`

`\spPcirc(<1 | 2 | 3 | ... | n>)`

`\spPcirc[<key = val>]`

`\spPcirc[<key = val>](<centro>)`

`\spPcirc[<key = val>](<1 | 2 | 3 | ... | n>)`

El comando `\spPcirc` trabaja en *modo matemático* y recibe el *argumento opcional delimitado por paréntesis* de la forma (`<centro>`) o (`<1 | 2 | 3 | ... | n>`), aplicando `\mathcal{P}` a la impresión.

Si no se proporciona el *argumento opcional* [`<key = val>`] ni el *argumento opcional delimitado por paréntesis*, `\spPcirc` imprimirá ' $\mathcal{P}$ ' en la salida PDF y NVDA/MathCAT verbalizará «perímetro de la circunferencia».

Si se proporciona el argumento ($\langle centro \rangle$), $\$ \backslash spPcirc(0) \$$ imprimirá ' $\mathcal{P}_{O(0)}$ ' en la salida PDF y NVDA/MathCAT verbalizará «perímetro de la circunferencia con centro en O».

Si se proporciona el argumento ($\langle 1 | 2 | 3 | \dots | n \rangle$), $\$ \backslash spPcirc(1) \$$ imprimirá ' $\mathcal{P}_{O_1}$ ' en la salida PDF y NVDA/MathCAT verbalizará «perímetro de la circunferencia uno».

- El argumento ($\langle centro \rangle$) acepta prima ' ' o subíndice '_{...}' y no acepta un $\langle radio \rangle$. Una coma ',' en el argumento siempre produce un "error".

Llaves para $\backslash spPcirc$

`read-cmd = {  $\langle school | mathcat \rangle$  }` default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará el argumento ($\langle centro \rangle$). Si se establece en *school*, NVDA/MathCAT verbaliza el argumento ($\langle centro \rangle$) sin el prefijo «mayúscula»; si se establece en *mathcat*, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix = {  $\langle prefijo \rangle$  }` default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «perímetro de ...» seguido por el «concepto» establecido por la llave `read-sty`. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`read-sty = {  $\langle circun | circle \rangle$  }` default: *circun*

Establece el *concepto* que verbalizará NVDA/MathCAT después de verbalizar el término «perímetro de ...» al ejecutar el comando sin modificar la impresión de este. Si se establece en *circun*, $\$ \backslash spPcirc[read-sty=circun](0) \$$ se verbalizará «perímetro de la circunferencia con centro en O» en NVDA/MathCAT; si se establece en *circle*, $\$ \backslash spPcirc[read-sty=circle](0) \$$ se verbalizará «perímetro del círculo con centro en O» en NVDA/MathCAT.

`concept = {  $\langle true | false \rangle$  }` default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «perímetro de ...» y del «concepto» establecido por la llave `read-sty` al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT verbalizará el término, el concepto y el argumento ($\langle centro \rangle$) o ($\langle 1 | 2 | 3 | \dots | n \rangle$); si se establece en *false*, NVDA/MathCAT solo verbalizará el argumento ($\langle centro \rangle$) acorde al valor establecido por la llave `read-cmd`.

- Utilizar `concept=false` sin el argumento ($\langle centro \rangle$) o con el argumento ($\langle 1 | 2 | 3 | \dots | n \rangle$) es equivalente a usar `silent=true`.

`read-arg = {  $\langle verbalización NVDA \rangle$  }` default: *no usado*

Sobrescribe el *concepto* establecido por la llave `read-sty` que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves '{ }' y se debe usar en conjunto con la llave `concept`. Por ejemplo: $\$ \backslash spPcirc[read-sty=circle, read-arg={disco}](0) \$$ se verbalizará «perímetro del disco con centro en O» en NVDA/MathCAT; $\$ \backslash spPcirc[concept=false, read-arg={disco}](0) \$$ se verbalizará «disco con centro en O» en NVDA/MathCAT.

- Utilizar esta llave con `concept=false` y el argumento ($\langle 1 | 2 | 3 | \dots | n \rangle$) es equivalente a usar `silent=true`.

`print-sym = {  $\langle usual | odot \rangle$  }` default: *odot*

Establece si se *usará* $\backslash bigodot$ o la letra 'C' en la impresión del comando sin modificar la verbalización de este. Si se establece en *usual*, $\$ \backslash spPcirc[print-sym=usual](0) \$$ imprimirá ' $\mathcal{P}_{C(0)}$ ' en la salida PDF; si se establece en *odot*, $\$ \backslash spPcirc[print-sym=odot](0) \$$ imprimirá ' $\mathcal{P}_{O(0)}$ ' en la salida PDF.

`sub-index = {  $\langle subíndice \rangle$  }` default: *no usado*

Establece un *subíndice* sobre $\backslash bigodot$ o la letra 'C' establecida por la llave `print-sym` cuando se utiliza el argumento ($\langle centro \rangle$). El valor de esta llave *siempre* debe pasarse entre llaves '{ }'. Por ejemplo: $\$ \backslash spPcirc[sub-index={1}](0) \$$ imprimirá ' $\mathcal{P}_{O_1(0)}$ ' en la salida PDF y NVDA/MathCAT verbalizará «perímetro de la circunferencia uno con centro en O».

`read-sub = {  $\langle true | false \rangle$  }` default: *false*

Establece si se *antepone* la palabra «sub» al verbalizar el subíndice establecida por la llave `sub-index` o al usar el argumento ($\langle 1 | 2 | 3 | \dots | n \rangle$) al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, $\$ \backslash spPcirc[read-sub=true](1) \$$ se verbalizará «perímetro de la circunferencia sub uno» en NVDA/MathCAT; si se establece en *false*, NVDA/MathCAT verbalizará «perímetro de la circunferencia uno».

`silent = {  $\langle true | false \rangle$  }` default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «perímetro de ...», del concepto establecido por la llave `read-sty` y del argumento ($\langle centro \rangle$) o ($\langle 1 | 2 | 3 | \dots | n \rangle$) pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spVol`

 $\spVol$ $\spVol$ $\spVol[\langle key = val \rangle]$

El comando `\spVol` trabaja en *modo matemático* y recibe el *argumento opcional* $[\langle key = val \rangle]$ mediante el cual se establece la «*forma*» en la que se verbalizará en NVDA/MathCAT.

Si no se proporciona el *argumento opcional* $[\langle key = val \rangle]$, $\$ \spVol \$$ imprimirá ‘ $\mathcal{V}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*volumen*».

Llaves para `\spVol`

`concept` = $\{\langle true | false \rangle\}$ default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «*volumen*» al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, $\$ \spVol \$$ se verbaliza como «*volumen*» en NVDA/MathCAT; si se establece en *false*, se verbalizará la letra ‘ $\mathcal{V}$ ’ acorde al valor establecido por la llave `read-cmd`.

`read-cmd` = $\{\langle school | mathcat \rangle\}$ default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará la letra ‘ $\mathcal{V}$ ’ cuando `concept=false`. Si se establece en *school*, NVDA/MathCAT verbalizará únicamente la letra ‘ $\mathcal{V}$ ’; si se establece en *mathcat*, NVDA/MathCAT aplica sus propias reglas sobre ‘ $\mathcal{V}$ ’.

`sub-index` = $\{\langle subíndice \rangle\}$ default: *no usado*

Establece un *subíndice* sobre ‘ $\mathcal{V}$ ’. El valor de esta llave *siempre* debe pasarse entre llaves ‘ $\{ \}$ ’. Por ejemplo: $\$ \spVol[\text{sub-index}=\{1\}] \$$ imprimirá ‘ $\mathcal{V}_1$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*volumen uno*».

`read-sub` = $\{\langle true | false \rangle\}$ default: *false*

Establece si se *antepone* la palabra «*sub*» al verbalizar el subíndice establecido por la llave `sub-index` al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, $\$ \spVol[\text{sub-index}=\{1\}, \text{read-sub}=\text{true}] \$$ se verbalizará «*volumen sub uno*» en NVDA/MathCAT; si se establece en *false*, NVDA/MathCAT verbalizará «*volumen uno*».

`read-arg` = $\{\langle verbalización NVDA \rangle\}$ default: *no usado*

Sobrescribe el *concepto* establecido por la llave `concept` que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves ‘ $\{ \}$ ’ y se debe usar en conjunto con la llave `concept`. Por ejemplo: $\$ \spVol[\text{read-arg}=\{\text{medida exacta}\}] \$$ imprimirá ‘ $\mathcal{V}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*volumen medida exacta*»; $\$ \spVol[\text{concept}=\text{false}, \text{read-arg}=\{\text{medida exacta}\}] \$$ imprimirá ‘ $\mathcal{V}$ ’ y NVDA/MathCAT verbalizará «*medida exacta*».

Las llaves `sub-index` y `read-sub` no tienen efecto al usar `read-arg`.

`silent` = $\{\langle true | false \rangle\}$ default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del *concepto* establecido por la llave `concept` y de ‘ $\mathcal{V}$ ’ al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

8 Utilidades para álgebra

El paquete `spintent` provee utilidades para trabajar con álgebra descritas en esta sección.

El comando `\spinterval`

 $\spinterval$ $\spinterval\{\langle intervalo \rangle\}$ $\spinterval[\langle key = val \rangle]\{\langle intervalo \rangle\}$

El comando `\spinterval` trabaja en *modo matemático* y recibe el *argumento* obligatorio $\{\langle intervalo \rangle\}$ de la forma $\{\langle (a; b) \rangle\}$, donde ‘*a*’ y ‘*b*’ son los *extremos* del intervalo separados por un punto y coma ‘;’ o, si no hay punto y coma, por una única coma ‘,’. El delimitador izquierdo puede ser ‘(’, ‘[’ o ‘]’ y el derecho ‘)’, ‘]’ o ‘[’, en cualquier combinación. Los extremos numéricos se procesan internamente bajo el comando `\spnum`; cualquier otro extremo, como una letra, un subíndice, una fracción o una raíz, se imprime tal cual y lo verbaliza NVDA/MathCAT.

Un delimitador ‘(’ o ‘]’ a la izquierda, y ‘)’ o ‘[’ a la derecha, *excluye* su extremo y el intervalo es *abierto* en él; los demás delimitadores lo *incluyen* y el intervalo es *cerrado* en él.

Si el intervalo es abierto en sus dos extremos, $\$ \spinterval\{(1,5)\} \$$ imprimirá ‘(1,5)’ en la salida PDF y NVDA/MathCAT verbalizará «*intervalo abierto de uno a cinco*»; la notación chilena $\$ \spinterval\{]1,5[\} \$$ imprimirá ‘]1,5[’ y se verbalizará igual.

Si el intervalo es cerrado en sus dos extremos, $\$ \spinterval\{[1,5]\} \$$ imprimirá ‘[1,5]’ y NVDA/MathCAT verbalizará «*intervalo cerrado de uno a cinco*».

Si el intervalo es abierto en un extremo y cerrado en el otro, $\$ \spinterval\{]1,5]\} \$$ imprimirá ‘]1,5]’ y NVDA/MathCAT verbalizará «*intervalo abierto en uno y cerrado en cinco*»; $\$ \spinterval\{[1,5[\} \$$ imprimirá ‘[1,5[’ y verbalizará «*intervalo cerrado en uno y abierto en cinco*».

Si un extremo es infinito, $\$ \backslash \text{spinterval}\{-\infty, 3\}$ imprimirá $]-\infty, 3]$ y NVDA/MathCAT verbalizará «intervalo abierto en menos infinito y cerrado en tres».

Si los extremos son números decimales con coma, el separador debe ser el punto y coma: $\$ \backslash \text{spinterval}\{3,5;4,2\}$ imprimirá $[3,5;4,2]$ y NVDA/MathCAT verbalizará «intervalo cerrado de tres coma cinco a cuatro coma dos».

El comando $\backslash \text{spinterval}$ devuelve un “error” y se detiene la compilación del documento cuando el $\{ \langle \text{intervalo} \rangle \}$ no comienza con un delimitador izquierdo y termina con uno derecho de los indicados, cuando un extremo está vacío, cuando no hay separador, cuando hay más de una coma sin punto y coma, cuando las llaves no están balanceadas o cuando el infinito está mal ubicado o se incluye en el intervalo.

- El argumento $\{ \langle \text{intervalo} \rangle \}$ siempre debe pasarse entre llaves ‘{ }’, incluso cuando comienza con un corchete: sin ellas, $\backslash \text{spinterval}[1,5]$ interpreta los corchetes como el argumento opcional $[\langle \text{key} = \text{val} \rangle]$.
- El infinito ‘ ∞ ’ nunca se incluye en el intervalo: ‘ $-\infty$ ’ solo puede ir en el extremo izquierdo y ‘ ∞ ’ en el derecho, ambos junto a un delimitador que los excluya.

Llaves para $\backslash \text{spinterval}$

$\text{read-cmd} = \{ \langle \text{school} \mid \text{mathcat} \rangle \}$ default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará los extremos del intervalo. Si se establece en *school*, $\$ \backslash \text{spinterval}[\text{read-cmd=school}]\{-2,+4\}$ se verbalizará «intervalo abierto de menos dos a más cuatro»; si se establece en *mathcat*, NVDA/MathCAT aplica sus propias reglas y $\$ \backslash \text{spinterval}[\text{read-cmd=mathcat}]\{-2,+4\}$ se verbalizará «intervalo abierto de menos dos a positivo cuatro».

$\text{prefix} = \{ \langle \text{prefijo} \rangle \}$ default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el «concepto» intervalo. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’. Por ejemplo: $\$ \backslash \text{spinterval}[\text{prefix}=\{\text{el}\}]\{(1,5)\}$ imprimirá ‘(1, 5)’ en la salida PDF y NVDA/MathCAT verbalizará «el intervalo abierto de uno a cinco».

$\text{read-arg} = \{ \langle \text{verbalización NVDA} \rangle \}$ default: *no usado*

Sobrescribe la verbalización del *intervalo* completo que será verbalizada por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’. Por ejemplo: $\$ \backslash \text{spinterval}[\text{read-arg}=\{\text{rango}\}]\{[1,5]\}$ imprimirá ‘[1, 5]’ en la salida PDF y NVDA/MathCAT verbalizará «rango».

$\text{concept} = \{ \langle \text{true} \mid \text{false} \rangle \}$ default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del «concepto» intervalo y del prefijo establecido por la llave *prefix* al ejecutar el comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT verbalizará el «concepto» y los extremos; si se establece en *false*, $\$ \backslash \text{spinterval}[\text{concept=false}]\{(1,5)\}$ se verbalizará «abierto de uno a cinco».

$\text{size} = \{ \langle \text{auto} \mid \text{none} \mid \text{big} \mid \text{Big} \mid \text{bigg} \mid \text{Bigg} \rangle \}$ default: *auto*

Establece el *tamaño* de los delimitadores del intervalo sin modificar la verbalización de este. Si se establece en *auto*, los delimitadores conservan su tamaño normal salvo que algún extremo contenga una fracción $\backslash \text{frac}$ o $\backslash \text{dfrac}$, en cualquier posición del extremo: en ese caso se usan $\backslash \text{bigl}$ y $\backslash \text{bigr}$ en estilo texto, $\backslash \text{Bigl}$ y $\backslash \text{Bigr}$ en estilo display, y siempre $\backslash \text{Bigl}$ y $\backslash \text{Bigr}$ cuando la fracción es $\backslash \text{dfrac}$. Si se establece en *none*, los delimitadores nunca cambian de tamaño; si se establece en *big*, *Big*, *bigg* o *Bigg*, se usan siempre $\backslash \text{bigl}$ y $\backslash \text{bigr}$, $\backslash \text{Bigl}$ y $\backslash \text{Bigr}$, $\backslash \text{biggl}$ y $\backslash \text{biggr}$, o $\backslash \text{Biggl}$ y $\backslash \text{Biggr}$, respectivamente.

$\text{silent} = \{ \langle \text{true} \mid \text{false} \rangle \}$ default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del $\{ \langle \text{intervalo} \rangle \}$ pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, NVDA/MathCAT sí verbalizará la salida del comando.

El comando $\backslash \text{spfun}$

$\backslash \text{spfun} \quad \backslash \text{spfun}\{ \langle \text{función} \rangle \} \quad \backslash \text{spfun}[\langle \text{key} = \text{val} \rangle]\{ \langle \text{función} \rangle \}$

El comando $\backslash \text{spfun}$ trabaja en *modo matemático* y recibe el *argumento* obligatorio $\{ \langle \text{función} \rangle \}$ de la forma $\{ \langle f(x) \rangle \}$, donde ‘*f*’ es el *nombre* de la función y lo que va entre paréntesis son uno o más *argumentos* separados por coma ‘,’. El nombre es una letra, mayúscula o minúscula, con prima(s) ‘*’*’ o subíndice ‘*_*’...’ opcionales; el subíndice solo admite letras y dígitos. Los argumentos pueden ser letras, números o expresiones.

$\$ \backslash \text{spfun}\{f(x)\}$ imprimirá ‘*f(x)*’ en la salida PDF y NVDA/MathCAT verbalizará «*f de x*»; $\$ \backslash \text{spfun}\{f_{-1}(x)\}$ imprimirá ‘*f₁(x)*’ y verbalizará «*f sub uno de x*»; $\$ \backslash \text{spfun}\{f'(x)\}$ imprimirá ‘*f'(x)*’ y verbalizará «*f prima de x*»; $\$ \backslash \text{spfun}\{f''(x)\}$ imprimirá ‘*f''(x)*’ y verbalizará «*f doble prima de x*».

Con dos o más argumentos, $\$ \backslash \text{spfun}\{f(x,y)\}$ imprimirá ‘*f(x,y)*’ y NVDA/MathCAT verbalizará «*f de x coma y*». Un argumento puede ser una expresión: $\$ \backslash \text{spfun}\{f(x+1)\}$ imprimirá ‘*f(x + 1)*’ y verbalizará «*f de x más 1*».

Un argumento puede ser a su vez una llamada, tantas veces como se quiera: $\$ \backslash \text{spfun}\{f(g(x))\}$ imprimirá ‘*f(g(x))*’ y verbalizará «*f de g de x*»; $\$ \backslash \text{spfun}\{f(g^{-1}(y))\}$ imprimirá ‘*f(f⁻¹(y))*’ y verbalizará «*f de f inversa de y*»; $\$ \backslash \text{spfun}\{f(g(x),h(y))\}$ imprimirá ‘*f(g(x), h(y))*’. Una llamada anidada se valida igual

que la exterior. Si el argumento es otra cosa, como $g(x)+1$, se imprime y se verbaliza con normalidad pero sin anotar la llamada que contiene. Las llaves `read-cmd` y `silent` valen para todos los niveles; `concept` y `prefix` solo para la llamada exterior.

Las comas que quedan dentro de llaves, de paréntesis o de corchetes no separan argumentos. Por eso, un número decimal con coma debe escribirse entre llaves: $\$ \backslash \text{spfun}\{f(1{,}5)\}$ tiene un solo argumento.

El comando `\spfun` devuelve un “error” y se detiene la compilación del documento cuando el $\{ \langle \text{función} \rangle \}$ no comienza con un nombre válido, cuando el subíndice contiene algo distinto de letras o dígitos, cuando el nombre no va seguido de paréntesis, cuando los paréntesis, las llaves o los corchetes no están balanceados, cuando un argumento está vacío o cuando hay algo después del paréntesis que cierra. Lo mismo vale para una llamada anidada en un argumento.

Llaves para `\spfun`

`read-cmd` = $\{ \langle \text{school} \mid \text{mathcat} \rangle \}$ default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará el nombre de la función. Si se establece en `school`, NVDA/MathCAT verbaliza el nombre sin el prefijo «mayúscula»: $\$ \backslash \text{spfun}\{F'(x)\}$ se verbaliza «*F doble prima de x*»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix` = $\{ \langle \text{prefijo} \rangle \}$ default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el concepto «función». El valor de esta llave *siempre* debe pasarse entre llaves $\{ \quad \}$. Solo se verbaliza cuando `concept` es `true`.

`concept` = $\{ \langle \text{true} \mid \text{false} \rangle \}$ default: *false*

Establece si se verbaliza el concepto «función» antes de la función. Si se establece en `true`, NVDA/MathCAT verbalizará el concepto, el prefijo establecido por la llave `prefix` y la función; si se establece en `false`, solo verbalizará la función.

`silent` = $\{ \langle \text{true} \mid \text{false} \rangle \}$ default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del $\{ \langle \text{función} \rangle \}$ pasado al comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, sí verbalizará la salida del comando. El argumento se valida igual en ambos casos.

Los comandos `\spdom`, `\sprec` y `\spcod`

`\spdom` $\backslash \text{spdom}\{ \langle \text{función} \rangle \}$ $\backslash \text{spdom}[\langle \text{key} = \text{val} \rangle]\{ \langle \text{función} \rangle \}$

`\sprec` Los comandos `\spdom`, `\sprec` y `\spcod` trabajan en *modo matemático* y reciben el *argumento* obligatorio $\{ \langle \text{función} \rangle \}$: el nombre de una función como en `\spfun` (una letra con prima(s) o subíndice opcionales y, si se quiere, el exponente $\wedge\{-1\}$), sin argumentos. Imprimen respectivamente Dom, Rec y Cod seguidos de la función entre paréntesis.

$\$ \backslash \text{spdom}\{f\}$ imprimirá ‘Dom(*f*)’ y NVDA/MathCAT verbalizará «*dominio de f*»; $\$ \backslash \text{sprec}\{f\}$ verbalizará «*recorrido de f*» y $\$ \backslash \text{spcod}\{f\}$ «*codominio de f*». Con la inversa, $\$ \backslash \text{spdom}\{f^{\wedge\{-1\}}\}$ verbalizará «*dominio de f inversa*». El nombre de la función se verbaliza sin anunciar la mayúscula.

Los comandos devuelven un “error” y se detiene la compilación cuando el argumento no es el nombre de una función.

Llaves para `\spdom`, `\sprec` y `\spcod`

`prefix` = $\{ \langle \text{prefijo} \rangle \}$ default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de la palabra del concepto. El valor de esta llave *siempre* debe pasarse entre llaves $\{ \quad \}$.

`silent` = $\{ \langle \text{true} \mid \text{false} \rangle \}$ default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del $\{ \langle \text{función} \rangle \}$ pasado al comando sin modificar la impresión de este.

El comando `\spfuncomp`

`\spfuncomp` $\backslash \text{spfuncomp}\{ \langle \text{composición} \rangle \}$ $\backslash \text{spfuncomp}[\langle \text{key} = \text{val} \rangle]\{ \langle \text{composición} \rangle \}$

El comando `\spfuncomp` trabaja en *modo matemático* y recibe el *argumento* obligatorio $\{ \langle \text{composición} \rangle \}$: dos o más *funciones* separadas por `\circ`, como en $\{ \langle f \circ g \rangle \}$ o $\{ \langle f \circ g \circ h \rangle \}$. Cada función es un nombre como en `\spfun`: una letra, mayúscula o minúscula, con prima(s) o subíndice opcionales, y el exponente $\wedge\{-1\}$ de la función inversa. La composición puede ir entre paréntesis, $\{ \langle (f \circ g) \rangle \}$, y puede aplicarse a uno o más argumentos entre paréntesis separados por coma, $\{ \langle (f \circ g)(x) \rangle \}$. Los argumentos siguen las reglas de `\spfun`: pueden ser letras, números, expresiones o llamadas.

$\$ \backslash \text{spfuncomp}\{f \circ g\}$ imprimirá ‘*f* ◦ *g*’ en la salida PDF y NVDA/MathCAT verbalizará «*f compuesta con g*»; $\$ \backslash \text{spfuncomp}\{f \circ g \circ h\}$ imprimirá ‘*f* ◦ *g* ◦ *h*’ y verbalizará «*f compuesta con g compuesta con h*»; $\$ \backslash \text{spfuncomp}\{(f \circ g)(x)\}$ imprimirá ‘(*f* ◦ *g*)(*x*)’ y verbalizará «*f compuesta con g de x*»;

`\spfuncomp{(f^{-1}\circ g)(x)}` imprimirá $(f^{-1} \circ g)(x)$ y verbalizará «*f inversa compuesta con g de x*». Un argumento puede ser una llamada: `\spfuncomp{(f\circ g)(h(x))}` verbalizará «*f compuesta con g de h de x*».

El comando `\spfuncomp` devuelve un “error” y se detiene la compilación del documento cuando hay menos de dos funciones, cuando algo entre los `\circ` no es el nombre de una función, cuando los paréntesis no están balanceados, cuando lo que sigue a la composición no es una lista de argumentos entre paréntesis o cuando un argumento está vacío. La forma `f\circ g(x)`, sin paréntesis alrededor de la composición, no es válida porque es ambigua.

Llaves para `\spfuncomp`

`silent = {\true | false}`

default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del `\{composición\}` pasado al comando sin modificar la impresión de este. Si se establece en *true*, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en *false*, sí verbalizará la salida del comando. El argumento se valida igual en ambos casos.

La composición no admite *prefix* ni *read-cmd*: no lleva palabra de concepto y su lectura es fija. Los nombres se verbalizan sin anunciar la mayúscula.

9 Referencias

- [1] ROBERTSON, WILL . “unicode-math - Unicode mathematics support for X_YTeX and LuaTeX”. Available from CTAN, <https://www.ctan.org/pkg/unicode-math>, 2023.
- [2] KRUEGER, MARCEL. “LuaMML - Automatically generate MathML from LuaTeX math mode material”. Available from CTAN, <https://ctan.org/pkg/luamml>, 2026.
- [3] The L^AT_EX Project. “fontspec - Advanced font selection for X_YL^AT_EX and LuaTeX”. Available from CTAN, <https://www.ctan.org/pkg/fontspec>, 2025.
- [4] VOß, HERBERT. “libertinus-otf - Support for Libertinus OpenType”. Available from CTAN, <https://www.ctan.org/pkg/libertinus-otf>, 2025.
- [5] FLIPO, DANIEL. “erewhon-math - Utopia based OpenType Math font”. Available from CTAN, <https://ctan.org/pkg/erewhon-math>, 2026.
- [6] HOFSTRA, SILKE. “sourcecodepro - Use SourceCodePro with T_EX(-alike) systems”. Available from CTAN, <https://ctan.org/pkg/sourcecodepro>, 2025.
- [7] GONZÁLEZ, PABLO. “scontents - Stores L^AT_EX contents in memory or files”. Available from CTAN, <https://www.ctan.org/pkg/scontents>, 2026.
- [8] GONZÁLEZ, PABLO. “enumext - Enumerate exercise sheets”. Available from CTAN, <https://www.ctan.org/pkg/enumext>, 2026.
- [9] BRAAMS, JOHANNES AND BEZOS, JAVIER. “babel - Multilingual support for L^AT_EX, LuaL^AT_EX, X_YL^AT_EX, and Plain T_EX”. Available from CTAN, <https://www.ctan.org/pkg/babel>, 2026.
- [10] BEZOS, JAVIER. “babel-spanish - Babel support for Spanish”. Available from CTAN, <https://www.ctan.org/pkg/babel-spanish>, 2026.
- [11] NV ACCESS. “NVDA - NonVisual Desktop Access screen reader”. Available from NV ACCESS, <https://www.nvaccess.org/>, 2026.
- [12] SOIFFER, NEIL. “MathCAT - Math to speech and braille translation”. Available from GITHUB, <https://nsoiffer.github.io/MathCAT/>, 2026.
- [13] ADOBE INC. “Adobe Acrobat Reader - Reliable PDF viewer”. Available from ADOBE, <https://get.adobe.com/reader/>, 2026.
- [14] ISO. “ISO 32000-2:2020 - Portable Document Format (PDF 2.0)”. Available from PDF ASSOCIATION, <https://pdfa.org/resource/iso-32000-2/>, 2020.
- [15] PDF ASSOCIATION. “WTPDF - Well-Tagged PDF specification”. Available from PDFA, <https://pdfa.org/wtpdf/>, 2024.
- [16] PDF ASSOCIATION. “PDF 2.0 Errata Collection 3”. Available from PDFA, <https://pdfa.org/PDF-2-0-errata-collection-3-now-available/>, 2026.
- [17] L^AT_EX PROJECT LWG. “Best Practice Guide: Math in PDF”. Available from PDFA, <https://pdfa.org/resource/best-practice-guide-math-in-PDF/>, 2026.
- [18] JOHNSON, DUFF. “PDF 2.0: New Features, Real-World Impact”. Available from PDFA, <https://pdfa.org/PDF-2-0-new-features-real-world-impact/>, 2026.
- [19] The L^AT_EX Project. “LaTeX Tagged PDF Project - Documentation and resources”. Available from L^AT_EX PROJECT, <https://latex3.github.io/tagging-project/>, 2026.
- [20] VERAPDF CONSORTIUM. “veraPDF - Industry supported PDF/A and PDF/UA validator”. Available from VERAPDF, <https://verapdf.org/>, 2026.
- [21] DUAL LAB. “ngPDF - Next Generation PDF demonstrator and validator”. Available from NGPDF, <https://ngpdf.com/>, 2026.
- [22] BERRY, KARL. “L^AT_EX 2_ε: An Unofficial Reference Manual”. Available from CTAN, <https://ctan.org/pkg/latex2e-help-texinfo>, 2026.
- [23] The L^AT_EX Project. “Extensive support for hypertext in L^AT_EX”. Available from CTAN, <https://www.ctan.org/pkg/hyperref>, 2026.
- [24] The L^AT_EX Project. “The expl3 package”. Available from CTAN, <https://www.ctan.org/pkg/l3kernel>, 2026.

- [25] The L^AT_EX Project. “The L^AT_EX₃ Interfaces”. Available from CTAN, <https://www.ctan.org/pkg/l3kernel>, 2026.
- [26] The L^AT_EX Project. “The L^AT_EX_{2_ε} sources”. Available from CTAN, <https://ctan.org/tex-archive/macros/latex/base>, 2026.
- [27] The L^AT_EX Project. “L^AT_EX News, Issue 41, June 2025”. Available from CTAN, <https://ctan.org/tex-archive/macros/latex/base>, 2025.
- [28] The L^AT_EX Project. “L^AT_EX for authors current version”. Available from CTAN, <https://ctan.org/pkg/latex-base>, 2026.
- [29] FISCHER, ULRIKE. “tagpdf – L^AT_EX kernel code for PDF tagging”. Available from CTAN, <https://www.ctan.org/pkg/tagpdf>, 2026.
- [30] The L^AT_EX Project. “latex-lab – L^AT_EX laboratory”. Available from CTAN, <https://www.ctan.org/pkg/latex-lab>, 2026.

10 Change history

v0.99 (ctan), 2026-10-07 – First public release.

11 Índice de la Documentación

Los números en cursiva indican las páginas donde se describe la entrada correspondiente.

C	K
Document class:	Keys for \spAcirc provide by spintent :
article 1	concept 32
book 1	prefix 32
letter 1	print-sym 32
report 1	read-arg 32
Commands provide by spintent :	read-cmd 32
\setspintent 3	read-sty 32
\spAcirc 31	read-sub 32
\spAfig 26	silent 32
\spPcirc 32	sub-index 32
\spPfig 28	Keys for \spAfig provide by spintent :
\spPoint 19	concept* 27
\spPoly 26	concept 27
\spProp 18	prefix 27
\spVol 34	print-sym 27
\spangle 23	read-arg 27
\spang 24	read-cmd 27
\sparc 25	read-sub 27
\spcirc 30	silent 27
\spcod 36	Keys for \spangle provide by spintent :
\spcoord 19	concept 23
\spdate 11	prefix 23
\spdiam 29	read-cmd 23
\spdiv 13	recto 23
\spdom 36	silent 23
\spdsym 12	Keys for \sparc provide by spintent :
\spfuncomp 36	concept 25
\spfun 35, 36	prefix 25
\spinterval 34	read-cmd 25
\spmQ 17	silent 25
\spmangle 24	Keys for \spcirc provide by spintent :
\spmarc 25	concept 31
\spmcd 15	diameter 30
\spmcm 16	prefix 30
\spmoney 6	print-sym 30
\spmside 22	radio-sty 31
\spmsym 13	read-arg 31
\spmul 14	read-cmd 30
\spnD 14	read-sty 30
\spnM 15	read-sub 31
\spnZ 17	silent 31
\spnewread 11	spmside 30, 31
\spnewunit 6	spside 30, 31
\spnum 3	sub-index 31
\sprad 29	Keys for \spcoord provide by spintent :
\sprayo 21	silent 19
\sprazon 18	Keys for \spdiam provide by spintent :
\spread 12	concept 29
\sprecta 20	print-sym 29
\sprec 36	read-arg 30
\spshort 8	read-cmd 29
\spside 21	read-sub 30
\spsiglo 10	silent 30
\sptime 11	sub-index 29
\spunitalias 6	upper-case 29
\spunit 4	Keys for \spdiv provide by spintent :
\spusep 12	cifras-sep 14
\spvec 20	notation-sym 14
	read-parens 14
	read-period-sep 14

read-sign	14	prefix	22
read-sym	14	print-m	23
set-sym	14	read-arg	23
wrap-parens	14	read-cmd	22
Keys for \spdom provide by spintent :		read-sty	22
prefix	36	silent	23
silent	36	Keys for \spmsym provide by spintent :	
Keys for \spdsym provide by spintent :		not-print	13
prefix	13	prefix	13
read-sym	13	read-sym	13
set-sym	13	set-sym	13
suffix	13	suffix	13
Keys for \spfuncomp provide by spintent :		Keys for \spmul provide by spintent :	
silent	37	cifras-sep	14
Keys for \spfun provide by spintent :		notation-sym	14
concept	36	read-parens	14
prefix	36	read-period-sep	14
read-cmd	36	read-sign	14
silent	36	read-sym	14
Keys for \spinterval provide by spintent :		set-sym	14
concept	35	wrap-parens	14
prefix	35	Keys for \spnD provide by spintent :	
read-arg	35	concept	14
read-cmd	35	prefix	14
silent	35	result-num	14
size	35	result	14
Keys for \spmangle provide by spintent :		silent	15
concept	24	Keys for \spnM provide by spintent :	
prefix	24	concept	15
print-m	24	prefix	15
read-cmd	24	result-num	15
recto	24	result	15
silent	24	silent	15
Keys for \spmarc provide by spintent :		Keys for \spnum provide by spintent :	
concept	25	cifras-sep	3
prefix	25	notation-sym	4
read-cmd	25	read-period-sep	3
silent	25	read-sign	3
Keys for \spmcd provide by spintent :		Keys for \spnZ provide by spintent :	
concept	16	cifras-sep	17
prefix	16	read-parens	17
read-cmd	16	read-sign	17
result	16	wrap-parens	17
silent	16	Keys for \spPc i rc provide by spintent :	
upper-case	16	concept	33
Keys for \spmcm provide by spintent :		prefix	33
concept	16	print-sym	33
prefix	16	read-arg	33
read-cmd	16	read-cmd	33
result	16	read-sty	33
silent	17	read-sub	33
upper-case	16	silent	33
Keys for \spmoney provide by spintent :		sub-index	33
cifras-sep	7	Keys for \spPfig provide by spintent :	
currency	7	concept*	28
money-math	7	concept	28
sub-units	7	prefix	28
Keys for \spmQ provide by spintent :		print-sym	29
join-word	17	read-arg	28
read-parens	18	read-cmd	28
use-dfrac	17	read-sub	28
use-sp nZ	17	silent	29
wrap-parens	17	Keys for \spPoint provide by spintent :	
Keys for \spmside provide by spintent :		concept	19
concept	22	prefix	19

read-cmd	19	Keys for \spsiglo provide by spintent :	
silent	19	print-era	11
Keys for \spPoly provide by spintent :		Keys for \spunit provide by spintent :	
concept	26	cifras-sep	4
prefix	26	notation-sym	4
print-sym	26	print-cdot	4
read-cmd	26	read-cdot	4
silent	26	read-period-sep	4
Keys for \spProp provide by spintent :		read-slash	4
inline	18	Keys for \spusep provide by spintent :	
silent	18	silent	12
use-dfrac	18	size	12
Keys for \sprad provide by spintent :		Keys for \spvec provide by spintent :	
concept	29	autofit	20
read-arg	30	concept	20
read-cmd	29	prefix	20
read-sub	30	read-cmd	20
silent	30	silent	20
sub-index	29	Keys for \spVol provide by spintent :	
upper-case	29	concept	34
Keys for \sprayo provide by spintent :		read-arg	34
concept	21	read-cmd	34
prefix	21	read-sub	34
read-cmd	21	silent	34
read-sty	21	sub-index	34
silent	21		
Keys for \sprazon provide by spintent :			
inline	18		
silent	18		
use-dfrac	18		
Keys for \spread provide by spintent :			
read-arg	12		
silent	12		
Keys for \sprecta provide by spintent :			
concept	21		
prefix	20		
read-cmd	20		
silent	21		
Keys for \spshort provide by spintent :			
read-arg	8		
small-caps	8		
Keys for \spside provide by spintent :			
concept	22		
prefix	22		
read-arg	22		
read-cmd	22		
read-sty	22		
silent	22		

M

Lua module provide by spintent :	
spintent.lua	6–11

P

Packages :	
babel-spanish	2
babel	2, 8
documentmetadata-support	1
enumext	1
expl3	3
fontspec	2
fourier-otf	2
hyperref	2
l3keys	3
latex-lab-mathintent	1
libertinus-otf	2
luamml	1
scontents	1
sourcecodepro	2
tagpdf	1, 8, 11
unicode-math	1

12 Implementation

The most recent publicly released version of `spintent` is available at CTAN: <https://www.ctan.org/pkg/spintent>. While general feedback via email is welcomed, specific bugs or feature requests should be reported through the issue tracker: <https://github.com/pablgonz/spintent/issues>.

- The documentation presented here is far from professional, it contains a lot of obvious information that to the eye of a TeXpert are superfluous, but, after so many years developing this project is the only way to remember what does what.

12.1 General conventions

Functions of the form `\__spintent_luafun_some:n` or `\__spintent_luafun_some:nn` are defined (created) in the Lua module `spintent.lua` and only their variants are declared in `expl3`.

Similarly, variables of the form `\l__spintent_luaset_some_tl` or `\l__spintent_luaset_some_str` are defined in the Lua module `spintent.lua` and only declared in `expl3`.

- All variables and functions defined in this package are private and are NOT intended to work or be used by another package or module.

12.2 Initial set up

Start the DocStrip guards.

```
1 (*package)
```

Identify the internal prefix (L^AT_EX3 DocStrip convention) for `l3doc` class.

```
2 (@@=spintent)
```

12.3 Declaration of the package

First we will make sure we have a minimum (super updated) release version of L^AT_EX to work correctly, otherwise, we will return an “error”.

```
3 \NeedsTeXFormat{LaTeX2e}[2026-11-01]
4 \IfFormatAtLeastF { 2026-11-01 }
5 {
6   \PackageError { spintent }
7     { LaTeX~kernel~too~old~need~2026-11-01~or~later }
8     {
9       spintent~requires~a~LaTeX2e~release~2026-11-01~or~
10      later.~Update~your~TeX~distribution.
11     }
12 }
```

Now declare the `spintent` package and add only two keys: `mathcal` and `AFghost`, which can be passed as package options.

```
13 \ProvidesExplPackage {spintent} {2026-10-07} {0.99} {Spanish parse intents}
14 \keys_define:nn { spintent }
15 {
16   mathcal .usage:n    = load,
17   mathcal .bool_set:N = \l__spintent_mathcal_bool,
18   mathcal .initial:n  = true,
19   mathcal .value_required:n = true,
20   AFghost .usage:n    = load,
21   AFghost .bool_set:N = \l__spintent_debug_ghost_bool,
22   AFghost .initial:n  = false,
23   AFghost .value_required:n = true,
24 }
25 \ProcessKeyOptions [ spintent ]
```

We check if the requirements for running the `spintent` package are met; that is: `\DocumentMetadata` must be active, Lua_T_EX must be running, and the `unicode-math` or `lua-unicode-math` package must be loaded. If neither is present, we will load `unicode-math`. If the conditions cannot be met, we will return a “fatal error”.

```
26 \IfDocumentMetadataTF
27 {
28   \sys_if_engine luatex:TF
29   {
30     \msg_new:nnn { spintent } { package-not-load }
31     {
32       The~'#1'~package~will~be~loaded~as~a~dependency.
33     }
34     \IfPackageLoadedF { unicode-math }
35     {
36       \IfPackageLoadedF { lua-unicode-math }
37       {
38         \msg_info:nnn { spintent } { package-not-load } { unicode-math }
39         \RequirePackage{unicode-math}

```

```

40     }
41   }
42   \msg_new:nnn { spintent } { env-success }
43   { Everything~looks~correct,~spintent~will~make~the~intent! }
44   \msg_info:nn { spintent } { env-success }
45 }
46 {
47   \msg_new:nnnn { spintent } { fatal-engine-requires-luatex }
48   { The~spintent~package~requires~LuaLaTeX. }
49   { This~package~relies~on~Lua~module.~Compile~with~lualatex. }
50   \msg_fatal:nn { spintent } { fatal-engine-requires-luatex }
51 }
52 }
53 {
54   \msg_new:nnnn { spintent } { fatal-metadata-missing }
55   { \token_to_str:N \DocumentMetadata \c_space_tl is-missing. }
56   {
57     The~spintent~package~requires~tagged~PDF~active.\\
58     You~must~declare~\token_to_str:N \DocumentMetadata { tagging=on } ~
59   }
60   \msg_fatal:nn { spintent } { fatal-metadata-missing }
61 }

```

12.4 Some kernel variants

```

\str_if_eq:nVTF \str_if_eq:nVF
\str_uppercase:V \str_lowercase:V
\seq_set_from_clist:Ne
\cs_generate_variant:Nn \str_if_eq:nnTF { V }
\cs_generate_variant:Nn \str_if_eq:nnF { V }
\cs_generate_variant:Nn \str_uppercase:n { V }
\cs_generate_variant:Nn \str_lowercase:n { V }
\cs_generate_variant:Nn \seq_set_from_clist:Nn { Ne }

```

Non-standard kernel variants.

(End of definition for `\str_if_eq:nVTF` and others.)

12.5 Variables for the key `\mathcal`, `\rightanglesqr` and `\attribute`

First, we declare the string variable `\l__spintent_luaset_cal_fam_ok_str` used by the function `\__spintent_luafun_classic_mathcal:nn` (controlled by the key `mathcal`), the string variable `\l__spintent_luafun_right_angle_sqr` used by the function `\__spintent_luafun_right_angle_sqr:` to copy `\rightanglesqr` alongside the string variable `\l__spintent_luaset_scope_attr_str` used by `\attribute` before loading the Lua module `spintent.lua`. This is necessary due to the implementation of the functions within the Lua module; otherwise, `expl3` would issue a warning regarding duplicate variable declarations.

```

67 \str_new:N \l__spintent_luaset_cal_fam_ok_str
68 \str_new:N \l__spintent_luaset_sym_fam_ok_str
69 \str_new:N \l__spintent_luaset_scope_attr_str

```

(End of definition for `\l__spintent_luaset_cal_fam_ok_str`, `\l__spintent_luaset_sym_fam_ok_str`, and `\l__spintent_luaset_scope_attr_str`.)

12.6 Loading the Lua module and functions

`spintent.lua` Loads the Lua module `spintent.lua`, where every `_luafun_` function used throughout the package is registered.

```

70 \lua_load_module:n { spintent.lua }

```

(End of definition for `spintent.lua`.)

`\__spintent_luafun_set_debug_ghost:n` Passes `AFghost` to Lua as a plain string. When `true`, both ghost characters are marked `sptmp="outer"` or `sptmp="inner"` in the file `-luamml-mathml.html`, so an external script can identify and strip them before generating the Associated~Files structure for `tagged PDF` — `math-AF` mode does not want either ghost present.

```

71 \bool_if:NTF \l__spintent_debug_ghost_bool
72 { \__spintent_luafun_set_debug_ghost:n { true } }
73 { \__spintent_luafun_set_debug_ghost:n { false } }

```

(End of definition for `\__spintent_luafun_set_debug_ghost:n`.)

`\l__spintent_scope_attr_int` A LuaTeX attribute number, allocated collision-free from Lua via `luatexbase.new_attribute` and exposed here as an integer. Every command that uses the shared function `\__spintent_luafun_mrow_intent_arg:nn` activates it, via `\attribute \l__spintent_scope_attr_int = 1`, inside its own group — marking every node typeset within as eligible for that queue to claim. Without this mark, the queue could claim content from an unrelated command sitting in the same formula.

```

74 \int_new:N \l__spintent_scope_attr_int
75 \int_set:Nn \l__spintent_scope_attr_int { \l__spintent_luaset_scope_attr_str }

```

(End of definition for `\__spintent_scope_attr_int`.)

`\__spintent_luafun_spunit_lookup_alias:n` Variants generated for `\__spintent_luafun_spunit_lookup_alias:n` (used by `\spunit`), `\__spintent_luafun_spmoney_lookup_data:n` (used by `\spmoney`), `\__spintent_luafun_spmQ_scale_int:nn` (used by `\spmQ`) and `\__spintent_luafun_clean_split_and_set:n` (used by `\spnum`).

```

76 \cs_generate_variant:Nn \__spintent_luafun_spunit_lookup_alias:n { V }
77 \cs_generate_variant:Nn \__spintent_luafun_spmoney_lookup_data:n { V }
78 \cs_generate_variant:Nn \__spintent_luafun_spmQ_scale_int:nn { Vn }
79 \cs_generate_variant:Nn \__spintent_luafun_clean_split_and_set:n { V }

```

(End of definition for `\__spintent_luafun_spunit_lookup_alias:n` and others.)

12.7 Internal implementation for `\mathcal` and `\rightanglesqr`

To simplify user configuration and obtain the expected output style pdf_{TeX} for `\mathcal`, we will provide a fallback to the font NewCMMath-Regular.otf, which contains the glyphs for these characters. Using this same font, we will obtain the symbol `\rightanglesqr` for the key `recto`.

`mathcal-fallback`
`right-angle-sqr-fallback`

First, we'll define two error messages in case our fallback fails.

```

80 \hook_gput_code:nnn { begindocument } { spintent }
81 {
82   \__spintent_luafun_init_math_fallbacks:
83 }
84 \msg_new:nnn { spintent } { mathcal-fallback }
85 {
86   Using~classic~\token_to_str:N \mathcal \c_space_tl as~fallback~
87   (font~not~found~or~no~free~math~family~available).~\msg_line_context:
88 }
89 \msg_new:nnn { spintent } { right-angle-sqr-fallback }
90 {
91   Using~\token_to_str:N \rightangle \c_space_tl as~fallback~
92   for~the~right-angle~ glyph.~\msg_line_context:
93 }

```

(End of definition for `mathcal-fallback` and `right-angle-sqr-fallback`.)

`\__spintent_mathcal:n`

Now we define the internal function `\__spintent_mathcal:n` for `\mathcal` which checks the state of the variable `\__spintent_mathcal_bool` set by the package key `mathcal`; if its state is true, we check the string variable `\__spintent_luaset_cal_fam_ok_str` and if the latter has the value true, we call the function `\__spintent_luafun_classic_mathcal:n` and otherwise (we cannot find the source in the system) we return a warning message and directly use `\mathcal`. Otherwise, that is, `mathcal=false` we simply use `\mathcal`.

```

94 \cs_new_protected:Nn \__spintent_mathcal:n
95 {
96   \bool_if:NTF \__spintent_mathcal_bool
97   {
98     \str_if_eq:VnTF \__spintent_luaset_cal_fam_ok_str { true }
99     { \__spintent_luafun_classic_mathcal:n {#1} }
100     {
101       \msg_warning:nn { spintent } { mathcal-fallback }
102       \mathcal {#1}
103     }
104   }
105   { \mathcal {#1} }
106 }

```

(End of definition for `\__spintent_mathcal:n`.)

`\__spintent_right_angle_sqr:`

The internal function `\__spintent_right_angle_sqr:` establishes the internal copy of the command `\rightanglesqr`. First, we check the string variable `\__spintent_luaset_sym_fam_ok_str`. If it is true, we call the function `\__spintent__luafun_right_angle_sqr:`; otherwise (we do not find the font in the system), we return a warning message and use `\rightangle` instead.

```

107 \cs_new_protected:Nn \__spintent_right_angle_sqr:
108 {
109   \str_if_eq:VnTF \__spintent_luaset_sym_fam_ok_str { true }
110   { \__spintent_luafun_right_angle_sqr: }
111   {
112     \msg_warning:nn { spintent } { right-angle-sqr-fallback }
113     \rightangle
114   }
115 }

```

(End of definition for `\__spintent_right_angle_sqr:`.)

12.8 Shared annotation mechanisms: the queue and the ghosts

Almost all commands in the `spintent` package share three annotation functions, the mechanism of which is defined in the Lua module `spintent.lua`.

`\__spintent_luafun_mrow_intent_arg:nn`

The internal function `\__spintent_luafun_mrow_intent_arg:nn` — called directly, with no `expl3` wrapper — pushes one entry onto a queue: a concept `intent` (`#1`, possibly empty) and a Lua table literal `arg=/intent=(#2)`. A `pre_mlist_to_hlist_filter` callback (installed once, before `luamml[2]`’s own) walks the formula’s top-level noads in order and, for each one whose nucleus or subscript was typeset inside a `spintent` command’s own group (marked via a LuaTeX `\attribute` activated by every command that uses this queue), claims it for the next pending entry: `arg`/a literal `intent` land on that leaf, and, when the concept `intent` is non-empty, the leaf is wrapped in a native `<mrow>` carrying it — no token, no ghost involved. Entries are consumed strictly in order; any left unclaimed when the formula ends are discarded rather than carried into the next one. A native `\MathMLarg/\MathMLintent` claim is invisible to this queue: mixing the two for the same content risks the queue claiming something already tagged natively, or never being claimed at all.

(End of definition for `\__spintent_luafun_mrow_intent_arg:nn`.)

The other two are “ghost” characters (an invisible U+2063, self-tagged `:silent` through their own `mathml_filter` — so the queue above never mistakes either for one of its own free targets) dropped in as an extra sibling so the resulting `<mrow>` is not the formula’s sole top-level content, which `luamml` would otherwise promote straight to `<math intent=...>` instead of leaving it nested (issue #26).

`\__spintent_invisible_sep:`

Placed once, before opening the formula’s own outer `\MathMLintent` — protects the *whole formula* from collapsing when that outer group is its only top-level content. Fires at most once per document: past the first call, in any command, it does nothing, since one sibling anywhere in the formula is already enough.

```
116 \cs_new_protected:Nn \__spintent_invisible_sep:
117 {
118   \__spintent_luafun_invisible_sep:
119 }
```

🔗 Executing this function writes the following line directly to the output file `-luamml-mathml.html`:

```
<mi intent=":silent" mathvariant="normal"></mi>
```

and if the package key `AFghost=true` is used, the output is:

```
<mi intent=":silent" mathvariant="normal" sptmp="outer"></mi>
```

If you wish to use math-AF mode, you can remove the line generated by `AFghost=true` using the Lua script `spiafcls`.

(End of definition for `\__spintent_invisible_sep:`.)

`\__spintent_inner_sep:`

Placed just inside a native `\MathMLintent` group — protects *that specific group* when it too would otherwise hold only one child. Never deduplicates: called once per such group, however many appear, nested or sibling, in the same formula.

```
120 \cs_new_protected:Nn \__spintent_inner_sep:
121 {
122   \__spintent_luafun_inner_sep:
123 }
```

🔗 Executing this function writes the following line directly to the output file `-luamml-mathml.html`:

```
<mi intent=":silent" mathvariant="normal"></mi>
```

and if the package key `AFghost=true` is used, the output is:

```
<mi intent=":silent" mathvariant="normal" sptmp="inner"></mi>
```

If you wish to use math-AF mode, you can remove the line generated by `AFghost=true` using the Lua script `spiafcls`.

(End of definition for `\__spintent_inner_sep:`.)

`\c__spintent_invisible_sep_int`

A raw, untagged copy of the same U+2063 character, via the LuaTeX primitive `\Umathchardef` — for use as ordinary content *inside* an explicit `\MathMLintent{<intent>}` call (typically `:silent`), not as a sibling on its own.

```
124 \hook_gput_code:nnn { begindocument } { spintent }
125 {
126   \Umathchardef \c__spintent_invisible_sep_int = "0 "0 "2063
127 }
```

🔗 Documented in `source3` on page 887 for `\int_const:Nn`: “As stated, most constants can be defined as `\chardef` or `\mathchardef` but that’s engine dependent.” —so this appears to be the correct option here.

(End of definition for `\c__spintent_invisible_sep_int`.)

12.9 Normalize argument and affixes

We normalize the $\{\langle argument \rangle\}$ to support affixes (prefixes/suffixes) by trimming leading and trailing spaces, converting internal spaces to hyphens, collapsing consecutive hyphens, and removing leading or trailing hyphens.

`\__spintent_collapse_hyphens:N` A recursive helper to collapse multiple hyphens into a single one.

```

128 \cs_new_protected:Nn \__spintent_collapse_hyphens:N
129 {
130   \tl_if_in:NnT #1 { -- }
131   {
132     \tl_replace_all:Nnn #1 { -- } { - }
133     \__spintent_collapse_hyphens:N #1
134   }
135 }
```

(End of definition for `\__spintent_collapse_hyphens:N`.)

`\__spintent_sanitize_affix:Nn`
`\__spintent_sanitize_affix:cn`

The function `\__spintent_sanitize_affix:Nn` checks if the $\{\langle argument \rangle\}$ passed in ‘#2’ is *blank*. If so, it clears the target token list variable passed in ‘#1’. Otherwise, it converts the $\{\langle argument \rangle\}$ into a string (so every character has category code 12 and it no longer matters how an underscore ‘_’ was typed), trims the bounds, converts spaces ‘ ’ and underscores ‘_’ to hyphens ‘-’, collapses multiple hyphens, and safely strips leading/trailing hyphens. Because underscores are converted *before* collapsing, any run mixing hyphens and underscores is reduced to a single hyphen, and the stored value never starts or ends with ‘-’ or ‘_’: the command that uses the value is the one that places the leading ‘_’ of the intent name. For example, if you pass a target variable and raw text:

```
\__spintent_sanitize_affix:Nn \__spintent_spsomesym_prefix_tl{ foo bar baz }
```

The function will “store” the normalized string foo-bar-baz inside `\__spintent_spsomesym_prefix_tl`, ready to be conditionally appended. In the same way, `{ __foo__bar__ }` is stored as foo-bar.

The only exception is the exact value ‘_’ (once the bounds are trimmed), which is stored as a single hyphen ‘-’. After the command prepends its own leading ‘_’, this yields the *silent* literal ‘_-’ documented for `\spread`. Any other value made only of hyphens, underscores and spaces is stored as *empty*.

```

136 \cs_new_protected:Nn \__spintent_sanitize_affix:Nn
137 {
138   \tl_if_blank:nTF {#2}
139   { \tl_clear:N #1 }
140   {
141     \tl_set:Nc #1 { \tl_to_str:e {#2} }
142     \tl_trim_spaces:N #1
143     \str_if_eq:VnTF #1 { _- }
144     { \tl_set:Nn #1 { - } }
145     {
146       \tl_replace_all:Nnn #1 { ~ } { - }
147       \tl_replace_all:Nee #1 { \tl_to_str:n { _ } } { - }
148       \__spintent_collapse_hyphens:N #1
149       \tl_put_left:Nn #1 { \q_mark }
150       \tl_put_right:Nn #1 { \q_stop }
151       \tl_replace_all:Nnn #1 { \q_mark - } { \q_mark }
152       \tl_replace_all:Nnn #1 { - \q_stop } { \q_stop }
153       \tl_remove_all:Nn #1 { \q_mark }
154       \tl_remove_all:Nn #1 { \q_stop }
155     }
156   }
157 }
158 \cs_generate_variant:Nn \__spintent_sanitize_affix:Nn { cn }
```

(End of definition for `\__spintent_sanitize_affix:Nn`.)

12.10 The command `\setspintent`

The `\setspintent` command is used to configure $\langle keys \rangle$ “globally” for the utilities provided by `spintent`.

12.10.1 Internal variables and error messages for `\setspintent`

`\__spintent_setspintent_input_arg_cmd_tl`
`unknown-command-setup`

The variable `\__spintent_setspintent_input_arg_cmd_tl` will *store* the command or command name passed to the “first” $\{\langle argument \rangle\}$ ‘#1’ of the command `\setspintent`.

```
159 \tl_new:N \__spintent_setspintent_input_arg_cmd_tl
```

Error message if the $\langle argument \rangle$ ‘#1’ passed to `\setspintent` is a “non-existent” command.

```

160 \msg_new:nnnn { spintent } { unknown-command-setup }
161 {
162   Cannot~configure~unknown~command~'\exp_not:c{#1}'.
163 }
164 {
165   You~attempted~to~use~\token_to_str:N \setspintent{#1}{...},~but~the~
166   command~\exp_not:c{#1}~is~not~defined.~Please~check~for~typos.
167 }

```

(End of definition for `\__spintent_setspintent_input_arg_cmd_tl` and `unknown-command-setup`.)

```

\__spintent_setspintent_set_keys:nn
\__spintent_setspintent_set_keys:en

```

The function `\__spintent_setspintent_set_keys:nn` will check if the command passed as $\langle argument \rangle$ exists in ‘#1’ and set the $\langle keys \rangle$ passed as $\langle argument \rangle$ in ‘#2’ to the correct path; otherwise, it will return an error.

```

168 \cs_new_protected:Nn \__spintent_setspintent_set_keys:nn
169 {
170   \cs_if_exist:cTF { #1 }
171   {
172     \keys_set:nn { spintent / #1 } { #2 }
173   }
174   {
175     \msg_error:nnn { spintent } { unknown-command-setup } { #1 }
176   }
177 }
178 \cs_generate_variant:Nn \__spintent_setspintent_set_keys:nn { en }

```

(End of definition for `\__spintent_setspintent_set_keys:nn`.)

```

\__spintent_setspintent_parse_args:nn
\__spintent_setspintent_parse_args:Vn

```

The function `\__spintent_setspintent_parse_args:nn` will check if the argument passed in #1 begins with ‘\’ or is just a *command name*. If it begins with ‘\’, it will remove it using `\cs_to_str:N` and then execute `\__spintent_setspintent_set_keys:en`. Otherwise, i.e., if ‘\’ was not passed, it will directly execute `\__spintent_setspintent_set_keys:nn`.

```

179 \cs_new_protected:Nn \__spintent_setspintent_parse_args:nn
180 {
181   \tl_if_single:nTF { #1 }
182   {
183     \token_if_cs:NTF #1
184     {
185       \__spintent_setspintent_set_keys:en { \cs_to_str:N #1 } { #2 }
186     }
187     {
188       \__spintent_setspintent_set_keys:nn { #1 } { #2 }
189     }
190   }
191   {
192     \__spintent_setspintent_set_keys:nn { #1 } { #2 }
193   }
194 }
195 \cs_generate_variant:Nn \__spintent_setspintent_parse_args:nn { Vn }

```

(End of definition for `\__spintent_setspintent_parse_args:nn`.)

\setspintent Now we define the user command `\setspintent`.

```

196 \NewDocumentCommand \setspintent { m m }
197 {
198   \tl_if_blank:nTF { #1 }
199   {
200     \msg_error:nnn { spintent } { unknown-command-setup } { <empty> }
201   }
202   {
203     \tl_set:Nn \__spintent_setspintent_input_arg_cmd_tl { #1 }
204     \tl_trim_spaces:N \__spintent_setspintent_input_arg_cmd_tl
205     \__spintent_setspintent_parse_args:Vn \__spintent_setspintent_input_arg_cmd_tl { #2 }
206   }
207 }

```

(End of definition for `\setspintent`. This function is documented on page 3.)

12.11 Handling unknown keys

```
\l__spintent_command_name_str
    unknown-choice
    cmd-key-unknown
    cmd-key-value-unknown
```

We define the variable `\l__spintent_command_name_str` which will *store* the “*name*” of each command within the implementation along with the messages `unknown-choice` used by *(keys)* that use `.choice:n` in their implementation and the messages `cmd-key-unknown`, `cmd-key-value-unknown` used by the *(key)* `unknown` in commands that use `[key = val]`.

```
208 \str_new:N \l__spintent_command_name_str
209 \msg_new:nnn { spintent } { unknown-choice }
210 {
211     The~value~'#3'~for~'#1'~key~is~invalid~use~('#2').
212 }
213 \msg_new:nnnn { spintent } { cmd-key-unknown }
214 {
215     The~key~'#1'~is~unknown~by~command~
216     \c_backslash_str \l__spintent_command_name_str \c_space_tl
217     and~is~being~ignored~\msg_line_context:.
218 }
219 {
220     The~command~\c_backslash_str \l__spintent_command_name_str \c_space_tl
221     does~not~have~a~key~called~'#1'.\\
222     Check~that~you~have~spelled~the~key~name~correctly.
223 }
224 \msg_new:nnnn { spintent } { cmd-key-value-unknown }
225 {
226     The~key~'#1=#2'~is~unknown~by~command~
227     \c_backslash_str \l__spintent_command_name_str \c_space_tl
228     and~is~being~ignored~\msg_line_context:.
229 }
230 {
231     The~command~\c_backslash_str \l__spintent_command_name_str \c_space_tl
232     does~not~have~a~key~called~'#1'.\\
233     Check~that~you~have~spelled~the~key~name~correctly.
234 }
```

(End of definition for `\l__spintent_command_name_str` and others.)

```
\__spintent_unknown_keys:nn
\__spintent_unknown_keys:Vn
\__spintent_unknown_keys:n
```

The internal functions `\__spintent_unknown_keys:nn` and `\__spintent_unknown_keys:n` used by the *(key)* `unknown`.

```
235 \cs_new_protected:Nn \__spintent_unknown_keys_cmd:nn
236 {
237     \tl_if_blank:nTF {#2}
238     { \msg_error:nnn { spintent } { cmd-key-unknown } {#1} }
239     { \msg_error:nnnn { spintent } { cmd-key-value-unknown } {#1} {#2} }
240 }
241 \cs_generate_variant:Nn \__spintent_unknown_keys_cmd:nn { Vn }
242 \cs_new_protected:Nn \__spintent_unknown_keys_cmd:n
243 {
244     \__spintent_unknown_keys_cmd:Vn \l_keys_key_str {#1}
245 }
```

(End of definition for `\__spintent_unknown_keys:nn` and `\__spintent_unknown_keys:n`.)

12.12 Definition of core base variables

```
\__spintent_luafun_clean_split_and_set:n
\__spintent_luafun_clean_split_and_set:V
```

The function `\__spintent_luafun_clean_split_and_set:n` defined in the Lua module `spintent.lua`, is common to several commands in this implementation. It analyzes, processes, cleans and splits the `{(argument)}` passed to commands using internal functions defined in the Lua module `spintent.lua`. It isolates signs, integer components, decimal separators, decimal parte, decimal periodic part, exponents and final physical “*units*”, assigning them directly to `expl3` variables.

This function assigns the states of the variables ‘`_str`’ or ‘`_tl`’ described below to `true`, `false`, `error` or *stores* information according to the role it fulfills within the implementation.

(End of definition for `\__spintent_luafun_clean_split_and_set:n`.)

```
\__spintent_luafun_sign_tl
\__spintent_luafun_part_int_tl
\__spintent_luafun_dec_sep_tl
\__spintent_luafun_part_dec_tl
\__spintent_luafun_part_period_tl
\__spintent_luafun_has_integer_str
\__spintent_luafun_has_natural_str
```

The variable `\__spintent_luafun_sign_tl` stores the explicit ‘`+`’ or ‘`-`’ sign if present. The variable `\__spintent_luafun_part_int_tl` stores the raw digit sequence of the “*integer part*”. The variable `\__spintent_luafun_dec_sep_tl` stores the matched ‘`,`’ or ‘`.`’ used as the decimal separator. The variable `\__spintent_luafun_part_dec_tl` stores the raw digits of the “*finite decimal part*” that follow the separator, and the variable `\__spintent_luafun_part_period_tl` stores the digits of the “*infinite periodic decimal part*” that follow a semicolon ‘`;`’.

The variable `\l__spintent_luaset_has_integer_str` store the state `true` when it is *only* an “integer”. The variable `\l__spintent_luaset_has_natural_str` store the state `true` when it is *only* an “natural number”.

```

246 \tl_new:N \l__spintent_luaset_sign_tl
247 \tl_new:N \l__spintent_luaset_part_int_tl
248 \tl_new:N \l__spintent_luaset_dec_sep_tl
249 \tl_new:N \l__spintent_luaset_part_dec_tl
250 \tl_new:N \l__spintent_luaset_part_period_tl
251 \str_new:N \l__spintent_luaset_has_integer_str
252 \str_new:N \l__spintent_luaset_has_natural_str

```

(End of definition for `\l__spintent_luaset_sign_tl` and others.)

`\l__spintent_luaset_only_part_int_str` The literal entries of the numbers that we will pass to the *intent function* :number of `\MathMLintent` are stored in `\l__spintent_luaset_only_part_int_str` for the “integer part”, in `\l__spintent_luaset_only_part_dec_str` for the “finite decimal part”, and in `\l__spintent_luaset_only_part_period_str` for the “infinite periodic decimal part”.

Formatted versions (`cifras-sep=true`) are stored in `\l__spintent_luaset_format_part_int_str` for the “integer part” and `\l__spintent_luaset_format_part_dec_str` for the “finite decimal part”.

```

253 \str_new:N \l__spintent_luaset_only_part_int_str
254 \str_new:N \l__spintent_luaset_only_part_dec_str
255 \str_new:N \l__spintent_luaset_only_part_period_str
256 \str_new:N \l__spintent_luaset_format_part_int_str
257 \str_new:N \l__spintent_luaset_format_part_dec_str

```

(End of definition for `\l__spintent_luaset_only_part_int_str` and others.)

`\l__spintent_luaset_millions_str` The flag variable `\l__spintent_luaset_millions_str` indicates whether the integer part translates exactly to a multiple of one million when passed to `\MathMLintent`. To properly structure the reading and writing of numbers in MathML, we use the flag variables `\l__spintent_luaset_decimal_period_str`, `\l__spintent_luaset_not_decimal_period_str`, and `\l__spintent_luaset_not_decimal_not_period_str`. The flag variable `\l__spintent_luaset_dec_is_all_zeros_str` monitors whether the “finite decimal part” is completely zero (useful for “pure periodic decimals”).

The flag variable `\l__spintent_luaset_has_exponent_str` indicates whether active “scientific notation” was detected at the end of the entered number, passed with ‘e’ or ‘E’. If true, it will store the digits of the exponent in `\l__spintent_luaset_exponent_tl`.

```

258 \str_new:N \l__spintent_luaset_millions_str
259 \str_new:N \l__spintent_luaset_decimal_period_str
260 \str_new:N \l__spintent_luaset_not_decimal_period_str
261 \str_new:N \l__spintent_luaset_not_decimal_not_period_str
262 \str_new:N \l__spintent_luaset_dec_is_all_zeros_str
263 \str_new:N \l__spintent_luaset_has_exponent_str
264 \tl_new:N \l__spintent_luaset_exponent_tl

```

(End of definition for `\l__spintent_luaset_millions_str` and others.)

`\l__spintent_luaset_has_units_str` Since the function `\__spintent_luafun_clean_split_and_set:n` is common to `\spnum` (§12.14) and `\spunit` (§12.15) (as well as others), it determines the following unit-related variables after processing the `{⟨arguments⟩}` via the Lua module `spintent.lua`.

First, it checks if there is any residual text *after* the processed number that is a candidate for physical “units”. It sets the flag variable `\l__spintent_luaset_has_units_str` accordingly. Following this, it verifies if this text contains more than one “/”. It then sets the variable `\l__spintent_luaset_status_str` to `valid` or `multislash`. If the structure is `valid`, it splits the unit into `\l__spintent_luaset_arg_above_tl` and `\l__spintent_luaset_arg_below_tl`. Finally, it analyzes `\l__spintent_luaset_arg_below_tl`; if the denominator starts with a number, it sets `\l__spintent_luaset_denom_has_number_str` to `true`, otherwise to `false`.

```

265 \str_new:N \l__spintent_luaset_has_units_str
266 \str_new:N \l__spintent_luaset_status_str
267 \tl_new:N \l__spintent_luaset_arg_above_tl
268 \tl_new:N \l__spintent_luaset_arg_below_tl
269 \str_new:N \l__spintent_luaset_denom_has_number_str

```

(End of definition for `\l__spintent_luaset_has_units_str` and others.)

12.13 Common messages for text mode and math mode

Messages shared by the different commands for text mode or math mode.

```

270 \msg_new:nnnn { spintent } { need-math-mode }
271 {
272   Math~mode~required~for~'#1'~\msg_line_context:.
273 }
274 {
275   Use~inside~$.~$.~$~or~in~math~environment.
276 }
277 \msg_new:nnnn { spintent } { need-text-mode }
278 {
279   Text~mode~required~for~'#1'~\msg_line_context:.
280 }
281 {
282   Use~outside~of~$.~$.~$~or~math~environment.
283 }
```

12.14 The command \spnum

The user command `\spnum` is first of “the big four” provided by the `spintent` package and is fundamental to the implementation of other commands. It handles the processing of numerical input, scientific notation, and units of measurement, using the Lua module `spintent.lua`, which is responsible for separating and assigning variables defined in `expl3`.

From the `expl3` side, we handle the printing logic and the `intent` functions passed to `\MathMLintent` to achieve a valid MathML structure that is accepted by MathCAT.

12.14.1 Definition of keys and variables for \spnum

The variable `\l__spintent_spnum_read_terse_bool` handled by the key `read-sign`, controls the `intent` passed to `\MathMLintent` to spoken the ‘+’ or ‘-’. If set to `clear`, it will say “positivo” or “negativo” after reading the number; if set to `terse`, it will say “más” or “menos” before reading the number.

The variable `\l__spintent_spnum_read_period_sep_str` handled by the key `read-period-sep`, controls how the decimal separator ‘;’ (periodic part) is printed and spoken when the `{(argument)}` is a *pure decimal periodic* and is passed the `intent` to `\MathMLintent`. If set to `coma`, it will say “coma” and print ‘,’; if set to `punto` it will say “punto” and print ‘.’.

The variable `\l__spintent_spnum__notation_sym_tl`, handled by the key `notation-sym`, determines whether ‘x’ will be printed when set to `times` or ‘.’ when set to `cdot`.

The variable `\l__spintent_spnum_intent_read_sign_str` will store the spoken translation used in the `intent` function passed to `\MathMLintent` for the ‘+’ or ‘-’ according to the value passed to the key `read-sign`.

The variable `\l__spintent_spnum_intent_read_dec_sep_str` stores the spoken translation that will be used in the `intent` function passed to `\MathMLintent` for reading the decimal separation as a “coma” or a “punto” according to the value found in the variable `\l__spintent_luaset_dec_sep_tl`.

```

284 \bool_new:N \l__spintent_spnum_read_terse_bool
285 \str_new:N \l__spintent_spnum_read_period_sep_str
286 \tl_new:N \l__spintent_spnum_notation_sym_tl
287 \str_new:N \l__spintent_spnum_intent_read_sign_str
288 \str_new:N \l__spintent_spnum_intent_read_dec_sep_str
```

(End of definition for `\l__spintent_spnum_read_terse_bool` and others.)

```

cifras-sep
read-sign
read-period-sep
notation-sym

Now we add the keys cifras-sep, read-sign, read-period-sep and notation-sym.

289 \keys_define:nn { spintent/spnum }
290 {
291   cifras-sep .bool_set:N = \l__spintent_spnum_cifras_sep_bool,
292   cifras-sep .initial:n = true,
293   cifras-sep .value_required:n = true,
294   read-sign .choices:nn =
295     { terse , clear }
296     {
297       \int_case:nn { \l_keys_choice_int }
298       {
299         {1} { \bool_set_true:N \l__spintent_spnum_read_terse_bool }
300         {2} { \bool_set_false:N \l__spintent_spnum_read_terse_bool }
301       }
302     },
303   read-sign .initial:n = terse,
304   read-sign .value_required:n = true,
305   read-sign / unknown .code:n =
```

```

306     \msg_error:nneee { spintent } { unknown-choice }
307     { read-sign } { terse , ~clear } { \exp_not:n {#1} },
308 read-period-sep                .choices:nn =
309     { coma , punto }
310     {
311         \int_case:nn { \l_keys_choice_int }
312         {
313             {1} { \str_set:Nn \l__spintent_spnun_read_period_sep_str { coma } }
314             {2} { \str_set:Nn \l__spintent_spnun_read_period_sep_str { punto } }
315         }
316     },
317 read-period-sep                .initial:n = coma,
318 read-period-sep                .value_required:n = true,
319 read-period-sep / unknown .code:n =
320     \msg_error:nneee { spintent } { unknown-choice }
321     { read-period-sep } { coma , ~punto } { \exp_not:n {#1} },
322 notation-sym                    .choices:nn =
323     { times , cdot }
324     {
325         \int_case:nn { \l_keys_choice_int }
326         {
327             {1} { \tl_set:Nn \l__spintent_spnun_notation_sym_tl { \times } }
328             {2} { \tl_set:Nn \l__spintent_spnun_notation_sym_tl { \cdot } }
329         }
330     },
331 notation-sym                    .initial:n = cdot,
332 notation-sym                    .value_required:n = true,
333 notation-sym / unknown .code:n =
334     \msg_error:nneee { spintent } { unknown-choice }
335     { notation-sym } { times , ~ cdot } { \exp_not:n {#1} },
336 unknown                          .code:n =
337     {
338         \str_set:Nn \l__spintent_command_name_str { spnum }
339         \__spintent_unknown_keys_cmd:n {#1}
340     },
341 }

```

(End of definition for *cifras-sep* and others.)

12.14.2 Internal functions for \spnum

```

\__spintent_spnun_render_part:n
\__spintent_spnun_render_int:
\__spintent_spnun_render_dec:

```

The function `\__spintent_spnun_render_part:n` processes the $\langle argument \rangle$ passed in ‘#1’ used by the functions `\__spintent_spnun_render_int:` and `\__spintent_spnun_render_dec:` to define the *intent function* :number for the integer and decimal part passed to `\MathMLintent`. It evaluates whether the key *cifras-sep* is active or not.

```

342 \cs_new_protected:Nn \__spintent_spnun_render_part:n
343 {
344     \MathMLintent { \str_use:c { \l__spintent_luaset_only_part_#1_str } :number }
345     {
346         \bool_if:NTF \l__spintent_spnun_cifras_sep_bool
347         { \str_use:c { \l__spintent_luaset_format_part_#1_str } }
348         { \tl_use:c { \l__spintent_luaset_part_#1_tl } }
349     }
350 }
351 \cs_new_protected:Nn \__spintent_spnun_render_int:
352 {
353     \__spintent_spnun_render_part:n { int }
354 }
355 \cs_new_protected:Nn \__spintent_spnun_render_dec:
356 {
357     \__spintent_spnun_render_part:n { dec }
358 }

```

(End of definition for `\__spintent_spnun_render_part:n`, `\__spintent_spnun_render_int:`, and `\__spintent_spnun_render_dec:`.)

```

\__spintent_spnun_render_only_period:
\__spintent_spnun_print_period_bar:
\__spintent_spnun_render_period_sep:

```

The function `\__spintent_spnun_render_only_period:` isolates the “infinite periodic decimal part” from the “finite decimal part”, assigning the *intent function* :number passed to `\MathMLintent` which will be used by `\MathMLarg` in the function `\__spintent_spnun_print_period_bar:`.

```

359 \cs_new_protected:Nn \__spintent_spnun_render_only_period:
360 {
361     \MathMLintent { \l__spintent_luaset_only_part_period_str :number }

```

```

362     {
363         \l__spintent_luaset_part_period_tl
364     }
365 }

```

The function `\__spintent_snum_print_period_bar:` inserts the function `\__spintent_snum_render_only_` as an argument within the *intent function* `_periódico:postfix($dp)` passed to `\MathMLintent`, generating the correct spoken for the “*infinite periodic decimal part*”.

```

366 \cs_new_protected:Nn \__spintent_snum_print_period_bar:
367 {
368     \MathMLintent { _periódico:postfix($dp) }
369     {
370         {
371             \overline
372             {
373                 \MathMLarg { dp } { \__spintent_snum_render_only_period: }
374             }
375         }
376     }
377 }

```

The function `\__spintent_snum_render_period_sep:` sets how the decimal separator handled by the key `read-period-sep` will be printed and spoken when a “*pure periodic decimal*” is passed.

```

378 \cs_new_protected:Nn \__spintent_snum_render_period_sep:
379 {
380     \str_if_eq:VnTF \l__spintent_snum_read_period_sep_str { coma }
381     {
382         \MathMLintent { _coma:prefix($pp) }
383         {
384             \mathord{,}
385             \MathMLarg { pp } { \__spintent_snum_print_period_bar: }
386         }
387     }
388     {
389         \MathMLintent { _punto:prefix($pp) }
390         {
391             \mathord{.}
392             \MathMLarg { pp } { \__spintent_snum_print_period_bar: }
393         }
394     }
395 }

```

(End of definition for `\__spintent_snum_render_only_period:`, `\__spintent_snum_print_period_bar:`, and `\__spintent_snum_render_period_sep:`.)

`\__spintent_snum_render_decimal_sep:`

The function `\__spintent_snum_render_decimal_sep:` parses the value of the variable `\l__spintent_luaset_` and sets the variable `\l__spintent_snum_intent_read_dec_sep_str` with the function `_coma:prefix($dp)` for reading a “*coma*” or `_punto:prefix($dp)` for reading a “*punto*” to `\MathMLintent`. We print the decimal separator stored in `\l__spintent_luaset_dec_sep_tl` using `\mathord` to remove spaces around it and then use the command `\MathMLarg` where we insert `\__spintent_invisible_sep:` for compatibility with MathML and finally call the function `\__spintent_snum_render_dec:` to print the finite decimal part.

```

396 \cs_new_protected:Nn \__spintent_snum_render_decimal_sep:
397 {
398     \str_if_eq:VnTF \l__spintent_luaset_dec_sep_tl { , }
399     { \str_set:Nn \l__spintent_snum_intent_read_dec_sep_str { _coma } }
400     { \str_set:Nn \l__spintent_snum_intent_read_dec_sep_str { _punto } }
401     \MathMLintent { \l__spintent_snum_intent_read_dec_sep_str :prefix($dp) }
402     {
403         \mathord { \l__spintent_luaset_dec_sep_tl }
404         \MathMLarg { dp }
405         {
406             \__spintent_inner_sep:
407             \__spintent_snum_render_dec:
408         }
409     }
410     % period part
411     \str_if_eq:VnT \l__spintent_luaset_decimal_period_str { true }
412     {
413         \str_if_eq:VnTF \l__spintent_luaset_dec_is_all_zeros_str { true }
414         {
415             \__spintent_inner_sep:
416             \__spintent_snum_print_period_bar:

```

```

417     }
418     {
419         \MathMLintent { _con:prefix($pnum) }
420         {
421             \__spintent_inner_sep:
422             \MathMLarg { pnum }
423             {
424                 \__spintent_snum_print_period_bar:
425             }
426         }
427     }
428 }
429 }

```

(End of definition for `\__spintent_snum_render_decimal_sep:`.)

`\__spintent_snum_print_part_dec:`
`\__spintent_snum_print_part_period:`

The function `\__spintent_snum_print_part_dec:` checks if a decimal part exists. If it does, it determines whether the decimal separator is a comma or a period to set the correct speech intent (“*coma*” or “*punto*”). It then prints the separator, renders the decimal digits, and if a periodic sequence follows, it decides how to attach the periodic bar.

The function `\__spintent_snum_print_part_period:` handles cases where a periodic part exists. It evaluates if it should render the periodic separator directly, particularly when the number has a periodic part but lacks a standard decimal part.

```

430 \cs_new_protected:Nn \__spintent_snum_print_part_dec:
431 {
432     \tl_if_empty:NF \l__spintent_luaset_part_dec_tl
433     {
434         \__spintent_snum_render_decimal_sep:
435     }
436 }
437 \cs_new_protected:Nn \__spintent_snum_print_part_period:
438 {
439     \tl_if_empty:NF \l__spintent_luaset_part_period_tl
440     {
441         \str_if_eq:VnTF \l__spintent_luaset_not_decimal_period_str { true }
442         { \__spintent_snum_render_period_sep: }
443         {
444             \tl_if_empty:NT \l__spintent_luaset_part_dec_tl
445             {
446                 \__spintent_snum_render_period_sep:
447             }
448         }
449     }
450 }

```

(End of definition for `\__spintent_snum_print_part_dec:` and `\__spintent_snum_print_part_period:`.)

`\__spintent_snum_print_integer:`
`\__spintent_snum_print_num_start:`
`\__spintent_snum_print_number_core:`

The function `\__spintent_snum_print_integer:` checks if the number consists only of an integer. If so, it simply renders it. If a decimal or periodic part exists, it sets up a `MathML` intent to link the integer part (respecting digit separators if enabled) to the subsequent decimal or periodic functions.

The function `\__spintent_snum_print_num_start:` checks if an explicit sign is present. If it finds one, it evaluates the terse reading flag to decide whether the sign should be read as a prefix (“*más*” or “*menos*”) or as a postfix (“*positivo*” or “*negativo*”), and wraps the intent around the rest of the number.

The function `\__spintent_snum_print_number_core:` acts as the main wrapper. It triggers the sign and base number evaluation, and then checks if a scientific exponent is present. If it is, it appends the correct base-10 notation symbol and exponent to complete the sequence.

📌 Note for `\__spintent_invisible_sep:`.

```

451 \cs_new_protected:Nn \__spintent_snum_print_integer:
452 {
453     \str_if_eq:VnTF \l__spintent_luaset_not_decimal_not_period_str { true }
454     { \__spintent_snum_render_int: }
455     {
456         \MathMLintent { \l__spintent_luaset_only_part_int_str :number:prefix($next) }
457         {
458             \bool_if:NTF \l__spintent_snum_cifras_sep_bool
459             { \l__spintent_luaset_format_part_int_str }
460             { \l__spintent_luaset_part_int_tl }
461             \MathMLarg { next }
462         }

```

```

463         \tl_if_empty:NTF \l__spintent_luaset_part_dec_tl
464         { \__spintent_spnum_print_part_period: }
465         { \__spintent_spnum_print_part_dec: }
466     }
467 }
468 }
469 }
470 \cs_new_protected:Nn \__spintent_spnum_print_num_start:
471 {
472     \tl_if_empty:NTF \l__spintent_luaset_sign_tl
473     {
474         \__spintent_invisible_sep: % need
475         \__spintent_spnum_print_integer:
476     }
477     {
478         \bool_if:NTF \l__spintent_spnum_read_terse_bool
479         {
480             \str_if_eq:VnTF \l__spintent_luaset_sign_tl { + }
481             { \str_set:Nn \l__spintent_spnum_intent_read_sign_str { _más:prefix($n)} }
482             { \str_set:Nn \l__spintent_spnum_intent_read_sign_str { _menos:prefix($n)} }
483         }
484         {
485             \str_if_eq:VnTF \l__spintent_luaset_sign_tl { + }
486             { \str_set:Nn \l__spintent_spnum_intent_read_sign_str { _positivo:postfix($n) } }
487             { \str_set:Nn \l__spintent_spnum_intent_read_sign_str { _negativo:postfix($n) } }
488         }
489         \__spintent_invisible_sep: % need
490         \MathMLintent { \l__spintent_spnum_intent_read_sign_str }
491         {
492             \l__spintent_luaset_sign_tl
493             \MathMLarg { n } { \__spintent_spnum_print_integer: }
494         }
495     }
496 }
497 \cs_new_protected:Nn \__spintent_spnum_print_number_core:
498 {
499     \__spintent_spnum_print_num_start:
500     \str_if_eq:VnT \l__spintent_luaset_has_exponent_str { true }
501     {
502         \MathMLintent { _por } { \l__spintent_spnum_notation_sym_tl }
503         10^{\l__spintent_luaset_exponent_tl }
504     }
505 }

```

(End of definition for `\__spintent_spnum_print_integer:`, `\__spintent_spnum_print_num_start:`, and `\__spintent_spnum_print_number_core:`.)

12.14.3 Definition of error messages

spnum-has-units
spnum-no-digits

We define two error messages for the `\spnum` command. The first “error” triggers if the user includes “units” in the $\langle argument \rangle$ passed in ‘#1’, advising them to use the `\spunit` command instead. The second “error” is raised if the provided $\langle argument \rangle$ contains no numerical digits.

```

506 \msg_new:nnnn { spintent } { spnum-has-units }
507 {
508     The~\token_to_str:N \spnum \c_space_tl command~does~not~accept~units~(' #1').
509 }
510 {
511     Use~\token_to_str:N \spunit \c_space_tl if~you~need~to~print~numbers~with~units.
512 }
513 \msg_new:nnnn { spintent } { spnum-no-digits }
514 {
515     The~\token_to_str:N \spnum \c_space_tl command~requires~a~valid~numerical~value.
516 }
517 {
518     You~provided~' #1',~which~contains~no~digits.
519 }

```

(End of definition for `spnum-has-units` and `spnum-no-digits`.)

12.14.4 Implementation of the `\spum` command

`\spnum` The public user command `\spnum` accepts an optional $[key = val]$ argument in ‘#1’ and a mandatory $\langle rational number \rangle$ in ‘#2’. It first checks if it is executed inside math mode, raising an error if it is

not. If correct, it opens a local group to apply the options and passes the input to Lua for parsing via `\__spintent_luafun_clean_split_and_set:n`.

Before printing, it validates the extracted data. It throws an error if it finds trailing units in the input or if the integer part is completely empty (no valid digits). If all checks pass, it calls `\__spintent_spnum_print_number_core:` to render the final number, and finally closes the group.

```

520 \NewDocumentCommand \spnum { O{} m }
521 {
522   \mode_if_math:TF
523   {
524     \group_begin:
525       \keys_set:nn { spintent/spnum } { #1 }
526       \__spintent_luafun_clean_split_and_set:n { \exp_not:n { #2 } }
527       \str_if_eq:VnTF \__spintent_luaset_has_units_str { true }
528       { \msg_error:nnn { spintent } { spnum-has-units } { #2 } }
529       {
530         \tl_if_empty:NTF \__spintent_luaset_part_int_tl
531         { \msg_error:nnn { spintent } { spnum-no-digits } { #2 } }
532         { \__spintent_spnum_print_number_core: }
533       }
534     \group_end:
535   }
536   { \msg_error:nnn { spintent } { need-math-mode } { \spnum } }
537 }

```

(End of definition for `\spnum`. This function is documented on page 3.)

12.15 The command `\spunit`

The user command `\spunit` is the *second* of “the big four” provided by the `spintent` package and is used to parse and print physical units and angular systems. It sends the `{\argument}` to Lua module `spintent.lua` via `\__spintent_luafun_clean_split_and_set:n` to separate and clean the `{\argument}`.

This ensures that all “units” (such as SI fractions, products, or sexagesimal angles) are processed consistently by the `spintent.lua`.

12.15.1 Definition of variables

These string variables store the data returned by the Lua module `spintent.lua`. They hold the canonical “unit symbol”, the dictionary lookup status, the correct speech text for NVDA/MathCAT, and specific flags that indicate if the “unit” is compact or represents a sexagesimal angle.

```

538 \str_new:N \__spintent_spunit_luaset_canonical_str
539 \str_new:N \__spintent_spunit_luaset_lookup_status_str
540 \str_new:N \__spintent_spunit_luaset_read_str
541 \str_new:N \__spintent_spunit_luaset_is_compact_str
542 \str_new:N \__spintent_spunit_luaset_compact_exp_str
543 \str_new:N \__spintent_spunit_luaset_is_sexagesimal_str
544 \str_new:N \__spintent_spunit_luaset_status_str

```

(End of definition for `\__spintent_spunit_luaset_canonical_str` and others.)

These variables are used to process and format the “unit”. The sequences `\__spintent_spunit_mult_seq` and `\__spintent_spunit_exp_seq` split compound “units” at multiplication ‘`*`’ or exponent ‘`^`’ boundaries.

The boolean variables control specific rendering options, such as whether the multiplication dot ‘`\cdot`’ is explicitly read aloud or silently skipped.

```

545 \tl_new:N \__spintent_spunit_exponent_tl
546 \tl_new:N \__spintent_spunit_unit_name_tl
547 \str_new:N \__spintent_spunit_read_slash_str
548 \seq_new:N \__spintent_spunit_exp_seq
549 \seq_new:N \__spintent_spunit_mult_seq
550 \bool_new:N \__spintent_spunit_is_mult_bool
551 \bool_new:N \__spintent_spunit_read_cdod_bool

```

(End of definition for `\__spintent_spunit_exponent_tl` and others.)

12.15.2 Definition of error messages

We define three error messages for the `\spunit` command. The first “error” triggers if the “unit expression” provided in ‘`#1`’ is unknown or malformed. The second “error” occurs if no “units” are found in the `{\argument}`, advising the user to use `\spnum` for pure numbers. The third “error” is raised if a numeric coefficient is illegally placed in the denominator of an inline fraction (after a slash).

```

552 \msg_new:nnnn { spintent } { spunit-invalid-unit }
553 {
554   The~expression~'#1'~contains~an~unknown~or~malformed~unit~structure.

```

```

555 }
556 {
557   Check~spelling~or~verify~the~unit~is~defined~in~the~dictionary.
558 }
559 \msg_new:nnnn { spintent } { spunit-no-units }
560 {
561   The~\token_to_str:N \spunit \c_space_tl command~requires~at~least~one~unit~('#1').
562 }
563 {
564   Use~\token_to_str:N \spnum \c_space_tl if~you~only~need~to~format~a~pure~number.
565 }
566 \msg_new:nnnn { spintent } { inline-numeric-denominator }
567 {
568   Illegal~numeric~coefficient~in~inline~denominator~'#1'.
569 }
570 {
571   The~inline~mode~of~\token_to_str:N \spunit \c_space_tl
572   does~not~allow~numbers~after~the~slash~(/).
573 }

```

(End of definition for spunit-invalid-unit, spunit-no-units, and inline-numeric-denominator.)

12.15.3 Definition of keys for \spunit

Now we add the keys `print-cdot`, `read-cdot`, `read-slash`, `cifras-sep`, `read-period-sep` and `notation-sym`.

```

574 \keys_define:nn { spintent/spunit }
575 {
576   print-cdot      .bool_set:N = \l_spintent_spunit_print_cdot_bool,
577   print-cdot      .initial:n  = false,
578   print-cdot      .value_required:n = true,
579   read-cdot       .bool_set:N = \l_spintent_spunit_read_cdot_bool,
580   read-cdot       .initial:n  = false,
581   read-cdot       .value_required:n = true,
582   read-slash      .choices:nn =
583     { por , partido-por }
584     {
585       \int_case:nn { \l_keys_choice_int }
586       {
587         {1} { \str_set:Nn \l__spintent_spunit_read_slash_str { _por } }
588         {2} { \str_set:Nn \l__spintent_spunit_read_slash_str { _partido-por } }
589       }
590     },
591   read-slash      .initial:n  = por,
592   read-slash      .value_required:n = true,
593   read-slash / unknown .code:n =
594     \msg_error:nneee { spintent } { unknown-choice }
595     { read-slash } { por , ~partido-por } { \exp_not:n {#1} },
596   cifras-sep      .meta:nn = { spintent/spnum } { cifras-sep = {#1} },
597   cifras-sep      .value_required:n = true,
598   read-period-sep .meta:nn = { spintent/spnum } { read-period-sep = {#1} },
599   read-period-sep .value_required:n = true,
600   notation-sym    .meta:nn = { spintent/spnum } { notation-sym = {#1} },
601   notation-sym    .value_required:n = true,
602   unknown         .code:n =
603     {
604       \str_set:Nn \l__spintent_command_name_str { spunit }
605       \__spintent_unknown_keys_cmd:n {#1}
606     },
607 }

```

(End of definition for read-cdot and others.)

12.15.4 Internal functions for \spunit

`\__spintent_spunit_format_canonical:` The function `\__spintent_spunit_format_canonical:` handles the visual formatting of the “unit symbol”. It specifically checks for the *liter symbol* ‘ℓ’ to print it as `\ell`, and formats all other valid “units” in standard upright text using `\mathrm`.

```

608 \cs_new_protected:Nn \__spintent_spunit_format_canonical:
609 {
610   \str_case:VnF \l__spintent_spunit_luaset_canonical_str
611   {
612     { ℓ } { \ell }

```

```

613     }
614     { \mathrm { \l__spintent_spunit_luaset_canonical_str } }
615 }

```

(End of definition for `\l__spintent_spunit_format_canonical:`)

`\l__spintent_spunit_process_base_exp:n`

The function `\l__spintent_spunit_process_base_exp:n` evaluates a “single unit” along with its exponent, ‘#1’. It first looks up the unit in the dictionary; if not found, flags it as invalid.

Otherwise, unless this is the first unit in a product (`\l__spintent_spunit_is_mult_bool`), it handles the spacing and multiplication dot: `\cdot` when `print-cdot` is `true` (read as `por` when `read-cdot` is also `true`, silent otherwise), or a thin space `\,` when `print-cdot` is `false` (still read as `por` under `read-cdot`).

```

616 \cs_new_protected:Nn \l__spintent_spunit_process_base_exp:n
617 {
618   \tl_clear:N \l__spintent_spunit_exponent_tl
619   \l__spintent_luafun_spunit_lookup_alias:n { #1 }
620   \str_if_eq:VnTF \l__spintent_spunit_luaset_lookup_status_str { notfound }
621   {
622     \str_set:Nn \l__spintent_luaset_status_str { invalid }
623   }
624   {
625     \bool_if:NF \l__spintent_spunit_is_mult_bool
626     {
627       \bool_if:NTF \l__spintent_spunit_print_cdot_bool
628       {
629         \bool_if:NTF \l__spintent_spunit_read_cdot_bool
630         { \MathMLint { _por } { \cdot } }
631         { \MathMLint { :silent } { \cdot } }
632       }
633       {
634         \bool_if:NTF \l__spintent_spunit_read_cdot_bool
635         { \MathMLint { _por } { \, } }
636         { \, }
637       }
638     }

```

Then renders the base unit and its exponent: either a pre-defined compact exponent from Lua (`\l__spintent_spunit_luafun_spunit_lookup_status_str`) or by manually splitting ‘#1’ at the ‘^’ symbol otherwise.

```

639   \str_if_eq:VnTF \l__spintent_spunit_luaset_is_compact_str { true }
640   {
641     {
642       \MathMLint { \l__spintent_spunit_luaset_read_str }
643       {
644         \l__spintent_spunit_format_canonical:
645       }
646     }
647     ^ { \l__spintent_spunit_luaset_compact_exp_str }
648   }
649   {
650     \seq_set_split:Nnn \l__spintent_spunit_exp_seq { ^ } { #1 }
651     \seq_pop_left:NN \l__spintent_spunit_exp_seq \l__spintent_spunit_unit_name_tl
652     \seq_if_empty:NF \l__spintent_spunit_exp_seq
653     {
654       \seq_pop_left:NN \l__spintent_spunit_exp_seq \l__spintent_spunit_exponent_tl
655     }
656     \MathMLint { \l__spintent_spunit_luaset_read_str }
657     {
658       \l__spintent_spunit_format_canonical:
659     }
660     \tl_if_empty:NF \l__spintent_spunit_exponent_tl
661     {
662       ^ { \l__spintent_spunit_exponent_tl }
663     }
664   }
665   \bool_set_false:N \l__spintent_spunit_is_mult_bool
666 }
667 }

```

(End of definition for `\l__spintent_spunit_process_base_exp:n`)

`\l__spintent_spunit_process_mult:n`

`\l__spintent_spunit_process_mult:V`

The function `\l__spintent_spunit_process_mult:n` handles compound “units” joined by an explicit multiplication star ‘*’. It splits the $\{ \langle argument \rangle \}$ at each “star symbol” and applies `\l__spintent_spunit_process_base_exp:n` to evaluate and render each resulting unit individually.

```

668 \cs_new_protected:Nn \l__spintent_spunit_process_mult:n

```

```

669 {
670   \bool_set_true:N \__spintent_spunit_is_mult_bool
671   \seq_set_split:Nnn \__spintent_spunit_mult_seq { * } { #1 }
672   \seq_map_function:NN \__spintent_spunit_mult_seq \__spintent_spunit_process_base_exp:n
673 }
674 \cs_generate_variant:Nn \__spintent_spunit_process_mult:n { V }

```

(End of definition for `\__spintent_spunit_process_mult:n`.)

`\__spintent_print_spunit_hierarchy:`

The function `\__spintent_print_spunit_hierarchy:` handles the layout of inline “unit” fractions. It first processes the “units” before the slash (the numerator). If a denominator exists, it prints the slash / ‘/’ with its corresponding speech intent, and then processes the remaining “units”.

```

675 \cs_new_protected:Nn \__spintent_print_spunit_hierarchy:
676 {
677   \tl_if_empty:NF \__spintent_luaset_arg_above_tl
678   {
679     \__spintent_spunit_process_mult:V \__spintent_luaset_arg_above_tl
680     \tl_if_empty:NF \__spintent_luaset_arg_below_tl
681     {
682       \MathMLintent { \__spintent_spunit_read_slash_str } { / }
683       \__spintent_spunit_process_mult:V \__spintent_luaset_arg_below_tl
684     }
685   }
686 }

```

(End of definition for `\__spintent_print_spunit_hierarchy:`.)

`\__spintent_spunit_safe_exec:n`

The function `\__spintent_spunit_safe_exec:n` performs the main validation and layout for “units”. It first checks if the `{⟨argument⟩}` has units, is a valid expression, and does not contain illegal numbers in the denominator, throwing the appropriate “errors” if any check fails.

If the unit is valid, it checks if it is a sexagesimal angle (degrees, minutes, or seconds). For sexagesimal “units”, it prints the number and the symbol “without any space in between”, applying a slight negative space ‘\!’ for arcseconds. When the number is exactly 1, with no decimal part, the reading becomes the singular literal `_grado:unit`, `_minuto:unit` or `_segundo:unit`, since neither the fixed plural names nor `MathCAT` can see the number. For standard units, it prints the number followed by a thin space ‘\,’ and then processes the unit sequence.

```

687 \cs_new_protected:Nn \__spintent_spunit_safe_exec:n
688 {
689   \str_if_eq:VnTF \__spintent_luaset_has_units_str { false }
690   { \msg_error:nnn { spintent } { spunit-no-units } { #1 } }
691   {
692     \str_if_eq:VnTF \__spintent_luaset_status_str { valid }
693     {
694       \str_if_eq:VnTF
695       \__spintent_luaset_denom_has_number_str { true }
696       {
697         \msg_error:nnn { spintent }
698         { inline-numeric-denominator } { #1 }
699       }
700       {
701         \__spintent_luafun_spunit_lookup_alias:V \__spintent_luaset_arg_above_tl
702         \str_if_eq:VnTF \__spintent_spunit_luaset_is_sexagesimal_str { true }
703         {
704           \tl_if_empty:NF \__spintent_luaset_part_int_tl
705           {
706             \__spintent_spnum_print_number_core:
707           }
708           \bool_lazy_and:nnT
709           { \str_if_eq_p:ee { \__spintent_luaset_part_int_tl } { 1 } }
710           { \tl_if_empty_p:N \__spintent_luaset_part_dec_tl }
711           {
712             \str_set:Ne \__spintent_spunit_luaset_read_str
713             {
714               \str_case:Vn \__spintent_spunit_luaset_canonical_str
715               {
716                 { ° } { _grado:unit }
717                 { ' } { _minuto:unit }
718                 { " } { _segundo:unit }
719               }
720             }
721           }
722         }
723       }
724     }
725   }

```

```

722         \MathMLintent { \l__spintent_spunit_luaset_read_str }
723         {
724             \mathord
725             {
726                 \str_case:Vn \l__spintent_spunit_luaset_canonical_str
727                 {
728                     { " } { \prime \! \prime }
729                     { ' } { \prime }
730                     { ° } { ° }
731                 }
732             }
733         }
734     }
735     {
736         \tl_if_empty:NF \l__spintent_luaset_part_int_tl
737         {
738             \__spintent_spnum_print_number_core: \,
739         }
740         \__spintent_print_spunit_hierarchy:
741     }
742 }
743 }
744 { \msg_error:nnn { spintent } { spunit-invalid-unit } { #1 } }
745 }
746 }

```

(End of definition for `\__spintent_spunit_safe_exec:n`.)

12.15.5 Implementation of the `\spunit` command

`\spunit` The public user command `\spunit` accepts an optional [*key* = *val*] argument in ‘#1’ and a mandatory {*unit expression*} in ‘#2’. It first checks if it is executed inside *math mode*, raising an “error” if it is not. If correct, it opens a local group to apply the options and passes the {*argument*} to Lua module for parsing via `\__spintent_luafun_clean_split_and_set:n` function.

Finally, it delegates the execution to the function `\__spintent_spunit_safe_exec:n` to perform the safety checks and render the parsed data, before closing the group.

```

747 \NewDocumentCommand \spunit { O{} m }
748 {
749     \mode_if_math:TF
750     {
751         \group_begin:
752         \keys_set:nn { spintent/spunit } { #1 }
753         \__spintent_luafun_clean_split_and_set:n { \exp_not:n { #2 } }
754         \__spintent_spunit_safe_exec:n { #2 }
755         \group_end:
756     }
757     { \msg_error:nnn { spintent } { need-math-mode } { \spunit } }
758 }

```

(End of definition for `\spunit`. This function is documented on page 4.)

12.16 The commands `\spnewunit` and `\spunitalias`

The user commands `\spnewunit` and `\spunitalias` allow users to add new custom “units” or “aliases” to the package’s dictionary. Similar to `\spunit`, these commands delegate the processing and validation to the `spintent.lua` via `\__spintent_luafun_define_custom_unit:nn` and `\__spintent_luafun_define_spunit`. During the definition, the Lua module checks if the new “unit” or “alias” is valid and ensures it does not conflict with existing “units”. It returns the result of this operation to T_EX using the `\l__spintent_spunit_luaset_status_s` variable. If a collision is detected or the {*argument*} is invalid, the package throws the corresponding “error” message.

12.16.1 Definition of error messages

spnewunit-duplicate
spunitalias-duplicate
spunitalias-invalid-expr

We define three error messages for the creation of “units” and “aliases”. The first two “errors” trigger if the user attempts to define a “new unit” or alias passed in ‘#1’ that already exists in the dictionary. The third “error” is raised by `\spunitalias` if the target expression provided in ‘#2’ is invalid (for example, if it contains unknown “units” or incorrect syntax).

```

759 \msg_new:nnnn { spintent } { spnewunit-duplicate }
760 {
761     The~unit~symbol~or~alias~'#1'~is~already~defined!
762 }
763 {
764     You~cannot~redefine~with~\token_to_str:N \spnewunit.

```

```

765 }
766 \msg_new:nnnn { spintenc } { spunitalias-duplicate }
767 {
768   The~alias~'#1'~is~already~defined~or~reserved!
769 }
770 {
771   You~cannot~redefine~an~existing~unit~or~another~alias.
772 }
773 \msg_new:nnnn { spintenc } { spunitalias-invalid-expr }
774 {
775   The~expression~'#2'~for~alias~'#1'~is~invalid.
776 }
777 {
778   It~must~contain~only~defined~units,~exponents~and~one~'/' .
779 }

```

(End of definition for `spnewunit-duplicate`, `spunitalias-duplicate`, and `spunitalias-invalid-expr`.)

`\spnewunit` The public command `\spnewunit` defines a new custom unit. It passes the new unit symbol ‘#1’ and its speech text ‘#2’ to Lua via `\__spintenc_luafun_define_custom_unit:nn`. If the module reports that the unit already exists, it throws the corresponding duplicate error.

```

780 \NewDocumentCommand \spnewunit { m m }
781 {
782   \mode_if_math:TF
783   {
784     \msg_error:nnn { spintenc } { need-text-mode } { \spnewunit }
785   }
786   {
787     \__spintenc_luafun_define_custom_unit:nn { #1 } { #2 }
788     \str_if_eq:VnT \__spintenc_spunit_luaset_status_str { duplicate }
789     {
790       \msg_error:nnn { spintenc } { spnewunit-duplicate } { #1 }
791     }
792   }
793 }

```

(End of definition for `\spnewunit`. This function is documented on page 6.)

`\spunitalias` The public command `\spunitalias` creates a shortcut for a complex unit expression. It passes the new alias name ‘#1’ and the target expression ‘#2’ to Lua via `\__spintenc_luafun_define_spunit_alias:nn`. It throws an error if the alias already exists, or if the provided target expression contains invalid syntax or unknown units.

```

794 \NewDocumentCommand \spunitalias { m m }
795 {
796   \mode_if_math:TF
797   {
798     \msg_error:nnn { spintenc } { need-text-mode } { \spunitalias }
799   }
800   {
801     \__spintenc_luafun_define_spunit_alias:nn { #1 } { #2 }
802     \str_if_eq:VnT \__spintenc_spunit_luaset_status_str { duplicate }
803     {
804       \msg_error:nnn { spintenc } { spunitalias-duplicate } { #1 }
805     }
806     \str_if_eq:VnT \__spintenc_spunit_luaset_status_str { invalid-expr }
807     {
808       \msg_error:nnnn { spintenc } { spunitalias-invalid-expr } { #1 } { #2 }
809     }
810   }
811 }

```

(End of definition for `\spunitalias`. This function is documented on page 6.)

12.17 The command `\spmoney`

The user command `\spmoney` is the *third* of “the big four” provided by the `spintenc` package, designed to format monetary values and currencies. It delegates `{(argument)}` parsing to the Lua module `spintenc.lua` via `\__spintenc_luafun_clean_split_and_set:n`. The module then retrieves the correct “currency symbol”, pluralization rules, and layout structures from the database.

The command accepts *optional arguments* to override the default “currency symbol” or enable “subunit” rendering. Finally, it constructs the MathML intent for MathCAT.

```

\l__spintent_spmoney_luaset_symbol_str
\l__spintent_spmoney_luaset_position_str
\l__spintent_spmoney_luaset_sing_str
\l__spintent_spmoney_luaset_plur_str
\l__spintent_spmoney_luaset_conde_str
\l__spintent_spmoney_luaset_subsing_str
\l__spintent_spmoney_luaset_subplur_str
\l__spintent_spmoney_luaset_print_iso_str
\l__spintent_spmoney_luaset_currency_arg_str
\l__spintent_spmoney_luaset_status_str

```

These string variables store the currency data returned by the Lua module `spintent.lua`. Upon a successful database lookup, the module populates these variables with the “*currency symbol*”, its layout position (pre or post), the appropriate singular and plural names for both the main unit and subunits, and the execution status flags. T_EX then uses this information to render the visual layout and construct the correct MathML speech intents.

```

812 \str_new:N \l__spintent_spmoney_luaset_symbol_str
813 \str_new:N \l__spintent_spmoney_luaset_position_str
814 \str_new:N \l__spintent_spmoney_luaset_sing_str
815 \str_new:N \l__spintent_spmoney_luaset_plur_str
816 \str_new:N \l__spintent_spmoney_luaset_conde_str
817 \str_new:N \l__spintent_spmoney_luaset_subsing_str
818 \str_new:N \l__spintent_spmoney_luaset_subplur_str
819 \str_new:N \l__spintent_spmoney_luaset_print_iso_str
820 \str_new:N \l__spintent_spmoney_luaset_currency_arg_str
821 \str_new:N \l__spintent_spmoney_luaset_status_str

```

(End of definition for `\l__spintent_spmoney_luaset_symbol_str` and others.)

```

\l__spintent_spmoney_sub_units_bool
\l__spintent_spmoney_intent_money_str
\l__spintent_spmoney_intent_sign_str
\l__spintent_spmoney_intent_sub_unit_str
\l__spintent_spmoney_res_main_voice_str
\l__spintent_spmoney_subnum_int
\l__spintent_spmoney_extracted_curr_str
\l__spintent_spmoney_extracted_num_tl

```

The boolean variable tracks whether subunit rendering is enabled. The string variables assemble the specific speech text (intents) for NVDA/MathCAT, including the sign, the main currency, and the subunits. Finally, the remaining variables temporarily store the numeric value and currency code extracted from the user’s input.

```

822 \bool_new:N \l__spintent_spmoney_sub_units_bool
823 \str_new:N \l__spintent_spmoney_intent_money_str
824 \str_new:N \l__spintent_spmoney_intent_sign_str
825 \str_new:N \l__spintent_spmoney_intent_sub_unit_str
826 \str_new:N \l__spintent_spmoney_res_main_voice_str
827 \int_new:N \l__spintent_spmoney_subnum_int
828 \str_new:N \l__spintent_spmoney_extracted_curr_str
829 \tl_new:N \l__spintent_spmoney_extracted_num_tl

```

(End of definition for `\l__spintent_spmoney_sub_units_bool` and others.)

12.17.1 Definition of error messages

```

currency-no-subunits
invalid-currency

```

We define two messages to handle invalid currency configurations. The first “error” warns the user if subunit rendering is requested for a currency that does not support fractional divisions (such as JPY or CLP), safely ignoring the flag. The second “error” is triggered if the provided currency identifier is not registered in the Lua database or is not written validly: an alias or symbol with any uppercase letter, or an ISO code that is neither all lowercase nor all uppercase.

```

830 \msg_new:nnnn { spintent } { currency-no-subunits }
831 {
832   The~currency~'#1'~does~not~support~subunits.~
833   The~option~'sub-units=true'~is~ignored.
834 }
835 {
836   Monies~like~the~Japanese~Yen~(JPY)~or~Chilean~Peso~(CLP)~
837   do~not~have~fractional~subunits~registered~in~Lua~module.~
838 }
839 \msg_new:nnnn { spintent } { invalid-currency }
840 {
841   Invalid~currency~identifier~'#1'~in~\token_to_str:N \spmoney .
842 }
843 {
844   The~provided~symbol,~alias,~or~ISO~code~does~not~exist~in~
845   the~Lua~module.
846 }

```

(End of definition for `currency-no-subunits` and `invalid-currency`.)

12.17.2 Auxiliary functions for `\spmoney`

```

\__spintent_spmoney_set_currency:n
\__spintent_spmoney_normalize_key:n
\__spintent_spmoney_normalize_key:V

```

The function `\__spintent_spmoney_set_currency:n` validates the currency identifier provided in ‘#1’ against the database. It delegates the lookup to the helper function `\__spintent_spmoney_normalize_key:n`, which interfaces with the Lua module `spintent.lua`. If the module returns a not found status, an “error” is triggered. Otherwise, it stores the valid currency identifier in `\l__spintent_spmoney_luaset_currency_arg_str` for subsequent processing.

```

847 \cs_new_protected:Nn \__spintent_spmoney_set_currency:n
848 {
849   \__spintent_spmoney_normalize_key:n {#1}
850   \str_if_eq:VnTF \l__spintent_spmoney_luaset_status_str { notfound }
851   {
852     \msg_error:nnn { spintent } { invalid-currency } {#1}

```

```

853     }
854     { \str_set:Nn \l__spintenc_spmoney_luaset_currency_arg_str {#1} }
855   }
856   \cs_new_protected:Nn \__spintenc_spmoney_normalize_key:n
857   {
858     \__spintenc_luafun_spmoney_normalize_key:n { \exp_not:n { #1 } }
859   }
860   \cs_generate_variant:Nn \__spintenc_spmoney_normalize_key:n { V }

```

(End of definition for `\__spintenc_spmoney_set_currency:n` and `\__spintenc_spmoney_normalize_key:n`.)

12.17.3 Definition of keys for `\spmoney`

Now we add the keys `currency`, `sub-units`, `money-math` and `cifras-sep`.

```

861 \keys_define:nn { spintenc / spmoney }
862 {
863   currency      .code:n      = { \__spintenc_spmoney_set_currency:n {#1} },
864   currency      .initial:n    = { pesos },
865   currency      .value_required:n = true,
866   sub-units     .bool_set:N   = \l__spintenc_spmoney_sub_units_bool,
867   sub-units     .value_required:n = true,
868   money-math    .bool_set:N   = \l__spintenc_spmoney_money_math_bool,
869   money-math    .initial:n    = { true },
870   money-math    .value_required:n = true,
871   cifras-sep    .meta:nn      = { spintenc/spnum } { cifras-sep = {#1} },
872   cifras-sep    .value_required:n = true,
873   unknown       .code:n      =
874   {
875     \str_set:Nn \l__spintenc_command_name_str { spmoney }
876     \__spintenc_unknown_keys_cmd:n {#1}
877   },
878 }

```

(End of definition for `currency` and others.)

12.17.4 Internal functions for `\spmoney`

`\__spintenc_spmoney_validate_money:` The function `\__spintenc_spmoney_validate_money:` verifies if the parsed input contains an embedded currency identifier by checking `\l__spintenc_luaset_has_units_str`. If a currency is detected, it extracts and normalizes the key from `\l__spintenc_luaset_arg_above_tl`. It throws an “error” if the currency is not found in the database. If valid, it updates `\l__spintenc_spmoney_luaset_currency_arg_str` and retrieves the full currency metadata from the `spintenc.lua` via `\__spintenc_luafun_spmoney_lookup_data:V`.

```

879 \cs_new_protected:Nn \__spintenc_spmoney_validate_money:
880 {
881   \str_if_eq:VnT \l__spintenc_luaset_has_units_str { true }
882   {
883     \__spintenc_spmoney_normalize_key:V \l__spintenc_luaset_arg_above_tl
884     \str_if_eq:VnTF \l__spintenc_spmoney_luaset_status_str { notfound }
885     { \msg_error:nne { spintenc } { invalid-currency } { \l__spintenc_luaset_arg_above_tl } }
886     {
887       \str_set:NV
888         \l__spintenc_spmoney_luaset_currency_arg_str
889         \l__spintenc_luaset_arg_above_tl
890     }
891   }
892   \str_set:Ne \l__spintenc_spmoney_luaset_status_str { found }
893   \__spintenc_luafun_spmoney_lookup_data:V \l__spintenc_spmoney_luaset_currency_arg_str
894 }

```

(End of definition for `\__spintenc_spmoney_validate_money:`.)

`\__spintenc_spmoney_print_symbol:` The function `\__spintenc_spmoney_print_symbol:` renders the “*currency symbol*”. If `\l__spintenc_spmoney_luaset_currency_arg_str` is `true`, it outputs the ISO code using `\mathrm`. Otherwise, it prints the “*standard symbol*”, directly in math mode when `\l__spintenc_spmoney_money_math_bool` is `true`, or wrapped inside `\text` when it is `false`. For dollar signs ‘\$’, the symbol is typeset through the control sequence `\$` in both cases.

```

895 \cs_new_protected:Nn \__spintenc_spmoney_print_symbol:
896 {
897   \str_if_eq:VnTF \l__spintenc_spmoney_luaset_print_iso_str { true }
898   { \mathrm { \l__spintenc_spmoney_luaset_currency_arg_str } }
899   {
900     \str_if_eq:VnTF \l__spintenc_spmoney_luaset_symbol_str { $ }
901     {
902       \bool_if:NTF \l__spintenc_spmoney_money_math_bool

```

```

903         { \ $ } { \text { \ $ } }
904     }
905     {
906         \bool_if:NTF \__spintent_spmoney_money_math_bool
907         { \__spintent_spmoney_luaset_symbol_str }
908         { \text { \__spintent_spmoney_luaset_symbol_str } }
909     }
910 }
911 }

```

(End of definition for `\__spintent_spmoney_print_symbol:`)

`\__spintent_spmoney_render_layout:`

The function `\__spintent_spmoney_render_layout:` determines the layout order of the “*currency symbol*” relative to the numeric value. It evaluates `\__spintent_spmoney_luaset_position_str`; if set to `pre`, it renders the “*currency symbol*” *before* the number node. Otherwise, it places the number first, followed by the “*currency symbol*”.

```

912 \cs_new_protected:Nn \__spintent_spmoney_render_layout:
913 {
914     \str_if_eq:VnTF \__spintent_spmoney_luaset_position_str { pre }
915     {
916         \__spintent_spmoney_print_symbol:
917         \MathMLarg { n } { \__spintent_spmoney_render_number_node: }
918     }
919     {
920         \MathMLarg { n } { \__spintent_spmoney_render_number_node: }
921         \__spintent_spmoney_print_symbol:
922     }
923 }

```

(End of definition for `\__spintent_spmoney_render_layout:`)

`\__spintent_spmoney_render_number_node:`
`\__spintent_spmoney_render_only_integer_node:`

The function `\__spintent_spmoney_render_number_node:` renders the full numeric value, encompassing both the integer and decimal components. Conversely, the function `\__spintent_spmoney_render_only_integer_node:` outputs exclusively the integer portion, omitting any decimal data.

```

924 \cs_new_protected:Nn \__spintent_spmoney_render_number_node:
925 {
926     \__spintent_spmoney_render_int:
927     \__spintent_spmoney_print_part_dec:
928 }
929 \cs_new_protected:Nn \__spintent_spmoney_render_only_integer_node:
930 {
931     \__spintent_spmoney_render_int:
932 }

```

(End of definition for `\__spintent_spmoney_render_number_node:` and `\__spintent_spmoney_render_only_integer_node:`)

`\__spintent_spmoney_print_int:`

The function `\__spintent_spmoney_print_int:` evaluates the presence of an explicit sign in `\__spintent_luaset_sign_tl`. If a sign is detected, it assigns a prefix speech intent ‘*más*’ or ‘*menos*’ and embeds the sign and subsequent layout inside a `\MathMLintent`. If no sign is present, it routes execution based on whether subunits are enabled.

```

933 \cs_new_protected:Nn \__spintent_spmoney_print_int:
934 {
935     \tl_if_empty:NTF \__spintent_luaset_sign_tl
936     {
937         \bool_if:NTF \__spintent_spmoney_sub_units_bool
938         { \__spintent_spmoney_set_and_print_sub_units: }
939         { \__spintent_spmoney_intent_int: }
940     }
941     {
942         \str_if_eq:VnTF \__spintent_luaset_sign_tl { + }
943         {
944             \str_set:Nn \__spintent_spmoney_intent_sign_str { _más:prefix ($n) }
945         }
946         {
947             \str_set:Nn \__spintent_spmoney_intent_sign_str { _menos:prefix ($n) }
948         }
949         \__spintent_invisible_sep:
950         \MathMLintent { \__spintent_spmoney_intent_sign_str }
951         {
952             \__spintent_luaset_sign_tl
953             \MathMLarg { n }

```

```

954         {
955             \bool_if:NTF \l__spintent_spmoney_sub_units_bool
956             { \__spintent_spmoney_set_and_print_sub_units: }
957             { \__spintent_spmoney_intent_int: }
958         }
959     }
960 }
961 }

```

(End of definition for `\__spintent_spmoney_print_int:`)

`\__spintent_spmoney_intent_int:`

The function `\__spintent_spmoney_intent_int:` determines the correct grammatical inflection for the currency string (singular, plural, or the prepositional ‘de’ variant for exact millions). It evaluates the integer component, decimal boundaries, and zero-fills to select the appropriate string register, applying it as a postfix MathML intent over the layout rendered by `\__spintent_spmoney_render_layout:`.

```

962 \cs_new_protected:Nn \__spintent_spmoney_intent_int:
963 {
964     \tl_if_empty:NTF \l__spintent_luaset_dec_sep_tl
965     {
966         \str_if_eq:VnTF \l__spintent_luaset_only_part_int_str { 1 }
967         {
968             \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_sing_str
969         }
970         {
971             \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_plur_str
972         }
973         \str_if_eq:VnT \l__spintent_luaset_millions_str { true }
974         {
975             \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_conde_s
976         }
977     }
978     {
979         \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_plur_str
980         \str_if_eq:VnT \l__spintent_luaset_millions_str { true }
981         {
982             \str_if_eq:VnTF \l__spintent_luaset_dec_is_all_zeros_str { true }
983             {
984                 \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_conde
985             }
986             {
987                 \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_plu
988             }
989         }
990     }
991     \__spintent_invisible_sep:
992     \MathMLintent { _ \l__spintent_spmoney_intent_money_str :postfix($n) }
993     {
994         \__spintent_spmoney_render_layout:
995     }
996 }

```

(End of definition for `\__spintent_spmoney_intent_int:`)

`\__spintent_spmoney_check_sub_units:`

The function `\__spintent_spmoney_check_sub_units:` verifies whether the selected currency supports “subunits”. If the database returns empty subunit data, it triggers a warning and automatically disables the “subunit” rendering flag. The flag is also disabled, without a warning, when the amount has no decimal part, since there are no “subunits” to voice.

The function `\__spintent_spmoney_set_and_print_sub_units:` processes and formats the speech intents when “subunits” are active. It normalizes the decimal component (for instance, padding a single-digit decimal like ‘.5’ to ‘.50’), evaluates the correct singular or plural inflections for both the main “currency” and the “subunits”, and delegates the final MathML tree assembly to `\__spintent_spmoney_build_tree:`.

```

997 \cs_new_protected:Nn \__spintent_spmoney_check_sub_units:
998 {
999     \str_if_empty:NT \l__spintent_spmoney_luaset_subsing_str
1000     {
1001         \msg_warning:nne { spintent } { currency-no-subunits }
1002         { \l__spintent_spmoney_luaset_currency_arg_str }
1003         \bool_set_false:N \l__spintent_spmoney_sub_units_bool
1004     }
1005     \tl_if_empty:NT \l__spintent_luaset_dec_sep_tl
1006     { \bool_set_false:N \l__spintent_spmoney_sub_units_bool }

```

```

1007   }
1008   \cs_new_protected:Nn \__spintent_spmoney_set_and_print_sub_units:
1009   {
1010     \int_compare:nTF { \str_count:N \l__spintent_luaset_only_part_dec_str == 1 }
1011     {
1012       \int_set:Nn \l__spintent_spmoney_subnum_int
1013       { \l__spintent_luaset_only_part_dec_str * 10 }
1014       \str_set:Ne \l__spintent_luaset_only_part_dec_str
1015       { \l__spintent_luaset_only_part_dec_str 0 }
1016     }
1017     {
1018       \int_set:Nn \l__spintent_spmoney_subnum_int
1019       { \l__spintent_luaset_only_part_dec_str }
1020     }
1021     \int_compare:nTF { \l__spintent_spmoney_subnum_int == 1 }
1022     {
1023       \str_set:Ne \l__spintent_spmoney_intent_sub_unit_str
1024       { _ \l__spintent_spmoney_luaset_subsing_str :postfix($sub) }
1025     }
1026     {
1027       \str_set:Ne \l__spintent_spmoney_intent_sub_unit_str
1028       { _ \l__spintent_spmoney_luaset_subplur_str :postfix($sub) }
1029     }
1030     \str_if_eq:VnTF \l__spintent_luaset_only_part_int_str { 1 }
1031     {
1032       \str_set:NV \l__spintent_spmoney_res_main_voice_str
1033       \l__spintent_spmoney_luaset_sing_str
1034     }
1035     {
1036       \str_set:NV \l__spintent_spmoney_res_main_voice_str
1037       \l__spintent_spmoney_luaset_plur_str
1038     }
1039     \str_if_eq:VnT \l__spintent_luaset_millions_str { true }
1040     {
1041       \str_set:NV \l__spintent_spmoney_res_main_voice_str
1042       \l__spintent_spmoney_luaset_conde_str
1043     }
1044     \__spintent_spmoney_build_tree:
1045   }

```

(End of definition for __spintent_spmoney_check_sub_units: and __spintent_spmoney_set_and_print_sub_units:.)

The function `\__spintent_spmoney_build_tree:` structures the final MathML tree assembly for fractional “currencies”. It arranges the integer component, the decimal separator, and the “subunit” component based on the layout position (pre or post) of the “currency symbol”.

Furthermore, `\__spintent_spmoney_print_sep_con:` applies the specific speech intent ‘_con’ to the decimal separator, while `\__spintent_spmoney_print_dec_sub_unit:` binds the computed “subunit” intent to the decimal digits, ensuring accurate accessibility output.

```

1046 \cs_new_protected:Nn \__spintent_spmoney_build_tree:
1047 {
1048   \str_if_eq:VnTF \l__spintent_spmoney_luaset_position_str { pre }
1049   {
1050     \MathMLintent { _ \l__spintent_spmoney_res_main_voice_str :postfix($n) }
1051     {
1052       \__spintent_spmoney_print_symbol:
1053       \MathMLarg { n } { \__spintent_spmoney_render_only_integer_node: }
1054     }
1055     \__spintent_spmoney_print_sep_con:
1056     \__spintent_spmoney_print_dec_sub_unit:
1057   }
1058   {
1059     \MathMLintent { _ \l__spintent_spmoney_res_main_voice_str :postfix($n) }
1060     {
1061       \__spintent_inner_sep:
1062       \MathMLarg { n } { \__spintent_spmoney_render_only_integer_node: }
1063     }
1064     \__spintent_spmoney_print_sep_con:
1065     \__spintent_spmoney_print_dec_sub_unit:
1066     \MathMLintent { :silent } { \__spintent_spmoney_print_symbol: }
1067   }
1068 }

```

```

1069 \cs_new_protected:Nn \__spintent_spmoney_print_sep_con:
1070 {
1071   \MathMLintent { _con }
1072   {
1073     \mathord { \l__spintent_luaset_dec_sep_tl }
1074   }
1075 }
1076 \cs_new_protected:Nn \__spintent_spmoney_print_dec_sub_unit:
1077 {
1078   \MathMLintent { \l__spintent_spmoney_intent_sub_unit_str }
1079   {
1080     \__spintent_inner_sep:
1081     \MathMLarg { sub }
1082     {
1083       \__spintent_spmoney_render_dec:
1084     }
1085   }
1086 }

```

(End of definition for `\__spintent_spmoney_build_tree:`, `\__spintent_spmoney_print_sep_con:`, and `\__spintent_spmoney_print_dec_sub_unit:`.)

`\__spintent_spmoney_parse_and_route:n`

The internal function `\__spintent_spmoney_parse_and_route:n` processes the $\{(\textit{argument})\}$ passed in ‘#1’ using Lua module to extract the numeric value and any “currency” identifier. If a “currency” is found (`\l__spintent_spmoney_extracted_curr_str`), it sets the corresponding variables. It then calls `\__spintent_spmoney_validate_money:` to verify the “currency”. If “subunit” rendering is enabled (`\l__spintent_spmoney_sub_units_bool`), it calls `\__spintent_spmoney_check_sub_units:`. Finally, it calls `\__spintent_spmoney_print_int:` to format the output.

```

1087 \cs_new_protected:Nn \__spintent_spmoney_parse_and_route:n
1088 {
1089   \__spintent_luafun_spmoney_prepare_input:n { \exp_not:n { #1 } }
1090   \__spintent_luafun_clean_split_and_set:n { \exp_not:V \l__spintent_spmoney_extracted_num_tl }
1091   \str_if_empty:NF \l__spintent_spmoney_extracted_curr_str
1092   {
1093     \str_set:Ne \l__spintent_luaset_has_units_str { true }
1094     \str_set:NV \l__spintent_luaset_arg_above_tl \l__spintent_spmoney_extracted_curr_str
1095   }
1096   \__spintent_spmoney_validate_money:
1097   \bool_if:NT \l__spintent_spmoney_sub_units_bool
1098   { \__spintent_spmoney_check_sub_units: }
1099   \__spintent_spmoney_print_int:
1100 }

```

(End of definition for `\__spintent_spmoney_parse_and_route:n`.)

`\spmoney`

The public document command `\spmoney` formats monetary values with accessible speech intents. It accepts an optional argument ‘#1’ for local key-value configurations and a mandatory argument ‘#2’ containing the numeric value and optional embedded currency identifier. It first evaluates the typesetting context, triggering a fatal error if executed outside of math mode. Upon validation, it establishes a local TeX group to safely isolate the key assignments, delegates the raw input string to `\__spintent_spmoney_parse_and_route:n` for parsing and rendering, and subsequently closes the group to prevent scope leakage.

```

1101 \NewDocumentCommand \spmoney { 0{} m }
1102 {
1103   \mode_if_math:TF
1104   {
1105     \group_begin:
1106     \keys_set:nn { spintent / spmoney } { #1 }
1107     \__spintent_spmoney_parse_and_route:n { #2 }
1108     \group_end:
1109   }
1110   { \msg_error:nnn { spintent } { need-math-mode } { \spmoney } }
1111 }

```

(End of definition for `\spmoney`. This function is documented on page 6.)

12.18 The command `\spshort`

The user command `\spshort` is the *fourth* and *final* of “the big four”. It manages the semantic injection and typographical formatting of *text mode* abbreviations, ordinals, and acronyms. Unlike *math mode* commands, which rely on MathML attributes, this command uses the PDF *tagging tools* provided by the `tagpdf`[29] package. Specifically, it uses the E (expanded) key to ensure that NVDA interpret the full form of the abbreviation, ordinal, or acronym in *text mode*.

```

\__spintent_spshort_luaset_status_str
\__spintent_spshort_luaset_layout_str
\__spintent_spshort_luaset_expand_str
\__spintent_spshort_luaset_output_str
\__spintent_spshort_luaset_base_str
\__spintent_spshort_luaset_suffix_str

```

These internal string variables serve as the primary data interface with the Lua module `spintent.lua`. During processing, the Lua module “stores” within these registers the execution status, the targeted typographical layout parameters, the phonetic replacement text for the `E` key intended for NVDA, the final sanitized output string, and the isolated structural components for ordinals (base and suffix).

```

1112 \str_new:N \__spintent_spshort_luaset_status_str
1113 \str_new:N \__spintent_spshort_luaset_layout_str
1114 \str_new:N \__spintent_spshort_luaset_expand_str
1115 \str_new:N \__spintent_spshort_luaset_output_str
1116 \str_new:N \__spintent_spshort_luaset_base_str
1117 \str_new:N \__spintent_spshort_luaset_suffix_str

```

(End of definition for `\__spintent_spshort_luaset_status_str` and others.)

12.18.1 Definition of error messages for `\spshort`

unknown-abbreviation

The internal “error” message `unknown-abbreviation` is triggered when the command `\spshort` encounters an invalid lookup state from the Lua module. It acts as a strict fallback exception when the provided `{⟨argument⟩}` cannot be mapped to any registered database entry in `spintent.lua` nor successfully resolved through the grammatical ordinal expansion rules.

```

1118 \msg_new:nnnn { spintent } { unknown-abbreviation }
1119 { The~abbreviation~or~ordinal~'#1'~is~invalid~or~not~recognized. }
1120 { Verify~the~spelling~or~check~the~grammar~rules~for~ordinals. }

```

(End of definition for `unknown-abbreviation`.)

12.18.2 Accessibility interface

`\__spintent_spshort_tag_span:nn`

`\__spintent_spshort_tag_span:en`

The internal function `\__spintent_spshort_tag_span:nn` processes the `{⟨argument⟩}` passed in ‘#2’ using the tools provided by `tagpdf`. By utilizing the `tag` and `E` keys, it creates a native PDF /E (`#1`) structure and expands the `{⟨argument⟩}` passed in ‘#1’ for proper reading with NVDA.

```

1121 \cs_new_protected:Nn \__spintent_spshort_tag_span:nn
1122 {
1123   \tag_mc_end_push:
1124   \tag_struct_begin:n { tag=Span, E={ #1 } }
1125   \tag_mc_begin:n { tag=Span }
1126   \tag_suspend:n { \spshort }
1127   #2
1128   \tag_resume:n { \spshort }
1129   \tag_mc_end:
1130   \tag_struct_end:
1131   \tag_mc_begin_pop:n {}
1132 }
1133 \cs_generate_variant:Nn \__spintent_spshort_tag_span:nn { en }

```

(End of definition for `\__spintent_spshort_tag_span:nn`.)

12.18.3 Definition of keys for `\spshort`

small-caps

read-arg

Now we add the keys `small-caps` and `read-arg`.

```

1134 \keys_define:nn { spintent / spshort }
1135 {
1136   small-caps .bool_set:N = \__spintent_spshort_smallcaps_bool,
1137   small-caps .initial:n = false,
1138   small-caps .value_required:n = true,
1139   read-arg .str_set:N = \__spintent_spshort_read_arg_str,
1140   read-arg .initial:n = {},
1141   read-arg .value_required:n = true,
1142   unknown .code:n =
1143   {
1144     \str_set:Nn \__spintent_command_name_str { spshort }
1145     \__spintent_unknown_keys_cmd:n {#1}
1146   },
1147 }

```

(End of definition for `small-caps` and `read-arg`.)

12.18.4 Internal functions for `\spshort`

`\__spintent_spshort_print:nn`

The internal function `\__spintent_spshort_print:nn` acts as the primary formatter for text abbreviations. It clears the string variable `\__spintent_spshort_read_arg_str` and evaluates the optional `⟨keys⟩` passed in ‘#1’. It then checks if a manual reading override was provided via those keys. If that string variable remains empty, it calls `\__spintent_spshort_process_and_render:n` to process the `{⟨argument⟩}` passed in ‘#2’. Otherwise, if an override is detected, it directly delegates the execution to `\__spintent_spshort_render_bypass:n`.

```

1148 \cs_new_protected:Nn \__spintent_spshort_print:nn

```

```

1149 {
1150   \str_clear:N \l__spintent_spshort_read_arg_str
1151   \keys_set:nn { spintent / spshort } {#1}
1152   \str_if_empty:NTF \l__spintent_spshort_read_arg_str
1153   {
1154     \__spintent_spshort_process_and_render:n {#2}
1155   }
1156   {
1157     \__spintent_spshort_render_bypass:n {#2}
1158   }
1159 }

```

(End of definition for `\__spintent_spshort_print:nn`.)

`\__spintent_spshort_process_and_render:n`

The internal function `\__spintent_spshort_process_and_render:n` interfaces with the Lua module `spintent.lua` function `\__spintent_luafun_spshort_process:n` to resolve the abbreviation or ordinal `{⟨argument⟩}` passed in ‘#1’.

It subsequently evaluates the execution state stored within the string variable `\l__spintent_spshort_luaset_status` and dynamically routes execution to the corresponding rendering subroutine success, fallback or `error`. Any unrecognized status automatically defaults to the “error” handler.

```

1160 \cs_new_protected:Nn \__spintent_spshort_process_and_render:n
1161 {
1162   \__spintent_luafun_spshort_process:n {#1}
1163   \str_case:VnF \l__spintent_spshort_luaset_status_str
1164   {
1165     { success } { \__spintent_spshort_render_success: }
1166     { fallback } { \__spintent_spshort_render_fallback: }
1167     { error } { \__spintent_spshort_render_error:n {#1} }
1168   }
1169   { \__spintent_spshort_render_error:n {#1} }
1170 }

```

(End of definition for `\__spintent_spshort_process_and_render:n`.)

`\__spintent_spshort_render_success:`

The function `\__spintent_spshort_render_success:` injects the key `E` and executes the targeted typographical layout. The `\__spintent_spshort_render_fallback:` function handles default cases, applying optional formatting such as SMALL CAPS to unrecognized or partially resolved terms.

`\__spintent_spshort_render_fallback:`

`\__spintent_spshort_render_bypass:n`

Conversely, the function `\__spintent_spshort_render_bypass:n` directly maps the user-defined “*speech*” to the `{⟨argument⟩}`, bypassing the automated lookup Lua module. Finally, the function `\__spintent_spshort_render_error:n` triggers a structural exception while yielding the unformatted `{⟨argument⟩}` to prevent fatal compilation.

`\__spintent_spshort_render_error:n`

```

1171 \cs_new_protected:Nn \__spintent_spshort_render_success:
1172 {
1173   \__spintent_spshort_tag_span:en
1174   { \l__spintent_spshort_luaset_expand_str }
1175   { \__spintent_spshort_execute_layout: }
1176 }
1177 \cs_new_protected:Nn \__spintent_spshort_render_fallback:
1178 {
1179   \__spintent_spshort_tag_span:en
1180   { \l__spintent_spshort_luaset_expand_str }
1181   {
1182     \bool_if:NTF \l__spintent_spshort_smallcaps_bool
1183     { \textsc{ \l__spintent_spshort_luaset_output_str } }
1184     { \l__spintent_spshort_luaset_output_str }
1185   }
1186 }
1187 \cs_new_protected:Nn \__spintent_spshort_render_bypass:n
1188 {
1189   \__spintent_spshort_tag_span:en { \l__spintent_spshort_read_arg_str } { #1 }
1190 }
1191 \cs_new_protected:Nn \__spintent_spshort_render_error:n
1192 {
1193   \msg_error:nnn { spintent } { unknown-abbreviation } {#1}
1194 }

```

(End of definition for `\__spintent_spshort_render_success:` and others.)

`\__spintent_spshort_execute_layout:`

The internal function `\__spintent_spshort_execute_layout:` executes the typographical layout. It evaluates the descriptor “*stored*” within the string variable `\l__spintent_spshort_luaset_layout_str` (such as `linear_regular`, `superscript`, or `small_caps_pure`) and maps it to the corresponding $\LaTeX$

formatting. Furthermore, it manages complex nested configurations, dynamically applying combinations such as superscripts integrated with conditional SMALL CAPS based on the active boolean variables.

```

1195 \cs_new_protected:Nn \__spintent_spshort_execute_layout:
1196 {
1197   \str_case:VnF \l__spintent_spshort_luaset_layout_str
1198   {
1199     { linear_regular }
1200     {
1201       \bool_if:NTF \l__spintent_spshort_smallcaps_bool
1202       { \textsc{ \l__spintent_spshort_luaset_output_str } }
1203       { \l__spintent_spshort_luaset_output_str }
1204     }
1205     { linear_caps }
1206     { \l__spintent_spshort_luaset_output_str }
1207     { linear_mixed }
1208     { \l__spintent_spshort_luaset_output_str }
1209     { superscript }
1210     {
1211       \l__spintent_spshort_luaset_base_str
1212       \bool_if:NTF \l__spintent_spshort_smallcaps_bool
1213       {
1214         \textsuperscript{
1215           \textsc{
1216             \str_lowercase:V \l__spintent_spshort_luaset_suffix_str
1217           }
1218         }
1219       }
1220       {
1221         \textsuperscript{ \l__spintent_spshort_luaset_suffix_str }
1222       }
1223     }
1224     { small_caps_pure }
1225     {
1226       \bool_if:NTF \l__spintent_spshort_smallcaps_bool
1227       {
1228         \textsc{
1229           \str_lowercase:V \l__spintent_spshort_luaset_output_str
1230         }
1231       }
1232       { \l__spintent_spshort_luaset_output_str }
1233     }
1234     { acronym_long }
1235     { \l__spintent_spshort_luaset_output_str }
1236   }
1237   { \l__spintent_spshort_luaset_output_str }
1238 }

```

(End of definition for __spintent_spshort_execute_layout:.)

\spshort The public document command `\spshort` manages the accessible formatting of *text mode* abbreviations, acronyms, and ordinals. It first triggers an error if invoked within a *math mode*, upon validation, puts `\mode_leave_vertical:` and establishes a local group to call the function `\__spintent_spshort_print:nn` for processing and printing.

```

1239 \NewDocumentCommand \spshort { 0{} m }
1240 {
1241   \mode_if_math:TF
1242   {
1243     \msg_error:nnn { spintent } { need-text-mode } { \spshort }
1244   }
1245   {
1246     \mode_leave_vertical:
1247     \group_begin:
1248       \__spintent_spshort_print:nn {#1} {#2}
1249     \group_end:
1250   }
1251 }

```

(End of definition for \spshort. This function is documented on page 8.)

12.19 The command `\spsiglo`

The public document command `\spsiglo` provides a semantic interface for typesetting century designations (e.g., siglo XXI). It accepts both Arabic and Roman numerals as $\langle \textit{argument} \rangle$, interfacing with the Lua module `spintent.lua` to validate, parse, and normalize chronological values.

This command work in dual-mode context awareness, operating uniformly across both *text mode* and *math mode*. When executed within *math mode*, it constructs an accessible `MathML` intent expression tree. Conversely, when invoked in standard *text mode*, it encapsulates the rendered output within a native PDF tagging structure (such as `/Alt (#1)`) containing an `alt` key.

```
\l__spintent_spsiglo_luaset_error_str
\l__spintent_spsiglo_luaset_arabic_str
\l__spintent_spsiglo_luaset_roman_min_str
\l__spintent_spsiglo_print_era_str
```

These internal string variables handle data transfer and synchronization with the Lua module `spintent.lua`. They isolate the exception evaluation states, the Arabic numeral required for speech synthesis, the lowercase Roman numeral designated for small caps typesetting via `\textsc`, and the localized chronological era modifier.

```
1252 \str_new:N \l__spintent_spsiglo_luaset_error_str
1253 \str_new:N \l__spintent_spsiglo_luaset_arabic_str
1254 \str_new:N \l__spintent_spsiglo_luaset_roman_min_str
1255 \str_new:N \l__spintent_spsiglo_print_era_str
```

(End of definition for `\l__spintent_spsiglo_luaset_error_str` and others.)

12.19.1 Definition of error messages for `\spsiglo`

```
invalid-century-format
```

This internal error message is issued when the provided century identifier fails both Arabic and Roman numeral validation. It is triggered when the Lua module `spintent.lua` cannot map the $\langle \textit{argument} \rangle$ to an integer within the valid range of `1` to `4000`, preventing incorrect speech synthesis.

```
1256 \msg_new:nnnn { spintent } { invalid-century-format }
1257 {
1258   Invalid~century~format~in~'#1'.
1259 }
1260 {
1261   The~argument~'#2'~is~not~a~valid~Arabic~or~Roman~numeral.~
1262   Ensure~the~value~is~between~1~and~4000.
1263 }
```

(End of definition for `invalid-century-format`.)

12.19.2 Definition of keys for `\spsiglo`

```
print-era
```

Now we add the key `print-era`. User configuration determining if an era designation is appended, available values are `none`, `ac` (antes de cristo) and `dc` (después de cristo).

```
1264 \keys_define:nn { spintent / spsiglo }
1265 {
1266   print-era .choices:nn =
1267     { none , ac , dc }
1268   {
1269     \int_case:nn { \l_keys_choice_int }
1270     {
1271       {1} { \str_set:Nn \l__spintent_spsiglo_print_era_str { none } }
1272       {2} { \str_set:Nn \l__spintent_spsiglo_print_era_str { ac } }
1273       {3} { \str_set:Nn \l__spintent_spsiglo_print_era_str { dc } }
1274     }
1275   },
1276   print-era .initial:n = none,
1277   print-era .value_required:n = true,
1278   print-era / unknown .code:n =
1279     \msg_error:nnee { spintent } { unknown-choice }
1280     { print-era } { none , ~ac , ~dc } { \exp_not:n {#1} },
1281   unknown .code:n =
1282     {
1283       \str_set:Nn \l__spintent_command_name_str { spsiglo }
1284       \__spintent_unknown_keys_cmd:n {#1}
1285     },
1286 }
```

(End of definition for `print-era`.)

12.19.3 Internal functions for \spsiglo

The functions `\__spintent_spsiglo_render_simple:nn` and `\__spintent_spsiglo_render_with_era:nnn` generate MathML structures when century designations are evaluated within *math mode*. They construct MathML intent structures, mapping the Arabic numerals to their phonetic equivalents. Furthermore, `\__spintent_spsiglo_render_with_era:nnn` integrates the era modifiers *ac* or *dc*. It adds invisible spacing and postfix intents to ensure correct NVDA/MathCAT pronunciation.

```

1287 \cs_new_protected:Nn \__spintent_spsiglo_render_simple:nn
1288 {
1289   \MathMLintent { #1:number } { \text { \textsc {#2} } }
1290 }
1291 \cs_generate_variant:Nn \__spintent_spsiglo_render_simple:nn { VV }
1292 \cs_new_protected:Nn \__spintent_spsiglo_render_with_era:nnn
1293 {
1294   \str_if_eq:nnTF { #1 } { ac }
1295   {
1296     \MathMLintent { _antes-de-cristo:postfix($z) }
1297     {
1298       \__spintent_inner_sep:
1299       \MathMLarg { z }
1300       {
1301         \MathMLintent { #2:number }
1302         {
1303           \text { \textsc {#3}~a.~C. }
1304         }
1305       }
1306     }
1307   }
1308   {
1309     \MathMLintent { _después-de-cristo:postfix($z) }
1310     {
1311       \__spintent_inner_sep:
1312       \MathMLarg { z }
1313       {
1314         \MathMLintent { #2:number }
1315         {
1316           \text { \textsc {#3}~d.~C. }
1317         }
1318       }
1319     }
1320   }
1321 }
1322 \cs_generate_variant:Nn \__spintent_spsiglo_render_with_era:nnn { VVV }

```

(End of definition for `\__spintent_spsiglo_render_simple:nn` and `\__spintent_spsiglo_render_with_era:nnn`.)

The function `\__spintent_spsiglo_tag_span:nn` handles century formatting within *text mode*. It utilizes the `tagpdf` package to enclose the content inside a PDF /Alt (#1) structure. By passing the `{⟨argument⟩}` to the `alt` key, it embeds the chronological value directly into the PDF, ensuring correct NVDA/MathCAT pronunciation without relying on MathML intents.

```

1323 \cs_new_protected:Nn \__spintent_spsiglo_tag_span:nn
1324 {
1325   \tag_mc_end_push:
1326   \tag_struct_begin:n { tag=Span, alt={ #1 } }
1327   \tag_mc_begin:n { tag=Span }
1328   \tag_suspend:n { \spsiglo }
1329   #2
1330   \tag_resume:n { \spsiglo }
1331   \tag_mc_end:
1332   \tag_struct_end:
1333   \tag_mc_begin_pop:n {}
1334 }
1335 \cs_generate_variant:Nn \__spintent_spsiglo_tag_span:nn { en }

```

(End of definition for `\__spintent_spsiglo_tag_span:nn`.)

The functions `\__spintent_spsiglo_print_math:n` and `\__spintent_spsiglo_print_text:n` format century designations based on the active mode. Both first check the “error” status in the variable `\l__spintent_spsiglo_luaset_error_str` returned by the Lua module `spintent.lua` and issue an “error” if validation fails.

If the $\langle argument \rangle$ is valid, the function `\__spintent_spsiglo_print_math:n` operates in *math mode*, applying MathML structures depending on whether an era modifier is present. Conversely, the function `\__spintent_spsiglo_print_text:n` operates in *text mode*, utilizing the `tag` and `alt` keys to expand the value of the `print-era` option into full text (e.g., “antes de cristo”) for NVDA/MathCAT.

```

1336 \cs_new_protected:Nn \__spintent_spsiglo_print_math:n
1337 {
1338   \str_if_eq:VnTF \__spintent_spsiglo_luaset_error_str { true }
1339   {
1340     \msg_error:nnnn { spintent } { invalid-century-format }
1341     { \spsiglo } { #1 }
1342   }
1343   {
1344     \str_if_eq:VnTF \__spintent_spsiglo_print_era_str { none }
1345     {
1346       \__spintent_spsiglo_render_simple:VV
1347       \__spintent_spsiglo_luaset_arabic_str
1348       \__spintent_spsiglo_luaset_roman_min_str
1349     }
1350     {
1351       \__spintent_spsiglo_render_with_era:VVV
1352       \__spintent_spsiglo_print_era_str
1353       \__spintent_spsiglo_luaset_arabic_str
1354       \__spintent_spsiglo_luaset_roman_min_str
1355     }
1356   }
1357 }
1358 \cs_new_protected:Nn \__spintent_spsiglo_print_text:n
1359 {
1360   \str_if_eq:VnTF \__spintent_spsiglo_luaset_error_str { true }
1361   {
1362     \msg_error:nnnn { spintent } { invalid-century-format }
1363     { \spsiglo } { #1 }
1364   }
1365   {
1366     \str_case:Vn \__spintent_spsiglo_print_era_str
1367     {
1368       { none }
1369       {
1370         \__spintent_spsiglo_tag_span:en
1371         { \__spintent_spsiglo_luaset_arabic_str }
1372         { \textsc { \__spintent_spsiglo_luaset_roman_min_str } }
1373       }
1374       { ac }
1375       {
1376         \__spintent_spsiglo_tag_span:en
1377         { \__spintent_spsiglo_luaset_arabic_str ~ antes ~ de ~ cristo }
1378         { \textsc { \__spintent_spsiglo_luaset_roman_min_str } ~ a.~C. }
1379       }
1380       { dc }
1381       {
1382         \__spintent_spsiglo_tag_span:en
1383         { \__spintent_spsiglo_luaset_arabic_str ~ después ~ de ~ cristo }
1384         { \textsc { \__spintent_spsiglo_luaset_roman_min_str } ~ d.~C. }
1385       }
1386     }
1387   }
1388 }

```

(End of definition for `\__spintent_spsiglo_print_math:n` and `\__spintent_spsiglo_print_text:n`)

`\spsiglo` The public command `\spsiglo` is the main interface for century typesetting. It opens a local group and evaluates any optional $\langle keys \rangle$ passed in ‘#1’, then executes the function `\__spintent_luafun_spsiglo_parse:n` defined in the Lua module `spintent.lua` passed in ‘#2’ and formats the output for *math mode* or *text mode*.

```

1389 \NewDocumentCommand \spsiglo { 0{} m }
1390 {
1391   \group_begin:
1392   \keys_set:nn { spintent / spsiglo } { #1 }
1393   \__spintent_luafun_spsiglo_parse:n { #2 }
1394   \mode_if_math:TF
1395   { \__spintent_spsiglo_print_math:n { #2 } }
1396   {

```

```

1397         \mode_leave_vertical:
1398         \__spintent_spsiglo_print_text:n { #2 }
1399     }
1400     \group_end:
1401 }

```

(End of definition for \spsiglo. This function is documented on page 10.)

12.20 The command \sptime

The command `\sptime` is the main public interface for typesetting and tagging time expressions. It parses time string formats (such as HH:MM or HH:MM:SS) using the Lua function `\__spintent_luafun_sptime_parse:n`. This separates the hours, minutes, seconds, and fractions to construct a MathML `intent` tree for NVDA/MathCAT.

```

\__spintent_sptime_luaset_seconds_str
\__spintent_sptime_luaset_has_seconds_str
\__spintent_sptime_luaset_error_str
\__spintent_sptime_luaset_is_fraction_str
\__spintent_sptime_luaset_base_str

```

These internal string variables store the values returned from the Lua module `spintent.lua`. They hold the extracted time units, error flags, and boolean indicators (such as the presence of seconds or decimal fractions) used during formatting.

```

1402 \str_new:N \__spintent_sptime_luaset_seconds_str
1403 \str_new:N \__spintent_sptime_luaset_has_seconds_str
1404 \str_new:N \__spintent_sptime_luaset_error_str
1405 \str_new:N \__spintent_sptime_luaset_is_fraction_str
1406 \str_new:N \__spintent_sptime_luaset_base_str

```

(End of definition for __spintent_sptime_luaset_seconds_str and others.)

12.20.1 Definition of error messages

```
invalid-time-format
```

The error message `invalid-time-format` defines the text displayed when an invalid time input is detected. It is triggered when an argument fails the format checks or contains values outside normal ranges, such as hours exceeding 23 or minutes and seconds exceeding 59.

```

1407 \msg_new:nnnn { spintent } { invalid-time-format }
1408 {
1409     Invalid~time~format~in~#1. \\\
1410     The~argument~'#2'~does~not~match~any~supported~time~format.
1411 }
1412 {
1413     Allowed~formats:~HH:MM,~HH:MM:SS,~or~HH:MM:SS.ss. \\\
1414     Please~verify~that~hours~are~0-23~and~minutes/seconds~are~0-59.
1415 }

```

(End of definition for `invalid-time-format`.)

12.20.2 Internal functions for \sptime

```
\__spintent_sptime_build_seconds:nn
```

The internal function `\__spintent_sptime_build_seconds:nn` builds the MathML structure for the seconds component of a time expression. It wraps the numeric value ‘#2’ in speech intent tags, applying a connector prefix (passed in ‘#1’) and the “segundos” postfix so it is read correctly by NVDA/MathCAT.

```

1416 \cs_new_protected:Nn \__spintent_sptime_build_seconds:nn
1417 {
1418     \MathMLintent { #1:prefix($z) }
1419     {
1420         \text { : }
1421         \MathMLarg { z }
1422         {
1423             \MathMLintent { _segundos:postfix($m) }
1424             {
1425                 \__spintent_inner_sep:
1426                 \MathMLarg { m }
1427                 {
1428                     \text { #2 }
1429                 }
1430             }
1431         }
1432     }
1433 }

```

(End of definition for `\__spintent_sptime_build_seconds:nn`.)

```
\__spintent_sptime_build_tree:
```

The internal function `\__spintent_sptime_build_tree:` builds the complete MathML structure for the time expression. It checks the string variables `\__spintent_sptime_luaset_has_seconds_str` and `\__spintent_sptime_luaset_is_fraction_str` to determine if the time includes seconds or decimal fractions. Based on these values, it inserts the base time string and either calls `\__spintent_sptime_build_seconds:`

with the appropriate prefix (`'con'` or `'minutos-con'`), or simply appends the `'minutos'` intent tag if no seconds are present.

```

1434 \cs_new_protected:Nn \__spintent_sptime_build_tree:
1435 {
1436   \str_if_eq:VnTF \l__spintent_sptime_luaset_has_seconds_str { true }
1437   {
1438     \MathMLintent { :time }
1439     {
1440       \text { \l__spintent_sptime_luaset_base_str }
1441     }
1442     \str_if_eq:VnTF \l__spintent_sptime_luaset_is_fraction_str { true }
1443     {
1444       \__spintent_sptime_build_seconds:nn { con }
1445       { \l__spintent_sptime_luaset_seconds_str }
1446     }
1447     {
1448       \__spintent_sptime_build_seconds:nn { _minutos-con }
1449       { \l__spintent_sptime_luaset_seconds_str }
1450     }
1451   }
1452   {
1453     \MathMLintent { :time }
1454     {
1455       \text { \l__spintent_sptime_luaset_base_str }
1456     }
1457     \str_if_eq:VnTF \l__spintent_sptime_luaset_is_fraction_str { false }
1458     {
1459       \MathMLintent { _minutos }
1460       {
1461         \c__spintent_invisible_sep_int
1462       }
1463     }
1464   }
1465 }

```

(End of definition for `\__spintent_sptime_build_tree:`.)

\sptime The command `\sptime` is the public interface for formatting time expressions. It first checks if it is being used in math mode, issuing an error if not. Inside a local group, it parses the input using `\__spintent_luafun_sptime_parse:n` and checks the error status in `\l__spintent_sptime_luaset_error_str`. If an invalid format is detected, it issues the `invalid-time-format` error; otherwise, it calls `\__spintent_sptime_build_tree:` to assemble the MathML output.

```

1466 \NewDocumentCommand \sptime { m }
1467 {
1468   \mode_if_math:TF
1469   {
1470     \group_begin:
1471     \__spintent_luafun_sptime_parse:n { #1 }
1472     \str_if_eq:VnTF \l__spintent_sptime_luaset_error_str { true }
1473     {
1474       \msg_error:nnnn { spintent } { invalid-time-format }
1475       { \sptime } { #1 }
1476     }
1477     {
1478       \__spintent_sptime_build_tree:
1479     }
1480     \group_end:
1481   }
1482   { \msg_error:nnn { spintent } { need-math-mode } { \sptime } }
1483 }

```

(End of definition for `\sptime`. This function is documented on page 11.)

12.21 The command `\spdate`

The command `\spdate` is the main public command for typesetting and tagging date expressions. It parses date string formats (such as `DD/MM/YYYY` or `YYYY-MM-DD`) using the Lua function `\__spintent_luafun_spdate_parse:n`. This verifies the input and constructs a MathML `intent` tree for NVDA/MathCAT.

`\l__spintent_spdate_luaset_output_str`
`\l__spintent_spdate_luaset_error_str`

These internal string variables store the values returned from the Lua module `spintent.lua`. They hold the formatted date string and the error status used during formatting.

```

1484 \str_new:N \l__spintent_spdate_luaset_output_str

```

```
1485 \str_new:N \__spintent_spdate_luaset_error_str
```

(End of definition for `\__spintent_spdate_luaset_output_str` and `\__spintent_spdate_luaset_error_str`.)

12.21.1 Definition of error messages

invalid-date-format

The error message `invalid-date-format` defines the text displayed when an invalid date input is detected. It is triggered when an argument fails the format checks or represents an invalid calendar date, such as `31/02/2024` or using an unsupported pattern like `YYYY/MM/DD`.

```
1486 \msg_new:nnnn { spintent } { invalid-date-format }
1487 {
1488   Invalid~date~or~unsupported~format~in~#1. \\\
1489   The~argument~'#2'~does~not~match~a~supported~format.
1490 }
1491 {
1492   Allowed~formats:~DD\slash MM\slash YYYY,~or~YYYY-MM-DD. \\\
1493   Ensure~the~day~and~month~are~valid~(e.g.,~avoid~31\slash 02\slash 2024).
1494 }
```

(End of definition for `invalid-date-format`.)

12.21.2 Internal functions for `\spdate`

`\__spintent_spdate_process:n`

The internal function `\__spintent_spdate_process:n` parses the input using `\__spintent_luafun_spdate_parse:n` and checks the error status in `\__spintent_spdate_luaset_error_str`. If an invalid format is detected, it issues the `invalid-date-format` error. Otherwise, it builds the `MathML intent` structure using the formatted date stored in `\__spintent_spdate_luaset_output_str`.

```
1495 \cs_new_protected:Nn \__spintent_spdate_process:n
1496 {
1497   \__spintent_luafun_spdate_parse:n { #1 }
1498   \str_if_eq:VnTF \__spintent_spdate_luaset_error_str { true }
1499   {
1500     \msg_error:nnnn { spintent } { invalid-date-format }
1501     { \spdate } { #1 }
1502   }
1503   {
1504     \MathMLintent { :date }
1505     {
1506       \text { \__spintent_spdate_luaset_output_str }
1507     }
1508   }
1509 }
```

(End of definition for `\__spintent_spdate_process:n`.)

`\spdate`

The command `\spdate` is the public interface for formatting date expressions. It first checks if it is being used in math mode, issuing an error if not. Inside a local group, it calls `\__spintent_spdate_process:n` to process the input.

```
1510 \NewDocumentCommand \spdate { m }
1511 {
1512   \mode_if_math:TF
1513   {
1514     \group_begin:
1515     \__spintent_spdate_process:n { #1 }
1516     \group_end:
1517   }
1518   { \msg_error:nnn { spintent } { need-math-mode } { \spdate } }
1519 }
```

(End of definition for `\spdate`. This function is documented on page 11.)

12.22 Semantic Operators

This section provides purely semantic operators. Unlike their formatting counterparts (which parse and format numbers), these commands do not process mandatory arguments. Instead, they solely inject a mathematical glyph wrapped in its corresponding regional `MathCAT` intent. This grants teachers absolute control when building complex algebraic expressions, mixed numbers, or fraction divisions.

12.23 The command `\spusep`

For the use of parentheses, we will provide the command `\spusep` with only two *keys* `size` and `silent`. Most of the time we can use `\left` and `\right` or `\Uleft` and `\Uright`, but both forms create a group, and this hinders the code for *tagged* PDF when we want to interrupt something between them.

spusep-empty-arg
spusep-invalid-arg

The first “error” message spusep-empty-arg is raised when `\spusep` receives an empty or blank $\{ \langle \textit{argument} \rangle \}$, the second this is when an input is invalid.

```

1520 \msg_new:nnnn { spintent } { spusep-empty-arg }
1521 {
1522   Empty~argument~'#1'~for~\token_to_str:N \spusep.
1523 }
1524 {
1525   \token_to_str:N \spusep~requires~a~non-empty~argument.
1526 }
1527 \msg_new:nnn { spintent } { spusep-invalid-arg }
1528 {
1529   Invalid~argument~'#1'~for~\token_to_str:N \spusep.
1530 }

```

(End of definition for spusep-empty-arg and spusep-invalid-arg.)

The variable `\l__spintent_spusep_arg_tl` stores the $\{ \langle \textit{argument} \rangle \}$, the variable `\l__spintent_spusep_left_tl` and `\l__spintent_spusep_right_tl` stores the parentheses.

```

1531 \tl_new:N \l__spintent_spusep_arg_tl
1532 \tl_new:N \l__spintent_spusep_left_tl
1533 \tl_new:N \l__spintent_spusep_right_tl

```

(End of definition for `\l__spintent_spusep_arg_tl`, `\l__spintent_spusep_left_tl`, and `\l__spintent_spusep_right_tl`.)

size
silent
unknown

We define the *keys* `size` and `silent`.

```

1534 \keys_define:nn { spintent / spusep }
1535 {
1536   size .choices:nn =
1537     { none , big , Big , bigg, Bigg }
1538     {
1539       \int_case:nn { \l_keys_choice_int }
1540       {
1541         { 1 }
1542         {
1543           \tl_clear:N \l__spintent_spusep_left_tl
1544           \tl_clear:N \l__spintent_spusep_right_tl
1545         }
1546         { 2 }
1547         {
1548           \tl_set:Nn \l__spintent_spusep_left_tl { \bigl }
1549           \tl_set:Nn \l__spintent_spusep_right_tl { \bigr }
1550         }
1551         { 3 }
1552         {
1553           \tl_set:Nn \l__spintent_spusep_left_tl { \Bigl }
1554           \tl_set:Nn \l__spintent_spusep_right_tl { \Bigr }
1555         }
1556         { 4 }
1557         {
1558           \tl_set:Nn \l__spintent_spusep_left_tl { \biggl }
1559           \tl_set:Nn \l__spintent_spusep_right_tl { \biggr }
1560         }
1561         { 5 }
1562         {
1563           \tl_set:Nn \l__spintent_spusep_left_tl { \Biggl }
1564           \tl_set:Nn \l__spintent_spusep_right_tl { \Biggr }
1565         }
1566       }
1567     },
1568   size .initial:n = none,
1569   size .value_required:n = true,
1570   size / unknown .code:n =
1571     \msg_error:nneee { spintent } { unknown-choice }
1572     { size } { none,~big,~Big,~bigg,~Bigg } { \exp_not:n {#1} },
1573   silent .bool_set:N = \l__spintent_spusep_silent_bool,
1574   silent .initial:n = false,

```

```

1575     silent           .value_required:n = true,
1576     unknown          .code:n =
1577     {
1578         \str_set:Nn \__spintent_command_name_str { spusep }
1579         \__spintent_unknown_keys_cmd:n {#1}
1580     },
1581 }

```

(End of definition for `size`, `silent`, and `unknown`.)

`\__spintent_spusep_set_parens:nn`

The function `\__spintent_spusep_set_parens:nn` will place the argument passed in ‘#2’ to the right of the variables `\__spintent_spusep_left_tl` and `\__spintent_spusep_right_tl` and then check the state of the variable `\__spintent_spusep_silent_bool` handled by the key `silent` to print.

```

1582 \cs_new_protected:Nn \__spintent_spusep_set_parens:nn
1583 {
1584     \tl_put_right:cn { \__spintent_spusep_ #1 _tl } { #2 }
1585     \bool_if:NTF \__spintent_spusep_silent_bool
1586     {
1587         \MathMLintenc { :silent } { \tl_use:c { \__spintent_spusep_ #1 _tl } }
1588     }
1589     {
1590         \tl_use:c { \__spintent_spusep_ #1 _tl }
1591     }
1592 }

```

(End of definition for `\__spintent_spusep_set_parens:nn`.)

`\__spintent_spusep_convert:n`

The function `\__spintent_spusep_convert:n` captures and stores the $\langle argument \rangle$ passed in ‘#1’ in the variable `\__spintent_spusep_arg_tl`, validates the cases by converting them to the macros provided by `unicode-math`, and passes them to the function `\__spintent_spusep_set_parens:nn` for printing. If the input is not supported or is empty, we return an “error”.

```

1593 \cs_new_protected:Nn \__spintent_spusep_convert:n
1594 {
1595     \tl_set:Ne \__spintent_spusep_arg_tl { \tl_trim_spaces:n {#1} }
1596     \tl_if_empty:NT \__spintent_spusep_arg_tl
1597     {
1598         \msg_error:nnn { spintenc } { spusep-empty-arg } {#1}
1599     }
1600     \str_case:VnF \__spintent_spusep_arg_tl
1601     {
1602         { ( }
1603         {
1604             \__spintent_spusep_set_parens:nn { left } { \lparen }
1605         }
1606         { ) }
1607         {
1608             \__spintent_spusep_set_parens:nn { right } { \rparen }
1609         }
1610         { [ }
1611         {
1612             \__spintent_spusep_set_parens:nn { left } { \lbrack }
1613         }
1614         { ] }
1615         {
1616             \__spintent_spusep_set_parens:nn { right } { \rbrack }
1617         }
1618         { \{ }
1619         {
1620             \__spintent_spusep_set_parens:nn { left } { \lbrace }
1621         }
1622         { \} }
1623         {
1624             \__spintent_spusep_set_parens:nn { right } { \rbrace }
1625         }
1626     }
1627     {
1628         \msg_error:nnn { spintenc } { spusep-invalid-arg } {#1}
1629     }
1630 }

```

(End of definition for `\__spintent_spusep_convert:n`.)

`\spusep` The command `\spusep` checks that it is running in math mode, opens a local group, sets the keys passed in ‘#1’ and processes the $\langle argument \rangle$ passed in ‘#2’ by means of the function `\__spintent_spusep_convert:n`.

```

1631 \NewDocumentCommand \spusep { O{} m }
1632 {
1633   \mode_if_math:TF
1634   {
1635     \group_begin:
1636     \keys_set:nn { spintent / spusep } {#1}
1637     \__spintent_spusep_convert:n {#2}
1638     \group_end:
1639   }
1640   { \msg_error:nnn { spintent } { need-math-mode } { \spusep } }
1641 }

```

(End of definition for `\spusep`. This function is documented on page 12.)

12.23.1 The command `\spread`

The command `\spread` provides a semantic read for MathCAT of mathematical symbols for situations in which it is necessary to specify gender or plurality in Spanish.

The “error” `spnewread-empty-args` is raised when `\spnewread` receives a blank symbol or a reading that is empty once normalized. The “error” `spnewread-duplicate` is raised when it registers a symbol already in the dictionary. The “error” `spread-unknown-sym` is raised when `\spread` is given a symbol that is neither registered nor accompanied by `read-arg`.

```

1642 \msg_new:nnnn { spintent } { spnewread-empty-args }
1643 {
1644   Empty~argument~in~\token_to_str:N \spnewread.
1645 }
1646 {
1647   \token_to_str:N \spnewread~requires~two~non-empty~arguments.
1648 }
1649 \msg_new:nnnn { spintent } { spnewread-duplicate }
1650 {
1651   The~symbol~'~#1'~is~already~defined!
1652 }
1653 {
1654   You~cannot~redefine~with~\token_to_str:N \spnewread.
1655 }
1656 \msg_new:nnnn { spintent } { spread-empty-sym }
1657 {
1658   Empty~symbol~argument~in~\token_to_str:N \spread.
1659 }
1660 {
1661   \token_to_str:N \spread~requires~a~non-empty~symbol.
1662 }
1663 \msg_new:nnnn { spintent } { spread-unknown-sym }
1664 {
1665   The~symbol~'~#1'~is~not~registered~and~no~'read-arg'~was~given.
1666 }
1667 {
1668   Register~it~first~with~\token_to_str:N \spnewread,~or~pass~'read-arg'.
1669 }

```

(End of definition for `spnewread-empty-args`, `spnewread-duplicate`, and `spread-unknown-sym`.)

The variables `\__spintent_spread_luaset_status_str` and `\__spintent_spread_luaset_reading_str` are set by `\__spintent_luafun_spread_new:nn/\__spintent_luafun_spread_lookup:n`. The variable `\__spintent_spread_read_arg_tl` stores `read-arg`.

```

1670 \str_new:N \__spintent_spread_luaset_status_str
1671 \str_new:N \__spintent_spread_luaset_reading_str
1672 \tl_new:N \__spintent_spread_read_arg_tl
1673 \tl_new:N \__spintent_spread_read_tmp_tl

```

(End of definition for `\__spintent_spread_luaset_status_str` and others.)

We define the $\langle keys \rangle$ `read-arg` and `silent`.

```

1674 \keys_define:nn { spintent / spread }
1675 {
1676   read-arg .code:n =
1677   {
1678     \__spintent_sanitizé_affix:Nn \__spintent_spread_read_arg_tl {#1}

```

```

1679     },
1680     read-arg      .initial:n = ,
1681     read-arg      .value_required:n = true,
1682     silent        .bool_set:N = \l__spintent_spread_silent_bool,
1683     silent        .initial:n = false,
1684     silent        .value_required:n = true,
1685     unknown       .code:n =
1686     {
1687       \str_set:Nn \l__spintent_command_name_str { spread }
1688       \__spintent_unknown_keys_cmd:n {#1}
1689     },
1690   }

```

(End of definition for `read-arg`, `silent`, and `unknown`.)

`\__spintent_spread_exec:nn`

The function `\__spintent_spread_exec:nn` receives the optional $\langle keys \rangle$ in ‘#1’ and the $\{\langle symbol \rangle\}$ in ‘#2’. With `silent true`, the symbol is typeset inside a literal `_- \MathMLintent`, nothing else evaluated; a composite symbol (such as x^2) is not reliably silenced by `:silent` alone. In every case the symbol is preceded by `\__spintent_inner_sep:` inside the group, so that a composite symbol (a sole child, such as x^2) keeps its `intent` instead of collapsing. Otherwise, with `read-arg` given, it is used directly, overriding any registered reading for this call alone. With `read-arg` empty, calls `\__spintent_luafun_spread_lookup:n`, using the registered reading when found, reporting `spread-unknown-sym` otherwise.

```

1691 \cs_new_protected:Nn \__spintent_spread_exec:nn
1692 {
1693   \keys_set:nn { spintent / spread } { #1 }
1694   \tl_if_blank:nTF { #2 }
1695   { \msg_error:nn { spintent } { spread-empty-sym } }
1696   {
1697     \__spintent_invisible_sep:
1698     \bool_if:NTF \l__spintent_spread_silent_bool
1699     {
1700       \MathMLintent { _- } { \__spintent_inner_sep: #2 }
1701     }
1702     {
1703       \tl_if_empty:NTF \l__spintent_spread_read_arg_tl
1704       {
1705         \__spintent_luafun_spread_lookup:n { \exp_not:n {#2} }
1706         \str_if_eq:VnTF \l__spintent_spread_luaaset_status_str { found }
1707         {
1708           \MathMLintent { _\l__spintent_spread_luaaset_reading_str } { \__spintent_inner_sep: #2 }
1709         }
1710         {
1711           \msg_error:nne { spintent } { spread-unknown-sym } { \exp_not:n {#2} }
1712         }
1713       }
1714       {
1715         \MathMLintent { _\l__spintent_spread_read_arg_tl } { \__spintent_inner_sep: #2 }
1716       }
1717     }
1718   }
1719 }

```

(End of definition for `\__spintent_spread_exec:nn`.)

`\spread` The public command `\spread` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_spread_exec:nn`.

```

1720 \NewDocumentCommand \spread { 0{} m }
1721 {
1722   \mode_if_math:TF
1723   {
1724     \group_begin:
1725     \__spintent_spread_exec:nn { #1 } { #2 }
1726     \group_end:
1727   }
1728   { \msg_error:nnn { spintent } { need-math-mode } { \spread } }
1729 }

```

(End of definition for `\spread`. This function is documented on page 12.)

`\spnewread`

The public command `\spnewread` registers a new symbol/reading pair. It first normalizes the reading ‘#2’ with `\__spintent_sanitize_affix:Nn`, storing the result in the scratch variable `\l__spintent_spread_read_tmp_tl`.

and reports `spnewread-empty-args` when the symbol ‘`#1`’ is blank or the normalized reading is empty. Otherwise it passes the symbol and the normalized reading to Lua via `\__spintent_luafun_spread_new:nn`, reporting `spnewread-duplicate` when the symbol is already registered.

```

1730 \NewDocumentCommand \spnewread { m m }
1731 {
1732   \mode_if_math:TF
1733   {
1734     \msg_error:nnn { spintent } { need-text-mode } { \spnewread }
1735   }
1736   {
1737     \__spintent_sanitizize_affix:Nn \__spintent_spread_read_tmp_tl { #2 }
1738     \bool_lazy_or:nnTF
1739     { \tl_if_blank_p:n { #1 } } { \tl_if_empty_p:N \__spintent_spread_read_tmp_tl }
1740     { \msg_error:nn { spintent } { spnewread-empty-args } }
1741     {
1742       \__spintent_luafun_spread_new:nn { \exp_not:n {#1} } { \exp_not:V \__spintent_spread
1743       \str_if_eq:VnT \__spintent_spread_luaseta_status_str { duplicate }
1744       {
1745         \msg_error:nnn { spintent } { spnewread-duplicate } { #1 }
1746       }
1747     }
1748   }
1749 }

```

(End of definition for `\spnewread`. This function is documented on page 11.)

12.23.2 The command `\spdsym`

Variables to store the regional reading string, the visual operator, and the optional semantic affixes specifically for `\spdsym`.

```

\__spintent_spdsym_read_sym_tl
\__spintent_spdsym_symbol_tl
\__spintent_spdsym_prefix_tl
\__spintent_spdsym_suffix_tl
1750 \tl_new:N \__spintent_spdsym_read_sym_tl
1751 \tl_new:N \__spintent_spdsym_symbol_tl
1752 \tl_new:N \__spintent_spdsym_prefix_tl
1753 \tl_new:N \__spintent_spdsym_suffix_tl

```

(End of definition for `\__spintent_spdsym_read_sym_tl` and others.)

```

prefix We define the (keys) prefix, suffix, set-sym and read-sym for \spdsym.
suffix
set-sym
read-sym
unknown
1754 \keys_define:nn { spintent / spdsym }
1755 {
1756   prefix .code:n =
1757   { \__spintent_sanitizize_affix:Nn \__spintent_spdsym_prefix_tl {#1} },
1758   prefix .initial:n = ,
1759   prefix .value_required:n = true,
1760   suffix .code:n =
1761   { \__spintent_sanitizize_affix:Nn \__spintent_spdsym_suffix_tl {#1} },
1762   suffix .initial:n = ,
1763   suffix .value_required:n = true,
1764   set-sym .choices:nn =
1765   { old-style , two-dots , slash }
1766   {
1767     \int_case:nn { \l_keys_choice_int }
1768     {
1769       {1} { \tl_set:Nn \__spintent_spdsym_symbol_tl { \div } }
1770       {2} { \tl_set:Nn \__spintent_spdsym_symbol_tl { : } }
1771       {3} { \tl_set:Nn \__spintent_spdsym_symbol_tl { / } }
1772     }
1773   },
1774   set-sym .initial:n = two-dots,
1775   set-sym .value_required:n = true,
1776   set-sym / unknown .code:n =
1777   \msg_error:nnee { spintent } { unknown-choice }
1778   { set-sym } { old-style, ~two-dots, ~slash } { \exp_not:n {#1} },
1779   read-sym .choices:nn =
1780   { dividido , dividido-por , dividido-entre }
1781   {
1782     \int_case:nn { \l_keys_choice_int }
1783     {
1784       {1} { \tl_set:Nn \__spintent_spdsym_read_sym_tl { dividido } }
1785       {2} { \tl_set:Nn \__spintent_spdsym_read_sym_tl { dividido-por } }
1786       {3} { \tl_set:Nn \__spintent_spdsym_read_sym_tl { dividido-entre } }
1787     }

```

```

1788     },
1789     read-sym          .initial:n = dividido,
1790     read-sym          .value_required:n = true,
1791     read-sym / unknown .code:n =
1792       \msg_error:nneee { spintent } { unknown-choice }
1793       { read-sym } { dividido, ~dividido-por, ~dividido-entre }
1794       { \exp_not:n {#1} },
1795     unknown          .code:n =
1796       {
1797         \str_set:Nn \l__spintent_command_name_str { spdsym }
1798         \__spintent_unknown_keys_cmd:n {#1}
1799       },
1800   }

```

(End of definition for *prefix* and others.)

\spdsym Defines the **\spdsym** operator. It takes a single optional argument for local setup and outputs the configured intent structure, safely attaching prefixes and suffixes if present.

```

1801 \NewDocumentCommand \spdsym { 0{} }
1802 {
1803   \mode_if_math:TF
1804   {
1805     \group_begin:
1806     \keys_set:nn { spintent / spdsym } { #1 }
1807     \MathMLintent
1808     {
1809       -
1810       \tl_if_empty:NF \l__spintent_spdsym_prefix_tl
1811       { \l__spintent_spdsym_prefix_tl - }
1812       \l__spintent_spdsym_read_sym_tl
1813       \tl_if_empty:NF \l__spintent_spdsym_suffix_tl
1814       { - \l__spintent_spdsym_suffix_tl }
1815     }
1816     { \l__spintent_spdsym_symbol_tl }
1817     \group_end:
1818   }
1819   { \msg_error:nnn { spintent } { need-math-mode } { \spdsym } }
1820 }

```

(End of definition for **\spdsym**. This function is documented on page 12.)

12.23.3 The command **\spmsym**

\l__spintent_spmsym_read_sym_tl Variables to store the regional reading string, the visual operator, and the optional semantic affixes specifically for **\spmsym**.

```

\l__spintent_spmsym_symbol_tl
\l__spintent_spmsym_prefix_tl
\l__spintent_spmsym_suffix_tl
1821 \tl_new:N \l__spintent_spmsym_read_sym_tl
1822 \tl_new:N \l__spintent_spmsym_symbol_tl
1823 \tl_new:N \l__spintent_spmsym_prefix_tl
1824 \tl_new:N \l__spintent_spmsym_suffix_tl

```

(End of definition for **\l__spintent_spmsym_read_sym_tl** and others.)

We define the *(keys)* prefix, suffix, set-sym, read-sym and not-print for **\spmsym**.

```

prefix
suffix
set-sym
read-sym
not-print
unknown
1825 \keys_define:nn { spintent / spmsym }
1826 {
1827   prefix          .code:n =
1828   { \__spintent_sanitize_affix:Nn \l__spintent_spmsym_prefix_tl {#1} },
1829   prefix          .initial:n = ,
1830   prefix          .value_required:n = true,
1831   suffix          .code:n =
1832   { \__spintent_sanitize_affix:Nn \l__spintent_spmsym_suffix_tl {#1} },
1833   suffix          .initial:n = ,
1834   suffix          .value_required:n = true,
1835   set-sym         .choices:nn =
1836   { cross , dot , asterisk }
1837   {
1838     \int_case:nn { \l_keys_choice_int }
1839     {
1840       {1} { \tl_set:Nn \l__spintent_spmsym_symbol_tl { \times } }
1841       {2} { \tl_set:Nn \l__spintent_spmsym_symbol_tl { \cdot } }
1842       {3} { \tl_set:Nn \l__spintent_spmsym_symbol_tl { \ast } }
1843     }
1844   },

```

```

1845   set-sym          .initial:n = dot,
1846   set-sym          .value_required:n = true,
1847   set-sym / unknown .code:n =
1848     \msg_error:nnee { spintent } { unknown-choice }
1849     { set-sym } { cross, ~dot, ~asterisk } { \exp_not:n {#1} },
1850   read-sym         .choices:nn =
1851     { por , multiplicado-por , veces }
1852     {
1853       \int_case:nn { \l_keys_choice_int }
1854       {
1855         {1} { \tl_set:Nn \l__spintent_spmsym_read_sym_tl { por } }
1856         {2} { \tl_set:Nn \l__spintent_spmsym_read_sym_tl { multiplicado-por } }
1857         {3} { \tl_set:Nn \l__spintent_spmsym_read_sym_tl { veces } }
1858       }
1859     },
1860   read-sym         .initial:n = por,
1861   read-sym         .value_required:n = true,
1862   read-sym / unknown .code:n =
1863     \msg_error:nnee { spintent } { unknown-choice }
1864     { read-sym } { por, ~multiplicado-por, ~veces } { \exp_not:n {#1} },
1865   not-print        .bool_set:N = \l__spintent_spmsym_not_print_bool,
1866   not-print        .initial:n = false,
1867   not-print        .value_required:n = true,
1868   unknown          .code:n =
1869     {
1870       \str_set:Nn \l__spintent_command_name_str { spmsym }
1871       \__spintent_unknown_keys_cmd:n {#1}
1872     },
1873 }

```

(End of definition for *prefix* and others.)

\spmsym Defines the **\spmsym** operator. Similar to **\spdsym**, it outputs the correct mathematical glyph based on the local *<keys>*, safely attaching prefixes and suffixes if present.

```

1874 \NewDocumentCommand \spmsym { 0{} }
1875 {
1876   \mode_if_math:TF
1877   {
1878     \group_begin:
1879     \keys_set:nn { spintent / spmsym } { #1 }
1880     \MathMLintent
1881     {
1882       -
1883       \tl_if_empty:NF \l__spintent_spmsym_prefix_tl
1884       { \l__spintent_spmsym_prefix_tl - }
1885       \l__spintent_spmsym_read_sym_tl
1886       \tl_if_empty:NF \l__spintent_spmsym_suffix_tl
1887       { - \l__spintent_spmsym_suffix_tl }
1888     }
1889     {
1890       \bool_if:NTF \l__spintent_spmsym_not_print_bool
1891       {
1892         \invisibletimes
1893       }
1894       { \l__spintent_spmsym_symbol_tl }
1895     }
1896     \group_end:
1897   }
1898   { \msg_error:nnn { spintent } { need-math-mode } { \spmsym } }
1899 }

```

(End of definition for **\spmsym**. This function is documented on page 13.)

12.24 The command **\spdiv**

The user-level command **\spdiv** formats division representations. It takes two mandatory arguments: the *<dividend>* and the *<divisor>*. Both *<arguments>* are internally processed by the **\spnum** command to ensure proper digit separation. This command enforces strict validation, ensuring that neither the *<dividend>* nor the *<divisor>* is *blank*.

12.24.1 Definition of keys for \spdiv

We define the *(keys)* `wrap-parens` and `read-parens` for `\spdiv` command. Number-related *(keys)* `cifras-sep`, `read-sign`, `read-period-sep` and `notation-sym` are forwarded to `\spnum` command using `.meta:nn`, while semantic operator *(keys)* `set-sym` and `read-sym` are forwarded to the `\spdsym` command using `.meta:nn`.

```

1900 \keys_define:nn { spintent / spdiv }
1901 {
1902   cifras-sep      .meta:nn = { spintent / spnum } { cifras-sep      = {#1} },
1903   read-sign       .meta:nn = { spintent / spnum } { read-sign       = {#1} },
1904   read-period-sep .meta:nn = { spintent / spnum } { read-period-sep = {#1} },
1905   notation-sym    .meta:nn = { spintent / spnum } { notation-sym    = {#1} },
1906   set-sym         .meta:nn = { spintent / spdsym } { set-sym         = {#1} },
1907   read-sym        .meta:nn = { spintent / spdsym } { read-sym        = {#1} },
1908   wrap-parens     .bool_set:N = \l__spintent_spdiv_wrap_parens_bool,
1909   wrap-parens     .initial:n  = false,
1910   wrap-parens     .value_required:n = true,
1911   read-parens     .bool_set:N = \l__spintent_spdiv_read_parens_bool,
1912   read-parens     .initial:n  = true,
1913   read-parens     .value_required:n = true,
1914   unknown         .code:n =
1915   {
1916     \str_set:Nn \l__spintent_command_name_str { spdiv }
1917     \l__spintent_unknown_keys_cmd:n {#1}
1918   },
1919 }

```

(End of definition for `cifras-sep` and others.)

12.24.2 Definition of error messages

`spdiv-empty-argument` Raised when either the *(dividend)* or the *(divisor)* is missing or *blank*.

```

1920 \msg_new:nnnn { spintent } { spdiv-empty-argument }
1921 { The~arguments~for~\token_to_str:N \spdiv~cannot~be~empty. }
1922 {
1923   You~must~provide~both~a~dividend~and~a~divisor.\\
1924 }

```

(End of definition for `spdiv-empty-argument`.)

12.24.3 Internal functions for \spdiv

`\__spintent_spdiv_render_op:nn` The function `\__spintent_spdiv_render_op:nn` parses both *(arguments)* through the `\spnum` command and embeds the semantic operator from `\spdsym`.

```

1925 \cs_new_protected:Nn \__spintent_spdiv_render_op:nn
1926 {
1927   \spnum {#1}
1928   \MathMLintent { _ \l__spintent_spdsym_read_sym_tl }
1929   {
1930     \l__spintent_spdsym_symbol_tl
1931   }
1932   \spnum {#2}
1933 }

```

(End of definition for `\__spintent_spdiv_render_op:nn`.)

`\__spintent_spdiv_render_parens:nn` The function `\__spintent_spdiv_render_parens:nn` handles the structural parentheses around the division.

```

1934 \cs_new_protected:Nn \__spintent_spdiv_render_parens:nn
1935 {
1936   \bool_if:NTF \l__spintent_spdiv_read_parens_bool
1937   {
1938     ( \__spintent_spdiv_render_op:nn {#1} {#2} )
1939   }
1940   {
1941     \MathMLintent { :silent } { ( }
1942     \__spintent_spdiv_render_op:nn {#1} {#2}
1943     \MathMLintent { :silent } { ) }
1944   }
1945 }

```

(End of definition for `\__spintent_spdiv_render_parens:nn`.)

`\__spintent_spdiv_process:nnn`

The function `\__spintent_spdiv_process:nnn` sets $\langle keys \rangle$, validates $\{ \langle arguments \rangle \}$, and decides on parenthesis wrapping.

```

1946 \cs_new_protected:Nn \__spintent_spdiv_process:nnn
1947 {
1948   \keys_set:nn { spintent / spdiv } { #1 }
1949   \tl_if_blank:nTF { #2 }
1950   { \msg_error:nnn { spintent } { spdiv-empty-argument } }
1951   {
1952     \tl_if_blank:nTF { #3 }
1953     { \msg_error:nnn { spintent } { spdiv-empty-argument } }
1954     {
1955       \bool_if:NTF \__spintent_spdiv_wrap_parens_bool
1956       { \__spintent_spdiv_render_parens:nn {#2} {#3} }
1957       { \__spintent_spdiv_render_op:nn {#2} {#3} }
1958     }
1959   }
1960 }

```

(End of definition for `\__spintent_spdiv_process:nnn`.)

`\spdiv` Defines the user command `\spdiv`. It strictly relies on *math mode*.

```

1961 \NewDocumentCommand \spdiv { 0{} m m }
1962 {
1963   \mode_if_math:TF
1964   {
1965     \group_begin:
1966     \__spintent_spdiv_process:nnn { #1 } { #2 } { #3 }
1967     \group_end:
1968   }
1969   { \msg_error:nnn { spintent } { need-math-mode } { \spdiv } }
1970 }

```

(End of definition for `\spdiv`. This function is documented on page 13.)

12.25 The command `\spmul`

The user-level command `\spmul` formats multiplication representations. It takes two mandatory arguments: the $\{ \langle first\ factor \rangle \}$ and the $\{ \langle second\ factor \rangle \}$. Both $\{ \langle arguments \rangle \}$ are internally processed by the `\spnum` command to ensure proper digit separation. This command enforces strict validation, ensuring that neither the $\{ \langle first\ factor \rangle \}$ nor the $\{ \langle second\ factor \rangle \}$ is *blank*.

12.25.1 Definition of keys for `\spmul`

We define the $\langle keys \rangle$ `wrap-parens` and `read-parens` for `\spmul` command. Number-related $\langle keys \rangle$ `cifras-sep`, `read-sign`, `read-period-sep` and `notation-sym` are forwarded to `\spnum` command using `.meta:nn`, while semantic operator $\langle keys \rangle$ `set-sym` and `read-sym` are forwarded to the `\spmsym` command using `.meta:nn`.

```

1971 \keys_define:nn { spintent / spmul }
1972 {
1973   cifras-sep      .meta:nn = { spintent / spnum } { cifras-sep      = {#1} },
1974   read-sign       .meta:nn = { spintent / spnum } { read-sign       = {#1} },
1975   read-period-sep .meta:nn = { spintent / spnum } { read-period-sep = {#1} },
1976   notation-sym    .meta:nn = { spintent / spnum } { notation-sym    = {#1} },
1977   set-sym         .meta:nn = { spintent / spmsym } { set-sym         = {#1} },
1978   read-sym        .meta:nn = { spintent / spmsym } { read-sym        = {#1} },
1979   wrap-parens     .bool_set:N = \__spintent_spmul_wrap_parens_bool,
1980   wrap-parens     .initial:n  = false,
1981   wrap-parens     .value_required:n = true,
1982   read-parens     .bool_set:N = \__spintent_spmul_read_parens_bool,
1983   read-parens     .initial:n  = true,
1984   read-parens     .value_required:n = true,
1985   unknown         .code:n =
1986   {
1987     \str_set:Nn \__spintent_command_name_str { spmul }
1988     \__spintent_unknown_keys_cmd:n {#1}
1989   },
1990 }

```

(End of definition for `cifras-sep` and others.)

12.25.2 Definition of error messages

`spmul-empty-argument` Raised when either the $\langle\textit{first factor}\rangle$ or the $\langle\textit{second factor}\rangle$ is missing or *blank*.

```

1991 \msg_new:nnnn { spintent } { spmul-empty-argument }
1992 { The~arguments~for~\token_to_str:N \spmul~cannot~be~empty. }
1993 {
1994   You~must~provide~both~first~factor~and~second~factor.\\
1995 }

```

(End of definition for `spmul-empty-argument`.)

12.25.3 Internal functions for `\spmul`

`\__spintent_spmul_render_op:nn` The function `\__spintent_spmul_render_op:nn` parses both $\langle\textit{arguments}\rangle$ through the `\spnum` command and embeds the semantic operator from `\spmsym`.

```

1996 \cs_new_protected:Nn \__spintent_spmul_render_op:nn
1997 {
1998   \spnum {#1}
1999   \MathMLintent { _ \__spintent_spmsym_read_sym_tl }
2000   {
2001     \__spintent_spmsym_symbol_tl
2002   }
2003   \spnum {#2}
2004 }

```

(End of definition for `\__spintent_spmul_render_op:nn`.)

`\__spintent_spmul_render_parens:nn` The function `\__spintent_spmul_render_parens:nn` handles the structural parentheses around the multiplication.

```

2005 \cs_new_protected:Nn \__spintent_spmul_render_parens:nn
2006 {
2007   \bool_if:NTF \__spintent_spmul_read_parens_bool
2008   {
2009     ( \__spintent_spmul_render_op:nn {#1} {#2} )
2010   }
2011   {
2012     \MathMLintent { :silent } { ( }
2013     \__spintent_spmul_render_op:nn {#1} {#2}
2014     \MathMLintent { :silent } { ) }
2015   }
2016 }

```

(End of definition for `\__spintent_spmul_render_parens:nn`.)

`\__spintent_spmul_process:nnn` The function `\__spintent_spmul_process:nnn` sets $\langle\textit{keys}\rangle$, validates $\langle\textit{arguments}\rangle$, and decides on parenthesis wrapping.

```

2017 \cs_new_protected:Nn \__spintent_spmul_process:nnn
2018 {
2019   \keys_set:nn { spintent / spmul } { #1 }
2020   \tl_if_blank:nTF { #2 }
2021   { \msg_error:nnn { spintent } { spmul-empty-argument } }
2022   {
2023     \tl_if_blank:nTF { #3 }
2024     { \msg_error:nnn { spintent } { spmul-empty-argument } }
2025     {
2026       \bool_if:NTF \__spintent_spmul_wrap_parens_bool
2027       { \__spintent_spmul_render_parens:nn {#2} {#3} }
2028       { \__spintent_spmul_render_op:nn {#2} {#3} }
2029     }
2030   }
2031 }

```

(End of definition for `\__spintent_spmul_process:nnn`.)

`\spmul` Defines the user command `\spmul`. It strictly relies on *math mode*.

```

2032 \NewDocumentCommand \spmul { 0{ } m m }
2033 {
2034   \mode_if_math:TF
2035   {
2036     \group_begin:
2037     \__spintent_spmul_process:nnn { #1 } { #2 } { #3 }
2038     \group_end:
2039   }
2040   { \msg_error:nnn { spintent } { need-math-mode } { \spmul } }
2041 }

```

(End of definition for `\spmul`. This function is documented on page 14.)

12.26 The commands `\spgcd` and `\spmc`

The user-level commands `\spgcd` and `\spmc` provide a semantic interface for the Greatest Common Divisor (MCD in Spanish) and the Least Common Multiple (MCM in Spanish). They delegate argument validation and calculation to the Lua module `spintent.lua`, constructing a valid `MathCAT` intent.

12.26.1 Variables for `\spgcd` and `\spmc`

Internal variables generated dynamically via a comma-separated list to avoid code duplication for both symmetric commands.

```
\__spintenc_spmcm_spmgcd_luaset_error_str
\__spintenc_mc_luaset_types_str
\__spintenc_mc_luaset_all_numeric_str
\__spintenc_mc_args_seq
\__spintenc_mc_raw_args_seq
\__spintenc_spmgcd_print_result_bool
\__spintenc_spmcm_print_result_bool
\__spintenc_spmgcd_read_terse_bool
\__spintenc_spmcm_read_terse_bool
\__spintenc_spmgcd_print_upper_bool
\__spintenc_spmcm_print_upper_bool
\__spintenc_spmgcd_prefix_tl
\__spintenc_spmcm_prefix_tl
\__spintenc_spmgcd_suffix_tl
\__spintenc_spmcm_suffix_tl
```

```
2042 \str_new:N \__spintenc_spmcm_spmgcd_luaset_error_str
2043 \clist_new:N \__spintenc_mc_luaset_types_clist
2044 \str_new:N \__spintenc_mc_luaset_all_numeric_str
2045 \seq_new:N \__spintenc_mc_args_seq
2046 \seq_new:N \__spintenc_mc_raw_args_seq
2047 \seq_new:N \__spintenc_mc_types_seq
2048 \int_new:N \__spintenc_mc_idx_int
2049 \clist_map_inline:nn { mcd , mcm }
2050 {
2051   \tl_new:c { \__spintenc_sp #1 _luaset_ #1 _value_tl }
2052   \bool_new:c { \__spintenc_sp #1 _read_terse_bool }
2053   \tl_new:c { \__spintenc_sp #1 _prefix_tl }
2054 }
```

(End of definition for `\__spintenc_spmcm_spmgcd_luaset_error_str` and others.)

12.26.2 Definition of error messages

mc-invalid-arguments

Exception triggered when the comma-separated sequence contains negative integers, floats, or alphabetical characters.

```
2055 \msg_new:nnnn { spintenc } { mc-invalid-arguments }
2056 {
2057   Invalid~arguments~provided~to~#1.
2058 }
2059 {
2060   The~arguments~must~be~a~comma-separated~list. \\\
2061 }
```

(End of definition for `mc-invalid-arguments`.)

12.26.3 Keys for `\spgcd` and `\spmc`

```
result
prefix
read-cmd
upper-case
concept
silent
unknown
```

```
2062 \clist_map_inline:nn { mcd , mcm }
2063 {
2064   \keys_define:nn { spintenc / sp #1 }
2065   {
2066     result      .bool_set:c = { \__spintenc_sp #1 _print_result_bool },
2067     result      .initial:n = false,
2068     result      .value_required:n = true,
2069     prefix      .code:n =
2070       { \__spintenc_sanitizet_affix:cn { \__spintenc_sp #1 _prefix_tl } {##1} },
2071     prefix      .initial:n = ,
2072     prefix      .value_required:n = true,
2073     read-cmd    .choices:nn =
2074       { terse , clear }
2075     {
2076       \int_case:nn { \_keys_choice_int }
2077       {
2078         {1} { \bool_set_true:c { \__spintenc_sp #1 _read_terse_bool } }
2079         {2} { \bool_set_false:c { \__spintenc_sp #1 _read_terse_bool } }
2080       }
2081     },
2082     read-cmd    .initial:n = clear,
2083     read-cmd    .value_required:n = true,
2084     read-cmd / unknown .code:n =
2085       \msg_error:nneee { spintenc } { unknown-choice }
2086       { read-cmd } { terse , ~clear } { \exp_not:n {##1} },
2087     upper-case  .bool_set:c = { \__spintenc_sp #1 _print_upper_bool },
2088     upper-case  .initial:n = false,
2089     upper-case  .value_required:n = true,
2090     concept     .bool_set:c = { \__spintenc_sp #1 _concept_bool },
2091     concept     .initial:n = true,
```

```

2092     concept                .value_required:n = true,
2093     silent                 .bool_set:c = { l__spintent_sp #1 _silent_bool },
2094     silent                 .initial:n = false,
2095     silent                 .value_required:n = true,
2096     unknown                .code:n =
2097     {
2098         \str_set:Nn \l__spintent_command_name_str { sp #1 }
2099         \__spintent_unknown_keys_cmd:n {##1}
2100     },
2101 }
2102 }

```

(End of definition for *result* and others.)

12.26.4 Internal functions for \spmcd and \spmcm

__spintent_mc_process_args:nn

The function `\__spintent_mc_process_args:nn` parses the operation ‘#1’ and its raw argument list ‘#2’. Calls `\__spintent_luafun_mc_classify_args:n`, splits ‘#2’ into `\l__spintent_mc_raw_args_seq` and `\l__spintent_mc_luaset_types_clist` into `\l__spintent_mc_types_seq`, then walks the former by position, fetching the matching type from the latter through `\seq_item:Nn`, to build `\l__spintent_mc_args_seq`: each argument wrapped through `\spnum` when its type is *number*, verbatim otherwise. Injects MathML intents around the already-formatted `\l__spintent_mc_args_seq`.

```

2103 \cs_new_protected:Nn \__spintent_mc_process_args:nn
2104 {
2105     \__spintent_luafun_mc_classify_args:n {#2}
2106     \seq_set_from_clist:Nn \l__spintent_mc_raw_args_seq {#2}
2107     \seq_set_from_clist:NN \l__spintent_mc_types_seq \l__spintent_mc_luaset_types_clist
2108     \seq_clear:N \l__spintent_mc_args_seq
2109     \int_zero:N \l__spintent_mc_idx_int
2110     \seq_map_inline:Nn \l__spintent_mc_raw_args_seq
2111     {
2112         \int_incr:N \l__spintent_mc_idx_int
2113         \str_if_eq:eeTF
2114         { \seq_item:Nn \l__spintent_mc_types_seq { \l__spintent_mc_idx_int } } { number }
2115         {
2116             \seq_put_right:Nn \l__spintent_mc_args_seq { \spnum {##1} }
2117         }
2118         { \seq_put_right:Nn \l__spintent_mc_args_seq {##1} }
2119     }

```

Injects MathML intents around the already-formatted `\l__spintent_mc_args_seq`.

```

2120     \bool_if:cTF { l__spintent_sp #1 _silent_bool }
2121     {
2122         \seq_set_map:Nnn \l__spintent_mc_args_seq \l__spintent_mc_args_seq
2123         {
2124             \MathMLintent { _- } { \__spintent_inner_sep: ##1 }
2125         }
2126         \MathMLintent { :silent } { \c__spintent_invisible_sep_int } % prevent "por"
2127         \MathMLintent { :silent } { ( }
2128         \MathMLintent { :silent } { \c__spintent_invisible_sep_int } % prevent "intervalo"
2129         \seq_use:Nnnn \l__spintent_mc_args_seq
2130         { \MathMLintent { :silent } { , } }
2131         { \MathMLintent { :silent } { , } }
2132         { \MathMLintent { :silent } { , } }
2133     }
2134     {
2135         \MathMLintent { :silent } { \c__spintent_invisible_sep_int } % prevent doble "de"
2136         \bool_if:cTF { l__spintent_sp #1 _concept_bool }
2137         {
2138             \MathMLintent { _entre } { ( }
2139         }
2140         {
2141             \MathMLintent { :silent } { ( }
2142         }
2143         \MathMLintent { :silent } { \c__spintent_invisible_sep_int } % prevent "intervalo"
2144         \seq_use:Nnnn \l__spintent_mc_args_seq
2145         {
2146             \MathMLintent { :silent } { , }
2147             \MathMLintent { _y- } { \c__spintent_invisible_sep_int } % "y" medio
2148         }
2149         {
2150             \MathMLintent { :silent } { , }

```

```

2151     }
2152     {
2153         \MathMLintent { :silent } { , }
2154         \MathMLintent { _y- } { \c__spintent_invisible_sep_int } % "y" final
2155     }
2156 }
2157 \MathMLintent { :silent } { ) }
2158 }

```

(End of definition for `\__spintent_mc_process_args:nn`.)

`\__spintent_spmc_process:nnnn`

The internal function `\__spintent_spmc_process:nnnn` is the shared handler for both `\spmc` and `\spmc`. Takes the short identifier in `#1`, its full reading string in `#2`, the local `(keys)` in `#3`, and the optional arguments in `#4`.

The operator name is silenced with `silent=true` — `\__spintent_mc_process_args:nn` silences the arguments too, in that case, each wrapped in a literal `_-` intent, and the printed result is silenced as well — or with `concept=false` alone, leaving the arguments untouched. With `concept=true`, it carries the literal intent instead — `#1` or `#2` per `read-cmd`, prefixed with `prefix` when given.

With `#4` given, classifies it through `\__spintent_luafun_mc_classify_args:n`. Only when every argument is a number does it call the Lua calculation (`calculate_#1`), reporting `mc-invalid-arguments` on failure, and, when `result` is `true`, printing the computed value. `\__spintent_mc_process_args:n` prints the argument list itself either way.

- We should use `\mathrm` instead of `\operatorname`, which generates unwanted spacing when `\c__spintent_invisible_sep_int` is applied.

```

2159 \cs_new_protected:Nn \__spintent_spmc_process:nnnn
2160 {
2161     \keys_set:nn { spintent / sp #1 } { #3 }
2162     \bool_if:cTF { \__spintent_sp #1 _silent_bool }
2163     {
2164         \MathMLintent { :silent }
2165         {
2166             \mathrm
2167             {
2168                 \bool_if:cTF { \__spintent_sp #1 _print_upper_bool }
2169                 { \text_uppercase:n {#1} } { #1 }
2170             }
2171         }
2172     }
2173     {
2174         \bool_if:cTF { \__spintent_sp #1 _concept_bool }
2175         {
2176             \MathMLintent
2177             {
2178                 -
2179                 \tl_if_empty:cF { \__spintent_sp #1 _prefix_tl }
2180                 {
2181                     \tl_use:c { \__spintent_sp #1 _prefix_tl } -
2182                 }
2183                 \bool_if:cTF { \__spintent_sp #1 _read_terse_bool }
2184                 { #1 } { #2 }
2185             }
2186             {
2187                 \mathrm
2188                 {
2189                     \bool_if:cTF { \__spintent_sp #1 _print_upper_bool }
2190                     { \text_uppercase:n {#1} } { #1 }
2191                 }
2192             }
2193         }
2194     }
2195     \MathMLintent { :silent }
2196     {
2197         \mathrm
2198         {
2199             \bool_if:cTF { \__spintent_sp #1 _print_upper_bool }
2200             { \text_uppercase:n {#1} } { #1 }
2201         }
2202     }
2203 }
2204 }

```

With ‘#4’ given, classifies it through `\__spintent_luafun_mc_classify_args:n`. Only when every argument is a number does it call the Lua calculation (`calculate_#1`), reporting mc-invalid-arguments on failure — which should not happen, since every argument already passed classification — and, when `result` is `true`, printing the computed value. With any non-numeric argument, no calculation is attempted at all. Either way, `\__spintent_mc_process_args:n` prints the argument list itself.

```

2205 \tl_if_novalue:nF { #4 }
2206 {
2207   \seq_set_from_clist:Nn \l__spintent_mc_raw_args_seq {#4}
2208   \int_compare:nNnTF { \seq_count:N \l__spintent_mc_raw_args_seq } < { 2 }
2209   {
2210     \msg_error:nne { spintent } { mc-too-few-arguments } { \exp_not:c { sp #1 } }
2211   }
2212   {
2213     \__spintent_luafun_mc_classify_args:n {#4}
2214     \str_if_eq:VnT \l__spintent_mc_luaset_all_numeric_str { true }
2215     {
2216       \use:c { __spintent_luafun_calculate_ #1 :n } { #4 }
2217       \str_if_eq:VnT \l__spintent_spmcm_spmcd_luaset_error_str { true }
2218       {
2219         \msg_error:nne { spintent } { mc-invalid-arguments } { \exp_not:c { sp #1 } }
2220       }
2221     }
2222     \__spintent_mc_process_args:nn {#1} {#4}
2223     \str_if_eq:VnT \l__spintent_mc_luaset_all_numeric_str { true }
2224     {
2225       \bool_if:cT { l__spintent_sp #1 _print_result_bool }
2226       {
2227         \bool_if:cTF { l__spintent_sp #1 _silent_bool }
2228         {
2229           \MathMLint { :silent } { = }
2230           \MathMLint { _- }
2231           {
2232             \__spintent_inner_sep:
2233             \tl_use:c { l__spintent_sp #1 _luaset_ #1 _value_tl }
2234           }
2235         }
2236         {
2237           \MathMLint { _es-igual-a } { = }
2238           \tl_use:c { l__spintent_sp #1 _luaset_ #1 _value_tl }
2239         }
2240       }
2241     }
2242   }
2243 }
2244 }

```

(End of definition for `\__spintent_spmc_process:nnnn`.)

`\spmc` The commands `\spmc` and `\spmcm` act as the public interface wrappers for typesetting the greatest common divisor and least common multiple. They enforce math mode, open a local group, and defer execution to the main processing macro `\__spintent_spmc_process:nnnn`, passing along the appropriate reading strings and arguments.

```

2245 \NewDocumentCommand \spmc { 0{} d() }
2246 {
2247   \mode_if_math:TF
2248   {
2249     \group_begin:
2250     \__spintent_spmc_process:nnnn { mcd } { máximo-común-divisor } {#1} {#2}
2251     \group_end:
2252   }
2253   { \msg_error:nnn { spintent } { need-math-mode } { \spmc } }
2254 }
2255 \NewDocumentCommand \spmcm { 0{} d() }
2256 {
2257   \mode_if_math:TF
2258   {
2259     \group_begin:
2260     \__spintent_spmc_process:nnnn { mcm } { mínimo-común-múltiplo } {#1} {#2}
2261     \group_end:
2262   }
2263   { \msg_error:nnn { spintent } { need-math-mode } { \spmcm } }

```

```
2264 }
```

(End of definition for `\spmc` and `\spmcm`. These functions are documented on page 15.)

12.27 The commands `\spnM` and `\spnD`

The user-level commands `\spnM` and `\spnD` provide a semantic interface for the Multiples of a number (“*múltiplos*” in Spanish) and the Divisors of a number (“*divisores*” in Spanish). They delegate argument validation and calculation to the Lua module `spintent.lua`, constructing a valid MathCAT intent.

12.27.1 Definition of variables for `\spnM` and `\spnD`

Internal variables generated dynamically for multiples and divisors.

```
\__spintent_md_out_seq
\__spintent_spm_md_luaset_is_number_str
\__spintent_spm_spnD_luaset_error_str
\__spintent_spm_luaset_result_tl
\__spintent_spnD_luaset_result_tl
\__spintent_spm_luaset_is_truncated_str
\__spintent_spnD_luaset_is_truncated_str
\__spintent_spm_prefix_tl
\__spintent_spnD_prefix_tl
\__spintent_spm_print_result_num_tl
\__spintent_spnD_print_result_num_tl
```

```
2265 \seq_new:N \__spintent_md_out_seq
2266 \str_new:N \__spintent_spm_spnD_luaset_error_str
2267 \str_new:N \__spintent_spm_md_luaset_is_number_str
2268 \clist_map_inline:nn { nM , nD }
2269 {
2270   \tl_new:c { \__spintent_sp #1 _luaset_result_tl }
2271   \str_new:c { \__spintent_sp #1 _luaset_is_truncated_str }
2272   \tl_new:c { \__spintent_sp #1 _prefix_tl }
2273 }
```

(End of definition for `\__spintent_md_out_seq` and others.)

12.27.2 Keys for `\spnM` and `\spnD`

We define the shared *keys* `result`, `silent`, `result-num`, `prefix` and `concept`.

```
result
silent
result-num
prefix
concept
unknown
```

```
2274 \clist_map_inline:nn { nM , nD }
2275 {
2276   \keys_define:nn { spintent / sp #1 }
2277   {
2278     result      .bool_set:c = { \__spintent_sp #1 _print_result_bool },
2279     result      .initial:n = false,
2280     result      .value_required:n = true,
2281     silent      .bool_set:c = { \__spintent_sp #1 _silent_bool },
2282     silent      .initial:n = false,
2283     silent      .value_required:n = true,
2284     result-num  .tl_set:c = { \__spintent_sp #1 _print_result_num_tl },
2285     result-num  .initial:n = ,
2286     prefix      .code:n =
2287       { \__spintent_sanitize_affix:cn { \__spintent_sp #1 _prefix_tl } {##1} },
2288     prefix      .initial:n = ,
2289     prefix      .value_required:n = true,
2290     concept     .bool_set:c = { \__spintent_sp #1 _concept_bool },
2291     concept     .initial:n = true,
2292     concept     .value_required:n = true,
2293     unknown     .code:n =
2294       {
2295         \str_set:Nn \__spintent_command_name_str { sp #1 }
2296         \__spintent_unknown_keys_cmd:n {##1}
2297       },
2298   }
2299 }
```

(End of definition for `result` and others.)

12.27.3 Definition of error messages for `\spnM` and `\spnD`

Exception triggered when the argument for multiples or divisors is invalid.

```
md-invalid-argument
mc-too-few-arguments
```

```
2300 \msg_new:nnnn { spintent } { md-invalid-argument }
2301 {
2302   Invalid~argument~provided~to~#1~\msg_line_context:.
2303 }
2304 {
2305   The~argument~must~be~a~single~positive~integer. \\\
2306 }
```

Exception triggered when fewer than two arguments are provided.

```
2307 \msg_new:nnnn { spintent } { mc-too-few-arguments }
2308 {
2309   Too~few~arguments~provided~to~#1~\msg_line_context:.
2310 }
2311 {
2312   At~least~two~arguments~are~required. \\\
2313 }
```

(End of definition for `md-invalid-argument` and `mc-too-few-arguments`.)

12.27.4 Internal functions for `\spnM` and `\spnD`

`\__spintent_spnM_spnD_process:nnnn`

The internal function `\__spintent_spnM_spnD_process:nnnn` acts as the primary entry point for the multiples and divisors commands. It takes the command identifier `#1`, its printed mathematical symbol `#2`, its base reading string `#3`, the local key-value options `#4`, and the optional argument `#5`. Its sole purpose is to evaluate and set the provided keys within the current scope before delegating the core branching and typesetting logic to `\__spintent_spn_M_and_D_exec:nnnn`.

```
2314 \cs_new_protected:Nn \__spintent_spnM_spnD_process:nnnn
2315 {
2316   \keys_set:nn { spintent / sp #1 } { #4 }
2317   \__spintent_spnM_spnD_exec:nnnn { #1 } { #2 } { #3 } { #5 }
2318 }
```

(End of definition for `\__spintent_spnM_spnD_process:nnnn`.)

`\__spintent_spnM_spnD_intent:nn`

The internal function `\__spintent_spnM_spnD_intent:nn` resolves the main reading intent string. It takes the command identifier `#1` (such as “nM” or “nD”) and the base reading string `#2`. It automatically prepends the localized prefix if defined, normalizes accentuation for “*múltiplos*”, and appends `-de`.

```
2319 \cs_new:Nn \__spintent_spnM_spnD_intent:nn
2320 {
2321   -
2322   \tl_if_empty:cF { \__spintent_sp #1 _prefix_tl }
2323   { \tl_use:c { \__spintent_sp #1 _prefix_tl } - }
2324   \str_case:nnF { #2 } { { múltiplos } { múltiplos } }
2325   { #2 }
2326   -de
2327 }
```

(End of definition for `\__spintent_spnM_spnD_intent:nn`.)

`\__spintent_spnM_spnD_silent:nn`

The internal function `\__spintent_spn_M_and_D_silent:nn` prints the atomized notation structure in full silent mode. It takes the mathematical symbol `#1` (such as “D” or “M”) and the variable or explicit argument `#2`. Every component is wrapped in a silent intent, and an invisible separator is injected between the parentheses to prevent assistive technologies from applying default contextual reading rules. The argument is wrapped in a literal `_-` intent, not in `:silent`, so that intents of its own (such as those of `\spnum`) are silenced too.

```
2328 \cs_new_protected:Nn \__spintent_spnM_spnD_silent:nn
2329 {
2330   \MathMLintent { :silent } { \operatorname { #1 } }
2331   \MathMLintent { :silent } { ( }
2332   \MathMLintent { :silent } { \c__spintent_invisible_sep_int } % prevent doble "de"
2333   \MathMLintent { _- } { \__spintent_inner_sep: #2 }
2334   \MathMLintent { :silent } { ) }
2335 }
```

(End of definition for `\__spintent_spnM_spnD_silent:nn`.)

`\__spintent_spnM_spnD_result:nn`

The internal function `\__spintent_spn_M_and_D_result:nn` handles the calculation and accessible formatting of the computed mathematical set. It takes the command identifier `#1` and the classified numeric argument `#2`. It invokes the corresponding Lua backend calculation, reporting `md-invalid-argument` if that fails — a safety net beyond `\__spintent_luafun_spnmd_classify:n`’s own check, since the Lua calculation never trusted that check blindly. On success, it maps the resulting clist into a sequence, and formats the final printed output utilizing custom intents for both separators and conjunctions.

```
2336 \cs_new_protected:Nn \__spintent_spnM_spnD_result:nn
2337 {
2338   \use:c { __spintent_luafun_calcular_ #1 :nn } { #2 }
2339   {
2340     \tl_use:c { \__spintent_sp #1 _print_result_num_tl }
2341   }
2342   \str_if_eq:VnTF \__spintent_spnM_spnD_luaset_error_str { true }
2343   {
2344     \msg_error:nne { spintent } { md-invalid-argument } { \exp_not:c { sp #1 } }
2345   }
2346   {
2347     \MathMLintent { :silent } { \c__spintent_invisible_sep_int } % prevent "por"
2348     \MathMLintent { _son } { = \{ }
2349     \seq_set_from_clist:Ne
2350     \l__spintent_md_out_seq { \tl_use:c { \__spintent_sp #1 _luaset_result_tl } }
2351     \seq_use:Nnnn \l__spintent_md_out_seq
```

```

2352     {
2353       \MathMLintent { :silent } { , }
2354       \MathMLintent { _y- } { \c__spintent_invisible_sep_int } % "y" (solo dos)
2355     }
2356     { \MathMLintent { :silent } { , } }
2357     {
2358       \MathMLintent { :silent } { , }
2359       \str_if_eq:vnT { l__spintent_sp #1 _luaset_is_truncated_str } { false }
2360       {
2361         \MathMLintent { _y- } { \c__spintent_invisible_sep_int } % "y" final
2362       }
2363     }
2364     \str_if_eq:vnT { l__spintent_sp #1 _luaset_is_truncated_str } { true }
2365     {
2366       \MathMLintent { :silent } { , } ~ \dots
2367     }
2368     \MathMLintent { :silent } { \} }
2369   }
2370 }

```

(End of definition for `\__spintent_spnM_spnD_result:nn`.)

`\__spintent_spnM_spnD_exec:nnnn`

The function `\__spintent_spnM_spnD_exec:nnnn` takes the command name in ‘#1’, the symbol in ‘#2’, the base reading string in ‘#3’, and the optional argument in ‘#4’. The parentheses are always silenced.

With ‘#4’ absent, the placeholder `n` stands in for the argument. With ‘#4’ given, `\__spintent_luafun_spnmd_classify:n` decides whether it is wrapped through `\spnum`—and, when it is, whether `\__spintent_spnM_spnD_result:nn` also prints the computed set, per `result`—or left verbatim, with no result attempted. A zero is flagged as an error by `\__spintent_luafun_spnmd_classify:n` and rejected right away with `md-invalid-argument`. With `silent=true` the printed result is wrapped in a literal `_-` intent, so it stays on the page but is not verbalized.

Either way, `silent` silences the whole thing through `\__spintent_spnM_spnD_silent:nn`; otherwise `concept=true` carries the literal intent from `\__spintent_spnM_spnD_intent:nn`, `false` silences only the operator symbol, leaving the argument to read normally.

```

2371 \cs_new_protected:Nn \__spintent_spnM_spnD_exec:nnnn
2372 {
2373   \tl_if_novalue:nTF { #4 }
2374   {
2375     \bool_if:cTF { l__spintent_sp #1 _silent_bool }
2376     { \__spintent_spnM_spnD_silent:nn { #2 } { n } }
2377     {
2378       \bool_if:cTF { l__spintent_sp #1 _concept_bool }
2379       {
2380         \MathMLintent
2381         { \__spintent_spnM_spnD_intent:nn { #1 } { #3 } }
2382         { \operatorname { #2 } }
2383       }
2384       { \MathMLintent { :silent } { \operatorname { #2 } } }
2385       \MathMLintent { :silent } { ( }
2386       \MathMLintent { :silent } { \c__spintent_invisible_sep_int } % prevent doble "de"
2387       n
2388       \MathMLintent { :silent } { ) }
2389     }
2390   }
2391   {
2392     \__spintent_luafun_spnmd_classify:n { \exp_not:n { #4 } }
2393     \str_if_eq:VnT \__spintent_spnM_spnD_luaset_error_str { true }
2394     { \msg_error:nne { spintent } { md-invalid-argument } { \exp_not:c { sp #1 } } }
2395     \bool_if:cTF { l__spintent_sp #1 _silent_bool }
2396     {
2397       \str_if_eq:VnTF \__spintent_spnmd_luaset_is_number_str { true }
2398       { \__spintent_spnM_spnD_silent:nn { #2 } { \spnum { #4 } } }
2399       { \__spintent_spnM_spnD_silent:nn { #2 } { #4 } }
2400     }
2401     {
2402       \bool_if:cTF { l__spintent_sp #1 _concept_bool }
2403       {
2404         \MathMLintent
2405         { \__spintent_spnM_spnD_intent:nn { #1 } { #3 } }
2406         { \operatorname { #2 } }
2407       }

```

```

2408         { \MathMLintent { :silent } { \operatorname { #2 } } }
2409     \MathMLintent { :silent } { ( }
2410     \MathMLintent { :silent } { \c__spintent_invisible_sep_int } % prevent doble "de"
2411     \str_if_eq:VnTF \l__spintent_spnmd_luaset_is_number_str { true }
2412     { \spnum { #4 } }
2413     { #4 }
2414     \MathMLintent { :silent } { ) }
2415 }
2416 \str_if_eq:VnT \l__spintent_spnmd_luaset_is_number_str { true }
2417 {
2418     \bool_if:cT { l__spintent_sp #1 _print_result_bool }
2419     {
2420         \bool_if:cTF { l__spintent_sp #1 _silent_bool }
2421         {
2422             \MathMLintent { :silent } { \c__spintent_invisible_sep_int } % prevent "por"
2423             \MathMLintent { _- }
2424             {
2425                 \__spintent_inner_sep:
2426                 \__spintent_spnM_spnD_result:nn { #1 } { #4 }
2427             }
2428         }
2429         {
2430             \__spintent_spnM_spnD_result:nn { #1 } { #4 }
2431         }
2432     }
2433 }
2434 }
2435 }

```

(End of definition for `\__spintent_spnM_spnD_exec:nnnn`.)

`\spnM` The commands `\spnM` and `\spnD` act as the public interface wrappers for typesetting multiples and divisors, respectively. They enforce math mode, open a local group, and defer execution to the function `\__spintent_spnM_spnD_process:nnnnn`, passing along the optional *keys* ‘#1’ and the numeric *argument* ‘#2’.

```

2436 \NewDocumentCommand \spnM { 0{} d() }
2437 {
2438     \mode_if_math:TF
2439     {
2440         \group_begin:
2441         \__spintent_spnM_spnD_process:nnnnn { nM } { M } { multiplos } { #1 } { #2 }
2442         \group_end:
2443     }
2444     { \msg_error:nnn { spintent } { need-math-mode } { \spnM } }
2445 }
2446 \NewDocumentCommand \spnD { 0{} d() }
2447 {
2448     \mode_if_math:TF
2449     {
2450         \group_begin:
2451         \__spintent_spnM_spnD_process:nnnnn { nD } { D } { divisores } { #1 } { #2 }
2452         \group_end:
2453     }
2454     { \msg_error:nnn { spintent } { need-math-mode } { \spnD } }
2455 }

```

(End of definition for `\spnM` and `\spnD`. These functions are documented on page 15.)

12.28 The command `\sprazon`

The user command `\sprazon` typesets a ratio $a:b$ as a fraction (`\dfrac` or `\frac`, per `use-dfrac`), read natively — no Lua parsing involved.

12.28.1 Definition of keys for `\sprazon`

We define the *keys* `silent`, `use-dfrac` and `inline`.

```

2456 \keys_define:nn { spintent / sprazon }
2457 {
2458     silent .bool_set:N = \l__spintent_sprazon_silent_bool,
2459     silent .initial:n = false,
2460     silent .value_required:n = true,
2461     use-dfrac .bool_set:N = \l__spintent_sprazon_use_dfrac_bool,
2462     use-dfrac .initial:n = false,

```

```

2463     use-dfrac    .value_required:n = true,
2464     inline      .bool_set:N = \l__spintent_sprazon_inline_bool,
2465     inline      .initial:n = false,
2466     inline      .value_required:n = true,
2467     unknown     .code:n =
2468     {
2469         \str_set:Nn \l__spintent_command_name_str { sprazon }
2470         \__spintent_unknown_keys_cmd:n {#1}
2471     },
2472 }

```

(End of definition for *silent* and others.)

`\__spintent_sprazon_exec:nnn` The function `\__spintent_sprazon_exec:nnn` receives the optional $\langle keys \rangle$ in ‘#1’, the numerator in ‘#2’ and the denominator in ‘#3’. Under *silent*, the whole thing is silenced as one unit — `\frac/\dfrac` (per `use-dfrac`) normally, or `#2 : #3` under `inline`.

Otherwise, with `inline=false` (the default), numerator and denominator are each tagged through a native `\MathMLarg` (num/denom) inside `\frac/\dfrac`, and the outer `\MathMLintent` reads `_es-a:infix($num,$denom)`. With `inline=true`, the same two references are tagged side by side instead — no `\frac/\dfrac` involved — with the `:` between them carrying the `_es-a:infix($num,$denom)` intent on its own.

```

2473 \cs_new_protected:Nn \__spintent_sprazon_exec:nnn
2474 {
2475     \keys_set:nn { spintent / sprazon } { #1 }
2476     \__spintent_invisible_sep:
2477     \bool_if:NTF \l__spintent_sprazon_silent_bool
2478     {
2479         \MathMLintent { _- }
2480         {
2481             \__spintent_inner_sep:
2482             \bool_if:NTF \l__spintent_sprazon_inline_bool
2483             { #2 : #3 }
2484             {
2485                 \bool_if:NTF \l__spintent_sprazon_use_dfrac_bool
2486                 { \dfrac { #2 } { #3 } }
2487                 { \frac { #2 } { #3 } }
2488             }
2489         }
2490     }
2491     {
2492         \bool_if:NTF \l__spintent_sprazon_inline_bool
2493         {
2494             \MathMLintent { _es-a:infix($num,$denom) }
2495             {
2496                 \MathMLarg { num } { #2 }
2497                 :
2498                 \MathMLarg { denom } { #3 }
2499             }
2500         }
2501         {
2502             \MathMLintent { _es-a:infix($num,$denom) }
2503             {
2504                 \__spintent_inner_sep:
2505                 \bool_if:NTF \l__spintent_sprazon_use_dfrac_bool
2506                 {
2507                     \dfrac { \MathMLarg { num } { #2 } } { \MathMLarg { denom } { #3 } }
2508                 }
2509                 { \frac { \MathMLarg { num } { #2 } } { \MathMLarg { denom } { #3 } } }
2510             }
2511         }
2512     }
2513 }

```

(End of definition for `\__spintent_sprazon_exec:nnn`.)

`\sprazon` The public command `\sprazon` enforces *math mode* and delegates execution to `\__spintent_sprazon_exec:nnn`.

```

2514 \NewDocumentCommand \sprazon { 0{} m m }
2515 {
2516     \mode_if_math:TF
2517     {
2518         \group_begin:
2519         \__spintent_sprazon_exec:nnn { #1 } { #2 } { #3 }

```

```

2520     \group_end:
2521   }
2522   { \msg_error:nnn { spintent } { need-math-mode } { \sprazon } }
2523 }

```

(End of definition for \sprazon. This function is documented on page 18.)

12.29 The command \spProp

The user command \spProp typesets a proportion $a:b = c:d$ as two fractions joined by $=$, reusing __spintent_sprazon_exec:nnn directly (not the public \sprazon command) for each ratio.

\spProp The public command \spProp enforces *math mode*, and forwards the same *⟨keys⟩* in ‘#1’ to both ratios: a/b (‘#2’/‘#3’) and c/d (‘#4’/‘#5’), joined by a native \MathMLintent reading _como over the $=$ sign.

```

2524 \NewDocumentCommand \spProp { O{} m m m m }
2525 {
2526   \mode_if_math:TF
2527   {
2528     \group_begin:
2529     \_\_spintent_sprazon_exec:nnn { #1 } { #2 } { #3 }
2530     \bool_if:NTF \_\_spintent_sprazon_silent_bool
2531     {
2532       \MathMLintent { :silent } { = }
2533     }
2534     { \MathMLintent { _como } { = } }
2535     \_\_spintent_sprazon_exec:nnn { #1 } { #4 } { #5 }
2536   \group_end:
2537   }
2538   { \msg_error:nnn { spintent } { need-math-mode } { \spProp } }
2539 }

```

(End of definition for \spProp. This function is documented on page 18.)

12.30 The command \spang

The command \spang is the main public command for typesetting and tagging sexagesimal angle expressions (degrees, minutes, and seconds). It parses the input using the Lua function __spintent_luafun_spang_parse:n. This separates the degrees, minutes, and seconds to construct a MathML intent tree with the correct postfixes for NVDA/MathCAT.

```

\_\_spintent_spang_luaset_grado_str
\_\_spintent_spang_luaset_minuto_str
\_\_spintent_spang_luaset_segundo_str
\_\_spintent_spang_luaset_error_str

```

These internal string variables store the values returned from the Lua module spintent.lua. They hold the extracted numerical values for degrees, minutes, and seconds, as well as the error status used during formatting.

```

2540 \str_new:N \_\_spintent_spang_luaset_grado_str
2541 \str_new:N \_\_spintent_spang_luaset_minuto_str
2542 \str_new:N \_\_spintent_spang_luaset_segundo_str
2543 \str_new:N \_\_spintent_spang_luaset_error_str

```

(End of definition for __spintent_spang_luaset_grado_str and others.)

12.30.1 Definition of error messages for \spang

invalid-angle-format The error message invalid-angle-format defines the text displayed when an invalid angle input is detected. It is triggered when an argument fails the format checks, does not match the expected sexagesimal pattern, or contains invalid measurement symbols.

```

2544 \msg_new:nnnn { spintent } { invalid-angle-format }
2545 { Invalid~angle~format~in~#1. }
2546 {
2547   The~argument~'~#2'~could~not~be~parsed. \
2548 }

```

(End of definition for invalid-angle-format.)

12.30.2 Internal functions for \spang

__spintent_spang_process:n The internal function __spintent_spang_process:n parses the angle input using __spintent_luafun_spang_parse:n after resetting the error flag in __spintent_spang_luaset_error_str. If an invalid format is detected, it issues the invalid-angle-format error. Otherwise, it checks the extracted string variables __spintent_spang_luaset_grado_str, __spintent_spang_luaset_minuto_str, and __spintent_spang_luaset_segundo_str. For each non-empty variable, it constructs the corresponding MathML intent structure with the correct measurement symbols and postfixes for NVDA/MathCAT. Each value is expanded with \exp_args:N before it reaches \spunit, which protects its *⟨argument⟩* from expansion.

```

2549 \cs_new_protected:Nn \_\_spintent_spang_process:n
2550 {

```

```

2551 \str_set:Nn \l__spintent_spang_luaset_error_str { false }
2552 \__spintent_luafun_spang_parse:n { #1 }
2553 \str_if_eq:VnTF \l__spintent_spang_luaset_error_str { true }
2554 {
2555   \msg_error:nnnn { spintent } { invalid-angle-format } { \spang } { #1 }
2556 }
2557 {
2558   \str_if_empty:NF \l__spintent_spang_luaset_grado_str
2559   {
2560     \exp_args:Ne \spunit { \l__spintent_spang_luaset_grado_str ° }
2561   }
2562   \str_if_empty:NF \l__spintent_spang_luaset_minuto_str
2563   {
2564     \MathMLintend { _minutos:postfix($m) }
2565     {
2566       \MathMLarg { m }
2567       {
2568         \exp_args:Ne \spunit { \l__spintent_spang_luaset_minuto_str ' }
2569       }
2570     }
2571   }
2572   \str_if_empty:NF \l__spintent_spang_luaset_segundo_str
2573   {
2574     \MathMLintend { _segundos:postfix($s) }
2575     {
2576       \MathMLarg { s }
2577       {
2578         \exp_args:Ne \spunit { \l__spintent_spang_luaset_segundo_str " }
2579       }
2580     }
2581   }
2582 }
2583 }

```

(End of definition for `\__spintent_spang_process:n`.)

`\spang` The command `\spang` is the public interface for formatting sexagesimal angle expressions. It first checks if it is being used in math mode, issuing an error if not. Inside a local group, it calls `\__spintent_spang_process:n` to process the $\langle argument \rangle$.

```

2584 \NewDocumentCommand \spang { m }
2585 {
2586   \mode_if_math:TF
2587   {
2588     \group_begin:
2589     \__spintent_spang_process:n { #1 }
2590     \group_end:
2591   }
2592   { \msg_error:nnn { spintent } { need-math-mode } { \spang } }
2593 }

```

(End of definition for `\spang`. This function is documented on page 24.)

12.31 The command `\spnZ`

The user-level command `\spnZ` formats integers (the set $\mathbb{Z}$). It includes specific options to automatically enclose numbers (particularly negative ones) in visual parentheses and allows you to control whether these parentheses are read by MathCAT.

This command verifies that the $\langle argument \rangle$ is indeed an *integer*, checking for the complete absence of decimal separators or attached units of measurement.

12.31.1 Definition of keys for `\spnZ`

Now we define the $\langle keys \rangle$ `cifras-sep` and `read-sign`. These are `.meta:nn` $\langle keys \rangle$ passed directly to the `\spnum` command. We also introduce `wrap-parens`, which determines whether the number is enclosed in parentheses, and `read-parens`, which controls if these parentheses are read by MathCAT.

```

2594 \keys_define:nn { spintent / spnZ }
2595 {
2596   cifras-sep .meta:nn = { spintent / spnum } { cifras-sep = {#1} },
2597   read-sign .meta:nn = { spintent / spnum } { read-sign = {#1} },
2598   wrap-parens .bool_set:N = \l__spintent_spnz_wrap_parens_bool,
2599   wrap-parens .initial:n = false,
2600   wrap-parens .value_required:n = true,

```

```

2601     read-parens .bool_set:N = \l__spintent_spnz_read_parens_bool,
2602     read-parens .initial:n = true,
2603     read-parens .value_required:n = true,
2604     unknown .code:n =
2605     {
2606         \str_set:Nn \l__spintent_command_name_str { spnZ }
2607         \__spintent_unknown_keys_cmd:n {#1}
2608     },
2609 }

```

(End of definition for *cifras-sep* and others.)

12.31.2 Definition of error messages

spnz-not-integer

Raised when the $\langle \textit{argument} \rangle$ contains decimal separators or attached units, which conflict with the constraints of a pure integer representation.

```

2610 \msg_new:nnnn { spintent } { spnz-not-integer }
2611 { The~value~'!#1'~is~not~a~valid~integer. }
2612 {
2613     Command~\token_to_str:N \spnZ~only~accepts~integers.\\
2614 }

```

(End of definition for *spnz-not-integer*.)

12.31.3 Internal functions for \spnZ

__spintent_spnz_render_parens:

The function `\__spintent_spnz_render_parens:` first checks the state of the boolean variable `\l__spintent_spnz_read_parens` (controlled by the `read-parens` key). If it is `true`, it prints the number surrounded by parentheses. Otherwise, it applies the `:silent function intent` to the parentheses while printing the number.

```

2615 \cs_new_protected:Nn \__spintent_spnz_render_parens:
2616 {
2617     \bool_if:NTF \l__spintent_spnz_read_parens_bool
2618     {
2619         ( \__spintent_spnum_print_num_start: )
2620     }
2621     {
2622         \MathMLintent { :silent } { ( }
2623         \__spintent_spnum_print_num_start:
2624         \MathMLintent { :silent } { ) }
2625     }
2626 }

```

(End of definition for `\__spintent_spnz_render_parens:`.)

__spintent_spnz_process:nn

The function `\__spintent_spnz_process:nn` first processes the $\langle \textit{keys} \rangle$ passed in ‘#1’ and then calls `\__spintent_luafun_clean_split_and_set:n` on ‘#2’. It verifies that the input is a valid integer by checking if the variable `\l__spintent_luaset_has_integer_str` is `true`. If not `true`, an “error” is raised.

If it is `true`, it checks whether `\l__spintent_luaset_part_int_tl` is *empty* and raises an “error” if so. Otherwise, it checks `\l__spintent_spnz_wrap_parens_bool` set by `wrap-parens` and calls either the function `\__spintent_spnz_render_parens:` or `\__spintent_spnum_print_num_start:`.

```

2627 \cs_new_protected:Nn \__spintent_spnz_process:nn
2628 {
2629     \keys_set:nn { spintent / spnZ } {#1}
2630     \__spintent_luafun_clean_split_and_set:n { \exp_not:n {#2} }
2631     \str_if_eq:VnTF \l__spintent_luaset_has_integer_str { true }
2632     {
2633         \tl_if_empty:NTF \l__spintent_luaset_part_int_tl
2634         { \msg_error:nnn { spintent } { spnum-no-digits } {#2} }
2635         {
2636             \bool_if:NTF \l__spintent_spnz_wrap_parens_bool
2637             { \__spintent_spnz_render_parens: }
2638             { \__spintent_spnum_print_num_start: }
2639         }
2640     }
2641     { \msg_error:nnn { spintent } { spnz-not-integer } {#2} }
2642 }

```

(End of definition for `\__spintent_spnz_process:nn`.)

`\spnZ` Defines the user command `\spnZ`, which is executed within a *group* and strictly enforces *math mode*.

```

2643 \NewDocumentCommand \spnZ { O{} m }
2644 {
2645   \mode_if_math:TF
2646   {
2647     \group_begin:
2648     \__spintent_spnz_process:nn { #1 } { #2 }
2649     \group_end:
2650   }
2651   { \msg_error:nnn { spintent } { need-math-mode } { \spnZ } }
2652 }

```

(End of definition for `\spnZ`. This function is documented on page 17.)

12.32 The command `\spmQ`

The command `\spmQ` typesets “mixed numbers” within Spanish mathematical contexts (e.g., $3\frac{1}{2}$). It delegates the scaling of the integer part to the Lua module `spintent.lua` via `\__spintent_luafun_spmQ_scale_int:Vn`, which measures both the integer glyph and the fraction box at the node level and computes a scaling factor to align their heights visually. Finally, it constructs the correct MathML intent for MathCAT.

12.32.1 Definition of variables for `\spmQ`

```

\l__spintent_spmQ_join_word_str
\l__spintent_spmQ_intent_read_sign_str
\l__spintent_spmQ_wrap_parens_l_tl
\l__spintent_spmQ_wrap_parens_r_tl
\l__spintent_saved_luamml_flag_int

```

The variable `\l__spintent_spmQ_join_word_str` stores the value set by the key `join-word` which will be verbalized by MathCAT as a grammatical connector between the integer and fractional parts.

The variable `\l__spintent_spmQ_intent_read_sign_str` stores the `:intent` passed to `\MathMLintent` when the signs ‘+’ or ‘-’ are present in the integer part, used by the function `\__spintent_spmQ_print_scale_int:n`.

The variables `\l__spintent_spmQ_wrap_parens_l_tl` and `\l__spintent_spmQ_wrap_parens_r_tl` store the sized parentheses set by the function `\__spintent_spmQ_set_wrap_parens_size:` and used by the key `wrap-parens`.

The integer variable `\l__spintent_saved_luamml_flag_int` temporarily stores the current value of `\l__luamml_flag_int` before disabling it during the injection of the scaled integer node, and restores it afterwards. This prevents `luamml` from generating a spurious `<math>` element around the `sub_box` processed in isolation, before `\frac` has expanded.

```

2653 \str_new:N \l__spintent_spmQ_join_word_str
2654 \str_new:N \l__spintent_spmQ_intent_read_sign_str
2655 \tl_new:N \l__spintent_spmQ_wrap_parens_l_tl
2656 \tl_new:N \l__spintent_spmQ_wrap_parens_r_tl
2657 \int_new:N \l__spintent_saved_luamml_flag_int

```

(End of definition for `\l__spintent_spmQ_join_word_str` and others.)

12.32.2 Definition of error messages for `\spmQ`

```

spmQ-zero-denominator
spmQ-not-integer

```

The error message `spmQ-zero-denominator` is raised when a zero denominator is passed to `\spmQ`. The second error `spmQ-not-integer` is returned when the integer part passed in `{\argument}` is not actually an integer.

```

2658 \msg_new:nnnn { spintent } { spmQ-zero-denominator }
2659 {
2660   Zero~denominator~in~\token_to_str:N \spmQ~\msg_line_context:.
2661 }
2662 {
2663   The~denominator~of~a~fraction~cannot~be~zero.\\
2664 }
2665 \msg_new:nnnn { spintent } { spmQ-not-integer }
2666 {
2667   The~value~'#1'~is~not~a~valid~integer~\msg_line_context:.
2668 }
2669 {
2670   Command~\token_to_str:N \spmQ \c_space_tl only~accepts~integers.\\
2671 }

```

(End of definition for `spmQ-zero-denominator` and `spmQ-not-integer`.)

12.32.3 Definition of keys for `\spmQ`

```

join-word
use-spnZ
wrap-parens
read-parens
use-dfrac
unknown

```

Now we add the keys `join-word`, `use-spnZ`, `wrap-parens`, `read-parens` and `use-dfrac`.

```

2672 \keys_define:nn { spintent / spmQ }
2673 {
2674   join-word .choices:nn =
2675   { entero , enteros , y }
2676   {
2677     \int_case:nn { \l_keys_choice_int }

```

```

2678     {
2679         {1} { \str_set:Nn \l__spintent_spmQ_join_word_str { entero } }
2680         {2} { \str_set:Nn \l__spintent_spmQ_join_word_str { enteros } }
2681         {3} { \str_set:Nn \l__spintent_spmQ_join_word_str { y } }
2682     }
2683 },
2684 join-word .initial:n = y,
2685 join-word .value_required:n = true,
2686 join-word/unknown .code:n =
2687     \msg_error:nneee { spintent } { unknown-choice }
2688     { join-word } { entero, ~enteros, ~y } { \exp_not:n {#1} },
2689 use-spnZ .bool_set:N = \l__spintent_spmQ_use_spnZ_bool,
2690 use-spnZ .initial:n = true,
2691 use-spnZ .value_required:n = true,
2692 use-dfrac .bool_set:N = \l__spintent_spmQ_use_dfrac_bool,
2693 use-dfrac .initial:n = false,
2694 use-dfrac .value_required:n = true,
2695 wrap-parens .bool_set:N = \l__spintent_spmQ_wrap_parens_bool,
2696 wrap-parens .initial:n = false,
2697 wrap-parens .value_required:n = true,
2698 read-parens .bool_set:N = \l__spintent_spmQ_read_parens_bool,
2699 read-parens .initial:n = true,
2700 read-parens .value_required:n = true,
2701 unknown .code:n =
2702     {
2703         \str_set:Nn \l__spintent_command_name_str { spmQ }
2704         \__spintent_unknown_keys_cmd:n {#1}
2705     },
2706 }

```

(End of definition for join-word and others.)

12.32.4 Internal functions for \spmQ

__spintent_spmQ_set_wrap_parens_size:

The function `\__spintent_spmQ_set_wrap_parens_size`: captures the current mathematical style using the primitive `\mathstyle` from LuaTeX and sets the variables `\l__spintent_spmQ_wrap_parens_l_tl` and `\l__spintent_spmQ_wrap_parens_r_tl` with the appropriate size for the parentheses. Display styles (0 and 1) receive `\Bigl\lparen` and `\Bigr\rparen`; text styles (2 and 3) receive `\bigl\lparen` and `\bigr\rparen`; any other style falls back to `\lparen` and `\rparen`.

If the key `use-dfrac` is active, the size is always forced to `\Bigl\lparen`/`\Bigr\rparen` regardless of the current mathematical style, since `\dfrac` always produces a display-height fraction.

```

2707 \cs_new_protected:Nn \__spintent_spmQ_set_wrap_parens_size:
2708 {
2709     \int_case:nnF { \mathstyle }
2710     {
2711         { 0 }
2712         {
2713             \tl_set:Nn \l__spintent_spmQ_wrap_parens_l_tl { \Bigl \lparen }
2714             \tl_set:Nn \l__spintent_spmQ_wrap_parens_r_tl { \Bigr \rparen }
2715         }
2716         { 1 }
2717         {
2718             \tl_set:Nn \l__spintent_spmQ_wrap_parens_l_tl { \Bigl \lparen }
2719             \tl_set:Nn \l__spintent_spmQ_wrap_parens_r_tl { \Bigr \rparen }
2720         }
2721         { 2 }
2722         {
2723             \tl_set:Nn \l__spintent_spmQ_wrap_parens_l_tl { \bigl \lparen }
2724             \tl_set:Nn \l__spintent_spmQ_wrap_parens_r_tl { \bigr \rparen }
2725         }
2726         { 3 }
2727         {
2728             \tl_set:Nn \l__spintent_spmQ_wrap_parens_l_tl { \bigl \lparen }
2729             \tl_set:Nn \l__spintent_spmQ_wrap_parens_r_tl { \bigr \rparen }
2730         }
2731     }
2732     {
2733         \tl_set:Nn \l__spintent_spmQ_wrap_parens_l_tl { \lparen }
2734         \tl_set:Nn \l__spintent_spmQ_wrap_parens_r_tl { \rparen }
2735     }

```

If the key `use-dfrac` is active, we must reset the variables `\l__spintenc_spmQ_wrap_parens_l_tl` and `\l__spintenc_spmQ_wrap_parens_r_tl` to `\Bigl\Bigr` unconditionally.

```

2736   \bool_if:NT \l__spintenc_spmQ_use_dfrac_bool
2737   {
2738     \tl_set:Nn \l__spintenc_spmQ_wrap_parens_l_tl { \Bigl \lparen }
2739     \tl_set:Nn \l__spintenc_spmQ_wrap_parens_r_tl { \Bigr \rparen }
2740   }
2741 }

```

(End of definition for `\__spintenc_spmQ_set_wrap_parens_size:`)

```

\__spintenc_spmQ_start_wrap_parens:
\__spintenc_spmQ_stop_wrap_parens:

```

The functions `\__spintenc_spmQ_start_wrap_parens:` and `\__spintenc_spmQ_stop_wrap_parens:` open and close the parentheses around the “mixed number” when the key `wrap-parens` is active. They first call `\__spintenc_spmQ_set_wrap_parens_size:` to determine the correct size for the current mathematical style, then print the appropriate parenthesis stored in `\l__spintenc_spmQ_wrap_parens_l_tl` or `\l__spintenc_spmQ_wrap_parens_r_tl`.

If the variable `\l__spintenc_spmQ_read_parens_bool`, controlled by the key `read-parens`, is *false*, each parenthesis is wrapped inside `\MathMLintent{:silent}` so that NVDA/MathCAT does not verbalize them.

```

2742 \cs_new_protected:Nn \__spintenc_spmQ_start_wrap_parens:
2743 {
2744   \bool_if:NT \l__spintenc_spmQ_wrap_parens_bool
2745   {
2746     \__spintenc_spmQ_set_wrap_parens_size:
2747     \bool_if:NTF \l__spintenc_spmQ_read_parens_bool
2748       { \l__spintenc_spmQ_wrap_parens_l_tl }
2749       { \MathMLintent { :silent } { \l__spintenc_spmQ_wrap_parens_l_tl } }
2750   }
2751 }
2752 \cs_new_protected:Nn \__spintenc_spmQ_stop_wrap_parens:
2753 {
2754   \bool_if:NT \l__spintenc_spmQ_wrap_parens_bool
2755   {
2756     \__spintenc_spmQ_set_wrap_parens_size:
2757     \bool_if:NTF \l__spintenc_spmQ_read_parens_bool
2758       { \l__spintenc_spmQ_wrap_parens_r_tl }
2759       { \MathMLintent { :silent } { \l__spintenc_spmQ_wrap_parens_r_tl } }
2760   }
2761 }

```

(End of definition for `\__spintenc_spmQ_start_wrap_parens:` and `\__spintenc_spmQ_stop_wrap_parens:`)

```
\__spintenc_spmQ_set_join_word:
```

The function `\__spintenc_spmQ_set_join_word:` injects the grammatical connector between the integer part and the fraction by dispatching on the value stored in `\l__spintenc_spmQ_join_word_str`, set by the key `join-word`. In all three cases the visual output is the Unicode invisible separator U+2063 via `\c__spintenc_invisible_sep_tl`, which produces a `<mi>` node in the MathML structure; what differs is the `:intent` text attached to it via `\MathMLintent`, which NVDA/MathCAT will verbalize as «entero», «enteros» or «y» respectively.

```

2762 \cs_new_protected:Nn \__spintenc_spmQ_set_join_word:
2763 {
2764   \str_case:Vn \l__spintenc_spmQ_join_word_str
2765   {
2766     { entero } { \MathMLintent { _entero } { \c__spintenc_invisible_sep_int } }
2767     { enteros } { \MathMLintent { _enteros } { \c__spintenc_invisible_sep_int } }
2768     { y } { \MathMLintent { _y- } { \c__spintenc_invisible_sep_int } }
2769   }
2770 }

```

(End of definition for `\__spintenc_spmQ_set_join_word:`)

```
\__spintenc_spmQ_print_scale_int:
```

The function `\__spintenc_spmQ_print_scale_int:` prints the integer part of the “mixed number” scaled to match the visual height of the accompanying fraction. Before calling the Lua function `\__spintenc_luafun_spmQ_sca` it temporarily disables `luamml` by saving the current value of `\l__luamml_flag_int` into `\l__spintenc_saved_luamml_flag_int` and setting it to 0 (skip completely). This prevents `luamml` from generating a spurious `<math>` wrapper around the `sub_box` node that holds the scaled integer, which is processed in isolation before `\frac` has expanded. Once the Lua function returns, the original flag value is restored via `\l__spintenc_saved_luamml_flag_int`.

The Lua function receives the integer digits from `\l__spintenc_luaset_part_int_tl` and the fraction command name (`\frac` or `\dfrac`, determined by the key `use-dfrac`) as its two arguments, and inserts the appropriately scaled `sub_box` node directly into the current horizontal list.

```
2771 \cs_new_protected:Nn \__spintenc_spmQ_print_scale_int:
```

```

2772 {
2773   \int_set_eq:Nc \l__spintrent_saved_luamml_flag_int { l__luamml_flag_int }
2774   \int_set:Nn \l__luamml_flag_int { 0 }
2775   \bool_if:NTF \l__spintrent_spmQ_use_dfrac_bool
2776     { \l__spintrent_luafun_spmQ_scale_int:Nn \l__spintrent_luaset_part_int_tl { dfrac } }
2777     { \l__spintrent_luafun_spmQ_scale_int:Nn \l__spintrent_luaset_part_int_tl { frac } }
2778   \int_set_eq:cN { l__luamml_flag_int } \l__spintrent_saved_luamml_flag_int
2779 }

```

(End of definition for `\l__spintrent_spmQ_print_scale_int:n`.)

`\l__spintrent_spmQ_print_scale_int:n` The function `\l__spintrent_spmQ_print_scale_int:n` takes the integer part of the “mixed number” passed in ‘#1’. It first calls `\l__spintrent_luafun_clean_split_and_set:n` on ‘#1’ to parse and populate the shared Lua variables, then checks the status of `\l__spintrent_luaset_has_integer_str`.

If the status is `true`, it inspects `\l__spintrent_luaset_sign_tl`. When the sign is *empty*, it delegates directly to `\l__spintrent_spmQ_print_scale_int:` to render the scaled integer. When a sign is present, it sets `\l__spintrent_spmQ_intent_read_sign_str` to either `_más:prefix($e)` or `_menos:prefix($e)` and wraps the sign glyph together with a MathML argument (`$e`) containing the scaled integer inside a `\MathMLintent`, so that NVDA/MathCAT verbalizes the sign as a prefix before the integer value. An `\l__spintrent_invisible_sep:` call between the sign and the `MathMLarg` forces the opening of a new `<mrow>` in the MathML structure, which is required for the intent tree to be well-formed.

If the status is `false`, the `spmQ-not-integer` “error” is raised immediately.

```

2780 \cs_new_protected:Nn \l__spintrent_spmQ_print_scale_int:n
2781 {
2782   \l__spintrent_luafun_clean_split_and_set:n { \exp_not:n {#1} }
2783   \str_if_eq:NnTF \l__spintrent_luaset_has_integer_str { true }
2784     {
2785       \tl_if_empty:NTF \l__spintrent_luaset_sign_tl
2786       {
2787         \l__spintrent_spmQ_print_scale_int:
2788       }
2789       {
2790         \str_if_eq:NnTF \l__spintrent_luaset_sign_tl { + }
2791         {
2792           \str_set:Nn
2793             \l__spintrent_spmQ_intent_read_sign_str { _más:prefix($e) }
2794         }
2795         {
2796           \str_set:Nn
2797             \l__spintrent_spmQ_intent_read_sign_str { _menos:prefix($e) }
2798         }
2799         \MathMLintent { \l__spintrent_spmQ_intent_read_sign_str }
2800         {
2801           \l__spintrent_luaset_sign_tl \c__spintrent_invisible_sep_int % \l__spintrent_invisible_s
2802           \MathMLarg { e } { \l__spintrent_spmQ_print_scale_int: }
2803         }
2804       }
2805     }
2806     { \msg_error:nnn { spintrent } { spmQ-not-integer } { #1 } }
2807 }

```

(End of definition for `\l__spintrent_spmQ_print_scale_int:n`.)

`\l__spintrent_spmQ_print_spnZ_dfrac:nn` The function `\l__spintrent_spmQ_print_spnZ_dfrac:nn` typesets the fractional part of the “mixed number”. It takes the numerator in ‘#1’ and the denominator in ‘#2’. It first calls `\l__spintrent_luafun_clean_split_and_set:n` on ‘#1’ to validate that the numerator is an integer; if `\l__spintrent_luaset_has_integer_str` is `false`, the `spmQ-not-integer` “error” is raised immediately.

If the validation passes, the function dispatches on the two boolean variables `\l__spintrent_spmQ_use_spnZ_bool` and `\l__spintrent_spmQ_use_dfrac_bool`, controlled respectively by the keys `use-spnZ` and `use-dfrac`. When `use-spnZ` is `true`, both the numerator and denominator are passed through `\spnZ` to gain MathML integer semantics; otherwise, the raw token lists ‘#1’ and ‘#2’ are used directly. In both branches, the fraction command is selected dynamically via `\use:c`: if `use-dfrac` is `true`, `dfrac` is assembled; otherwise, `frac` is used.

```

2808 \cs_new_protected:Nn \l__spintrent_spmQ_print_spnZ_dfrac:nn
2809 {
2810   \l__spintrent_luafun_clean_split_and_set:n { \exp_not:n {#1} }
2811   \str_if_eq:NnTF \l__spintrent_luaset_has_integer_str { false }
2812     {
2813       \msg_error:nnn { spintrent } { spmQ-not-integer } { #1 }

```

```

2814     }
2815     \bool_if:NTF \__spintest_spmQ_use_spnZ_bool
2816     {
2817         \use:c { \bool_if:NT \__spintest_spmQ_use_dfrac_bool { d } frac }
2818         { \spnZ { #1 } } { \spnZ { #2 } }
2819     }
2820     {
2821         \use:c { \bool_if:NT \__spintest_spmQ_use_dfrac_bool { d } frac }
2822         { #1 } { #2 }
2823     }
2824 }

```

(End of definition for `\__spintest_spmQ_print_spnZ_dfrac:nn`.)

`\__spintest_spmQ_print:nnn`

The function `\__spintest_spmQ_print:nnn` is the main rendering orchestrator for `\spmQ`. It takes three arguments: the integer part `#1`, the numerator `#2`, and the denominator `#3`. It opens the optional parentheses via `\__spintest_spmQ_start_wrap_parens:`, then calls `\__spintest_spmQ_print_scale_int:n` to scale and render the integer part. Next, it injects the grammatical connector between the integer and the fraction via `\__spintest_spmQ_set_join_word:`. Then it calls `\__spintest_spmQ_print_spnZ_dfrac:nn` to typeset the fraction using the keys `use-dfrac` and `use-spnZ`. Finally, it closes the optional parentheses via `\__spintest_spmQ_stop_wrap_parens:`.

```

2825 \cs_new_protected:Nn \__spintest_spmQ_print:nnn
2826 {
2827     \__spintest_spmQ_start_wrap_parens:
2828     \__spintest_spmQ_print_scale_int:n { #1 }
2829     \__spintest_spmQ_set_join_word:
2830     \__spintest_spmQ_print_spnZ_dfrac:nn { #2 } { #3 }
2831     \__spintest_spmQ_stop_wrap_parens:
2832 }

```

(End of definition for `\__spintest_spmQ_print:nnn`.)

`\__spintest_spmQ_exec:nnnn`

The internal function `\__spintest_spmQ_exec:nnnn` takes four arguments: the optional *(keys)* `#1`, the integer `#2`, the numerator `#3`, and the denominator `#4`. First sets the optional *(keys)* from `#1`, then processes the denominator input in `#4` using `\__spintest_luafun_clean_split_and_set:n` to validate it is a non-zero integer.

It checks the status of `\__spintest_luaaset_has_integer_str`. If `true` and the integer part is exactly zero, it issues the `spmQ-zero-denominator` “error”. Otherwise, whether the status is `false` or the integer part is *non-zero*, it calls `\__spintest_spmQ_print:nnn` to format the “mixed number”.

```

2833 \cs_new_protected:Nn \__spintest_spmQ_exec:nnnn
2834 {
2835     \keys_set:nn { spintest / spmQ } { #1 }
2836     \__spintest_luafun_clean_split_and_set:n { \exp_not:n { #4 } }
2837     \str_if_eq:VnTF \__spintest_luaaset_has_integer_str { true }
2838     {
2839         \int_compare:nNnTF { \__spintest_luaaset_part_int_tl } = { 0 }
2840         { \msg_error:nn { spintest } { spmQ-zero-denominator } }
2841         { \__spintest_spmQ_print:nnn { #2 } { #3 } { #4 } }
2842     }
2843     { \__spintest_spmQ_print:nnn { #2 } { #3 } { #4 } }
2844 }

```

(End of definition for `\__spintest_spmQ_exec:nnnn`.)

`\spmQ`

The command `\spmQ` is the public interface for formatting “mixed numbers”. It first checks if it is being used in *math mode*, issuing an “error” if not. Inside a local group, it calls `\__spintest_spmQ_exec:nnnn` to process the optional *(keys)* and the *{(arguments)}*.

```

2845 \NewDocumentCommand \spmQ { 0{} m m m }
2846 {
2847     \mode_if_math:TF
2848     {
2849         \group_begin:
2850         \__spintest_spmQ_exec:nnnn { #1 } { #2 } { #3 } { #4 }
2851         \group_end:
2852     }
2853     { \msg_error:nnn { spintest } { need-math-mode } { \spmQ } }
2854 }

```

(End of definition for `\spmQ`. This function is documented on page 17.)

12.33 The commands of the geo family

The commands described in this section provide a semantic interface for geometric objects in Spanish mathematical contexts. They all delegate parsing to `spintent.lua`, sharing the variables `\l__spintent_geo_luaset_error_str`, `\l__spintent_geo_luaset_intent_str`, `\l__spintent_geo_luaset_print_tl`, `\l__spintent_geo_luaset_sub_str` and `\l__spintent_geo_luaset_tipo_str` — constructing a valid MathCAT intent — and a single point is always annotated through `\l__spintent_mathmlarg_point:nnn`. Every command also activates `\l__spintent_scope_attr_int` within its own group, so the shared queue never claims content typeset outside a geo command in the same formula.

12.33.1 Definition of shared variables

Internal variables shared across all commands of the family to avoid repeated calls into Lua.

```

\l__spintent_geo_luaset_error_str
\l__spintent_geo_luaset_intent_str
\l__spintent_geo_luaset_print_tl
\l__spintent_geo_luaset_is_name_str
\l__spintent_geo_luaset_poly_sym_str
\l__spintent_geo_luaset_angulo_case_str
\l__spintent_geo_luaset_concept_str
\l__spintent_geo_luaset_letra_word_str
\l__spintent_geo_luaset_sub_spoken_str
\l__spintent_geo_luaset_nombre_es_num_str
\l__spintent_geo_luaset_letra_str
\l__spintent_geo_luaset_sub_str
\l__spintent_geo_luaset_tipo_str
2855 \str_new:N \l__spintent_geo_luaset_error_str
2856 \str_new:N \l__spintent_geo_luaset_intent_str
2857 \tl_new:N \l__spintent_geo_luaset_print_tl
2858 \clist_new:N \l__spintent_geo_luaset_points_clist
2859 \str_new:N \l__spintent_geo_luaset_is_name_str
2860 \str_new:N \l__spintent_geo_luaset_poly_sym_str
2861 \str_new:N \l__spintent_geo_luaset_angulo_case_str
2862 \str_new:N \l__spintent_geo_luaset_concept_str
2863 \str_new:N \l__spintent_geo_luaset_letra_word_str
2864 \str_new:N \l__spintent_geo_luaset_sub_spoken_str
2865 \str_new:N \l__spintent_geo_luaset_nombre_es_num_str
2866 \str_new:N \l__spintent_geo_luaset_letra_str
2867 \str_new:N \l__spintent_geo_luaset_sub_str
2868 \str_new:N \l__spintent_geo_luaset_tipo_str

```

(End of definition for `\l__spintent_geo_luaset_error_str` and others.)

12.34 The command `\spcoord`

The user command `\spcoord` typesets an ordered pair (x, y) , with the comma (or semicolon) between them carrying its own `:infix` intent, and each side wrapped with `:prefix($vp)/:prefix($vn)` when it starts with $+/-$, so the sign is always read as *más/menos* — never *positivo/negativo*, which is not how MathCAT reads a bare sign in this position.

12.34.1 Definition of variables for `\spcoord`

Variables set by the function `\l__spintent_luafun_spcoord_parse_and_set:n`. The string variable `\l__spintent_spcoord_luaset_sep_str` is `,` or `;`, whichever the argument actually used. For the abscissa and ordinate values, the variables `\l__spintent_spcoord_luaset_#1_sign_str` is `+`, `-`, or empty; the variables `\l__spintent_spcoord_luaset_#1_type_str` is `number` or `raw`; the variables `\l__spintent_spcoord_luaset_#1_value_str` is the value with the sign already removed; and the variables `\l__spintent_spcoord_luaset_#1_has_frac_str` is `none`, `frac`, or `dfrac`; and the variables `\l__spintent_spcoord_luaset_#1_composite_str` is `true` when the value is a sole subscripted, superscripted or radical node, `false` otherwise.

```

\l__spintent_spcoord_luaset_error_str
\l__spintent_spcoord_luaset_sep_str
\l__spintent_spcoord_luaset_x_sign_str
\l__spintent_spcoord_luaset_x_type_str
\l__spintent_spcoord_luaset_x_value_str
\l__spintent_spcoord_luaset_x_has_frac_str
\l__spintent_spcoord_luaset_y_sign_str
\l__spintent_spcoord_luaset_y_type_str
\l__spintent_spcoord_luaset_y_value_str
\l__spintent_spcoord_luaset_y_has_frac_str
2869 \str_new:N \l__spintent_spcoord_luaset_error_str
2870 \str_new:N \l__spintent_spcoord_luaset_sep_str
2871 \clist_map_inline:nn { x , y }
2872 {
2873   \str_new:c { \l__spintent_spcoord_luaset_#1_sign_str }
2874   \str_new:c { \l__spintent_spcoord_luaset_#1_type_str }
2875   \str_new:c { \l__spintent_spcoord_luaset_#1_value_str }
2876   \str_new:c { \l__spintent_spcoord_luaset_#1_has_frac_str }
2877   \str_new:c { \l__spintent_spcoord_luaset_#1_composite_str }
2878 }

```

(End of definition for `\l__spintent_spcoord_luaset_error_str` and others.)

The variable `\l__spintent_spcoord_raw_arg_tl` holds the `{(argument)}` exactly as given. The variable `\l__spintent_spcoord_intent_str` holds the `:infix` word for the separator `_coma:infix` or `_punto-y-coma:infix`. The variables `\l__spintent_spcoord_wrap_parens_l_tl`/`\l__spintent_spcoord_wrap_parens_r_tl` hold the opening/closing delimiter, sized by the function `\l__spintent_spcoord_set_wrap_parens_size:.`

```

2879 \tl_new:N \l__spintent_spcoord_raw_arg_tl
2880 \str_new:N \l__spintent_spcoord_intent_str
2881 \tl_new:N \l__spintent_spcoord_wrap_parens_l_tl
2882 \tl_new:N \l__spintent_spcoord_wrap_parens_r_tl

```

(End of definition for `\l__spintent_spcoord_raw_arg_tl` and others.)

12.34.2 Definition of error messages for \spcoord

spcoord-invalid-arg

```
2883 \msg_new:nnn { spintent } { spcoord-invalid-arg }
2884 {
2885   Invalid~argument~'#1'~for~\token_to_str:N \spcoord.
2886 }
```

(End of definition for spcoord-invalid-arg.)

12.34.3 Definition of keys for \spcoord

silent

We define the key `silent`.

```
2887 \keys_define:nn { spintent / spcoord }
2888 {
2889   silent .bool_set:N = \l__spintent_spcoord_silent_bool,
2890   silent .initial:n = false,
2891   silent .value_required:n = true,
2892   unknown .code:n =
2893     {
2894       \str_set:Nn \l__spintent_command_name_str { spcoord }
2895       \__spintent_unknown_keys_cmd:n {#1}
2896     },
2897 }
```

(End of definition for silent.)

12.34.4 Internal functions for \spcoord

__spintent_spcoord_set_wrap_parens_size:

With neither side carrying a fraction, the parentheses stay unsized. A fraction counts wherever it appears in the value (`\frac`, `\dfrac` or a `\spmQ`, which prints one), except inside a subscript or superscript, where it does not make the line taller. Otherwise, `\mathstyle` picks `\Bigl/\Bigr` for display, `\bigl/\bigr` for text. When either side is `\dfrac`, the parentheses are forced to `\Bigl/\Bigr` regardless of `\mathstyle`, since `\dfrac` always renders at display height.

```
2898 \cs_new_protected:Nn \__spintent_spcoord_set_wrap_parens_size:
2899 {
2900   \tl_set:Nn \l__spintent_spcoord_wrap_parens_l_tl { \lparen }
2901   \tl_set:Nn \l__spintent_spcoord_wrap_parens_r_tl { \rparen }
2902   \bool_if:nT
2903     {
2904       \bool_lazy_or_p:nn
2905         { \bool_not_p:n { \str_if_eq_p:Vn \l__spintent_spcoord_luaset_x_has_frac_str { none } } }
2906         { \bool_not_p:n { \str_if_eq_p:Vn \l__spintent_spcoord_luaset_y_has_frac_str { none } } }
2907     }
2908     {
2909       \int_case:nnF { \mathstyle }
2910       {
2911         { 0 }
2912         {
2913           \tl_set:Nn \l__spintent_spcoord_wrap_parens_l_tl { \Bigl \lparen }
2914           \tl_set:Nn \l__spintent_spcoord_wrap_parens_r_tl { \Bigr \rparen }
2915         }
2916         { 1 }
2917         {
2918           \tl_set:Nn \l__spintent_spcoord_wrap_parens_l_tl { \Bigl \lparen }
2919           \tl_set:Nn \l__spintent_spcoord_wrap_parens_r_tl { \Bigr \rparen }
2920         }
2921         { 2 }
2922         {
2923           \tl_set:Nn \l__spintent_spcoord_wrap_parens_l_tl { \bigl \lparen }
2924           \tl_set:Nn \l__spintent_spcoord_wrap_parens_r_tl { \bigr \rparen }
2925         }
2926         { 3 }
2927         {
2928           \tl_set:Nn \l__spintent_spcoord_wrap_parens_l_tl { \bigl \lparen }
2929           \tl_set:Nn \l__spintent_spcoord_wrap_parens_r_tl { \bigr \rparen }
2930         }
2931       }
2932     }
2933   \bool_if:nT
2934     {
2935       \bool_lazy_or_p:nn
2936         { \str_if_eq_p:Vn \l__spintent_spcoord_luaset_x_has_frac_str { dfrac } } }
```

```

2937         { \str_if_eq_p:Nn \__spintrent_spcoord_luaset_y_has_frac_str { dfraction } }
2938     }
2939     {
2940         \tl_set:Nn \__spintrent_spcoord_wrap_parens_l_tl { \Bigl \lparen }
2941         \tl_set:Nn \__spintrent_spcoord_wrap_parens_r_tl { \Bigr \rparen }
2942     }
2943 }
2944 }

```

(End of definition for `\__spintrent_spcoord_set_wrap_parens_size:n`.)

`\__spintrent_spcoord_value:n`

With `\__spintrent_spcoord_luaset_ #1 _type_str` equal to `number`, calls `\spnum` on `\__spintrent_spcoord_value:n`. Otherwise, typesets it as given — a `\frac{\dfrac{}` or a letter expression, untouched. Receives `x` or `y` in ‘#1’.

```

2945 \cs_new_protected:Nn \__spintrent_spcoord_value:n
2946 {
2947     \str_if_eq:eeTF
2948     { \str_use:c { \__spintrent_spcoord_luaset_ #1 _type_str } } { number }
2949     {
2950         \exp_args:Ne \spnum { \str_use:c { \__spintrent_spcoord_luaset_ #1 _value_str } }
2951     }
2952     { \str_use:c { \__spintrent_spcoord_luaset_ #1 _value_str } }
2953 }

```

(End of definition for `\__spintrent_spcoord_value:n`.)

`\__spintrent_spcoord_print:n`

With `\__spintrent_spcoord_luaset_ #1 _sign_str` equal to `+` or `-`, wraps the sign and `\__spintrent_spcoord_value:n` together with `_más:prefix($vp)/_menos:prefix($vn)`, referencing the value through `read` as `más/menos`. With no sign, delegates directly to `\__spintrent_spcoord_value:n`. Receives `x` or `y` in ‘#1’.

```

2954 \cs_new_protected:Nn \__spintrent_spcoord_print:n
2955 {
2956     \str_case:enF { \str_use:c { \__spintrent_spcoord_luaset_ #1 _sign_str } }
2957     {
2958         { + }
2959         {
2960             \MathMLint { _más:prefix ($vp) }
2961             {
2962                 \str_use:c { \__spintrent_spcoord_luaset_ #1 _sign_str }
2963                 \MathMLarg{ vp } { \__spintrent_spcoord_value:n { #1 } }
2964             }
2965         }
2966         { - }
2967         {
2968             \MathMLint { _menos :prefix($vn) }
2969             {
2970                 \str_use:c { \__spintrent_spcoord_luaset_ #1 _sign_str }
2971                 \MathMLarg{ vn } { \__spintrent_spcoord_value:n { #1 } }
2972             }
2973         }
2974     }
2975     { \__spintrent_spcoord_value:n { #1 } }
2976 }

```

(End of definition for `\__spintrent_spcoord_print:n`.)

`\__spintrent_spcoord_inner_sep:n`

Receives `x` or `y` in ‘#1’. With `\__spintrent_spcoord_luaset_ #1 _composite_str` equal to `true`, places `\__spintrent_inner_sep:` first, so that the group `\MathMLarg` wraps holds two children and keeps its `arg`: `luamml` drops it from a sole node that carries a subscript, a superscript or a radical.

```

2977 \cs_new_protected:Nn \__spintrent_spcoord_inner_sep:n
2978 {
2979     \str_if_eq:eeT
2980     { \str_use:c { \__spintrent_spcoord_luaset_ #1 _composite_str } } { true }
2981     { \__spintrent_inner_sep: }
2982 }

```

(End of definition for `\__spintrent_spcoord_inner_sep:n`.)

`\__spintrent_spcoord_intent_infix:`

Sets `\__spintrent_spcoord_intent_str` to `_coma:infix` or `_punto-y-coma:infix`, per `\__spintrent_spcoord_intent_str`, then wraps both sides — each referenced through with it, `$x/$y` standing for `\__spintrent_spcoord_print:n`’s own output.

```

2983 \cs_new_protected:Nn \__spintrent_spcoord_intent_infix:
2984 {
2985     \str_if_eq:NnTF \__spintrent_spcoord_luaset_sep_str { ; }

```

```

2986 { \str_set:Nn \__spintent_spcoord_intent_str { _punto-y-coma:infix } }
2987 { \str_set:Nn \__spintent_spcoord_intent_str { _coma:infix } }
2988 \MathMLintent{ \__spintent_spcoord_intent_str ($x, $y)}
2989 {
2990   \MathMLarg{x}{ \__spintent_spcoord_inner_sep:n { x } \__spintent_spcoord_print:n { x } }
2991   \MathMLintent { :silent } { \__spintent_spcoord_luaset_sep_str }
2992   \MathMLarg{y}{ \__spintent_spcoord_inner_sep:n { y } \__spintent_spcoord_print:n { y } }
2993 }
2994 }

```

(End of definition for __spintent_spcoord_intent_infix:.)

__spintent_spcoord_mathml: With `silent=true`, everything — parentheses and argument alike — is wrapped in a single `_-`, reading nothing at all: `\l__spintent_spcoord_raw_arg_tl` is typeset exactly as given, untouched by `\__spintent_spcoord_print:n/\__spintent_spcoord_intent_infix:`. Otherwise, the parentheses are silenced individually and `\__spintent_spcoord_intent_infix:` provides the reading in between.

```

2995 \cs_new_protected:Nn \__spintent_spcoord_mathml:
2996 {
2997   \bool_if:NTF \__spintent_spcoord_silent_bool
2998   {
2999     \MathMLintent { _- }
3000     {
3001       \l__spintent_spcoord_wrap_parens_l_tl
3002       \l__spintent_spcoord_raw_arg_tl
3003       \l__spintent_spcoord_wrap_parens_r_tl
3004     }
3005   }
3006   {
3007     \MathMLintent { :silent } { \l__spintent_spcoord_wrap_parens_l_tl }
3008     \__spintent_spcoord_intent_infix:
3009     \MathMLintent { :silent } { \l__spintent_spcoord_wrap_parens_r_tl }
3010   }
3011 }

```

(End of definition for __spintent_spcoord_mathml:.)

__spintent_spcoord_exec:nn Sets the `<keys>` in ‘#1’ and `\l__spintent_spcoord_raw_arg_tl` to ‘#2’. Calls `\__spintent_luafun_spcoord_parse_and_set:n` and reports `spcoord-invalid-arg` on error, whatever the value of `silent`: an invalid `{<argument>}` is always rejected. Then sizes the parentheses, and delegates to `\__spintent_spcoord_mathml:` unless there was an error, preceded by an invisible separator — placed *before* the opening parenthesis, as `\__spintent_sptvec_exec:nnn` and `\__spintent_spPoint_exec:nnn` do — and the whole pair is wrapped in `\mathord`. The wrapper is what keeps MathCAT from reading `(1,2)` as an interval when an equals sign or `\in` follows it; a silent separator after the closing parenthesis does not help, as it makes the relation disappear.

```

3012 \cs_new_protected:Nn \__spintent_spcoord_exec:nn
3013 {
3014   \keys_set:nn { spintent / spcoord } {#1}
3015   \tl_set:Nn \l__spintent_spcoord_raw_arg_tl {#2}
3016   \__spintent_luafun_spcoord_parse_and_set:n { \exp_not:n {#2} }
3017   \str_if_eq:VnT \l__spintent_spcoord_luaset_error_str { true }
3018   { \msg_error:nnn { spintent } { spcoord-invalid-arg } { \exp_not:n {#2} } }
3019   \__spintent_spcoord_set_wrap_parens_size:
3020   \str_if_eq:VnF \l__spintent_spcoord_luaset_error_str { true }
3021   {
3022     \mathord
3023     {
3024       \MathMLintent { :silent } { \c__spintent_invisible_sep_int } % previene lectura de intervalos
3025       \__spintent_spcoord_mathml:
3026     }
3027   }
3028 }

```

(End of definition for __spintent_spcoord_exec:nn.)

\spcoord The public command enforces *math mode*, opens a local group, and delegates to `\__spintent_spcoord_exec:nn`.

```

3029 \NewDocumentCommand \spcoord { 0{} m }
3030 {
3031   \mode_if_math:TF
3032   {
3033     \group_begin:
3034     \attribute \l__spintent_scope_attr_int = 1

```

```

3035         \__spintent_spcoord_exec:nn {#1} {#2}
3036     \group_end:
3037 }
3038 { \msg_error:nnn { spintent } { need-math-mode } { \spcoord } }
3039 }

```

(End of definition for `\spcoord`. This function is documented on page 19.)

12.35 The command `\spinterval`

The user command `\spinterval` receives the $\langle argument \rangle$ written as a left delimiter (, [or], two endpoints separated by a semicolon or by a single comma, and a right delimiter),] or [. A delimiter (or] on the left, and) or [on the right, excludes its endpoint; the others include it. The type of interval, the endpoints and the separator are automatically detected by the Lua module.

12.35.1 Definition of variables for `\spinterval`

Variables set by the function `\__spintent_luafun_spinterval_parse_and_set:n`.

```

\__spintent_spinterval_luaset_error_str
spintest_spinterval_luaset_error_code_str
__spintest_spinterval_luaset_left_sym_str
spintest_spinterval_luaset_right_sym_str
spintest_spinterval_luaset_left_open_str
spintest_spinterval_luaset_right_open_str
\__spintest_spinterval_luaset_type_str
\__spintest_spinterval_luaset_sep_str
\__spintest_spinterval_luaset_tall_str

```

```

3040 \str_new:N \__spintest_spinterval_luaset_error_str
3041 \str_new:N \__spintest_spinterval_luaset_error_code_str
3042 \str_new:N \__spintest_spinterval_luaset_left_sym_str
3043 \str_new:N \__spintest_spinterval_luaset_right_sym_str
3044 \str_new:N \__spintest_spinterval_luaset_left_open_str
3045 \str_new:N \__spintest_spinterval_luaset_right_open_str
3046 \str_new:N \__spintest_spinterval_luaset_type_str
3047 \str_new:N \__spintest_spinterval_luaset_sep_str
3048 \str_new:N \__spintest_spinterval_luaset_tall_str

```

(End of definition for `\__spintest_spinterval_luaset_error_str` and others.)

```

spintest_spinterval_luaset_ #1 _sign_str
spintest_spinterval_luaset_ #1 _type_str
spintest_spinterval_luaset_ #1 _value_str
__spintest_spinterval_luaset_ #1 _inf_str
spintest_spinterval_luaset_ #1 _tall_str
spintest_spinterval_luaset_ #1 _composite_str

```

The same variables for each endpoint, a and b.

```

3049 \clist_map_inline:nn { a , b }
3050 {
3051     \str_new:c { \__spintest_spinterval_luaset_ #1 _sign_str }
3052     \str_new:c { \__spintest_spinterval_luaset_ #1 _type_str }
3053     \str_new:c { \__spintest_spinterval_luaset_ #1 _value_str }
3054     \str_new:c { \__spintest_spinterval_luaset_ #1 _inf_str }
3055     \str_new:c { \__spintest_spinterval_luaset_ #1 _tall_str }
3056     \str_new:c { \__spintest_spinterval_luaset_ #1 _composite_str }
3057 }

```

(End of definition for `\__spintest_spinterval_luaset_ #1 _sign_str` and others.)

```

\__spintest_spinterval_raw_arg_tl
\__spintest_spinterval_prefix_tl
\__spintest_spinterval_read_arg_tl
\__spintest_spinterval_mode_str
\__spintest_spinterval_size_str
\__spintest_spinterval_reason_str
\__spintest_spinterval_word_w_tl
\__spintest_spinterval_word_g_tl
\__spintest_spinterval_intent_arg_tl
\__spintest_spinterval_intent_group_tl
\__spintest_spinterval_sym_l_tl
\__spintest_spinterval_sym_r_tl
\__spintest_spinterval_size_l_tl
\__spintest_spinterval_size_r_tl
\__spintest_spinterval_delim_l_tl
\__spintest_spinterval_delim_r_tl
\__spintest_spinterval_tmp_tl

```

The variable `\__spintest_spinterval_raw_arg_tl` stores the $\langle argument \rangle$ as it was received. The variables `\__spintest_spinterval_mode_str`, `\__spintest_spinterval_prefix_tl`, `\__spintest_spinterval_read_arg_tl` and `\__spintest_spinterval_size_str` store the values set by the keys `read-cmd`, `prefix`, `read-arg` and `size`, and `\__spintest_spinterval_reason_str` the text of the error reason. The variables `\__spintest_spinterval_word_w_tl` and `\__spintest_spinterval_word_g_tl` store the word that names the interval and the word that joins its endpoints, and `\__spintest_spinterval_intent_arg_tl` and `\__spintest_spinterval_intent_group_tl` the intents of the whole interval and of the group of endpoints. The variables `\__spintest_spinterval_sym_l_tl`, `\__spintest_spinterval_sym_r_tl`, `\__spintest_spinterval_size_l_tl`, `\__spintest_spinterval_size_r_tl`, `\__spintest_spinterval_delim_l_tl` and `\__spintest_spinterval_delim_r_tl` store the delimiter symbols, their size commands and the delimiters ready to be typeset; `\__spintest_spinterval_tmp_tl` is a scratch variable.

```

3058 \tl_new:N \__spintest_spinterval_raw_arg_tl
3059 \tl_new:N \__spintest_spinterval_prefix_tl
3060 \tl_new:N \__spintest_spinterval_read_arg_tl
3061 \str_new:N \__spintest_spinterval_mode_str
3062 \str_new:N \__spintest_spinterval_size_str
3063 \str_new:N \__spintest_spinterval_reason_str
3064 \tl_new:N \__spintest_spinterval_word_w_tl
3065 \tl_new:N \__spintest_spinterval_word_g_tl
3066 \tl_new:N \__spintest_spinterval_intent_arg_tl
3067 \tl_new:N \__spintest_spinterval_intent_group_tl
3068 \tl_new:N \__spintest_spinterval_sym_l_tl
3069 \tl_new:N \__spintest_spinterval_sym_r_tl
3070 \tl_new:N \__spintest_spinterval_size_l_tl
3071 \tl_new:N \__spintest_spinterval_size_r_tl
3072 \tl_new:N \__spintest_spinterval_delim_l_tl
3073 \tl_new:N \__spintest_spinterval_delim_r_tl
3074 \tl_new:N \__spintest_spinterval_tmp_tl

```

(End of definition for `\__spintest_spinterval_raw_arg_tl` and others.)

12.35.2 Definition of error messages for \spinterval

spinterval-invalid-arg

The “error” spinterval-invalid-arg is raised when the Lua module rejects the $\langle \textit{argument} \rangle$; it reports the argument and the reason set by `\__spinttent_spinterval_set_reason:`.

```

3075 \msg_new:nnnn { spintent } { spinterval-invalid-arg }
3076 {
3077   Invalid~format~in~\token_to_str:N \spinterval:~'#1'~(#{2})~\msg_line_context:.
3078 }
3079 {
3080   See~the~documentation~of~\token_to_str:N \spinterval~for~the~accepted~argument~forms.
3081 }

```

(End of definition for spinterval-invalid-arg.)

12.35.3 Definition of keys for \spinterval

read-cmd

We define the $\langle \textit{keys} \rangle$ read-cmd, prefix, read-arg, size, silent and concept.

prefix

read-arg

size

silent

concept

unknown

```

3082 \keys_define:nn { spintent / spinterval }
3083 {
3084   read-cmd          .choices:nn = { school , mathcat }
3085   {
3086     \int_case:nn { \l_keys_choice_int }
3087     {
3088       {1} { \str_set:Nn \__spinttent_spinterval_mode_str { school } }
3089       {2} { \str_set:Nn \__spinttent_spinterval_mode_str { mathcat } }
3090     }
3091   },
3092   read-cmd          .initial:n = school,
3093   read-cmd          .value_required:n = true,
3094   read-cmd / unknown .code:n =
3095     \msg_error:nneee { spintent } { unknown-choice }
3096     { read-cmd } { school,~mathcat } { \exp_not:n {#1} },
3097   prefix            .code:n =
3098     { \__spinttent_sanitiz_ affix:Nn \__spinttent_spinterval_prefix_tl {#1} },
3099   prefix            .initial:n = ,
3100   prefix            .value_required:n = true,
3101   read-arg          .code:n =
3102     { \__spinttent_sanitiz_ affix:Nn \__spinttent_spinterval_read_arg_tl {#1} },
3103   read-arg          .initial:n = ,
3104   read-arg          .value_required:n = true,
3105   size              .choices:nn = { auto , none , big , Big , bigg , Bigg }
3106   {
3107     \int_case:nn { \l_keys_choice_int }
3108     {
3109       {1} { \str_set:Nn \__spinttent_spinterval_size_str { auto } }
3110       {2} { \str_set:Nn \__spinttent_spinterval_size_str { none } }
3111       {3} { \str_set:Nn \__spinttent_spinterval_size_str { big } }
3112       {4} { \str_set:Nn \__spinttent_spinterval_size_str { Big } }
3113       {5} { \str_set:Nn \__spinttent_spinterval_size_str { bigg } }
3114       {6} { \str_set:Nn \__spinttent_spinterval_size_str { Bigg } }
3115     }
3116   },
3117   size              .initial:n = auto,
3118   size              .value_required:n = true,
3119   size / unknown    .code:n =
3120     \msg_error:nneee { spintent } { unknown-choice }
3121     { size } { auto,~none,~big,~Big,~bigg,~Bigg } { \exp_not:n {#1} },
3122   silent            .bool_set:N = \__spinttent_spinterval_silent_bool,
3123   silent            .initial:n = false,
3124   silent            .value_required:n = true,
3125   concept           .bool_set:N = \__spinttent_spinterval_concept_bool,
3126   concept           .initial:n = true,
3127   concept           .value_required:n = true,
3128   unknown          .code:n =
3129     {
3130       \str_set:Nn \__spinttent_command_name_str { spinterval }
3131       \__spinttent_unknown_keys_cmd:n {#1}
3132     },
3133 }

```

(End of definition for read-cmd and others.)

12.35.4 Internal functions for \spinterval

__spintenc_spinterval_set_reason:

The internal function `\__spintenc_spinterval_set_reason:` stores in `\l__spintenc_spinterval_reason_str` the text that describes the error code returned by the Lua module, or `invalid` argument when the code is not known.

```

3134 \cs_new_protected:Nn \__spintenc_spinterval_set_reason:
3135 {
3136   \str_case:VnF \l__spintenc_spinterval_luaset_error_code_str
3137   {
3138     { delim }
3139     {
3140       \str_set:Nn \l__spintenc_spinterval_reason_str
3141       { it~must~start~with~'(',~'['~or~']'~and~end~with~')',~']'~or~'[' }
3142     }
3143     { vacio }
3144     { \str_set:Nn \l__spintenc_spinterval_reason_str { an~endpoint~is~empty } }
3145     { sep }
3146     {
3147       \str_set:Nn \l__spintenc_spinterval_reason_str
3148       { there~is~no~separator:~use~a~comma~or~a~semicolon }
3149     }
3150     { ambiguo }
3151     {
3152       \str_set:Nn \l__spintenc_spinterval_reason_str
3153       { more~than~one~separator:~use~a~single~semicolon~and~the~commas~are~decimal }
3154     }
3155     { llaves }
3156     { \str_set:Nn \l__spintenc_spinterval_reason_str { unbalanced~braces } }
3157     { inf-orden }
3158     {
3159       \str_set:Nn \l__spintenc_spinterval_reason_str
3160       { minus~infinity~goes~on~the~left~and~infinity~on~the~right }
3161     }
3162     { inf-cerrado }
3163     {
3164       \str_set:Nn \l__spintenc_spinterval_reason_str
3165       { an~infinite~endpoint~cannot~be~included:~use~'('~or~']'~on~the~left,~')'~or~'['~or~']' }
3166     }
3167   }
3168   { \str_set:Nn \l__spintenc_spinterval_reason_str { invalid~argument } }
3169 }

```

(End of definition for __spintenc_spinterval_set_reason:.)

__spintenc_spinterval_set_delims:

The internal function `\__spintenc_spinterval_set_delims:` sets the symbols of the left and right delimiters from the characters returned by the Lua module, and their size commands from the key `size`. With `big`, `Big`, `bigg` and `Bigg` the size is fixed, and with `none` there is no size command. With `auto` there is none unless an endpoint contains a fraction, in which case it is `\Bigl/\Bigr` in display style and `\bigl/\bigr` in text style, and always `\Bigl/\Bigr` when the fraction is a `\dfrac`. Without a size command each delimiter is wrapped in `\mathopen` or `\mathclose`; with one, the delimiter follows the size command.

```

3170 \cs_new_protected:Nn \__spintenc_spinterval_set_delims:
3171 {
3172   \str_case:Vn \l__spintenc_spinterval_luaset_left_sym_str
3173   {
3174     { ( ) } { \tl_set:Nn \l__spintenc_spinterval_sym_l_tl { \lparen } }
3175     { [ ] } { \tl_set:Nn \l__spintenc_spinterval_sym_l_tl { \lbrack } }
3176     { } } { \tl_set:Nn \l__spintenc_spinterval_sym_l_tl { \rbrack } }
3177   }
3178   \str_case:Vn \l__spintenc_spinterval_luaset_right_sym_str
3179   {
3180     { ) } { \tl_set:Nn \l__spintenc_spinterval_sym_r_tl { \rparen } }
3181     { [ ] } { \tl_set:Nn \l__spintenc_spinterval_sym_r_tl { \lbrack } }
3182     { } } { \tl_set:Nn \l__spintenc_spinterval_sym_r_tl { \rbrack } }
3183   }
3184   \tl_clear:N \l__spintenc_spinterval_size_l_tl
3185   \tl_clear:N \l__spintenc_spinterval_size_r_tl
3186   \str_case:Vn \l__spintenc_spinterval_size_str
3187   {
3188     { big }
3189     {
3190       \tl_set:Nn \l__spintenc_spinterval_size_l_tl { \bigl }

```

```

3191         \tl_set:Nn \l__spintent_spinterval_size_r_tl { \bigr }
3192     }
3193     { Big }
3194     {
3195         \tl_set:Nn \l__spintent_spinterval_size_l_tl { \Bigl }
3196         \tl_set:Nn \l__spintent_spinterval_size_r_tl { \Bigr }
3197     }
3198     { bigg }
3199     {
3200         \tl_set:Nn \l__spintent_spinterval_size_l_tl { \biggl }
3201         \tl_set:Nn \l__spintent_spinterval_size_r_tl { \biggr }
3202     }
3203     { Bigg }
3204     {
3205         \tl_set:Nn \l__spintent_spinterval_size_l_tl { \Biggl }
3206         \tl_set:Nn \l__spintent_spinterval_size_r_tl { \Biggr }
3207     }
3208     { auto }
3209     {
3210         \str_if_eq:VnF \l__spintent_spinterval_luaset_tall_str { none }
3211         {
3212             \int_case:nnF { \mathstyle }
3213             {
3214                 { 0 }
3215                 {
3216                     \tl_set:Nn \l__spintent_spinterval_size_l_tl { \Bigl }
3217                     \tl_set:Nn \l__spintent_spinterval_size_r_tl { \Bigr }
3218                 }
3219                 { 1 }
3220                 {
3221                     \tl_set:Nn \l__spintent_spinterval_size_l_tl { \Bigl }
3222                     \tl_set:Nn \l__spintent_spinterval_size_r_tl { \Bigr }
3223                 }
3224                 { 2 }
3225                 {
3226                     \tl_set:Nn \l__spintent_spinterval_size_l_tl { \bigl }
3227                     \tl_set:Nn \l__spintent_spinterval_size_r_tl { \bigr }
3228                 }
3229                 { 3 }
3230                 {
3231                     \tl_set:Nn \l__spintent_spinterval_size_l_tl { \bigl }
3232                     \tl_set:Nn \l__spintent_spinterval_size_r_tl { \bigr }
3233                 }
3234             }
3235         }
3236         \str_if_eq:VnT \l__spintent_spinterval_luaset_tall_str { dfrac }
3237         {
3238             \tl_set:Nn \l__spintent_spinterval_size_l_tl { \Bigl }
3239             \tl_set:Nn \l__spintent_spinterval_size_r_tl { \Bigr }
3240         }
3241     }
3242 }
3243 }
3244 \tl_if_empty:NTF \l__spintent_spinterval_size_l_tl
3245 {
3246     \tl_set:Nc \l__spintent_spinterval_delim_l_tl
3247     { \exp_not:N \mathopen { \exp_not:V \l__spintent_spinterval_sym_l_tl } }
3248     \tl_set:Nc \l__spintent_spinterval_delim_r_tl
3249     { \exp_not:N \mathclose { \exp_not:V \l__spintent_spinterval_sym_r_tl } }
3250 }
3251 {
3252     \tl_set:Nc \l__spintent_spinterval_delim_l_tl
3253     { \exp_not:V \l__spintent_spinterval_size_l_tl \exp_not:V \l__spintent_spinterval_sym_l_tl }
3254     \tl_set:Nc \l__spintent_spinterval_delim_r_tl
3255     { \exp_not:V \l__spintent_spinterval_size_r_tl \exp_not:V \l__spintent_spinterval_sym_r_tl }
3256 }
3257 }

```

(End of definition for `\l__spintent_spinterval_set_delims:`)

`\l__spintent_spinterval_set_intents:`

The internal function `\l__spintent_spinterval_set_intents:` sets the two words used in the intents from the type returned by the Lua module: `abierto-de` and `a` for an open interval, `cerrado-de` and `a` for a

closed one, abierto-en and y-cerrado-en for an open-closed one, and cerrado-en and y-abierto-en for a closed-open one. The intent of the group of endpoints is `_<word>:infix($a,$b)` with the second word. The intent of the whole interval is `_` - when `silent` is `true`, `_` followed by the value of `read-arg` when it is not empty, and otherwise `_<word>:prefix($ab)` with the first word, preceded by the `prefix`, if any, and `intervalo-` when `concept` is `true`.

```

3258 \cs_new_protected:Nn \__spintent_spinterval_set_intents:
3259 {
3260   \str_case:Vn \__spintent_spinterval_luaset_type_str
3261   {
3262     { open }
3263     {
3264       \tl_set:Nn \__spintent_spinterval_word_w_tl { abierto-de }
3265       \tl_set:Nn \__spintent_spinterval_word_g_tl { a }
3266     }
3267     { closed }
3268     {
3269       \tl_set:Nn \__spintent_spinterval_word_w_tl { cerrado-de }
3270       \tl_set:Nn \__spintent_spinterval_word_g_tl { a }
3271     }
3272     { open-closed }
3273     {
3274       \tl_set:Nn \__spintent_spinterval_word_w_tl { abierto-en }
3275       \tl_set:Nn \__spintent_spinterval_word_g_tl { y-cerrado-en }
3276     }
3277     { closed-open }
3278     {
3279       \tl_set:Nn \__spintent_spinterval_word_w_tl { cerrado-en }
3280       \tl_set:Nn \__spintent_spinterval_word_g_tl { y-abierto-en }
3281     }
3282   }
3283   \tl_set:Nn \__spintent_spinterval_intent_group_tl
3284   { _ \__spintent_spinterval_word_g_tl :infix($a,$b) }
3285   \bool_if:NTF \__spintent_spinterval_silent_bool
3286   { \tl_set:Nn \__spintent_spinterval_intent_arg_tl { _- } }
3287   {
3288     \tl_if_empty:NTF \__spintent_spinterval_read_arg_tl
3289     {
3290       \tl_set:Nn \__spintent_spinterval_intent_arg_tl
3291       {
3292         -
3293         \bool_if:NT \__spintent_spinterval_concept_bool
3294         {
3295           \tl_if_empty:NF \__spintent_spinterval_prefix_tl
3296           { \tl_use:N \__spintent_spinterval_prefix_tl - }
3297           intervalo-
3298         }
3299         \__spintent_spinterval_word_w_tl :prefix($ab)
3300       }
3301     }
3302     {
3293       \tl_set:Nn \__spintent_spinterval_intent_arg_tl
3294       { _ \__spintent_spinterval_read_arg_tl }
3305     }
3306   }
3307 }

```

(End of definition for `\__spintent_spinterval_set_intents:`)

`\__spintent_spinterval_inner_sep:n`

The internal function `\__spintent_spinterval_inner_sep:n` calls `\__spintent_inner_sep:` when the endpoint `a` or `b` passed in `#1` is composite.

```

3308 \cs_new_protected:Nn \__spintent_spinterval_inner_sep:n
3309 {
3310   \str_if_eq:eeT
3311   { \str_use:c { \__spintent_spinterval_luaset_ #1 _composite_str } } { true }
3312   { \__spintent_inner_sep: }
3313 }

```

(End of definition for `\__spintent_spinterval_inner_sep:n`)

`\__spintent_spinterval_plain_number:n`

The internal function `\__spintent_spinterval_plain_number:n` typesets the value of the endpoint passed in `#1` as a number, preceded by `\__spintent_inner_sep:` and with each comma protected by

braces.

```

3314 \cs_new_protected:Nn \__spintnt_spinterval_plain_number:n
3315 {
3316   \__spintnt_inner_sep:
3317   \tl_set:Nc \__spintnt_spinterval_tmp_tl
3318     { \str_use:c { l__spintnt_spinterval_luaset_ #1 _value_str } }
3319   \tl_replace_all:Nnn \__spintnt_spinterval_tmp_tl { , } { {}, }
3320   \tl_use:N \__spintnt_spinterval_tmp_tl
3321 }

```

(End of definition for __spintnt_spinterval_plain_number:n.)

__spintnt_spinterval_value:n The internal function __spintnt_spinterval_value:n typesets the value of the endpoint passed in ‘#1’ according to its type: \infty for infinity; for a number, __spintnt_inner_sep: followed by \spnum when read-cmd is school, or __spintnt_spinterval_plain_number:n otherwise; and the value as written for any other type.

```

3322 \cs_new_protected:Nn \__spintnt_spinterval_value:n
3323 {
3324   \str_case:enF { \str_use:c { l__spintnt_spinterval_luaset_ #1 _type_str } }
3325   {
3326     { infinity } { \infty }
3327     { number }
3328     {
3329       \str_if_eq:VnTF \__spintnt_spinterval_mode_str { school }
3330       {
3331         \__spintnt_inner_sep:
3332         \exp_args:Nc \spnum { \str_use:c { l__spintnt_spinterval_luaset_ #1 _value_str } }
3333       }
3334       { \__spintnt_spinterval_plain_number:n { #1 } }
3335     }
3336   }
3337   { \str_use:c { l__spintnt_spinterval_luaset_ #1 _value_str } }
3338 }

```

(End of definition for __spintnt_spinterval_value:n.)

__spintnt_spinterval_print:n The internal function __spintnt_spinterval_print:n typesets the endpoint passed in ‘#1’ with its sign. With read-cmd=school a sign + or - is wrapped, with its value, in a native \MathMLintnt with the intent _más:prefix(\$vp) or _menos:prefix(\$vn), the value tagged with \MathMLarg and preceded by __spintnt_spinterval_inner_sep:n; an endpoint without sign is typeset through __spintnt_spinterval_value:n. With read-cmd=mathcat the sign is typeset as is, followed by the value.

```

3339 \cs_new_protected:Nn \__spintnt_spinterval_print:n
3340 {
3341   \str_if_eq:VnTF \__spintnt_spinterval_mode_str { school }
3342   {
3343     \str_case:enF { \str_use:c { l__spintnt_spinterval_luaset_ #1 _sign_str } }
3344     {
3345       { + }
3346       {
3347         \MathMLintnt { _más:prefix($vp) }
3348         {
3349           +
3350           \MathMLarg { vp }
3351           { \__spintnt_spinterval_inner_sep:n { #1 } \__spintnt_spinterval_value:n }
3352         }
3353       }
3354       { - }
3355       {
3356         \MathMLintnt { _menos:prefix($vn) }
3357         {
3358           -
3359           \MathMLarg { vn }
3360           { \__spintnt_spinterval_inner_sep:n { #1 } \__spintnt_spinterval_value:n }
3361         }
3362       }
3363     }
3364     { \__spintnt_spinterval_value:n { #1 } }
3365   }
3366   {
3367     \str_use:c { l__spintnt_spinterval_luaset_ #1 _sign_str }

```

```

3368     \__spintenz_spinterval_value:n { #1 }
3369   }
3370 }

```

(End of definition for __spintenz_spinterval_print:n.)

__spintenz_spinterval_mathml: The internal function __spintenz_spinterval_mathml: builds the annotated interval with a native `\MathMLintenz` that carries the intent of the whole interval. It contains the left delimiter silenced, a `\MathMLarg` named `ab` that wraps __spintenz_inner_sep: and a `\MathMLintenz` with the intent of the group of endpoints, and the right delimiter silenced. The group holds the endpoints `a` and `b`, each tagged with `\MathMLarg` and typeset by __spintenz_spinterval_print:n, separated by the silenced separator.

```

3371 \cs_new_protected:Nn \__spintenz_spinterval_mathml:
3372 {
3373   \MathMLintenz { \l__spintenz_spinterval_intenz_arg_tl }
3374   {
3375     \MathMLintenz { :silenz } { \l__spintenz_spinterval_delim_l_tl }
3376     \MathMLarg { ab }
3377     {
3378       \__spintenz_inner_sep:
3379       \MathMLintenz { \l__spintenz_spinterval_intenz_group_tl }
3380       {
3381         \MathMLarg { a }
3382         { \__spintenz_spinterval_inner_sep:n { a } \__spintenz_spinterval_print:n { a } }
3383         \MathMLintenz { :silenz } { \l__spintenz_spinterval_luaset_sep_str }
3384         \MathMLarg { b }
3385         { \__spintenz_spinterval_inner_sep:n { b } \__spintenz_spinterval_print:n { b } }
3386       }
3387     }
3388     \MathMLintenz { :silenz } { \l__spintenz_spinterval_delim_r_tl }
3389   }
3390 }

```

(End of definition for __spintenz_spinterval_mathml:.)

__spintenz_spinterval_exec:nn The internal function __spintenz_spinterval_exec:nn receives the optional `(keys)` passed in ‘#1’ and the `{(argument)}` passed in ‘#2’. It calls __spintenz_luafun_spinterval_parse_and_set:n; if the Lua module reports an error, it reports `spinterval-invalid-arg` whatever the value of `silenz`: an invalid `{(argument)}` is always rejected. When there is no error, it calls __spintenz_spinterval_set_delims: and __spintenz_spinterval_set_intents:, places __spintenz_invisible_sep: and delegates the rendering to __spintenz_spinterval_mathml:.

```

3391 \cs_new_protected:Nn \__spintenz_spinterval_exec:nn
3392 {
3393   \keys_set:nn { spintenz / spinterval } {#1}
3394   \tl_set:Nn \l__spintenz_spinterval_raw_arg_tl {#2}
3395   \__spintenz_luafun_spinterval_parse_and_set:n { \exp_not:n {#2} }
3396   \str_if_eq:VnTF \l__spintenz_spinterval_luaset_error_str { true }
3397   {
3398     \__spintenz_spinterval_set_reason:
3399     \msg_error:nnee { spintenz } { spinterval-invalid-arg }
3400     { \exp_not:n {#2} } { \l__spintenz_spinterval_reason_str }
3401   }
3402   {
3403     \__spintenz_spinterval_set_delims:
3404     \__spintenz_spinterval_set_intents:
3405     \__spintenz_invisible_sep:
3406     \__spintenz_spinterval_mathml:
3407   }
3408 }

```

(End of definition for __spintenz_spinterval_exec:nn.)

\spinterval The public command enforces *math mode*, opens a local group, and delegates to __spintenz_spinterval_exec:nn.

```

3409 \NewDocumentCommand \spinterval { O{} m }
3410 {
3411   \mode_if_math:TF
3412   {
3413     \group_begin:
3414     \attribute \l__spintenz_scope_attr_intenz = 1
3415     \__spintenz_spinterval_exec:nn {#1} {#2}
3416     \group_end:
3417   }

```

```

3418     { \msg_error:nnn { spintent } { need-math-mode } { \spinterval } }
3419   }

```

(End of definition for `\spinterval`. This function is documented on page 34.)

12.36 The command `\spfun`

The user command `\spfun` typesets a function call such as $f(x)$, $f_{\{1\}}(x)$, $f'(x,y)$. The Lua function `\__spintent_luafun_spfun_parse_and_set:n` splits the $\langle argument \rangle$ into the function name (a letter with an optional subscript or primes, left in the shared `\l_@@_geo_luaset_*` variables) and the list of arguments, cut at the top-level commas, each one stored in `\l_@@_spfun_luaset_arg_<i>_tl` together with a flag `\l_@@_spfun_luaset_simple_<i>_str` telling whether it is a single node. The whole call is tagged with a native `\MathMLintent` whose `intent` is $\$f(\$a1,\$a2,...)$; the name is tagged `arg="f"` and each argument `arg="a<i>"`, while the parentheses and the commas are silenced. The name may carry the inverse exponent $^{-1}$ (flag `\l_@@_spfun_luaset_inv_str`); in that case the literal `intent` of the name ends in `-inversa`, the exponent is typeset silenced and the parentheses may be omitted (no arguments, $\$f$ as the whole `intent`). An argument that is itself a call, $f(g(x))$ or $f(g^{-1}(y))$, is analysed recursively by Lua (kind `call`) and typeset by `\__spintent_spfun_nested:n`, which runs the same code in a group, one level deeper; the `arg` names carry the depth.

12.36.1 Definition of variables for `\spfun`

The variable `\l__spintent_spfun_mode_str` stores the value set by the key `read-cmd` and `\l__spintent_spfun_prefix_tl` the one set by `prefix`. The two `luaset` variables are set from Lua.

```

3420 \str_new:N \l__spintent_spfun_mode_str
3421 \tl_new:N \l__spintent_spfun_prefix_tl
3422 \tl_new:N \l__spintent_spfun_intent_arg_tl
3423 \tl_new:N \l__spintent_spfun_kv_tl
3424 \tl_new:N \l__spintent_spfun_refs_tl
3425 \str_new:N \l__spintent_spfun_tmp_str
3426 \int_new:N \l__spintent_spfun_nargs_int
3427 \bool_new:N \l__spintent_spfun_silent_bool
3428 \bool_new:N \l__spintent_spfun_concept_bool
3429 \str_new:N \l__spintent_spfun_luaset_error_str
3430 \str_new:N \l__spintent_spfun_luaset_nargs_str
3431 \str_new:N \l__spintent_spfun_luaset_inv_str
3432 \tl_new:N \l__spintent_spfun_call_tl
3433 \int_new:N \l__spintent_spfun_depth_int

```

(End of definition for `\l__spintent_spfun_mode_str` and others.)

12.36.2 Definition of error messages for `\spfun`

`spfun-invalid-arg`

The “error” `spfun-invalid-arg` is raised when the argument fails to parse: no valid name, no parentheses (except for the inverse function), a malformed or misplaced inverse exponent, unbalanced delimiters, an empty argument or text after the closing parenthesis. It is raised whatever the value of `silent`.

```

3434 \msg_new:nnn { spintent } { spfun-invalid-arg }
3435 {
3436   Invalid~argument~'!#1'~for~\token_to_str:N \spfun.
3437 }

```

(End of definition for `spfun-invalid-arg`.)

12.36.3 Definition of keys for `\spfun`

`read-cmd`
`prefix`
`concept`
`silent`
`unknown`

We define the $\langle keys \rangle$ `read-cmd`, `prefix`, `concept` and `silent`. Unlike the geometry commands, `concept` is `false` by default: *f de x* is already how the function is read.

```

3438 \keys_define:nn { spintent / spfun }
3439 {
3440   read-cmd .choices:nn = { school , mathcat }
3441   {
3442     \int_case:nn { \l_keys_choice_int }
3443     {
3444       {1} { \str_set:Nn \l__spintent_spfun_mode_str { school } }
3445       {2} { \str_set:Nn \l__spintent_spfun_mode_str { mathcat } }
3446     }
3447   },
3448   read-cmd .initial:n = school,
3449   read-cmd .value_required:n = true,
3450   read-cmd / unknown .code:n =
3451     \msg_error:nneee { spintent } { unknown-choice }
3452     { read-cmd } { school,~mathcat } { \exp_not:n {#1} },
3453   prefix .code:n =
3454     { \__spintent_sanitizе_affix:Nn \l__spintent_spfun_prefix_tl {#1} },

```

```

3455     prefix                .initial:n = ,
3456     prefix                .value_required:n = true,
3457     silent                .bool_set:N = \l__spintent_spfun_silent_bool,
3458     silent                .initial:n = false,
3459     silent                .value_required:n = true,
3460     concept               .bool_set:N = \l__spintent_spfun_concept_bool,
3461     concept               .initial:n = false,
3462     concept               .value_required:n = true,
3463     unknown               .code:n =
3464     {
3465         \str_set:Nn \l__spintent_command_name_str { spfun }
3466         \__spintent_unknown_keys_cmd:n {#1}
3467     },
3468 }

```

(End of definition for `read-cmd` and others.)

12.36.4 Internal functions for `\spfun`

`\__spintent_spfun_fname:` Names of the `arg` attributes at the current nesting depth `\l__spintent_spfun_depth_int`. At depth 1 they are `f` and `a<i>`; at a deeper level `f<d>` and `a<d>x<i>`, so that the references of a call nested in an argument never clash with those of the call around it.

```

3469 \cs_new:Nn \__spintent_spfun_fname:
3470 {
3471     f
3472     \int_compare:nNnF { \l__spintent_spfun_depth_int } = { 1 }
3473     { \int_use:N \l__spintent_spfun_depth_int }
3474 }
3475 \cs_new:Nn \__spintent_spfun_aname:n
3476 {
3477     a
3478     \int_compare:nNnF { \l__spintent_spfun_depth_int } = { 1 }
3479     { \int_use:N \l__spintent_spfun_depth_int x }
3480     #1
3481 }

```

(End of definition for `\__spintent_spfun_fname:` and `\__spintent_spfun_aname:n`.)

`\__spintent_spfun_name_plain:` Typesets the name without any annotation: the letter, followed by the subscript or by the primes when there are any.

```

3482 \cs_new_protected:Nn \__spintent_spfun_name_plain:
3483 {
3484     \l__spintent_geo_luaset_letra_str
3485     \str_case:VnF \l__spintent_geo_luaset_tipo_str
3486     {
3487         { sub } { \c_math_subscript_token { \l__spintent_geo_luaset_sub_str } }
3488         { prima } { \l__spintent_geo_luaset_sub_str }
3489     }
3490     { }
3491     \str_if_eq:VnT \l__spintent_spfun_luaset_inv_str { true }
3492     { \c_math_superscript_token { -1 } }
3493 }

```

(End of definition for `\__spintent_spfun_name_plain:`.)

`\__spintent_spfun_name_school:` For `read-cmd=school` the name is tagged `arg="f"` with a literal `intent(_f-sub-uno-, _F-doble-prima-, ...)` through the shared queue, so `MathCAT` neither announces a capital letter nor reads the Unicode double prime as seconds. A subscript is typeset silenced; a prime is not, since it is not spoken on the base. The inverse exponent `^{-1}`, when present, is typeset silenced too: the literal `intent(_f-inversa-)` already says everything.

```

3494 \cs_new_protected:Nn \__spintent_spfun_name_school:
3495 {
3496     \tl_set:Ne \l__spintent_spfun_kv_tl
3497     { arg = " \__spintent_spfun_fname: ", intent = " \l__spintent_geo_luaset_intent_str -" }
3498     \__spintent_luafun_mrow_intent_arg:nn { } { \l__spintent_spfun_kv_tl }
3499     \str_case:VnF \l__spintent_geo_luaset_tipo_str
3500     {
3501         { sub }
3502         {
3503             \l__spintent_geo_luaset_letra_str
3504             \c_math_subscript_token
3505             { \MathMLintent { :silent } { \l__spintent_geo_luaset_sub_str } }

```

```

3506     }
3507     { prima }
3508     { \l__spintent_geo_luaset_letra_str \l__spintent_geo_luaset_sub_str }
3509   }
3510   { \l__spintent_geo_luaset_letra_str }
3511   \str_if_eq:VnT \l__spintent_sfun_luaset_inv_str { true }
3512   {
3513     \c_math_superscript_token
3514     { \MathMLint { :silent } { -1 } }
3515   }
3516 }

```

(End of definition for \l__spintent_sfun_name_school:.)

\l__spintent_sfun_name_mathcat: For `read-cmd=mathcat` the name is tagged `arg="f"` through a native `\MathMLarg`, never mixed with the shared queue for the same name. With a subscript or primes the inner separator keeps the `arg` on a wrapping `<mrow>`.

```

3517 \cs_new_protected:Nn \l__spintent_sfun_name_mathcat:
3518 {
3519   \bool_lazy_and:nnTF
3520   { \str_if_eq_p:Vn \l__spintent_geo_luaset_tipo_str { } }
3521   { \str_if_eq_p:Vn \l__spintent_sfun_luaset_inv_str { false } }
3522   { \exp_args:Ne \MathMLarg { \l__spintent_sfun_fname: } { \l__spintent_sfun_name_plain: } }
3523   { \exp_args:Ne \MathMLarg { \l__spintent_sfun_fname: } { \l__spintent_inner_sep: \l__spintent_
3524 }

```

(End of definition for \l__spintent_sfun_name_mathcat:.)

\l__spintent_sfun_arg:n Tags argument number `#1` with `arg="a<i>"` (see `\l__spintent_sfun_aname:n`). A single node (letter, number, a command such as `\alpha`) carries its `arg` directly; any other argument starts with the inner separator, which forces an `<mrow>` to hold the `arg`. An argument that is itself a call (`g(x)`, `g-1(y)`) is typeset by `\l__spintent_sfun_nested:n`.

```

3525 \cs_new_protected:Nn \l__spintent_sfun_arg:n
3526 {
3527   \str_set_eq:Nc \l__spintent_sfun_tmp_str { \l__spintent_sfun_luaset_simple_ #1 _str }
3528   \str_case:VnF \l__spintent_sfun_tmp_str
3529   {
3530     { true }
3531     {
3532       \exp_args:Ne \MathMLarg { \l__spintent_sfun_aname:n {#1} }
3533       { \tl_use:c { \l__spintent_sfun_luaset_arg_ #1 _tl } }
3534     }
3535     { call }
3536     {
3537       \exp_args:Ne \MathMLarg { \l__spintent_sfun_aname:n {#1} }
3538       { \l__spintent_inner_sep: \l__spintent_sfun_nested:n {#1} }
3539     }
3540   }
3541   {
3542     \exp_args:Ne \MathMLarg { \l__spintent_sfun_aname:n {#1} }
3543     { \l__spintent_inner_sep: \tl_use:c { \l__spintent_sfun_luaset_arg_ #1 _tl } }
3544   }
3545 }

```

(End of definition for \l__spintent_sfun_arg:n.)

\l__spintent_sfun_args: Walks the arguments, silencing the comma between them.

```

3546 \cs_new_protected:Nn \l__spintent_sfun_args:
3547 {
3548   \int_step_inline:nn { \l__spintent_sfun_nargs_int }
3549   {
3550     \int_compare:nNnF {##1} = {1}
3551     { \MathMLint { :silent } { , } }
3552     \l__spintent_sfun_arg:n {##1}
3553   }
3554 }

```

(End of definition for \l__spintent_sfun_args:.)

`\__spintent_spfun_args_plain:` Typesets the arguments with no annotation (`silent`).

```

3555 \cs_new_protected:Nn \__spintent_spfun_args_plain:
3556 {
3557   \int_step_inline:nn { \__spintent_spfun_nargs_int }
3558   {
3559     \int_compare:nNnF {##1} = {1} { , }
3560     \tl_use:c { \__spintent_spfun_luaset_arg_ ##1 _tl }
3561   }
3562 }

```

(End of definition for `\__spintent_spfun_args_plain:`.)

`\__spintent_spfun_set_intent:` Builds the concept `intent` in `\__spintent_spfun_intent_arg_tl: $f($a1,$a2,...)` when `concept` is `false`, and `_prefix-función:prefix($f($a1,$a2,...))` otherwise. Without arguments (the inverse function written bare) the call is just `$f`.

```

3563 \cs_new_protected:Nn \__spintent_spfun_set_intent:
3564 {
3565   \tl_clear:N \__spintent_spfun_refs_tl
3566   \int_step_inline:nn { \__spintent_spfun_nargs_int }
3567   {
3568     \tl_put_right:Ne \__spintent_spfun_refs_tl
3569     { \int_compare:nNnF {##1} = {1} { , } $ \__spintent_spfun_ane:n {##1} }
3570   }
3571   \int_compare:nNnTF { \__spintent_spfun_nargs_int } = { 0 }
3572   { \tl_set:Ne \__spintent_spfun_call_tl { $ \__spintent_spfun_fname: } }
3573   { \tl_set:Ne \__spintent_spfun_call_tl { $ \__spintent_spfun_fname: ( \__spintent_spfun_r
3574   \bool_if:NTF \__spintent_spfun_concept_bool
3575   {
3576     \tl_set:Ne \__spintent_spfun_intent_arg_tl
3577     {
3578       -
3579       \tl_if_empty:NF \__spintent_spfun_prefix_tl
3580       { \tl_use:N \__spintent_spfun_prefix_tl - }
3581       función:prefix( \__spintent_spfun_call_tl )
3582     }
3583   }
3584   {
3585     \tl_set_eq:NN \__spintent_spfun_intent_arg_tl \__spintent_spfun_call_tl
3586   }
3587 }

```

(End of definition for `\__spintent_spfun_set_intent:`.)

`\__spintent_spfun_mathml:` Calls `\__spintent_invisible_sep:` first, then wraps name, parentheses and arguments in a native `\MathMLintent`. With `silent` set to `true` the intent is `_-` and nothing inside is annotated.

```

3588 \cs_new_protected:Nn \__spintent_spfun_mathml:
3589 {
3590   \__spintent_invisible_sep:
3591   \__spintent_spfun_body:
3592 }

```

(End of definition for `\__spintent_spfun_mathml:`.)

`\__spintent_spfun_body:` The call itself, without the outer separator: used by the top-level call and by every call nested in an argument.

```

3593 \cs_new_protected:Nn \__spintent_spfun_body:
3594 {
3595   \bool_if:NTF \__spintent_spfun_silent_bool
3596   {
3597     \MathMLintent { _- }
3598     {
3599       \__spintent_inner_sep:
3600       \__spintent_spfun_name_plain:
3601       \int_compare:nNnF { \__spintent_spfun_nargs_int } = { 0 }
3602       {
3603         (
3604         \__spintent_spfun_args_plain:
3605         )
3606       }
3607     }
3608   }
3609 }

```

```

3610     \__spintent_sfun_set_intent:
3611     \MathMLintent { \__spintent_sfun_intent_arg_tl }
3612     {
3613         \int_compare:nNtT { \__spintent_sfun_nargs_int } = { 0 }
3614         { \__spintent_inner_sep: }
3615         \str_if_eq:VnTF \__spintent_sfun_mode_str { school }
3616         { \__spintent_sfun_name_school: }
3617         { \__spintent_sfun_name_mathcat: }
3618         \int_compare:nNf { \__spintent_sfun_nargs_int } = { 0 }
3619         {
3620             \MathMLintent { :silent } { ( }
3621             \__spintent_sfun_args:
3622             \MathMLintent { :silent } { ) }
3623         }
3624     }
3625 }
3626 }

```

(End of definition for `\__spintent_sfun_body:`.)

`\__spintent_sfun_nested:n` Typesets argument number **#1** as a call of its own. Opens a group, so the values that Lua leaves in the variables for the nested call (name, arguments, number of arguments) vanish when it ends and the outer call keeps its own. The nested call raises the depth, never applies the `concept` and goes through `\__spintent_sfun_body:` with no outer separator. It cannot fail to parse: the first analysis already validated every level.

```

3627 \cs_new_protected:Nn \__spintent_sfun_nested:n
3628 {
3629     \group_begin:
3630     \exp_args:Ne \__spintent_luafun_sfun_parse_and_set:n
3631     { \exp_not:v { \__spintent_sfun_luaaset_arg_ #1 _tl } }
3632     \int_incr:N \__spintent_sfun_depth_int
3633     \bool_set_false:N \__spintent_sfun_concept_bool
3634     \int_set:Nn \__spintent_sfun_nargs_int { \__spintent_sfun_luaaset_nargs_str }
3635     \__spintent_sfun_body:
3636     \group_end:
3637 }

```

(End of definition for `\__spintent_sfun_nested:n`.)

`\__spintent_sfun_exec:nn` Receives the optional `<keys>` in ‘**#1**’ and the `{<argument>}` in ‘**#2**’. Parses first, reporting `spfun-invalid-arg` on error whatever `silent` is.

```

3638 \cs_new_protected:Nn \__spintent_sfun_exec:nn
3639 {
3640     \keys_set:nn { spintent / sfun } { #1 }
3641     \__spintent_luafun_sfun_parse_and_set:n { \exp_not:n { #2 } }
3642     \str_if_eq:VnTF \__spintent_sfun_luaaset_error_str { true }
3643     {
3644         \msg_error:nne { spintent } { spfun-invalid-arg } { \exp_not:n { #2 } }
3645     }
3646     {
3647         \int_set:Nn \__spintent_sfun_nargs_int { \__spintent_sfun_luaaset_nargs_str }
3648         \int_set:Nn \__spintent_sfun_depth_int { 1 }
3649         \__spintent_sfun_mathml:
3650     }
3651 }

```

(End of definition for `\__spintent_sfun_exec:nn`.)

`\spfun` The public command enforces *math mode*, opens a local group, and delegates to `\__spintent_sfun_exec:nn`.

```

3652 \NewDocumentCommand \spfun { 0{ } m }
3653 {
3654     \mode_if_math:TF
3655     {
3656         \group_begin:
3657         \attribute \__spintent_scope_attr_int = 1
3658         \__spintent_sfun_exec:nn { #1 } { #2 }
3659         \group_end:
3660     }
3661     { \msg_error:nnn { spintent } { need-math-mode } { \spfun } }
3662 }

```

(End of definition for `\spfun`. This function is documented on page 35.)

12.37 The command `\spfuncomp`

The user command `\spfuncomp` typesets the linear composition of functions: $f \circ g$, $f \circ g \circ h$, $(f \circ g)$ and the composition applied to arguments, $(f \circ g)(x)$. The Lua function `\__spintent_luafun_sp` splits the $\{ \langle \textit{argument} \rangle \}$ at the top-level `\circ` (or the Unicode character U+2218); every function is a name as in `\spfun` and gets its literal `intent` (`_f-`, `_f-inversa-`, ...), so `MathCAT` neither announces a capital letter nor reads the letter `h` (U+210E) as Planck's constant. The arguments of the application, if any, are left in the `\spfun` variables and typeset with its own functions, so an argument may be a call. The whole is tagged with a native `\MathMLintent` whose `intent` is `_compuesta-con:infix($k1,$k2,...)`; the application adds `$c($a1,...)` around it, with the composition tagged `arg="c"`. With two or more arguments they are grouped under their own `intent`, `_coma:infix($a1,$a2,...)`, tagged `arg="l"`, because `MathCAT` drops the commas of a call whose head is not a single letter. The `\circ` and the parentheses are silenced.

12.37.1 Definition of variables for `\spfuncomp`

Number of functions, the references and the `intent` being built, and the values that Lua leaves: error flag, number of functions, whether the composition is in parentheses and whether it is applied to arguments.

```

3663 \int_new:N \__spintent_spfuncomp_n_int
3664 \tl_new:N \__spintent_spfuncomp_refs_tl
3665 \tl_new:N \__spintent_spfuncomp_intent_tl
3666 \bool_new:N \__spintent_spfuncomp_silent_bool
3667 \tl_new:N \__spintent_spfuncomp_kv_tl
3668 \str_new:N \__spintent_spfuncomp_tipo_str
3669 \str_new:N \__spintent_spfuncomp_luaset_error_str
3670 \str_new:N \__spintent_spfuncomp_luaset_n_str
3671 \str_new:N \__spintent_spfuncomp_luaset_paren_str
3672 \str_new:N \__spintent_spfuncomp_luaset_apply_str

```

(End of definition for `\__spintent_spfuncomp_n_int` and others.)

12.37.2 Definition of error messages for `\spfuncomp`

Raised when the argument is not a composition of at least two function names, optionally in parentheses and optionally applied to arguments written in parentheses, whatever the value of `silent`.

```

3673 \msg_new:nnn { spintent } { spfuncomp-invalid-arg }
3674 {
3675     Invalid~argument~'#1'~for~\token_to_str:N \spfuncomp.
3676 }

```

(End of definition for `spfuncomp-invalid-arg`.)

12.37.3 Definition of keys for `\spfuncomp`

`silent` Only `silent`: the composition has no concept word and its reading is fixed, so `prefix` and `read-cmd` would have nothing to act on.

```

3677 \keys_define:nn { spintent / spfuncomp }
3678 {
3679     silent .bool_set:N = \__spintent_spfuncomp_silent_bool,
3680     silent .initial:n = false,
3681     silent .value_required:n = true,
3682     unknown .code:n =
3683     {
3684         \str_set:Nn \__spintent_command_name_str { spfuncomp }
3685         \__spintent_unknown_keys_cmd:n {#1}
3686     },
3687 }

```

(End of definition for `silent` and `unknown`.)

12.37.4 Internal functions for `\spfuncomp`

Function number `#1` of the chain. Its first letter is tagged `arg="k<i>"` with the literal `intent` of the name (`_f-`, `_f-inversa-`, ...) through the shared queue, exactly as `\spfun` does with its name: a native `\MathMLintent` around the name would leave its letter free for the queue to claim by mistake when the arguments contain calls. A subscript is typeset silenced; a prime is not; the inverse exponent $\wedge_{-1}$ is silenced too.

```

3688 \cs_new_protected:Nn \__spintent_spfuncomp_fun:n
3689 {
3690     \tl_set:Nc \__spintent_spfuncomp_kv_tl
3691     {
3692         arg = " k #1 ",
3693         intent = "_ \use:c { \__spintent_spfuncomp_luaset_intent_ #1 _str } -"
3694     }
3695     \__spintent_luafun_mrow_intent_arg:nn { } { \__spintent_spfuncomp_kv_tl }
3696     \str_set_eq:Nc \__spintent_spfuncomp_tipo_str
3697     { \__spintent_spfuncomp_luaset_tipo_ #1 _str }

```

```

3698 \str_case:VnF \l__spintent_sfuncomp_tipo_str
3699 {
3700   { sub }
3701   {
3702     \use:c { l__spintent_sfuncomp_luaset_letra_ #1 _str }
3703     \c_math_subscript_token
3704     {
3705       \MathMLintent { :silent }
3706       { \use:c { l__spintent_sfuncomp_luaset_sub_ #1 _str } }
3707     }
3708   }
3709   { prima }
3710   {
3711     \use:c { l__spintent_sfuncomp_luaset_letra_ #1 _str }
3712     \use:c { l__spintent_sfuncomp_luaset_sub_ #1 _str }
3713   }
3714 }
3715 { \use:c { l__spintent_sfuncomp_luaset_letra_ #1 _str } }
3716 \str_set_eq:Nc \l__spintent_sfuncomp_tipo_str
3717 { l__spintent_sfuncomp_luaset_inv_ #1 _str }
3718 \str_if_eq:VnT \l__spintent_sfuncomp_tipo_str { true }
3719 {
3720   \c_math_superscript_token
3721   { \MathMLintent { :silent } { -1 } }
3722 }
3723 }

```

(End of definition for `\l__spintent_sfuncomp_fun:n`.)

`\l__spintent_sfuncomp_group:` The composition, with its own `intent`, in parentheses when the argument was written in parentheses.

```

3724 \cs_new_protected:Nn \l__spintent_sfuncomp_group:
3725 {
3726   \tl_clear:N \l__spintent_sfuncomp_refs_tl
3727   \int_step_inline:nn { \l__spintent_sfuncomp_n_int }
3728   {
3729     \tl_put_right:Ne \l__spintent_sfuncomp_refs_tl
3730     { \int_compare:nNnF {##1} = {1} { , } $k ##1 }
3731   }
3732   \tl_set:Ne \l__spintent_sfuncomp_intent_tl
3733   { _compuesta-con:infix( \l__spintent_sfuncomp_refs_tl ) }
3734   \exp_args:NV \MathMLintent \l__spintent_sfuncomp_intent_tl
3735   {
3736     \str_if_eq:VnT \l__spintent_sfuncomp_luaset_paren_str { true }
3737     { \MathMLintent { :silent } { ( ) } }
3738     \int_step_inline:nn { \l__spintent_sfuncomp_n_int }
3739     {
3740       \int_compare:nNnF {##1} = {1}
3741       { \MathMLintent { :silent } { \circ } }
3742       \l__spintent_sfuncomp_fun:n {##1}
3743     }
3744     \str_if_eq:VnT \l__spintent_sfuncomp_luaset_paren_str { true }
3745     { \MathMLintent { :silent } { ) } }
3746   }
3747 }

```

(End of definition for `\l__spintent_sfuncomp_group:.`)

`\l__spintent_sfuncomp_plain:` Typesets the whole with no annotation (`silent`).

```

3748 \cs_new_protected:Nn \l__spintent_sfuncomp_plain:
3749 {
3750   \str_if_eq:VnT \l__spintent_sfuncomp_luaset_paren_str { true } { ( ) }
3751   \int_step_inline:nn { \l__spintent_sfuncomp_n_int }
3752   {
3753     \int_compare:nNnF {##1} = {1} { \circ }
3754     \tl_use:c { l__spintent_sfuncomp_luaset_print_ ##1 _tl }
3755   }
3756   \str_if_eq:VnT \l__spintent_sfuncomp_luaset_paren_str { true } { ) }
3757   \str_if_eq:VnT \l__spintent_sfuncomp_luaset_apply_str { true }
3758   {
3759     ( \l__spintent_sfun_args_plain: )
3760   }
3761 }

```

(End of definition for `\__spintent_sfuncomp_plain:`)

`\__spintent_sfuncomp_mathml:`

Calls `\__spintent_invisible_sep:` first. With `silent` the `intent` is `_-`. Otherwise an application wraps the composition (`arg="c"`) and the arguments in `$c($a1,...)`; the arguments reuse the `\spfun` machinery, including calls nested in them.

```

3762 \cs_new_protected:Nn \__spintent_sfuncomp_mathml:
3763 {
3764   \__spintent_invisible_sep:
3765   \str_if_eq:VnT \__spintent_sfuncomp_luaset_apply_str { true }
3766   {
3767     \int_set:Nn \__spintent_sfun_nargs_int { \__spintent_sfun_luaset_nargs_str }
3768   }
3769   \int_set:Nn \__spintent_sfun_depth_int { 1 }
3770   \str_set:Nn \__spintent_sfun_mode_str { school }
3771   \bool_set_false:N \__spintent_sfun_silent_bool
3772   \bool_set_false:N \__spintent_sfun_concept_bool
3773   \bool_if:NTF \__spintent_sfuncomp_silent_bool
3774   {
3775     \MathMLintent { _- }
3776     { \__spintent_inner_sep: \__spintent_sfuncomp_plain: }
3777   }
3778   {
3779     \str_if_eq:VnTF \__spintent_sfuncomp_luaset_apply_str { true }
3780     {
3781       \tl_clear:N \__spintent_sfun_refs_tl
3782       \int_step_inline:nn { \__spintent_sfun_nargs_int }
3783       {
3784         \tl_put_right:Ne \__spintent_sfun_refs_tl
3785         { \int_compare:nNnF {##1} = {1} { , } $ \__spintent_sfun_aname:n {##1} }
3786       }
3787       \int_compare:nNnTF { \__spintent_sfun_nargs_int } > { 1 }
3788       {
3789         \tl_set:Ne \__spintent_sfun_call_tl { $c ( $l ) }
3790         \exp_args:NV \MathMLintent \__spintent_sfun_call_tl
3791         {
3792           \MathMLarg { c } { \__spintent_inner_sep: \__spintent_sfuncomp_group: }
3793           \MathMLintent { :silent } { ( }
3794           \MathMLarg { l }
3795           {
3796             \__spintent_inner_sep:
3797             \exp_args:Ne \MathMLintent
3798             { _coma:infix ( \__spintent_sfun_refs_tl ) }
3799             { \__spintent_sfun_args: }
3800           }
3801           \MathMLintent { :silent } { ) }
3802         }
3803       }
3804     }
3805     \tl_set:Ne \__spintent_sfun_call_tl { $c ( \__spintent_sfun_refs_tl ) }
3806     \exp_args:NV \MathMLintent \__spintent_sfun_call_tl
3807     {
3808       \MathMLarg { c } { \__spintent_inner_sep: \__spintent_sfuncomp_group: }
3809       \MathMLintent { :silent } { ( }
3810       \__spintent_sfun_args:
3811       \MathMLintent { :silent } { ) }
3812     }
3813   }
3814 }
3815 { \__spintent_sfuncomp_group: }
3816 }
3817 }

```

(End of definition for `\__spintent_sfuncomp_mathml:`)

`\__spintent_sfuncomp_exec:nn`

Receives the optional `<keys>` in ‘#1’ and the `{<argument>}` in ‘#2’. Parses first, reporting `spfuncomp-invalid-arg` on error whatever `silent` is.

```

3818 \cs_new_protected:Nn \__spintent_sfuncomp_exec:nn
3819 {
3820   \keys_set:nn { spintent / spfuncomp } {#1}
3821   \__spintent_luafun_sfuncomp_parse_and_set:n { \exp_not:n {#2} }
3822   \str_if_eq:VnTF \__spintent_sfuncomp_luaset_error_str { true }

```

```

3823     {
3824         \msg_error:nne { spintent } { spfuncomp-invalid-arg }
3825         { \exp_not:n {#2} }
3826     }
3827     {
3828         \int_set:Nn \l__spintent_spfuncomp_n_int { \l__spintent_spfuncomp_luaset_n_str }
3829         \__spintent_spfuncomp_mathml:
3830     }
3831 }

```

(End of definition for `\__spintent_spfuncomp_exec:nn`.)

`\spfuncomp` The public command enforces *math mode*, opens a local group (with the scope attribute, for the calls that an argument may contain) and delegates to `\__spintent_spfuncomp_exec:nn`.

```

3832 \NewDocumentCommand \spfuncomp { O{} m }
3833 {
3834     \mode_if_math:TF
3835     {
3836         \group_begin:
3837         \attribute \l__spintent_scope_attr_int = 1
3838         \__spintent_spfuncomp_exec:nn {#1} {#2}
3839         \group_end:
3840     }
3841     { \msg_error:nnn { spintent } { need-math-mode } { \spfuncomp } }
3842 }

```

(End of definition for `\spfuncomp`. This function is documented on page 36.)

12.38 The commands `\spdom`, `\sprec` and `\spcod`

The user commands `\spdom`, `\sprec` and `\spcod` typeset the domain `\operatorname{Dom}(f)`, the range `\operatorname{Rec}(f)` and the codomain `\operatorname{Cod}(f)` of a function. They share one implementation, `\__spintent_spdom_exec:nnnnn`, which receives the command name, the operator, the spoken word, the *keys* and the *argument*. The Lua function `\__spintent_luafun_spdom_parse_and_set:n` accepts exactly a function name, as `\spfun` does (a letter with an optional subscript or primes, and the optional exponent `^{\text{-1}}`), and leaves it in the shared `\l_@@_geo_luaset_*` variables. The whole is tagged with a native `\MathMLintent` whose *intent* is `[_prefix-]dominio-de:prefix($f)` (recorrido, codominio); the operator, the parentheses are silenced and the name carries `arg="f"` wrapped in its own literal *intent* (`_f-`, `_F-doble-prima-`, `_f-inversa-`, ...) so `MathCAT` neither announces a capital letter nor reads the inverse as a power. No `read-cmd` nor `concept` keys exist: the native reading of `Dom(f)` is not useful, and the concept is already the spoken word.

12.38.1 Definition of variables for `\spdom`

```

\l__spintent_spdom_cmd_str
\l__spintent_spdom_op_tl
\l__spintent_spdom_word_tl
\l__spintent_spdom_prefix_tl
\l__spintent_spdom_intent_arg_tl
\l__spintent_spdom_luaset_error_str

```

Command name (for the messages), operator and word (set by each public command), the value of the key `prefix`, the *intent* being built and the error flag set from Lua.

```

3843 \str_new:N \l__spintent_spdom_cmd_str
3844 \tl_new:N \l__spintent_spdom_op_tl
3845 \tl_new:N \l__spintent_spdom_word_tl
3846 \tl_new:N \l__spintent_spdom_prefix_tl
3847 \tl_new:N \l__spintent_spdom_intent_arg_tl
3848 \bool_new:N \l__spintent_spdom_silent_bool
3849 \str_new:N \l__spintent_spdom_luaset_error_str

```

(End of definition for `\l__spintent_spdom_cmd_str` and others.)

12.38.2 Definition of error messages for `\spdom`

`spdom-invalid-arg`

The “error” `spdom-invalid-arg` is raised when the argument is not a function name. It receives the command name and the argument, and is raised whatever the value of `silent`.

```

3850 \msg_new:nnn { spintent } { spdom-invalid-arg }
3851 {
3852     Invalid~argument~!#2'~for~\c_backslash_str #1.
3853 }

```

(End of definition for `spdom-invalid-arg`.)

12.38.3 Definition of keys for \spdom

prefix We define the *(keys)* prefix and silent.

```

silent 3854 \keys_define:n { spintent / spdom }
unknown 3855 {
3856     prefix .code:n =
3857     { \__spintent_sanitize_affix:Nn \l__spintent_spdom_prefix_tl {#1} },
3858     prefix .initial:n = ,
3859     prefix .value_required:n = true,
3860     silent .bool_set:N = \l__spintent_spdom_silent_bool,
3861     silent .initial:n = false,
3862     silent .value_required:n = true,
3863     unknown .code:n =
3864     {
3865         \str_set_eq:NN \l__spintent_command_name_str \l__spintent_spdom_cmd_str
3866         \__spintent_unknown_keys_cmd:n {#1}
3867     },
3868 }
```

(End of definition for prefix, silent, and unknown.)

12.38.4 Internal functions for \spdom

__spintent_spdom_name: Tags the name with `arg="f"` through a native `\MathMLarg`; inside it, a native `\MathMLintent` gives the name its literal `intent`. Both groups start with the inner separator, so each one gets its own `<mrow>` even when the name is a single node.

```

3869 \cs_new_protected:Nn \__spintent_spdom_name:
3870 {
3871     \MathMLarg { f }
3872     {
3873         \__spintent_inner_sep:
3874         \MathMLintent { _ \l__spintent_geo_luaset_intent_str - }
3875         { \__spintent_inner_sep: \__spintent_spfun_name_plain: }
3876     }
3877 }
```

(End of definition for __spintent_spdom_name:.)

__spintent_spdom_mathml: Calls `\__spintent_invisible_sep:` first, then wraps operator, parentheses and name in a native `\MathMLintent`. With `silent` set to `true` the `intent` is `_-` and nothing inside is annotated.

```

3878 \cs_new_protected:Nn \__spintent_spdom_mathml:
3879 {
3880     \__spintent_invisible_sep:
3881     \bool_if:NTF \l__spintent_spdom_silent_bool
3882     {
3883         \MathMLintent { _- }
3884         {
3885             \__spintent_inner_sep:
3886             \operatorname { \l__spintent_spdom_op_tl }
3887             ( \__spintent_spfun_name_plain: )
3888         }
3889     }
3890     {
3891         \tl_set:Ne \l__spintent_spdom_intent_arg_tl
3892         {
3893             -
3894             \tl_if_empty:NF \l__spintent_spdom_prefix_tl
3895             { \tl_use:N \l__spintent_spdom_prefix_tl - }
3896             \l__spintent_spdom_word_tl -de:prefix( $f )
3897         }
3898         \MathMLintent { \l__spintent_spdom_intent_arg_tl }
3899         {
3900             \MathMLintent { :silent } { \operatorname { \l__spintent_spdom_op_tl } }
3901             \MathMLintent { :silent } { ( }
3902             \__spintent_spdom_name:
3903             \MathMLintent { :silent } { ) }
3904         }
3905     }
3906 }
```

(End of definition for __spintent_spdom_mathml:.)

`\__spintent_spdom_exec:nnnnn`

Receives the command name in ‘#1’, the operator in ‘#2’, the spoken word in ‘#3’, the optional $\langle keys \rangle$ in ‘#4’ and the $\{\langle argument \rangle\}$ in ‘#5’. Parses first, reporting `spdom-invalid-arg` on error whatever `silent` is.

```

3907 \cs_new_protected:Nn \__spintent_spdom_exec:nnnnn
3908 {
3909   \str_set:Nn \l__spintent_spdom_cmd_str {#1}
3910   \tl_set:Nn \l__spintent_spdom_op_tl {#2}
3911   \tl_set:Nn \l__spintent_spdom_word_tl {#3}
3912   \keys_set:nn { spintent / spdom } {#4}
3913   \__spintent_luafun_spdom_parse_and_set:n { \exp_not:n {#5} }
3914   \str_if_eq:VnTF \l__spintent_spdom_luaset_error_str { true }
3915   {
3916     \msg_error:nnee { spintent } { spdom-invalid-arg }
3917     {#1} { \exp_not:n {#5} }
3918   }
3919   { \__spintent_spdom_mathml: }
3920 }

```

(End of definition for `\__spintent_spdom_exec:nnnnn`.)

`\spdom` The public commands enforce *math mode*, open a local group, and delegate to `\__spintent_spdom_exec:nnnnn`.

```

\sprec
\spcod
3921 \NewDocumentCommand \spdom { O{} m }
3922 {
3923   \mode_if_math:TF
3924   {
3925     \group_begin:
3926     \__spintent_spdom_exec:nnnnn { spdom } { Dom } { dominio } {#1} {#2}
3927     \group_end:
3928   }
3929   { \msg_error:nnn { spintent } { need-math-mode } { \spdom } }
3930 }
3931 \NewDocumentCommand \sprec { O{} m }
3932 {
3933   \mode_if_math:TF
3934   {
3935     \group_begin:
3936     \__spintent_spdom_exec:nnnnn { sprec } { Rec } { recorrido } {#1} {#2}
3937     \group_end:
3938   }
3939   { \msg_error:nnn { spintent } { need-math-mode } { \sprec } }
3940 }
3941 \NewDocumentCommand \spcod { O{} m }
3942 {
3943   \mode_if_math:TF
3944   {
3945     \group_begin:
3946     \__spintent_spdom_exec:nnnnn { spcod } { Cod } { codominio } {#1} {#2}
3947     \group_end:
3948   }
3949   { \msg_error:nnn { spintent } { need-math-mode } { \spcod } }
3950 }

```

(End of definition for `\spdom`, `\sprec`, and `\spcod`. These functions are documented on page 36.)

12.39 The function `\__spintent_mathmlarg_point:nnn`

`\l__spintent_mathmlarg_point_ref_tl`
`\l__spintent_mathmlarg_point_mode_str`
`\l__spintent_mathmlarg_point_kv_tl`
`\l__spintent_mathmlarg_point_silent_bool`

Scratch variables for `\__spintent_mathmlarg_point:nnn`: the reference name (#1), and the calling command’s own `read-cmd/silent` state, looked up by name (#2).

```

3951 \tl_new:N \l__spintent_mathmlarg_point_ref_tl
3952 \str_new:N \l__spintent_mathmlarg_point_mode_str
3953 \tl_new:N \l__spintent_mathmlarg_point_kv_tl
3954 \bool_new:N \l__spintent_mathmlarg_point_silent_bool

```

(End of definition for `\l__spintent_mathmlarg_point_ref_tl` and others.)

`\__spintent_mathmlarg_point:nnn`
`\__spintent_mathmlarg_point:nnV`

Annotates one point (#3) with `arg` and, when applicable, a literal `intent`, without wrapping it in its own `<mrow>`. Concept `intent` is passed empty on purpose: the caller applies it externally, via a native `\MathMLintent`.

With `\l__spintent_geo_luaset_tipo_str` equal to `sub`, for `read-cmd=school` the letter alone is tagged through `\__spintent_luafun_mrow_intent_arg:nn`, and the subscript is appended separately, silenced. A subscript of more than one digit is not annotated natively — the queue would hand its digits to the next

point's claim — but typeset plain, with one extra silent claim per digit. For `read-cmd=mathcat`, letter and subscript are typeset together, tagged as a whole through a native `\MathMLarg`.

```

3955 \cs_new_protected:Nn \__spintent_mathmlarg_point:nnn
3956 {
3957   \tl_set:Nn \__spintent_mathmlarg_point_ref_tl {#1}
3958   \__spintent_luafun_geo_point_parse_and_set:n { \exp_not:n {#3} }
3959   \bool_set_eq:Nc \__spintent_mathmlarg_point_silent_bool { \__spintent_ #2 _silent_bool }
3960   \str_set_eq:Nc \__spintent_mathmlarg_point_mode_str { \__spintent_ #2 _mode_str }
3961   \__spintent_invisible_sep:
3962   \str_case:VnF \__spintent_geo_luaset_tipo_str
3963   {
3964     { sub }
3965     {
3966       \str_if_eq:VnTF \__spintent_mathmlarg_point_mode_str { school }
3967       {
3968         \tl_set:Nc \__spintent_mathmlarg_point_kv_tl
3969         { arg = "\__spintent_mathmlarg_point_ref_tl", intent = "_ \__spintent_geo_lua
3970         \__spintent_luafun_mrow_intent_arg:nn
3971         { } { \__spintent_mathmlarg_point_kv_tl }
3972         \int_compare:nNnTF { \str_count:N \__spintent_geo_luaset_sub_str } > { 1 }
3973         {
3974           \int_step_inline:nnnn { 1 } { 1 } { \str_count:N \__spintent_geo_luaset_sub_
3975           { \__spintent_luafun_mrow_intent_arg:nn { } { intent = ":silent" } }
3976           {
3977             \__spintent_geo_luaset_letra_str
3978             \c_math_subscript_token { \__spintent_geo_luaset_sub_str }
3979           }
3980         }
3981         {
3982           {
3983             \__spintent_geo_luaset_letra_str
3984             \c_math_subscript_token
3985             { \MathMLintent { :silent } { \__spintent_geo_luaset_sub_str } }
3986           }
3987         }
3988       }
3989     }
3990     {
3991       \__spintent_inner_sep:
3992       \MathMLarg { \__spintent_mathmlarg_point_ref_tl }
3993       {
3994         {
3995           \__spintent_geo_luaset_letra_str
3996           \c_math_subscript_token { \__spintent_geo_luaset_sub_str }
3997         }
3998       }
3999     }
4000   }
4001 }

```

With `\__spintent_geo_luaset_tipo_str` equal to `prima`, the same distinction applies: `read-cmd=school` tags the letter with a literal `intent` and appends the mark unsilenced (a prime is not spoken on the base, so nothing to repeat); `read-cmd=mathcat` tags letter and mark together through a native `\MathMLarg`.

```

4002 { prima }
4003 {
4004   \str_if_eq:VnTF \__spintent_mathmlarg_point_mode_str { school }
4005   {
4006     \tl_set:Nc \__spintent_mathmlarg_point_kv_tl
4007     { arg = "\__spintent_mathmlarg_point_ref_tl", intent = "_ \__spintent_geo_lua
4008     \__spintent_luafun_mrow_intent_arg:nn
4009     { } { \__spintent_mathmlarg_point_kv_tl }
4010     { \__spintent_geo_luaset_letra_str \__spintent_geo_luaset_sub_str }
4011   }
4012   {
4013     {
4014       \__spintent_inner_sep:
4015       \MathMLarg { \__spintent_mathmlarg_point_ref_tl }
4016       {
4017         { \__spintent_geo_luaset_letra_str \__spintent_geo_luaset_sub_str }
4018       }
4019     }

```

```

4020     }
4021   }
4022 }

```

Otherwise (no subscript, no prime), the same distinction applies to the combined `\l__spintent_geo_luaset_print_tl` as a whole — the shared queue for `school` (or when `silent` is `true`), a native `\MathMLarg` for `mathcat`, never mixed within the same formula, since the queue does not recognize a native `\MathMLarg` claim as already taken.

```

4023   {
4024     \bool_if:NTF \l__spintent_mathmlarg_point_silent_bool
4025     {
4026       \tl_set:Nc \l__spintent_mathmlarg_point_kv_tl
4027       { arg = "\l__spintent_mathmlarg_point_ref_tl", intent = "_" }
4028       \__spintent_luafun_mrow_intent_arg:nn
4029       { } % debe ir vacío
4030       { \l__spintent_mathmlarg_point_kv_tl }
4031       { \l__spintent_geo_luaset_print_tl }
4032     }
4033   {
4034     \str_if_eq:VnTF \l__spintent_mathmlarg_point_mode_str { school }
4035     {
4036       \tl_set:Nc \l__spintent_mathmlarg_point_kv_tl
4037       { arg = "\l__spintent_mathmlarg_point_ref_tl", intent = "_ \l__spintent_geo_luas"
4038       \__spintent_luafun_mrow_intent_arg:nn
4039       { } % debe ir vacío
4040       { \l__spintent_mathmlarg_point_kv_tl }
4041       { \l__spintent_geo_luaset_print_tl }
4042     }
4043     {
4044       \MathMLarg { \l__spintent_mathmlarg_point_ref_tl }
4045       { \l__spintent_geo_luaset_print_tl }
4046     }
4047   }
4048 }
4049 }
4050 \cs_generate_variant:Nn \__spintent_mathmlarg_point:nnn { nnV }

```

(End of definition for `\l__spintent_mathmlarg_point:nnn`.)

12.40 The command `\spPoint`

The user-level command `\spPoint` provides a semantic interface for a single geometric point. It delegates argument validation to the Lua module `spintent.lua`, constructing a valid MathCAT intent.

12.40.1 Definition of variables for `\spPoint`

Internal variables holding the values set by `read-cmd` and `prefix`.

```

\l__spintent_spPoint_mode_str
\l__spintent_spPoint_prefix_tl
\l__spintent_spPoint_intent_arg_tl
4051 \str_new:N \l__spintent_spPoint_mode_str
4052 \tl_new:N \l__spintent_spPoint_prefix_tl
4053 \tl_new:N \l__spintent_spPoint_intent_arg_tl

```

(End of definition for `\l__spintent_spPoint_mode_str`, `\l__spintent_spPoint_prefix_tl`, and `\l__spintent_spPoint_intent_arg_tl`.)

12.40.2 Definition of error messages for `\spPoint`

`spPoint-invalid-point`

The “error” `spPoint-invalid-point` is raised when the argument passed in ‘`#1`’ is not a single uppercase letter with an optional subscript or prime modifier.

```

4054 \msg_new:nnnn { spintent } { spPoint-invalid-point }
4055 {
4056   Invalid~point~format~in~\token_to_str:N \spPoint:~'#1'.
4057 }
4058 {
4059   The~argument~must~be~a~single~uppercase~letter~with~an~optional~
4060   subscript~(\_{...})~or~prime~(').\
4061 }

```

(End of definition for `spPoint-invalid-point`.)

12.40.3 Definition of keys for \spPoint

We define the *(keys)* `read-cmd`, `prefix`, `silent` and `concept`.

```

4062 \keys_define:nn { spintent / spPoint }
4063 {
4064   read-cmd      .choices:nn = { school , mathcat }
4065   {
4066     \int_case:nn { \l_keys_choice_int }
4067     {
4068       {1} { \str_set:Nn \l__spintent_spPoint_mode_str { school } }
4069       {2} { \str_set:Nn \l__spintent_spPoint_mode_str { mathcat } }
4070     }
4071   },
4072   read-cmd      .initial:n = school,
4073   read-cmd      .value_required:n = true,
4074   read-cmd / unknown .code:n =
4075     \msg_error:nneee { spintent } { unknown-choice }
4076     { read-cmd } { school,~mathcat } { \exp_not:n {#1} },
4077   prefix        .code:n =
4078     { \__spintent_sanitize_affix:Nn \l__spintent_spPoint_prefix_tl {#1} },
4079   prefix        .initial:n = ,
4080   prefix        .value_required:n = true,
4081   silent        .bool_set:N = \l__spintent_spPoint_silent_bool,
4082   silent        .initial:n = false,
4083   silent        .value_required:n = true,
4084   concept       .bool_set:N = \l__spintent_spPoint_concept_bool,
4085   concept       .initial:n = true,
4086   concept       .value_required:n = true,
4087   unknown      .code:n =
4088     {
4089       \str_set:Nn \l__spintent_command_name_str { spPoint }
4090       \__spintent_unknown_keys_cmd:n {#1}
4091     },
4092 }

```

(End of definition for `read-cmd` and others.)

12.40.4 Internal functions for \spPoint

__spintent_spPoint_print:n

The internal function `\__spintent_spPoint_print:n` processes the *(argument)* passed in ‘#1’. Calls `\__spintent_luafun_geo_point_parse_and_set:n`, reporting `spPoint-invalid-point` on error. If `silent` is `true` or `concept` is `false`, sets `\l__spintent_spPoint_intent_arg_tl` to `_ $\$pt$` — forced further to `_-` when `silent` is `true`, whatever `read-cmd` is: a composite point (one with a subscript) is not reliably silenced by the per-point `:silent` alone. Otherwise sets it to `_punto:prefix( $\$pt$ )`, with `prefix` placed after the leading `_` when given. Finally calls `\__spintent_spPoint_mathml:n`.

```

4093 \cs_new_protected:Nn \__spintent_spPoint_print:n
4094 {
4095   \__spintent_luafun_geo_point_parse_and_set:n { \exp_not:n {#1} }
4096   \str_if_eq:VnT \l__spintent_geo_luaseta_error_str { true }
4097   {
4098     \msg_error:nne { spintent } { spPoint-invalid-point } { \exp_not:n {#1} }
4099   }
4100   \bool_if:nTF
4101   {
4102     \bool_lazy_or:pnn
4103     { \l__spintent_spPoint_silent_bool }
4104     { ! \l__spintent_spPoint_concept_bool }
4105   }
4106   {
4107     \tl_set:Nn \l__spintent_spPoint_intent_arg_tl { _( $\$pt$ ) }
4108     \bool_if:NT \l__spintent_spPoint_silent_bool
4109     {
4110       \tl_set:Nn \l__spintent_spPoint_intent_arg_tl { _- }
4111     }
4112   }
4113   {
4114     \tl_set:Nn \l__spintent_spPoint_intent_arg_tl
4115     {
4116       -
4117       \tl_if_empty:NF \l__spintent_spPoint_prefix_tl
4118       {
4119         \tl_use:N \l__spintent_spPoint_prefix_tl -

```

```

4120         }
4121         punto:prefix($pt)
4122     }
4123 }
4124 \__spintenc_spPoint_mathml:n { #1 }
4125 }

```

(End of definition for __spintenc_spPoint_print:n.)

__spintenc_spPoint_mathml:n

The function `\__spintenc_spPoint_mathml:n` calls `\__spintenc_invisible_sep:` first, then wraps `\__spintenc_mathmlarg_point:nnn` — which parses ‘#1’, tags it `arg="pt"`, and silences a real subscript or prime on its own — inside a native `\MathMLintent` carrying `\__spintenc_spPoint_intent_arg_tl` as the concept `intent`, with `\__spintenc_inner_sep:` first inside that group.

```

4126 \cs_new_protected:Nn \__spintenc_spPoint_mathml:n
4127 {
4128     \__spintenc_invisible_sep:
4129     \MathMLintent { \__spintenc_spPoint_intent_arg_tl }
4130     {
4131         \__spintenc_inner_sep:
4132         \__spintenc_mathmlarg_point:nnn { pt } { spPoint } { #1 }
4133     }
4134 }

```

(End of definition for __spintenc_spPoint_mathml:n.)

__spintenc_spPoint_exec:nnn

The function `\__spintenc_spPoint_exec:nnn` receives the optional `{<keys>}` in ‘#1’ and the point `{<argument>}` in ‘#2’, delegating to `\__spintenc_spPoint_print:n`. With ‘#3’ given (the optional coordinate), silences the gap between the point and the coordinate first (otherwise read as an implicit product), then temporarily aliases `\__spintenc_spcoord_silent_bool` to `\__spintenc_spPoint_silent_bool` and calls `\__spintenc_luafun_spcoord_parse_and_set:n` directly, reporting `spcoord-invalid-arg` on error (whatever `silent` says), then `\__spintenc_spcoord_set_wrap_parens_size:` and `\__spintenc_spcoord_math`

```

4135 \cs_new_protected:Nn \__spintenc_spPoint_exec:nnn
4136 {
4137     \keys_set:nn { spintenc / spPoint } { #1 }
4138     \__spintenc_spPoint_print:n { #2 }
4139     \tl_if_novalue:nF {#3}
4140     {
4141         \MathMLintent { :silent } { \c__spintenc_invisible_sep_int } % silencio el "por"
4142         \bool_set_eq:NN \__spintenc_spcoord_silent_bool \__spintenc_spPoint_silent_bool
4143         \tl_set:Nn \__spintenc_spcoord_raw_arg_tl {#3}
4144         \__spintenc_luafun_spcoord_parse_and_set:n { \exp_not:n {#3} }
4145         \str_if_eq:VnT \__spintenc_spcoord_luaerror_str { true }
4146         { \msg_error:nnn { spintenc } { spcoord-invalid-arg } { \exp_not:n {#3} } }
4147         \__spintenc_spcoord_set_wrap_parens_size:
4148         \str_if_eq:VnF \__spintenc_spcoord_luaerror_str { true }
4149         { \__spintenc_spcoord_mathml: }
4150     }
4151 }

```

(End of definition for __spintenc_spPoint_exec:nnn.)

\spoint

The public command `\spoint` enforces *math mode*, opens a local group, and delegates execution to `\__spintenc_spPoint_exec:nnn`.

```

4152 \NewDocumentCommand \spoint { 0{} m d() }
4153 {
4154     \mode_if_math:TF
4155     {
4156         \group_begin:
4157         \attribute \__spintenc_scope_attr_int = 1
4158         \__spintenc_spPoint_exec:nnn { #1 } { #2 } { #3 }
4159         \group_end:
4160     }
4161     { \msg_error:nnn { spintenc } { need-math-mode } { \spoint } }
4162 }

```

(End of definition for \spoint. This function is documented on page 19.)

12.41 The command \spvec

The user command `\spvec` supports two types of notation for the input `{<argument>}`: a pair of points (with `\overrightarrow` applied), or a one-letter name (with `\vec` applied instead). Both notations are automatically detected by the Lua module. An optional coordinate, in parentheses, may follow the `{<argument>}`.

12.41.1 Definition of variables for \spvec

The variable `\__spintent_spvec_mode_str` stores the value set by the key `read-cmd` and the variable `\__spintent_spvec_prefix_tl` stores the value set by the key `prefix`.

```

4163 \str_new:N \__spintent_spvec_mode_str
4164 \tl_new:N \__spintent_spvec_prefix_tl
4165 \tl_new:N \__spintent_spvec_intent_arg_tl
4166 \tl_new:N \__spintent_spvec_kv_tl
4167 \seq_new:N \__spintent_spvec_pt_seq
4168 \tl_new:N \__spintent_spvec_refs_tl
4169 \int_new:N \__spintent_spvec_pt_idx_int

```

(End of definition for `\__spintent_spvec_mode_str` and others.)

12.41.2 Definition of error messages for \spvec

spvec-invalid-arg

The “error” `spvec-invalid-arg` is raised when the argument fails to parse, when a one-letter name carries a prime, or when the point list is not exactly two points.

```

4170 \msg_new:nnn { spintent } { spvec-invalid-arg }
4171 {
4172   Invalid~argument~'#1'~for~\token_to_str:N \spvec.
4173 }

```

(End of definition for `spvec-invalid-arg`.)

12.41.3 Definition of keys for \spvec

silent
concept
autofit
read-cmd
prefix
unknown

We define the *(keys)* `silent`, `concept`, `autofit`, `read-cmd` and `prefix`.

```

4174 \keys_define:nn { spintent / spvec }
4175 {
4176   silent      .bool_set:N = \__spintent_spvec_silent_bool,
4177   silent      .initial:n = false,
4178   silent      .value_required:n = true,
4179   concept     .bool_set:N = \__spintent_spvec_concept_bool,
4180   concept     .initial:n = true,
4181   concept     .value_required:n = true,
4182   autofit     .bool_set:N = \__spintent_spvec_autofit_bool,
4183   autofit     .initial:n = false,
4184   autofit     .value_required:n = true,
4185   read-cmd    .choices:nn = { school , mathcat }
4186   {
4187     \int_case:nn { \_keys_choice_int }
4188     {
4189       {1} { \str_set:Nn \__spintent_spvec_mode_str { school } }
4190       {2} { \str_set:Nn \__spintent_spvec_mode_str { mathcat } }
4191     }
4192   },
4193   read-cmd     .initial:n = school,
4194   read-cmd     .value_required:n = true,
4195   read-cmd / unknown .code:n =
4196     \msg_error:nneee { spintent } { unknown-choice }
4197     { read-cmd } { school,~mathcat } { \exp_not:n {##1} },
4198   prefix       .code:n =
4199     { \__spintent_sanitize_affix:Nn \__spintent_spvec_prefix_tl {##1} },
4200   prefix       .initial:n = { },
4201   prefix       .value_required:n = true,
4202   unknown      .code:n =
4203     {
4204       \str_set:Nn \__spintent_command_name_str { spvec }
4205       \__spintent_unknown_keys_cmd:n {##1}
4206     },
4207 }

```

(End of definition for `silent` and others.)

12.41.4 Internal functions for \spvec

__spintent_spvec_print:n

The internal function `\__spintent_spvec_print:n` processes the *{(argument)}* passed in ‘#1’. Calls `\__spintent_luafun_geo_parse_and_set:n`, reporting `spvec-invalid-arg` on error, on a proper name with `\__spintent_geo_luasets_tipo_str` equal to `prima` (a one-letter vector cannot carry a prime), or on a point list that is not exactly two points. Otherwise calls `\__spintent_spvec_mathml:n`.

```

4208 \cs_new_protected:Nn \__spintent_spvec_print:n
4209 {
4210   \__spintent_luafun_geo_parse_and_set:n { \exp_not:n {##1} }

```

```

4211 \str_if_eq:VnT \l__spintent_geo_luaset_is_name_str { true }
4212 {
4213   \str_if_eq:VnT \l__spintent_geo_luaset_tipo_str { prima }
4214   { \str_set:Nn \l__spintent_geo_luaset_error_str { true } }
4215 }
4216 \str_if_eq:VnT \l__spintent_geo_luaset_is_name_str { false }
4217 {
4218   \int_compare:nT { \clist_count:N \l__spintent_geo_luaset_points_clist != 2 }
4219   { \str_set:Nn \l__spintent_geo_luaset_error_str { true } }
4220 }
4221 \str_if_eq:VnTF \l__spintent_geo_luaset_error_str { true }
4222 { \msg_error:nne { spintent } { spvec-invalid-arg } { \exp_not:n {#1} } }
4223 { \l__spintent_spvec_mathml:n { #1 } }
4224 }

```

(End of definition for `\l__spintent_spvec_print:n`.)

`\l__spintent_spvec_mathml:n`

The function `\l__spintent_spvec_mathml:n` branches on `\l__spintent_geo_luaset_is_name_str`. For a proper name (a single letter, no prime), the concept `intent` is `_ $\$pt$` when `silent` is `true` or `concept` is `false` — forced further to `_` — when `silent` is `true`, whatever `read-cmd` is; otherwise `_vector:prefix( $\$pt$ )`, with `prefix` placed after the leading `_` when given. With `\l__spintent_geo_luaset_tipo` equal to `sub`, `read-cmd=school` tags `\vec{letra}` through `\l__spintent_luafun_mrow_intent_arg:nn` with a literal `kv`, the subscript appended and silenced; `read-cmd=mathcat` tags `\vec{...}` through a native `\MathMLarg` instead, the subscript silenced only under `silent`. `autofit` inserts a `\vphantom{b}` inside `\vec{...}` to normalize its height. Otherwise (no subscript), the same `kv` logic wraps `\vec{print\_tl}` as a whole.

```

4225 \cs_new_protected:Nn \l__spintent_spvec_mathml:n
4226 {
4227   \str_if_eq:VnTF \l__spintent_geo_luaset_is_name_str { true }
4228   {
4229     \bool_if:nTF
4230     {
4231       \bool_lazy_or_p:nn
4232       { \l__spintent_spvec_silent_bool }
4233       { ! \l__spintent_spvec_concept_bool }
4234     }
4235     {
4236       \tl_set:Nn \l__spintent_spvec_intent_arg_tl { _( $\$pt$ ) }
4237       \bool_if:NT \l__spintent_spvec_silent_bool
4238       { \tl_set:Nn \l__spintent_spvec_intent_arg_tl { _- } }
4239     }
4240     {
4241       \tl_set:Ne \l__spintent_spvec_intent_arg_tl
4242       {
4243         -
4244         \tl_if_empty:NF \l__spintent_spvec_prefix_tl
4245         { \tl_use:N \l__spintent_spvec_prefix_tl - }
4246         vector:prefix( $\$pt$ )
4247       }
4248     }
4249   }
4250   \str_case:VnF \l__spintent_geo_luaset_tipo_str
4251   {
4252     sub
4253     {
4254       \str_if_eq:VnTF \l__spintent_spvec_mode_str { school }
4255       {
4256         \bool_if:NTF \l__spintent_spvec_silent_bool
4257         { \tl_set:Nn \l__spintent_spvec_kv_tl { arg = "pt", intent = "_-" } }
4258         {
4259           \tl_set:Ne \l__spintent_spvec_kv_tl
4260           { arg = "pt", intent = "_ \l__spintent_geo_luaset_intent_str -" }
4261         }
4262         \l__spintent_invisible_sep:
4263         \MathMLintent { \l__spintent_spvec_intent_arg_tl }
4264         {
4265           \l__spintent_inner_sep:
4266           \l__spintent_luafun_mrow_intent_arg:nn
4267           { } { \l__spintent_spvec_kv_tl }
4268           {
4269             \vec

```

```

4270         \bool_if:NT \l__spintent_spmat_autofit_bool
4271         { \vphantom{ b } }
4272         \l__spintent_geo_luaset_letra_str
4273     }
4274     \c_math_subscript_token
4275     { \MathMLintent { :silent } { \l__spintent_geo_luaset_sub_str } }
4276 }
4277 }
4278 }
4279 {
4280     \__spintent_invisible_sep:
4281     \MathMLintent { \l__spintent_spmat_intent_arg_tl }
4282     {
4283         \__spintent_inner_sep:
4284         \vec
4285         {
4286             \bool_if:NT \l__spintent_spmat_autofit_bool
4287             { \vphantom{ b } }
4288             \MathMLarg{ pt } { \l__spintent_geo_luaset_letra_str }
4289         }
4290         \bool_if:NT \l__spintent_spmat_silent_bool
4291         { \c_math_subscript_token { \l__spintent_geo_luaset_sub_str } }
4292     }
4293     \bool_if:NF \l__spintent_spmat_silent_bool
4294     { \c_math_subscript_token { \l__spintent_geo_luaset_sub_str } }
4295 }
4296 }
4297 }
4298 {
4299     \bool_if:NTF \l__spintent_spmat_silent_bool
4300     { \tl_set:Nn \l__spintent_spmat_kv_tl { arg = "pt", intent = "- " } }
4301     {
4302         \tl_set:Nn \l__spintent_spmat_kv_tl
4303         { arg = "pt", intent = "- \l__spintent_geo_luaset_intent_str - " }
4304     }
4305     \__spintent_invisible_sep:
4306     \MathMLintent { \l__spintent_spmat_intent_arg_tl }
4307     {
4308         \__spintent_inner_sep:
4309         \__spintent_luafun_mrow_intent_arg:nn
4310         { } { \l__spintent_spmat_kv_tl }
4311     }
4312     \vec
4313     {
4314         \bool_if:NT \l__spintent_spmat_autofit_bool
4315         { \vphantom{ b } }
4316         \l__spintent_geo_luaset_print_tl
4317     }
4318 }
4319 }
4320 }
4321 }

```

For a pair of points, `\l__spintent_geo_luaset_points_clist` is split into `\l__spintent_spmat_pt_seq` and walked to build `\l__spintent_spmat_refs_tl`, the `$pts1`, `$pts2` reference list. The concept `intent` is `-` when `silent` is `true`, `_(refs)` when `concept` is `false` or `_vector:prefix(refs)` otherwise, applied with a native `\MathMLintent` wrapping `\overrightarrow`. The sequence is walked again, calling `\__spintent_mathmlarg_point:nnn` once per point.

```

4322     {
4323         \seq_set_from_clist:NN \l__spintent_spmat_pt_seq \l__spintent_geo_luaset_points_clist
4324         \tl_clear:N \l__spintent_spmat_refs_tl
4325         \int_zero:N \l__spintent_spmat_pt_idx_int
4326         \seq_map_inline:Nn \l__spintent_spmat_pt_seq
4327         {
4328             \int_incr:N \l__spintent_spmat_pt_idx_int
4329             \tl_if_empty:NF \l__spintent_spmat_refs_tl
4330             { \tl_put_right:Nn \l__spintent_spmat_refs_tl { , } }
4331             \tl_put_right:Nn \l__spintent_spmat_refs_tl { $ pts \int_use:N \l__spintent_spmat_pt_idx_int }
4332         }
4333         \bool_if:NTF
4334         {

```

```

4335     \bool_lazy_or_p:nn
4336     { \l__spintent_svec_silent_bool }
4337     { ! \l__spintent_svec_concept_bool }
4338   }
4339   {
4340     \tl_set:Nx \l__spintent_svec_intent_arg_tl { _ ( \l__spintent_svec_refs_tl ) }
4341     \bool_if:NT \l__spintent_svec_silent_bool
4342     { \tl_set:Nn \l__spintent_svec_intent_arg_tl { _- } }
4343   }
4344   {
4345     \tl_set:Nx \l__spintent_svec_intent_arg_tl
4346     {
4347       -
4348       \tl_if_empty:NF \l__spintent_svec_prefix_tl
4349       { \tl_use:N \l__spintent_svec_prefix_tl - }
4350       vector:prefix( \l__spintent_svec_refs_tl )
4351     }
4352   }
4353   \__spintent_invisible_sep:
4354   \MathMLintent { \l__spintent_svec_intent_arg_tl }
4355   {
4356     \__spintent_inner_sep:
4357     \overrightarrow
4358     {
4359       \int_zero:N \l__spintent_svec_pt_idx_int
4360       \seq_map_inline:Nn \l__spintent_svec_pt_seq
4361       {
4362         \int_incr:N \l__spintent_svec_pt_idx_int
4363         \__spintent_mathmlarg_point:nnn
4364         { pts \int_use:N \l__spintent_svec_pt_idx_int }
4365         { svec }
4366         { ##1 }
4367       }
4368     }
4369   }
4370 }
4371 }

```

(End of definition for `\__spintent_svec_mathml:n`.)

`\__spintent_svec_exec:nnn`

The function `\__spintent_svec_exec:nnn` receives the optional *keys* in ‘#1’ and the $\langle argument \rangle$ in ‘#2’, delegating to `\__spintent_svec_print:n`. With ‘#3’ given (the optional coordinate), aliases `\l__spintent_spcoord_silent_bool` to `\l__spintent_svec_silent_bool` — reverted when the calling group closes — and calls `\__spintent_luafun_spcoord_parse_and_set:n` directly, reporting `spcoord-invalid-arg` on error (whatever `silent` says), then `\__spintent_spcoord_set_wrap_parens_size:` and `\__spintent_spcoord_mathml:` — the same rendering functions `\spcoord` itself uses.

```

4372 \cs_new_protected:Nn \__spintent_svec_exec:nnn
4373 {
4374   \keys_set:nn { spintent / svec } { #1 }
4375   \__spintent_svec_print:n { #2 }
4376   \tl_if_novalue:NF {#3}
4377   {
4378     \bool_if:NTF \l__spintent_svec_silent_bool
4379     { \MathMLintent { :silent } { \equal } }
4380     { \equal } % notación de vector =( , )
4381     \MathMLintent { :silent } { \c__spintent_invisible_sep_int } % previene lectura de intervalo
4382     \bool_set_eq:NN \l__spintent_spcoord_silent_bool \l__spintent_svec_silent_bool
4383     \tl_set:Nn \l__spintent_spcoord_raw_arg_tl {#3}
4384     \__spintent_luafun_spcoord_parse_and_set:n { \exp_not:n {#3} }
4385     \str_if_eq:VnT \l__spintent_spcoord_luaerror_str { true }
4386     { \msg_error:nnn { spintent } { spcoord-invalid-arg } { \exp_not:n {#3} } }
4387     \__spintent_spcoord_set_wrap_parens_size:
4388     \str_if_eq:VnF \l__spintent_spcoord_luaerror_str { true }
4389     { \__spintent_spcoord_mathml: }
4390   }
4391 }

```

(End of definition for `\__spintent_svec_exec:nnn`.)

`\svec`

The public command `\svec` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_svec_exec:nnn`.

```

4392 \NewDocumentCommand \svec { 0{ } m d() }

```

```

4393 {
4394   \mode_if_math:TF
4395   {
4396     \group_begin:
4397       \attribute \l__spintent_scope_attr_int = 1
4398       \__spintent_sprvec_exec:nnn { #1 } { #2 } { #3 }
4399     \group_end:
4400   }
4401   { \msg_error:nnn { spintent } { need-math-mode } { \sprvec } }
4402 }

```

(End of definition for \sprvec. This function is documented on page 20.)

12.42 The command \sprecta

The user command `\sprecta` supports two types of notation for the input $\langle argument \rangle$: when the argument contains two points (separated by an optional comma), `\overleftarrow{\rightarrow}` is applied; when the argument is a single point (without comma) or a lowercase letter, it is formatted without any visual decorator. Both notations are automatically detected by the Lua module.

12.42.1 Definition of variables for \sprecta

The variable `\l__spintent_sprecta_mode_str` store the values set by the key `read-cmd` and the variable `\l__spintent_sprayo_prefix_tl` store the value set by the key `prefix`.

```

\l__spintent_sprecta_mode_str
\l__spintent_sprecta_prefix_tl
\l__spintent_sprecta_pt_seq
\l__spintent_sprecta_refs_tl
\l__spintent_sprecta_intent_arg_tl
\l__spintent_sprecta_kv_tl
\l__spintent_sprecta_pt_idx_int
4403 \str_new:N \l__spintent_sprecta_mode_str
4404 \tl_new:N \l__spintent_sprecta_prefix_tl
4405 \seq_new:N \l__spintent_sprecta_pt_seq
4406 \tl_new:N \l__spintent_sprecta_refs_tl
4407 \tl_new:N \l__spintent_sprecta_intent_arg_tl
4408 \tl_new:N \l__spintent_sprecta_kv_tl
4409 \int_new:N \l__spintent_sprecta_pt_idx_int

```

(End of definition for \l__spintent_sprecta_mode_str and others.)

12.42.2 Definition of error messages for \sprecta

The “error” `sprecta-invalid-points` is raised when the argument matches neither the two-point notation nor the name notation.

```

4410 \msg_new:nnnn { spintent } { sprecta-invalid-points }
4411 {
4412   Invalid~format~in~\token_to_str:N \sprecta:~'#1'~\msg_line_context:.
4413 }
4414 {
4415   See~the~documentation~of~\token_to_str:N \sprecta~for~the~accepted~argument~forms.
4416 }

```

(End of definition for sprecta-invalid-points.)

12.42.3 Definition of keys for \sprecta

We define the $\langle keys \rangle$ `read-cmd`, `prefix` and `silent`.

```

read-cmd
prefix
silent
unknown
4417 \keys_define:nn { spintent / sprecta }
4418 {
4419   read-cmd .choices:nn = { school , mathcat }
4420   {
4421     \int_case:nn { \l_keys_choice_int }
4422     {
4423       {1} { \str_set:Nn \l__spintent_sprecta_mode_str { school } }
4424       {2} { \str_set:Nn \l__spintent_sprecta_mode_str { mathcat } }
4425     }
4426   },
4427   read-cmd .initial:n = school,
4428   read-cmd .value_required:n = true,
4429   read-cmd / unknown .code:n =
4430     \msg_error:nnee { spintent } { unknown-choice }
4431     { read-cmd } { school,~mathcat } { \exp_not:n {#1} },
4432   prefix .code:n =
4433     { \__spintent_sanitize_affix:Nn \l__spintent_sprecta_prefix_tl {#1} },
4434   prefix .initial:n = ,
4435   prefix .value_required:n = true,
4436   silent .bool_set:N = \l__spintent_sprecta_silent_bool,
4437   silent .initial:n = false,
4438   silent .value_required:n = true,
4439   concept .bool_set:N = \l__spintent_sprecta_concept_bool,
4440   concept .initial:n = true,

```

```

4441     concept          .value_required:n = true,
4442     unknown          .code:n =
4443     {
4444         \str_set:Nn \l__spintent_command_name_str { sprecta }
4445         \__spintent_unknown_keys_cmd:n {#1}
4446     },
4447 }

```

(End of definition for read-cmd and others.)

12.42.4 Internal functions for \sprecata

`\__spintent_sprecata_print:n` The internal function `\__spintent_sprecata_print:n` processes the $\langle argument \rangle$ passed in ‘#1’. Calls `\__spintent_luafun_geo_parse_and_set:n`, reporting sprecta-invalid-points on error, then `\__spintent_`

```

4448 \cs_new_protected:Nn \__spintent_sprecata_print:n
4449 {
4450     \__spintent_luafun_geo_parse_and_set:n { \exp_not:n {#1} }
4451     \str_if_eq:VnT \l__spintent_geo_luaset_error_str { true }
4452     {
4453         \msg_error:nne { spintent } { sprecta-invalid-points } { \exp_not:n {#1} }
4454     }
4455     \__spintent_sprecata_mathml:n { #1 }
4456 }

```

(End of definition for `\__spintent_sprecata_print:n`.)

`\__spintent_sprecata_mathml:n` The function `\__spintent_sprecata_mathml:n` branches on `\l__spintent_geo_luaset_is_name_str`. For a proper name (m, l, ...), the concept `intent` is `_( $\$pt$ )` when `silent` is `true` or `concept` is `false` — forced further to `_-` when `silent` is `true` and `read-cmd` is not `school`; otherwise `_recta:prefix( $\$pt$ )`, with `prefix` placed after the leading `_` when given. With `read-cmd=school`, the sub and prima cases tag the letter with a literal kv that is `_-` when `silent` is `true`.

With `\l__spintent_geo_luaset_tipo_str` equal to `sub`, `read-cmd=school` tags the base through `\__spintent_luafun_mrow_intent_arg:nn` with a literal kv, the subscript appended and silenced; `read-cmd=mathcat` tags base and subscript together through a native `\MathMLarg` instead.

```

4457 \cs_new_protected:Nn \__spintent_sprecata_mathml:n
4458 {
4459     \str_if_eq:VnTF \l__spintent_geo_luaset_is_name_str { true }
4460     {
4461         \__spintent_invisible_sep:
4462         \bool_if:nTF
4463         {
4464             \bool_lazy_or_p:nn
4465             { \l__spintent_sprecata_silent_bool }
4466             { ! \l__spintent_sprecata_concept_bool }
4467         }
4468         {
4469             \tl_set:Nn \l__spintent_sprecata_intent_arg_tl { _( $\$pt$ ) }
4470             \bool_if:NT \l__spintent_sprecata_silent_bool
4471             {
4472                 \str_if_eq:VnF \l__spintent_sprecata_mode_str { school }
4473                 { \tl_set:Nn \l__spintent_sprecata_intent_arg_tl { _- } }
4474             }
4475         }
4476         {
4477             \tl_set:Ne \l__spintent_sprecata_intent_arg_tl
4478             {
4479                 -
4480                 \tl_if_empty:NF \l__spintent_sprecata_prefix_tl
4481                 { \tl_use:N \l__spintent_sprecata_prefix_tl - }
4482                 recta:prefix( $\$pt$ )
4483             }
4484         }
4485         \str_case:VnF \l__spintent_geo_luaset_tipo_str
4486         {
4487             { sub }
4488             {
4489                 \str_if_eq:VnTF \l__spintent_sprecata_mode_str { school }
4490                 {
4491                     \bool_if:NTF \l__spintent_sprecata_silent_bool
4492                     { \tl_set:Nn \l__spintent_sprecata_kv_tl { arg = "pt", intent = "_-" } }
4493                     {
4494                         \tl_set:Ne \l__spintent_sprecata_kv_tl

```

```

4495         { arg = "pt", intent = "_ \l__spintent_geo_luaset_intent_str -" }
4496     }
4497     \MathMLintent { \l__spintent_spectra_intent_arg_tl }
4498     {
4499         \__spintent_inner_sep:
4500         \__spintent_luafun_mrow_intent_arg:nn
4501         { } { \l__spintent_spectra_kv_tl }
4502     }
4503     \l__spintent_geo_luaset_letra_str
4504     \c_math_subscript_token
4505     { \MathMLintent { :silent } { \l__spintent_geo_luaset_sub_str } }
4506 }
4507 }
4508 }

```

For `read-cmd=mathcat`, the same sub case tags base and subscript together through a native `\MathMLarg` instead — never mixed with the shared queue, since the queue does not recognize a native `\MathMLarg` claim as already taken.

```

4509     {
4510         \MathMLintent { \l__spintent_spectra_intent_arg_tl }
4511         {
4512             {
4513                 \__spintent_inner_sep:
4514                 \MathMLarg { pt }
4515                 {
4516                     {
4517                         \l__spintent_geo_luaset_letra_str
4518                         \c_math_subscript_token { \l__spintent_geo_luaset_sub_str }
4519                     }
4520                 }
4521             }
4522         }
4523     }
4524 }

```

With `\l__spintent_geo_luaset_tipo_str` equal to `prima`, the same distinction applies: `read-cmd=school` tags the base with the literal kv and appends the mark, unsilenced (a prime is not spelled out on the base, so nothing to repeat); `read-cmd=mathcat` tags base and mark together through a native `\MathMLarg`.

```

4525     { prima }
4526     {
4527         \str_if_eq:VnTF \l__spintent_spectra_mode_str { school }
4528         {
4529             \bool_if:NTF \l__spintent_spectra_silent_bool
4530             { \tl_set:Nn \l__spintent_spectra_kv_tl { arg = "pt", intent = "- " } }
4531             {
4532                 \tl_set:Ne \l__spintent_spectra_kv_tl
4533                 { arg = "pt", intent = "_ \l__spintent_geo_luaset_intent_str -" }
4534             }
4535             \MathMLintent { \l__spintent_spectra_intent_arg_tl }
4536             {
4537                 \__spintent_inner_sep:
4538                 \__spintent_luafun_mrow_intent_arg:nn
4539                 { } { \l__spintent_spectra_kv_tl }
4540                 { \l__spintent_geo_luaset_letra_str \l__spintent_geo_luaset_sub_str }
4541             }
4542         }
4543         {
4544             \MathMLintent { \l__spintent_spectra_intent_arg_tl }
4545             {
4546                 {
4547                     \__spintent_inner_sep:
4548                     \MathMLarg { pt }
4549                     {
4550                         { \l__spintent_geo_luaset_letra_str \l__spintent_geo_luaset_sub_str }
4551                     }
4552                 }
4553             }
4554         }
4555     }
4556 }

```

Otherwise (no subscript, no prime), the same kv logic wraps the combined `\l__spintent_geo_luaset_print_tl` as a whole — the shared queue for `school`, its own `arg` alone for `mathcat`.

```

4557     {
4558       \bool_if:NTF \l__spintent_specta_silent_bool
4559       {
4560         \tl_set:Nn \l__spintent_specta_kv_tl { arg = "pt", intent = "-"} }
4561       }
4562       {
4563         \str_if_eq:VnTF \l__spintent_specta_mode_str { school }
4564         {
4565           \tl_set:Ne \l__spintent_specta_kv_tl
4566             { arg = "pt", intent = "- \l__spintent_geo_luaset_intent_str -" }
4567         }
4568         {
4569           \tl_set:Nn \l__spintent_specta_kv_tl { arg = "pt" }
4570         }
4571       }
4572       \__spintent_luafun_mrow_intent_arg:nn
4573       { \l__spintent_specta_intent_arg_tl }
4574       { \l__spintent_specta_kv_tl }
4575       { \l__spintent_geo_luaset_print_tl }
4576     }
4577   }

```

For a list of points, `\l__spintent_geo_luaset_points_clist` is split into `\l__spintent_specta_pt_seq` and walked to build `\l__spintent_specta_refs_tl`, the `$pt1, $pt2, ...` reference list. The concept `intent` is `_` when `silent` is `true`, `_` (refs) when `concept` is `false` or `_recta:prefix` (refs) otherwise, applied with a native `\MathMLintent` wrapping `\overleftrightharrow`. The sequence is walked again, calling `\__spintent_mathmlarg_point:nnn` once per point.

```

4578     {
4579       \seq_set_from_clist:NN \l__spintent_specta_pt_seq \l__spintent_geo_luaset_points_clist
4580       \tl_clear:N \l__spintent_specta_refs_tl
4581       \int_zero:N \l__spintent_specta_pt_idx_int
4582       \seq_map_inline:Nn \l__spintent_specta_pt_seq
4583       {
4584         \int_incr:N \l__spintent_specta_pt_idx_int
4585         \tl_if_empty:NF \l__spintent_specta_refs_tl
4586           { \tl_put_right:Nn \l__spintent_specta_refs_tl { , } }
4587         \tl_put_right:Ne \l__spintent_specta_refs_tl { $ pt \int_use:N \l__spintent_specta_
4588       }
4589       \bool_if:nTF
4590       {
4591         \bool_lazy_or:p:nn
4592         { \l__spintent_specta_silent_bool }
4593         { ! \l__spintent_specta_concept_bool }
4594       }
4595       {
4596         \tl_set:Ne \l__spintent_specta_intent_arg_tl { _ ( \l__spintent_specta_refs_tl ) }
4597         \bool_if:NT \l__spintent_specta_silent_bool
4598           { \tl_set:Nn \l__spintent_specta_intent_arg_tl { _- } }
4599       }
4600       {
4601         \tl_set:Ne \l__spintent_specta_intent_arg_tl
4602         {
4603           -
4604           \tl_if_empty:NF \l__spintent_specta_prefix_tl
4605             { \tl_use:N \l__spintent_specta_prefix_tl - }
4606           recta:prefix( \l__spintent_specta_refs_tl )
4607         }
4608       }
4609       \__spintent_invisible_sep:
4610       \MathMLintent { \l__spintent_specta_intent_arg_tl }
4611       {
4612         \__spintent_inner_sep:
4613         \overleftrightharrow
4614         {
4615           \int_zero:N \l__spintent_specta_pt_idx_int
4616           \seq_map_inline:Nn \l__spintent_specta_pt_seq
4617           {
4618             \int_incr:N \l__spintent_specta_pt_idx_int
4619             \__spintent_mathmlarg_point:nnn

```

```

4620         { pt \int_use:N \__spintent_sprecta_pt_idx_int }
4621         { sprecta }
4622         { ##1 }
4623     }
4624 }
4625 }
4626 }
4627 }

```

(End of definition for `\__spintent_sprecta_mathml:n`.)

`\__spintent_sprecta_exec:nn`

The function `\__spintent_sprecta_exec:nn` receives the optional `(keys)` passed in ‘#1’ and the `{(argument)}` passed in ‘#2’ delegating the rendering of the latter to `\__spintent_sprecta_print:n`.

```

4628 \cs_new_protected:Nn \__spintent_sprecta_exec:nn
4629 {
4630     \keys_set:nn { spintent / sprecta } { #1 }
4631     \__spintent_sprecta_print:n { #2 }
4632 }

```

(End of definition for `\__spintent_sprecta_exec:nn`.)

`\sprecta`

The public command `\sprecta` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_sprecta_exec:nn`.

```

4633 \NewDocumentCommand \sprecta { 0{} m }
4634 {
4635     \mode_if_math:TF
4636     {
4637         \group_begin:
4638         \attribute \l__spintent_scope_attr_int = 1
4639         \__spintent_sprecta_exec:nn { #1 } { #2 }
4640         \group_end:
4641     }
4642     { \msg_error:nnn { spintent } { need-math-mode } { \sprecta } }
4643 }

```

(End of definition for `\sprecta`. This function is documented on page 20.)

12.43 The command `\sprayo`

The user command `\sprayo` provides a semantic interface for a ray. The key `read-sty` selects the verbalized term, `rayo` or `semirrecta`, both rendered identically with `\overrightarrow`.

12.43.1 Definition of variables for `\sprayo`

`\l__spintent_sprayo_mode_str`
`\l__spintent_sprayo_prefix_tl`
`\l__spintent_sprayo_read_sty_str`

The variable `\l__spintent_sprayo_mode_str` store the values set by the key `read-cmd`; the variable `\l__spintent_sprayo_prefix_tl` store the value set by the key `prefix` and the variable `\l__spintent_sprayo_read_sty_str` stores the value set by the key `read-sty`.

🟢 The variables defined here are assigned using `.choices:nn` or `.code:n` within `\keys_define:nn`.

```

4644 \str_new:N \l__spintent_sprayo_mode_str
4645 \tl_new:N \l__spintent_sprayo_prefix_tl
4646 \str_new:N \l__spintent_sprayo_read_sty_str

```

(End of definition for `\l__spintent_sprayo_mode_str`, `\l__spintent_sprayo_prefix_tl`, and `\l__spintent_sprayo_read_sty_str`.)

12.43.2 Definition of error messages for `\sprayo`

`sprayo-invalid-points`

Raised when the argument is not a comma-separated pair of uppercase letters.

```

4647 \msg_new:nnnn { spintent } { sprayo-invalid-points }
4648 {
4649     Invalid~point~format~in~\token_to_str:N \sprayo:~'##1'~\msg_line_context:.
4650 }
4651 {
4652     See~the~documentation~of~\token_to_str:N \sprayo~for~the~accepted~point~formats.
4653 }

```

(End of definition for `sprayo-invalid-points`.)

12.43.3 Definition of keys for \sprayo

We define the *(keys)* read-cmd, prefix, silent, concept and read-sty.

```

read-cmd
prefix 4654 \keys_define:nn { spintent / sprayo }
silent 4655 {
read-sty 4656   read-cmd           .choices:nn = { school , mathcat }
unknown 4657   {
4658     \int_case:nn { \l_keys_choice_int }
4659     {
4660       {1} { \str_set:Nn \l__spintent_sprayo_mode_str { school } }
4661       {2} { \str_set:Nn \l__spintent_sprayo_mode_str { mathcat } }
4662     }
4663   },
4664   read-cmd           .initial:n = school,
4665   read-cmd           .value_required:n = true,
4666   read-cmd / unknown .code:n =
4667     \msg_error:nneee { spintent } { unknown-choice }
4668     { read-cmd } { school,~mathcat } { \exp_not:n {#1} },
4669   prefix             .code:n =
4670     { \__spintent_sanitize_affix:Nn \l__spintent_sprayo_prefix_tl {#1} },
4671   prefix             .initial:n = ,
4672   prefix             .value_required:n = true,
4673   silent             .bool_set:N = \l__spintent_sprayo_silent_bool,
4674   silent             .initial:n = false,
4675   silent             .value_required:n = true,
4676   concept            .bool_set:N = \l__spintent_sprayo_concept_bool,
4677   concept            .initial:n = true,
4678   concept            .value_required:n = true,
4679   read-sty           .choices:nn = { rayo , semirrecta }
4680   {
4681     \int_case:nn { \l_keys_choice_int }
4682     {
4683       {1} { \str_set:Nn \l__spintent_sprayo_read_sty_str { rayo } }
4684       {2} { \str_set:Nn \l__spintent_sprayo_read_sty_str { semirrecta } }
4685     }
4686   },
4687   read-sty           .initial:n = rayo,
4688   read-sty           .value_required:n = true,
4689   read-sty / unknown .code:n =
4690     \msg_error:nneee { spintent } { unknown-choice }
4691     { read-sty } { rayo,~semirrecta } { \exp_not:n {#1} },
4692   unknown           .code:n =
4693   {
4694     \str_set:Nn \l__spintent_command_name_str { sprayo }
4695     \__spintent_unknown_keys_cmd:n {#1}
4696   },
4697 }

```

(End of definition for read-cmd and others.)

12.43.4 Internal functions for \sprayo

Auxiliary variables used by __spintent_sprayo_mathml: to split \l__spintent_geo_luaset_points_clist and build the \$pts1,\$pts2,... reference list.

```

\l__spintent_sprayo_pt_seq
\l__spintent_sprayo_refs_tl
\l__spintent_sprayo_intent_arg_tl
\l__spintent_sprayo_pt_idx_int
4702 \seq_new:N \l__spintent_sprayo_pt_seq
4703 \tl_new:N \l__spintent_sprayo_refs_tl
4704 \tl_new:N \l__spintent_sprayo_intent_arg_tl
4705 \int_new:N \l__spintent_sprayo_pt_idx_int

```

(End of definition for \l__spintent_sprayo_pt_seq and others.)

The internal function __spintent_sprayo_print:n process the *{(argument)}* passed in ‘#1’. It calls __spintent_luafun_geo_parse_and_set:n to process the *{(argument)}*, returning an “error” if the input is invalid, then calls __spintent_sprayo_mathml:. Unlike \sprecta, \sprayo has no proper-name argument: it always takes a list of points, so no intent is built here.

```

4702 \cs_new_protected:Nn \__spintent_sprayo_print:n
4703 {
4704   \__spintent_luafun_geo_parse_and_set:n { \exp_not:n {#1} }
4705   \str_if_eq:VnT \l__spintent_geo_luaset_error_str { true }
4706   {
4707     \msg_error:nne { spintent } { sprayo-invalid-points } { \exp_not:n {#1} }
4708   }
4709   \__spintent_sprayo_mathml:

```

```
4710 }
```

(End of definition for `\__spintent_sprayo_print:n`.)

`\__spintent_sprayo_mathml:` The function `\__spintent_sprayo_mathml:` splits `\l__spintent_geo_luaset_points_clist` (set by Lua, one raw point per element) into `\l__spintent_sprayo_pt_seq` and walks it once to build `\l__spintent_sprayo_refs_tl`, the comma-separated `$pts1,$pts2,...` reference list, naming each point by position.

The concept `intent` is `_` when `silent` is `true`, `_`(refs) when `concept` is `false`; otherwise it is `_\l__spintent_sprayo_read_sty_str:prefix(refs)` — the concept word itself (`rayo` or `semirrecta`) comes from the `read-sty` key, not a literal, since `\sprayo` verbalizes under two names — prefixed with `prefix` when given. It is applied with a native `\MathMLintent` wrapping `\overrightarrow`, preceded by `\__spintent_inner_sep:` (a second sibling so the sole `<mrow>` does not collapse into `<math>`). The sequence is walked a second time, calling `\__spintent_mathmlarg_point:nnn` once per point with its `ptsn` reference name and `silent/read-cmd` looked up under `sprayo:` it alone decides `arg`/literal `intent` per point, so `concept` plays no role at that level.

```
4711 \cs_new_protected:Nn \__spintent_sprayo_mathml:
4712 {
4713   \seq_set_from_clist:NN \l__spintent_sprayo_pt_seq \l__spintent_geo_luaset_points_clist
4714   \tl_clear:N \l__spintent_sprayo_refs_tl
4715   \int_zero:N \l__spintent_sprayo_pt_idx_int
4716   \seq_map_inline:Nn \l__spintent_sprayo_pt_seq
4717   {
4718     \int_incr:N \l__spintent_sprayo_pt_idx_int
4719     \tl_if_empty:NF \l__spintent_sprayo_refs_tl
4720     { \tl_put_right:Nn \l__spintent_sprayo_refs_tl { , } }
4721     \tl_put_right:Ne \l__spintent_sprayo_refs_tl { $ pts \int_use:N \l__spintent_sprayo_pt_idx_int }
4722   }
4723   \bool_if:nTF
4724   {
4725     \bool_lazy_or_p:nn
4726     { \l__spintent_sprayo_silent_bool }
4727     { ! \l__spintent_sprayo_concept_bool }
4728   }
4729   {
4730     \tl_set:Ne \l__spintent_sprayo_intent_arg_tl { _ ( \l__spintent_sprayo_refs_tl ) }
4731     \bool_if:NT \l__spintent_sprayo_silent_bool
4732     { \tl_set:Nn \l__spintent_sprayo_intent_arg_tl { _- } }
4733   }
4734   {
4735     \tl_set:Ne \l__spintent_sprayo_intent_arg_tl
4736     {
4737       -
4738       \tl_if_empty:NF \l__spintent_sprayo_prefix_tl
4739       { \tl_use:N \l__spintent_sprayo_prefix_tl - }
4740       \l__spintent_sprayo_read_sty_str :prefix( \l__spintent_sprayo_refs_tl )
4741     }
4742   }
4743   \__spintent_invisible_sep:
4744   \MathMLintent { \l__spintent_sprayo_intent_arg_tl }
4745   {
4746     \__spintent_inner_sep:
4747     \overrightarrow
4748     {
4749       \int_zero:N \l__spintent_sprayo_pt_idx_int
4750       \seq_map_inline:Nn \l__spintent_sprayo_pt_seq
4751       {
4752         \int_incr:N \l__spintent_sprayo_pt_idx_int
4753         \__spintent_mathmlarg_point:nnn
4754         { pts \int_use:N \l__spintent_sprayo_pt_idx_int }
4755         { sprayo }
4756         { ##1 }
4757       }
4758     }
4759   }
4760 }
```

(End of definition for `\__spintent_sprayo_mathml:`.)

`\__spintent_sprayo_exec:nn` The function `\__spintent_sprayo_exec:nn` receives the optional `<keys>` passed in ‘#1’ and the `{<argument>}` passed in ‘#2’ delegating the rendering of the latter to `\__spintent_sprayo_print:n`.

```
4761 \cs_new_protected:Nn \__spintent_sprayo_exec:nn
```

```

4762 {
4763   \keys_set:nn { spintent / sprayo } { #1 }
4764   \__spintent_sprayo_print:n { #2 }
4765 }

```

(End of definition for __spintent_sprayo_exec:nn.)

\sprayo The public command `\sprayo` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_sprayo_exec:nn`.

```

4766 \NewDocumentCommand \sprayo { O{} m }
4767 {
4768   \mode_if_math:TF
4769   {
4770     \group_begin:
4771     \attribute \__spintent_scope_attr_int = 1
4772     \__spintent_sprayo_exec:nn { #1 } { #2 }
4773     \group_end:
4774   }
4775   { \msg_error:nnn { spintent } { need-math-mode } { \sprayo } }
4776 }

```

(End of definition for `\sprayo`. This function is documented on page 21.)

12.44 The commands `\spside` and `\spmside`

The user commands `\spside` and `\spmside` provide a semantic interface for a side, segment or trace determined by two points. The key `read-sty` selects the verbalized term — `lado`, `segmento`, or `trazo`, the latter used in straightedge-and-compass constructions — and the key `read-arg` overrides that term entirely with a free-form string. Additionally, the command `\spmside` provides the key `print-m`, controlling whether `\operatorname{m}` is typeset before the argument.

12.44.1 Definition of variables for `\spside` and `\spmside`

Each pair of variables is declared once for both `\spside` and `\spmside` via `\clist_map_inline:nn`. The variables `\__spintent_spside_mode_str` and `\__spintent_spside_mode_str` store the values set by the key `read-cmd`; the variables `\__spintent_spside_prefix_tl` and `\__spintent_spmside_prefix_tl` store the values set by the key `prefix`.

The variables `\__spintent_spside_read_sty_str` and `\__spintent_spmside_read_sty_str` stores the value set by the key `read-sty`.

The variables `\__spintent_spside_read_arg_tl` and `\__spintent_spmside_read_arg_tl` stores the value set by the key `read-arg`.

- The variables are defined here as they are assigned using `.choices:nn` or `.code:n` rather than a direct property within `\keys_define:nn`.

```

4777 \clist_map_inline:nn { side , mside }
4778 {
4779   \str_new:c { \__spintent_sp #1 _mode_str }
4780   \tl_new:c { \__spintent_sp #1 _prefix_tl }
4781   \str_new:c { \__spintent_sp #1 _read_sty_str }
4782   \tl_new:c { \__spintent_sp #1 _read_arg_tl }
4783 }

```

(End of definition for `\__spintent_spside_mode_str` and others.)

12.44.2 Definition of error messages for `\spside` and `\spmside`

`spside-invalid-points` `spmside-invalid-points` We define an error message for the commands `\spside` and `\spmside`. The “error” is triggered if the user includes a invalid “*point*” in the `{\argument}` passed in ‘#1’ to the command.

```

4784 \clist_map_inline:nn { side , mside }
4785 {
4786   \msg_new:nnnn { spintent } { sp #1 -invalid-points }
4787   {
4788     Invalid~point~format~in~##1:~'##2'~\msg_line_context:.
4789   }
4790   {
4791     See~the~documentation~of~##1~for~the~accepted~point~formats.
4792   }
4793 }

```

(End of definition for `spside-invalid-points` and `spmside-invalid-points`.)

12.44.3 Definition of keys for \spside and \spmside

We define the shared *(keys)* read-cmd, prefix, silent, concept, read-sty, read-arg and print-m.

```

4794 \clist_map_inline:nn { side , mside }
4795 {
4796   \keys_define:nn { spintent / sp #1 }
4797   {
4798     read-cmd      .choices:nn = { school , mathcat }
4799     {
4800       \int_case:nn { \l_keys_choice_int }
4801       {
4802         {1} { \str_set:cn { l__spintent_sp #1 _mode_str } { school } }
4803         {2} { \str_set:cn { l__spintent_sp #1 _mode_str } { mathcat } }
4804       }
4805     },
4806     read-cmd      .initial:n = school,
4807     read-cmd      .value_required:n = true,
4808     read-cmd / unknown .code:n =
4809       \msg_error:nneee { spintent } { unknown-choice }
4810       { read-cmd } { school,~mathcat } { \exp_not:n {##1} },
4811     prefix        .code:n =
4812       { \__spintent_sanitize_affix:cn { l__spintent_sp #1 _prefix_tl } {##1} },
4813     prefix        .initial:n = ,
4814     prefix        .value_required:n = true,
4815     silent        .bool_set:c = { l__spintent_sp #1 _silent_bool },
4816     silent        .initial:n = false,
4817     silent        .value_required:n = true,
4818     concept       .bool_set:c = { l__spintent_sp #1 _concept_bool },
4819     concept       .initial:n = true,
4820     concept       .value_required:n = true,
4821     read-sty      .choices:nn = { lado , segmento , trazo }
4822     {
4823       \int_case:nn { \l_keys_choice_int }
4824       {
4825         {1} { \str_set:cn { l__spintent_sp #1 _read_sty_str } { lado } }
4826         {2} { \str_set:cn { l__spintent_sp #1 _read_sty_str } { segmento } }
4827         {3} { \str_set:cn { l__spintent_sp #1 _read_sty_str } { trazo } }
4828       }
4829     },
4830     read-sty      .initial:n = lado,
4831     read-sty      .value_required:n = true,
4832     read-sty / unknown .code:n =
4833       \msg_error:nneee { spintent } { unknown-choice }
4834       { read-sty } { lado,~segmento,~trazo } { \exp_not:n {##1} },
4835     read-arg      .code:n =
4836     {
4837       \__spintent_sanitize_affix:cn { l__spintent_sp #1 _read_arg_tl } {##1}
4838     },
4839     read-arg      .initial:n = ,
4840     read-arg      .value_required:n = true,
4841     print-m       .bool_set:c = { l__spintent_sp #1 _print_m_bool },
4842     print-m       .initial:n = false,
4843     print-m       .value_required:n = true,
4844     unknown       .code:n =
4845     {
4846       \str_set:Nn \l__spintent_command_name_str { sp #1 }
4847       \__spintent_unknown_keys_cmd:n {##1}
4848     },
4849   }
4850 }

```

🔗 The keys `print-m` and `concept` are defined for both commands for simplicity, but has no effect on `\spside`.

(End of definition for read-cmd and others.)

```

\l__spintent_spside_intent_tl
\l__spintent_spside_pt_seq
\l__spintent_spside_refs_tl
\l__spintent_spside_intent_arg_tl
\l__spintent_spside_pt_idx_int
4851 \tl_new:N \l__spintent_spside_intent_tl
4852 \seq_new:N \l__spintent_spside_pt_seq
4853 \tl_new:N \l__spintent_spside_refs_tl
4854 \tl_new:N \l__spintent_spside_intent_arg_tl
4855 \int_new:N \l__spintent_spside_pt_idx_int

```

The variable `\l__spintent_spside_intent_tl` holds the concept word (`read-arg` or `read-sty`). The rest are scratch variables shared by both commands (the function itself is shared) to split `\l__spintent_geo_luaseta_point` and build the `$pts1, $pts2, ...` reference list.

(End of definition for `\l__spintent_spside_intent_tl` and others.)

`\__spintent_spside_process:nn`

The internal function `\__spintent_spside_process:nn` receives the short name of the command (`side` or `mside`) in ‘#1’ and the $\{\langle argument \rangle\}$ in ‘#2’. It calls `\__spintent_luafun_geo_parse_and_set:n` to process ‘#2’, returning an “error” if invalid, then builds `\l__spintent_spside_intent_tl`. When `read-arg` is empty, it is set from `read-sty`, prefixed with ‘medida-del-’ for `\spmside`. When `read-arg` is given, it is used as is, and ‘medida-del-’ is prepended only if `concept` is `true`: with `concept=false`, `read-arg` is the verbatim escape hatch for a full user-written phrase (e.g. `read-arg={medida de la hipotenusa}`), gender and article left to the user. No `intent` is built here: it needs the reference list, which `\__spintent_spside_mathml` builds after splitting the points.

```

4856 \cs_new_protected:Nn \__spintent_spside_process:nn
4857 {
4858   \__spintent_luafun_geo_parse_and_set:n { \exp_not:n {#2} }
4859   \str_if_eq:VnT \l__spintent_geo_luaset_error_str { true }
4860   {
4861     \msg_error:nnee { spintent } { sp #1 -invalid-points }
4862     { \c_backslash_str sp #1 } { \exp_not:n {#2} }
4863   }
4864   \tl_if_empty:cTF { \l__spintent_sp #1 _read_arg_tl }
4865   {
4866     \tl_set:Ne \l__spintent_spside_intent_tl { \str_use:c { \l__spintent_sp #1 _read_sty_str }
4867     \str_if_eq:eeT {#1} { mside }
4868     {
4869       \tl_put_left:Nn \l__spintent_spside_intent_tl { medida-del- }
4870     }
4871   }
4872   {
4873     \tl_set:Ne \l__spintent_spside_intent_tl { \tl_use:c { \l__spintent_sp #1 _read_arg_tl } }
4874     \bool_if:cT { \l__spintent_sp #1 _concept_bool }
4875     {
4876       \str_if_eq:eeT {#1} { mside }
4877       {
4878         \tl_put_left:Nn \l__spintent_spside_intent_tl { medida-del- }
4879       }
4880     }
4881   }
4882   \__spintent_spside_mathml:n { #1 }
4883 }

```

(End of definition for `\__spintent_spside_process:nn`.)

`\__spintent_spside_mathml:n`

The function `\__spintent_spside_mathml:n` receives the short command name in ‘#1’. It splits `\l__spintent_geo_luaset_points_clist` into `\l__spintent_spside_pt_seq` and walks it once to build `\l__spintent_spside_refs_tl`, the comma-separated $\$pts1, \$pts2, \dots$ reference list, naming each point by position.

```

4884 \cs_new_protected:Nn \__spintent_spside_mathml:n
4885 {
4886   \seq_set_from_clist:NN \l__spintent_spside_pt_seq \l__spintent_geo_luaset_points_clist
4887   \tl_clear:N \l__spintent_spside_refs_tl
4888   \int_zero:N \l__spintent_spside_pt_idx_int
4889   \seq_map_inline:Nn \l__spintent_spside_pt_seq
4890   {
4891     \int_incr:N \l__spintent_spside_pt_idx_int
4892     \tl_if_empty:NF \l__spintent_spside_refs_tl
4893     { \tl_put_right:Nn \l__spintent_spside_refs_tl { , } }
4894     \tl_put_right:Ne \l__spintent_spside_refs_tl { $ pts \int_use:N \l__spintent_spside_pt_idx_int }
4895   }

```

The concept `intent` is `_` when `silent` is `true`; otherwise, if `read-arg` is empty, it is `_(refs)` when `concept` is `false` or `\l__spintent_spside_intent_tl:prefix(refs)` when `true`; if `read-arg` is given, it is always `\l__spintent_spside_intent_tl:prefix(refs)` — `concept` never silences a user-given `read-arg` (see `\__spintent_spside_process:nn`). The `:prefix(...)` form is prefixed with `prefix` when given.

```

4896 \bool_if:cTF { \l__spintent_sp #1 _silent_bool }
4897 {
4898   \tl_set:Nn \l__spintent_spside_intent_arg_tl { _- }
4899 }
4900 {
4901   \tl_if_empty:cTF { \l__spintent_sp #1 _read_arg_tl }
4902   {

```

```

4903 \bool_if:cTF { l__spintent_sp #1 _concept_bool }
4904 {
4905   \tl_set:Ne \l__spintent_spside_intent_arg_tl
4906   {
4907     -
4908     \tl_if_empty:cF { l__spintent_sp #1 _prefix_tl }
4909     { \tl_use:c { l__spintent_sp #1 _prefix_tl } - }
4910     \l__spintent_spside_intent_tl :prefix( \l__spintent_spside_refs_tl )
4911   }
4912 }
4913 {
4914   \tl_set:Ne \l__spintent_spside_intent_arg_tl { _( \l__spintent_spside_refs_tl ) }
4915 }
4916 }
4917 {
4918   \tl_set:Ne \l__spintent_spside_intent_arg_tl
4919   {
4920     -
4921     \tl_if_empty:cF { l__spintent_sp #1 _prefix_tl }
4922     { \tl_use:c { l__spintent_sp #1 _prefix_tl } - }
4923     \l__spintent_spside_intent_tl :prefix( \l__spintent_spside_refs_tl )
4924   }
4925 }
4926 }

```

Applied with a native `\MathMLintent`, preceded by `\__spintent_inner_sep:` (a second sibling so the sole `<mrow>` does not collapse into `<math>`). Inside, for `#1=side` the points are always wrapped in `\overline`; for `#1=mside`, when `print-m` is true a silent `\operatorname{m}` precedes the `\overline`-wrapped points; when `false`, the points are typeset bare, with no `\overline` at all — the recursive lookup in the shared `mathml_filter` callback finds each point either way, whether nested inside `\overline` or sitting directly in the `\MathMLintent` group. In every case the sequence is walked a second time, calling `\__spintent_mathmlarg_point:nnn` once per point with its `ptsn` reference name and `silent/read-cmd` looked up under `sp#1`: it alone decides `arg/literal intent` per point, so `concept` plays no role at that level.

```

4927 \__spintent_invisible_sep: % (luamml issues #26)
4928 \MathMLintent { \l__spintent_spside_intent_arg_tl }
4929 {
4930   \__spintent_inner_sep:
4931   \str_if_eq:eeTF {#1} { side }
4932   {
4933     \overline { \__spintent_spside_points_loop:n {#1} }
4934   }
4935   {
4936     \bool_if:cTF { l__spintent_sp #1 _print_m_bool }
4937     {
4938       \__spintent_luafun_mrow_intent_arg:nn
4939       { } { intent = ":silent" }
4940       { \operatorname { m } }
4941       \overline { \__spintent_spside_points_loop:n {#1} }
4942     }
4943     {
4944       \__spintent_spside_points_loop:n {#1}
4945     }
4946   }
4947 }
4948 }

```

(End of definition for `\__spintent_spside_mathml:n`.)

`\__spintent_spside_points_loop:n`

Walks `\__spintent_spside_pt_seq` a second time, calling `\__spintent_mathmlarg_point:nnn` once per point. Factored out of `\__spintent_spside_mathml:n` since it is needed identically in three places there (`side`, `mside` with and without `print-m`); it takes the short command name in ‘`#1`’ again since a separate macro does not inherit its caller’s own `#1`.

```

4949 \cs_new_protected:Nn \__spintent_spside_points_loop:n
4950 {
4951   \int_zero:N \l__spintent_spside_pt_idx_int
4952   \seq_map_inline:Nn \l__spintent_spside_pt_seq
4953   {
4954     \int_incr:N \l__spintent_spside_pt_idx_int
4955     \__spintent_mathmlarg_point:nnn
4956     { pts \int_use:N \l__spintent_spside_pt_idx_int }
4957     { sp #1 }

```

```

4958         { ##1 }
4959     }
4960 }

```

(End of definition for `\__spintent_spside_points_loop:n`.)

`\__spintent_spside_exec:nnn` The function `\__spintent_spside_exec:nnn` receives the *short name* of the command passed in ‘#1’, the optional *keys* passed in ‘#2’ and the *{argument}* passed in ‘#3’, delegating the rendering to `\__spintent_spside_process:nn`.

```

4961 \cs_new_protected:Nn \__spintent_spside_exec:nnn
4962 {
4963     \keys_set:nn { spintent / sp #1 } { #2 }
4964     \__spintent_spside_process:nn { #1 } { #3 }
4965 }

```

(End of definition for `\__spintent_spside_exec:nnn`.)

`\spside` Both public commands `\spside` and `\spmside` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_spside_exec:nnn`.

```

4966 \NewDocumentCommand \spside { 0{} m }
4967 {
4968     \mode_if_math:TF
4969     {
4970         \group_begin:
4971         \attribute \__spintent_scope_attr_int = 1
4972         \__spintent_spside_exec:nnn { side } { #1 } { #2 }
4973         \group_end:
4974     }
4975     { \msg_error:nnn { spintent } { need-math-mode } { \spside } }
4976 }
4977 \NewDocumentCommand \spmside { 0{} m }
4978 {
4979     \mode_if_math:TF
4980     {
4981         \group_begin:
4982         \attribute \__spintent_scope_attr_int = 1
4983         \__spintent_spside_exec:nnn { mside } { #1 } { #2 }
4984         \group_end:
4985     }
4986     { \msg_error:nnn { spintent } { need-math-mode } { \spmside } }
4987 }

```

(End of definition for `\spside` and `\spmside`. These functions are documented on page 21.)

12.45 The commands `\spangle` and `\spmangle`

The user commands `\spangle` and `\spmangle`, provide a semantic interface for angles, following the same generic pattern as `\spside/\spmside`. The key `print-m` chooses between the `\angle/\rightanglesqr` pair, typeset together with an explicit `\operatorname{m}`, and the `\measuredangle/\measuredrightangle` pair, which already conveys the notion of measure on its own; the key `recto` independently chooses the right-angle variant of whichever pair is active.

12.45.1 Definition of variables for `\spangle` and `\spmangle`

```

\l__spintent_spangle_mode_str
\l__spintent_spmangle_mode_str
\l__spintent_spangle_prefix_tl
\l__spintent_spmangle_prefix_tl

```

Each pair of variables is declared once for both `\spangle` and `\spmangle` via `\clist_map_inline:nn`. The variables `\l__spintent_spangle_mode_str` and `\l__spintent_spmangle_mode_str` store the values set by the key `read-cmd`; the variables `\l__spintent_spangle_prefix_tl` and `\l__spintent_spmangle_prefix_tl` store the values set by the key `prefix`.

🍎 The variables defined here are assigned using `.choices:nn` or `.code:n` within `\keys_define:nn`.

```

4988 \clist_map_inline:nn { angle , mangle }
4989 {
4990     \str_new:c { l__spintent_sp #1 _mode_str }
4991     \tl_new:c { l__spintent_sp #1 _prefix_tl }
4992 }

```

(End of definition for `\l__spintent_spangle_mode_str` and others.)

12.45.2 Definition of error messages for \spangle and \spmangle

spangle-invalid-arg
spmangle-invalid-arg

We define an “error” message for the commands `\spangle` and `\spmangle`. The “error” is triggered if the user includes a invalid `{⟨argument⟩}` passed in ‘#1’ to the command.

```

4993 \clist_map_inline:nn { angle , mangle }
4994 {
4995     \msg_new:nnnn { spintent } { sp #1 -invalid-arg }
4996     {
4997         Invalid~format~in~##1:~'##2'~\msg_line_context:.
4998     }
4999     {
5000         See~the~documentation~of~##1~for~the~accepted~argument~forms.
5001     }
5002 }
```

(End of definition for spangle-invalid-arg and spmangle-invalid-arg.)

12.45.3 Definition of keys for \spangle and \spmangle

read-cmd
prefix
recto
print-m
silent
concept
unknown

We define the shared `⟨keys⟩` `read-cmd`, `prefix`, `recto`, `print-m`, `silent` and `concept`,

```

5003 \clist_map_inline:nn { angle , mangle }
5004 {
5005     \keys_define:nn { spintent / sp #1 }
5006     {
5007         read-cmd          .choices:nn = { school , mathcat }
5008         {
5009             \int_case:nn { \l_keys_choice_int }
5010             {
5011                 {1} { \str_set:cn { l__spintent_sp #1 _mode_str } { school } }
5012                 {2} { \str_set:cn { l__spintent_sp #1 _mode_str } { mathcat } }
5013             }
5014         },
5015         read-cmd          .initial:n = school,
5016         read-cmd          .value_required:n = true,
5017         read-cmd / unknown .code:n =
5018             \msg_error:nneee { spintent } { unknown-choice }
5019             { read-cmd } { school,~mathcat } { \exp_not:n {##1} },
5020         prefix            .code:n =
5021             { \__spintent_sanitize_affix:cn { l__spintent_sp #1 _prefix_tl } {##1} },
5022         prefix            .initial:n = ,
5023         prefix            .value_required:n = true,
5024         recto              .bool_set:c = { l__spintent_sp #1 _recto_bool },
5025         recto              .initial:n = false,
5026         recto              .value_required:n = true,
5027         print-m            .bool_set:c = { l__spintent_sp #1 _print_m_bool },
5028         print-m            .initial:n = false,
5029         print-m            .value_required:n = true,
5030         silent             .bool_set:c = { l__spintent_sp #1 _silent_bool },
5031         silent             .initial:n = false,
5032         silent             .value_required:n = true,
5033         concept            .bool_set:c = { l__spintent_sp #1 _concept_bool },
5034         concept            .initial:n = true,
5035         concept            .value_required:n = true,
5036         unknown            .code:n =
5037             {
5038                 \str_set:Nn \l__spintent_command_name_str { sp #1 }
5039                 \__spintent_unknown_keys_cmd:n {##1}
5040             },
5041     }
5042 }
```

• The keys `print-m` and `concept` are defined for both commands for simplicity, but has no effect on `\spangle`.

(End of definition for read-cmd and others.)

12.45.4 Internal functions for \spangle and \spmangle

__spintent_spangle_sym:n

The internal function `\__spintent_spangle_sym:n` dispatches on #1, `print-m` and `recto` to select one of four symbols: `\angle/\rightanglesqr` when `print-m` is true; `\measuredangle/\measuredrightangle` otherwise. When #1 is `angle`, only `recto` applies, choosing between `\angle` and `\rightangle`.

```

5043 \cs_new_protected:Nn \__spintent_spangle_sym:n
5044 {
5045     \str_if_eq:eeTF {#1} { mangle }
5046     {
```

```

5047     \bool_if:cTF { l__spintent_sp #1 _print_m_bool }
5048     {
5049         \bool_if:cTF { l__spintent_sp #1 _recto_bool }
5050         { \__spintent_right_angle_sqr: } { \angle }
5051     }
5052     {
5053         \bool_if:cTF { l__spintent_sp #1 _recto_bool }
5054         { \measuredrightangle } { \measuredangle }
5055     }
5056 }
5057 {
5058     \bool_if:cTF { l__spintent_sp #1 _recto_bool }
5059     { \__spintent_right_angle_sqr: } { \angle }
5060 }
5061 }

```

(End of definition for `\__spintent_spangle_sym:n`.)

The variable `\__spintent_spangle_intent_tl` holds the concept word (ángulo or medida-del-ángulo, ‘-recto’-suffixed when `recto` is `true`). The rest are scratch variables shared by both commands to split `\__spintent_geo_luaset_points_clist` and build the reference list — used only in the tres-puntos and un-punto cases (see `\__spintent_spangle_mathml:n`).

```

5062 \tl_new:N \__spintent_spangle_intent_tl
5063 \seq_new:N \__spintent_spangle_pt_seq
5064 \tl_new:N \__spintent_spangle_refs_tl
5065 \tl_new:N \__spintent_spangle_intent_arg_tl
5066 \int_new:N \__spintent_spangle_pt_idx_int
5067 \str_new:N \__spintent_spangle_kv_tl
5068 \bool_new:N \__spintent_spangle_print_m_now_bool

```

(End of definition for `\__spintent_spangle_intent_tl` and others.)

The internal function `\__spintent_spangle_process:nn` receives the short name of the command (`angle` or `mangle`) in ‘#1’ and the `{\argument}` in ‘#2’. Calls `\__spintent_luafun_geo_angulo_parse_and_set:n`, reporting `sp#1-invalid-arg` on error. For a proper name that is a single letter (not numeric), also reports the same error when that letter is lowercase — angle names take an uppercase letter, number, or Greek letter, never a lowercase Latin one; the length check keeps a Greek letter command such as `\alpha` out of this comparison. Then sets `\__spintent_spangle_intent_tl` to ángulo (or medida-del-ángulo for `\spmangle`), suffixed with `-recto` when `recto` is `true`, and calls `\__spintent_spangle_mathml:n`.

```

5069 \cs_new_protected:Nn \__spintent_spangle_process:nn
5070 {
5071     \__spintent_luafun_geo_angulo_parse_and_set:n { \exp_not:n {#2} }
5072     \str_if_eq:VnT \__spintent_geo_luaset_error_str { true }
5073     {
5074         \msg_error:nnee { spintent } { sp #1 -invalid-arg }
5075         { \c_backslash_str sp #1 } { \exp_not:n {#2} }
5076     }
5077     \str_if_eq:VnT \__spintent_geo_luaset_angulo_case_str { nombre }
5078     {
5079         \str_if_eq:VnF \__spintent_geo_luaset_nombre_es_num_str { true }
5080         {
5081             \int_compare:nT { \str_count:N \__spintent_geo_luaset_letra_str = 1 }
5082             {
5083                 \str_if_eq:eeF { \__spintent_geo_luaset_letra_str } { \str_uppercase:n { \__spintent_geo_luaset_letra_str } }
5084                 {
5085                     \msg_error:nnee { spintent } { sp #1 -invalid-arg }
5086                     { \c_backslash_str sp #1 } { \exp_not:n {#2} }
5087                 }
5088             }
5089         }
5090     }
5091     \str_if_eq:eeTF {#1} { mangle }
5092     {
5093         \tl_set:Nn \__spintent_spangle_intent_tl { medida-del-ángulo }
5094     }
5095     {
5096         \tl_set:Nn \__spintent_spangle_intent_tl { ángulo }
5097     }
5098     \bool_if:cT { l__spintent_sp #1 _recto_bool }
5099     {
5100         \tl_put_right:Nn \__spintent_spangle_intent_tl { -recto }

```

```

5101     }
5102     \__spintent_spangle_mathml:n {#1}
5103 }

```

(End of definition for __spintent_spangle_process:nn.)

__spintent_spangle_mathml:n

The function `\__spintent_spangle_mathml:n` branches on `\l__spintent_geo_luaset_angulo_case_str`. For `nombre`, the symbol (`\__spintent_spangle_sym:n`) is tagged through `\__spintent_luafun_mrow_intent_arg` on its own, with `\l__spintent_spangle_intent_tl` alone (bare, no `$`-reference: `nombre` never refers to more than one thing) — `_` when `silent` is `true` or `concept` is `false`. With `silent true`, the name that follows is silenced as a whole.

```

5104 \cs_new_protected:Nn \__spintent_spangle_mathml:n
5105 {
5106     \__spintent_invisible_sep: % (luamml issues #26)
5107     \str_if_eq:VnTF \l__spintent_geo_luaset_angulo_case_str { nombre }
5108     {
5109         \bool_if:cTF { \l__spintent_sp #1 _silent_bool }
5110         {
5111             \tl_set:Nn \l__spintent_spangle_intent_arg_tl { _- }
5112         }
5113         {
5114             \bool_if:cTF { \l__spintent_sp #1 _concept_bool }
5115             {
5116                 \tl_set:Nn \l__spintent_spangle_intent_arg_tl
5117                 {
5118                     -
5119                     \tl_if_empty:cF { \l__spintent_sp #1 _prefix_tl }
5120                     { \tl_use:c { \l__spintent_sp #1 _prefix_tl } - }
5121                     \l__spintent_spangle_intent_tl
5122                 }
5123             }
5124             {
5125                 \tl_set:Nn \l__spintent_spangle_intent_arg_tl { _- }
5126             }
5127         }
5128         \tl_set:Nn \l__spintent_spangle_kv_tl
5129         { intent = "\l__spintent_spangle_intent_arg_tl" }
5130         \__spintent_luafun_mrow_intent_arg:nn
5131         { } { \l__spintent_spangle_kv_tl }
5132         { \__spintent_spangle_sym:n {#1} }
5133         \bool_if:cTF { \l__spintent_sp #1 _silent_bool }
5134         {
5135             \__spintent_luafun_mrow_intent_arg:nn { } { intent = "- " }
5136             { \l__spintent_geo_luaset_print_tl }
5137         }
5138     }
5139     {
5140         \str_if_eq:VnTF \l__spintent_geo_luaset_nombre_es_num_str { true }
5141         {
5142             \tl_set:Nn \l__spintent_spangle_kv_tl
5143             { intent = "_ \l__spintent_geo_luaset_intent_str - " }
5144             \__spintent_luafun_mrow_intent_arg:nn
5145             { } { \l__spintent_spangle_kv_tl }
5146             { \l__spintent_geo_luaset_print_tl }
5147         }
5148     }
5149 }

```

Otherwise, a numeric name is tagged as a whole, through `\__spintent_luafun_mrow_intent_arg:nn`, with a literal `intent` built from `\l__spintent_geo_luaset_intent_str`.

```

5138     {
5139         \str_if_eq:VnTF \l__spintent_geo_luaset_nombre_es_num_str { true }
5140         {
5141             \tl_set:Nn \l__spintent_spangle_kv_tl
5142             { intent = "_ \l__spintent_geo_luaset_intent_str - " }
5143             \__spintent_luafun_mrow_intent_arg:nn
5144             { } { \l__spintent_spangle_kv_tl }
5145             { \l__spintent_geo_luaset_print_tl }
5146         }
5147     }
5148 }

```

With `\l__spintent_geo_luaset_tipo_str` equal to `sub`, `read-cmd=school` tags the letter through the same function, with a literal `intent`, and the subscript is appended and silenced; `read-cmd=mathcat` leaves both letter and subscript untouched, for MathCAT to read the whole `<msub>` natively (no `arg` is set in either case: there is no `$`-reference to resolve).

```

5147     {
5148         \str_case:VnF \l__spintent_geo_luaset_tipo_str
5149         {
5150             { sub }
5151             {
5152                 \str_if_eq:eeTF { \use:c { \l__spintent_sp #1 _mode_str } } { school }
5153                 {
5154                     \tl_set:Nn \l__spintent_spangle_kv_tl
5155                     { intent = "_ \l__spintent_geo_luaset_intent_str - " }
5156                     \__spintent_luafun_mrow_intent_arg:nn

```

```

5157         { } { \l__spintent_spangle_kv_tl }
5158     {
5159         \l__spintent_geo_luaset_letra_str
5160         \c_math_subscript_token
5161         { \MathMLintent { :silent } { \l__spintent_geo_luaset_sub_str } }
5162     }
5163 }
5164 {
5165     \l__spintent_geo_luaset_letra_str
5166     \c_math_subscript_token { \l__spintent_geo_luaset_sub_str }
5167 }
5168 }

```

With `\l__spintent_geo_luaset_tipo_str` equal to `prima`, the prime is always read — spoken as part of the literal for `read-cmd=school`, native for `read-cmd=mathcat` — since a prime is not a repeated word to silence.

```

5169     { prima }
5170     {
5171         \str_if_eq:eeTF { \use:c { l__spintent_sp #1 _mode_str } } { school }
5172     {
5173         \tl_set:Ne \l__spintent_spangle_kv_tl
5174         { intent = "_ \l__spintent_geo_luaset_intent_str -" }
5175         \__spintent_luafun_mrow_intent_arg:nn
5176         { } { \l__spintent_spangle_kv_tl }
5177         { \l__spintent_geo_luaset_letra_str \l__spintent_geo_luaset_sub_str }
5178     }
5179     {
5180         \l__spintent_geo_luaset_letra_str \l__spintent_geo_luaset_sub_str
5181     }
5182 }
5183 }
5184 { \l__spintent_geo_luaset_print_tl }
5185 }
5186 }
5187 }

```

For `tres-puntos/un-punto`, `\l__spintent_geo_luaset_points_clist` is split into `\l__spintent_spangle_pt_seq` and walked to build `\l__spintent_spangle_refs_tl`, the `$pt1, $pt2, ...` reference list. A single native `\MathMLintent` wraps the symbol and the points together, with `\l__spintent_spangle_intent_tl:prefix(ref)` — `_` — when `silent` is `true`, `_` (refs) when `concept` is `false` alone. For `\spmangle` with `print-m` `true`, a silent `\operatorname{m}` precedes `\__spintent_spangle_sym:n` inside the same `\MathMLintent`. The sequence is walked again, calling `\__spintent_mathmlarg_point:nnn` once per point.

```

5188 {
5189     \seq_set_from_clist:NN \l__spintent_spangle_pt_seq \l__spintent_geo_luaset_points_clist
5190     \tl_clear:N \l__spintent_spangle_refs_tl
5191     \int_zero:N \l__spintent_spangle_pt_idx_int
5192     \seq_map_inline:Nn \l__spintent_spangle_pt_seq
5193     {
5194         \int_incr:N \l__spintent_spangle_pt_idx_int
5195         \tl_if_empty:NF \l__spintent_spangle_refs_tl
5196         { \tl_put_right:Nn \l__spintent_spangle_refs_tl { , } }
5197         \tl_put_right:Ne \l__spintent_spangle_refs_tl { $ pt \int_use:N \l__spintent_spangle_pt_idx_int }
5198     }
5199     \bool_if:cTF { l__spintent_sp #1 _silent_bool }
5200     {
5201         \tl_set:Nn \l__spintent_spangle_intent_arg_tl { _- }
5202     }
5203     {
5204         \bool_if:cTF { l__spintent_sp #1 _concept_bool }
5205         {
5206             \tl_set:Ne \l__spintent_spangle_intent_arg_tl
5207             {
5208                 -
5209                 \tl_if_empty:cF { l__spintent_sp #1 _prefix_tl }
5210                 { \tl_use:c { l__spintent_sp #1 _prefix_tl } - }
5211                 \l__spintent_spangle_intent_tl :prefix( \l__spintent_spangle_refs_tl )
5212             }
5213         }
5214         {
5215             \tl_set:Ne \l__spintent_spangle_intent_arg_tl { _ ( \l__spintent_spangle_refs_tl )
5216         }

```

```

5217     }
5218     \bool_set_false:N \l__spintent_spangle_print_m_now_bool
5219     \str_if_eq:eeT {#1} { mangle }
5220     {
5221         \bool_if:cT { \l__spintent_sp #1 _print_m_bool }
5222         { \bool_set_true:N \l__spintent_spangle_print_m_now_bool }
5223     }
5224     \MathMLintent { \l__spintent_spangle_intent_arg_tl }
5225     {
5226         \__spintent_inner_sep:
5227         \bool_if:NT \l__spintent_spangle_print_m_now_bool
5228         {
5229             \__spintent_luafun_mrow_intent_arg:nn
5230             { } { intent = ":silent" }
5231             { \operatorname { m } }
5232         }
5233         \__spintent_luafun_mrow_intent_arg:nn
5234         { } { intent = ":silent" }
5235         { \__spintent_spangle_sym:n {#1} }
5236         \int_zero:N \l__spintent_spangle_pt_idx_int
5237         \seq_map_inline:Nn \l__spintent_spangle_pt_seq
5238         {
5239             \int_incr:N \l__spintent_spangle_pt_idx_int
5240             \__spintent_mathmlarg_point:nnn
5241             { pt \int_use:N \l__spintent_spangle_pt_idx_int }
5242             { sp#1 }
5243             { ##1 }
5244         }
5245     }
5246 }
5247 }

```

(End of definition for `\__spintent_spangle_mathml:n`.)

`\__spintent_spangle_exec:nnn`

The function `\__spintent_spangle_exec:nnn` receives the *short name* of the command passed in ‘#1’, the optional *⟨keys⟩* passed in ‘#2’ and the *⟨argument⟩* passed in ‘#3’, delegating the rendering to `\__spintent_spangle_process:nn`.

```

5248 \cs_new_protected:Nn \__spintent_spangle_exec:nnn
5249 {
5250     \keys_set:nn { spintent / sp #1 } { #2 }
5251     \__spintent_spangle_process:nn { #1 } { #3 }
5252 }

```

(End of definition for `\__spintent_spangle_exec:nnn`.)

`\spangle`
`\spmangle`

Both public commands `\spangle` and `\spmangle` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_spangle_exec:nnn`.

```

5253 \NewDocumentCommand \spangle { 0{} m }
5254 {
5255     \mode_if_math:TF
5256     {
5257         \group_begin:
5258         \attribute \l__spintent_scope_attr_int = 1
5259         \__spintent_spangle_exec:nnn { angle } { #1 } { #2 }
5260         \group_end:
5261     }
5262     { \msg_error:nnn { spintent } { need-math-mode } { \spangle } }
5263 }
5264 \NewDocumentCommand \spmangle { 0{} m }
5265 {
5266     \mode_if_math:TF
5267     {
5268         \group_begin:
5269         \attribute \l__spintent_scope_attr_int = 1
5270         \__spintent_spangle_exec:nnn { mangle } { #1 } { #2 }
5271         \group_end:
5272     }
5273     { \msg_error:nnn { spintent } { need-math-mode } { \spmangle } }
5274 }

```

(End of definition for `\spangle` and `\spmangle`. These functions are documented on page 23.)

12.46 The commands `\sparc` and `\spmarc`

The user commands `\sparc` and `\spmarc` provide a semantic interface for arcs. Unlike `\spmside` and `\spmangle`, `\spmarc` has no `print-m` key: `\operatorname{m}` is always present, since $m\overline{AB}$ is the only standard notation for the measure of an arc.

12.46.1 Definition of variables for `\sparc` and `\spmarc`

Each pair of variables is declared once for both `\sparc` and `\spmarc` via `\clist_map_inline:nn`. The variables `\l__spintenc_sparc_mode_str` and `\l__spintenc_spmarc_mode_str` store the values set by the key `read-cmd`; the variables `\l__spintenc_sparc_prefix_tl` and `\l__spintenc_spmarc_prefix_tl` store the values set by the key `prefix`.

```
5275 \clist_map_inline:nn { arc , marc }
5276 {
5277   \str_new:c { l__spintenc_sp #1 _mode_str }
5278   \tl_new:c { l__spintenc_sp #1 _prefix_tl }
5279 }
```

(End of definition for `\l__spintenc_sparc_mode_str` and others.)

12.46.2 Definition of error messages for `\sparc` and `\spmarc`

We define an “error” message for the commands `\sparc` and `\spmarc`. The “error” is triggered if the user includes a invalid `{\langle argument \rangle}` passed in ‘#1’ to the command.

```
5280 \clist_map_inline:nn { arc , marc }
5281 {
5282   \msg_new:nnnn { spintenc } { sp #1 -invalid-points }
5283   {
5284     Invalid~point~format~in~##1:~'##2'~\msg_line_context:.
5285   }
5286   {
5287     See~the~documentation~of~##1~for~the~accepted~point~formats.
5288   }
5289 }
```

(End of definition for `sparc-invalid-points` and `spmarc-invalid-points`.)

12.46.3 Definition of keys for `\sparc` and `\spmarc`

We define the shared `\keys` `read-cmd`, `prefix`, `silent` and `concept`.

```
read-cmd 5290 \clist_map_inline:nn { arc , marc }
prefix   5291 {
silent   5292   \keys_define:nn { spintenc / sp #1 }
concept  5293   {
unknown  5294     read-cmd      .choices:nn = { school , mathcat }
          5295     {
          5296       \int_case:nn { \l_keys_choice_int }
          5297       {
          5298         {1} { \str_set:cn { l__spintenc_sp #1 _mode_str } { school } }
          5299         {2} { \str_set:cn { l__spintenc_sp #1 _mode_str } { mathcat } }
          5300       }
          5301     },
          5302     read-cmd      .initial:n = school,
          5303     read-cmd      .value_required:n = true,
          5304     read-cmd / unknown .code:n =
          5305       \msg_error:nneee { spintenc } { unknown-choice }
          5306       { read-cmd } { school,~mathcat } { \exp_not:n {##1} },
          5307     prefix      .code:n =
          5308       { \__spintenc_sanitize_affix:cn { l__spintenc_sp #1 _prefix_tl } {##1} },
          5309     prefix      .initial:n = ,
          5310     prefix      .value_required:n = true,
          5311     silent      .bool_set:c = { l__spintenc_sp #1 _silent_bool },
          5312     silent      .initial:n = false,
          5313     silent      .value_required:n = true,
          5314     concept      .bool_set:c = { l__spintenc_sp #1 _concept_bool },
          5315     concept      .initial:n = true,
          5316     concept      .value_required:n = true,
          5317     unknown      .code:n =
          5318       {
          5319         \str_set:Nn \l__spintenc_command_name_str { sp #1 }
          5320         \__spintenc_unknown_keys_cmd:n {##1}
          5321       },
          5322     }
          5323   }
```

(End of definition for `read-cmd` and others.)

12.46.4 Internal functions for `\sparc` and `\spmarc`

The function `\__spintent_sparc_wide_arc:n` acts as a fallback when the mathematical font used in the document does not provide the command `\widearc`. The definition of this function is a direct adaptation of a response given by Clea F. Rees (@cfr) to a query in the chat room of TeX-SX.

```
5324 \cs_new_protected_nopar:Nn \__spintent_sparc_wide_arc:n
5325 {
5326   \Umathaccent 7 \symoperators "023DC \scan_stop: {#1}
5327 }
```

(End of definition for `\__spintent_sparc_wide_arc:n`.)

The function `\__spintent_sparc_print_arc:n` applies the command `\widearc` to the argument passed to `\sparc` or `\spmarc`. If the mathematical font set in the document does not provide the command `\widearc`, we will use the built-in function `\__spintent_sparc_wide_arc:n`.

```
5328 \cs_new_protected:Nn \__spintent_sparc_print_arc:n
5329 {
5330   \cs_if_exist:NTF \widearc
5331   { \widearc {#1} }
5332   { \__spintent_sparc_wide_arc:n {#1} }
5333 }
```

(End of definition for `\__spintent_sparc_print_arc:n`.)

The variable `\__spintent_sparc_intent_tl` holds the concept word (arco or medida-del-arco). The rest are scratch variables to split `\__spintent_geo_luaset_points_clist` and build the `$pt1, $pt2, ...` reference list used in the concept intent.

```
5334 \tl_new:N \__spintent_sparc_intent_tl
5335 \seq_new:N \__spintent_sparc_pt_seq
5336 \tl_new:N \__spintent_sparc_refs_tl
5337 \tl_new:N \__spintent_sparc_intent_arg_tl
5338 \int_new:N \__spintent_sparc_pt_idx_int
```

(End of definition for `\__spintent_sparc_intent_tl` and others.)

The internal function `\__spintent_sparc_process:nn` receives the short name of the command (arc or marc) in ‘#1’ and the `{\argument}` in ‘#2’. It calls `\__spintent_luafun_geo_parse_and_set:n` to process ‘#2’, returning an “error” if invalid, then sets `\__spintent_sparc_intent_tl` to arco (or medida-del-arco for `\spmarc`). No intent is built here: it needs the reference list, which `\__spintent_sparc_mathml:n` builds after splitting the points.

```
5339 \cs_new_protected:Nn \__spintent_sparc_process:nn
5340 {
5341   \__spintent_luafun_geo_parse_and_set:n { \exp_not:n {#2} }
5342   \str_if_eq:VnT \__spintent_geo_luaset_error_str { true }
5343   {
5344     \msg_error:nnee { spintent } { sp #1 -invalid-points }
5345     { \c_backslash_str sp #1 } { \exp_not:n {#2} }
5346   }
5347   \str_if_eq:eeTF {#1} { marc }
5348   { \tl_set:Nn \__spintent_sparc_intent_tl { medida-del-arco } }
5349   { \tl_set:Nn \__spintent_sparc_intent_tl { arco } }
5350   \__spintent_sparc_mathml:n {#1}
5351 }
```

(End of definition for `\__spintent_sparc_process:nn`.)

The function `\__spintent_sparc_mathml:n` receives the short command name in ‘#1’. It splits `\__spintent_geo_luaset_points_clist` into `\__spintent_sparc_pt_seq` and walks it once to build `\__spintent_sparc_refs_tl`, the comma-separated `$pt1, $pt2, ...` reference list, naming each point by position.

```
5352 \cs_new_protected:Nn \__spintent_sparc_mathml:n
5353 {
5354   \seq_set_from_clist:NN \__spintent_sparc_pt_seq \__spintent_geo_luaset_points_clist
5355
5356   \tl_clear:N \__spintent_sparc_refs_tl
5357   \int_zero:N \__spintent_sparc_pt_idx_int
5358   \seq_map_inline:Nn \__spintent_sparc_pt_seq
5359   {
5360     \int_incr:N \__spintent_sparc_pt_idx_int
5361     \tl_if_empty:NF \__spintent_sparc_refs_tl
5362     { \tl_put_right:Nn \__spintent_sparc_refs_tl { , } }
5363     \tl_put_right:Ne \__spintent_sparc_refs_tl { $ pt \int_use:N \__spintent_sparc_pt_idx_int
5364   }
```

The concept `intent` is `_` - when `silent` is `true`, `_(refs)` when `concept` is `false`; otherwise `_\l__spintent_sparc_` with `prefix` placed after the leading `_` when given.

```

5365 \bool_if:cTF { \l__spintent_sp #1 _silent_bool }
5366 {
5367   \tl_set:Nn \l__spintent_sparc_intent_arg_tl { _- }
5368 }
5369 {
5370   \bool_if:cTF { \l__spintent_sp #1 _concept_bool }
5371   {
5372     \tl_set:Ne \l__spintent_sparc_intent_arg_tl
5373     {
5374       -
5375       \tl_if_empty:cF { \l__spintent_sp #1 _prefix_tl }
5376       { \tl_use:c { \l__spintent_sp #1 _prefix_tl } - }
5377       \l__spintent_sparc_intent_tl :prefix( \l__spintent_sparc_refs_tl )
5378     }
5379   }
5380   {
5381     \tl_set:Ne \l__spintent_sparc_intent_arg_tl { _ ( \l__spintent_sparc_refs_tl ) }
5382   }
5383 }

```

It is applied with a single native `\MathMLintent` wrapping `_\l__spintent_sparc_print_arc:n` directly (same shape as `_\l__spintent_spside_mathml:n`), preceded by `_\l__spintent_inner_sep:` (a second sibling so the sole `<mrow>` does not collapse into `<math>`, luamml issue #26). A silent `\operatorname{m}` precedes it for `\spmarc` (always, no `print-m` key here). The sequence is walked a second time, calling `_\l__spintent_mathmlarg_point:nnn` once per point with its `ptn` reference name, looked up under `sp#1`.

```

5384 \l__spintent_invisible_sep: % need by MathCAT (bug in luamml issues #26)
5385 \MathMLintent { \l__spintent_sparc_intent_arg_tl }
5386 {
5387   \l__spintent_inner_sep:
5388   \str_if_eq:eeT {#1} { marc }
5389   {
5390     \l__spintent_luafun_mrow_intent_arg:nn
5391     { } { intent = ":silent" }
5392     { \operatorname { m } } }
5393   }
5394   \l__spintent_sparc_print_arc:n
5395   {
5396     \int_zero:N \l__spintent_sparc_pt_idx_int
5397     \seq_map_inline:Nn \l__spintent_sparc_pt_seq
5398     {
5399       \int_incr:N \l__spintent_sparc_pt_idx_int
5400       \l__spintent_mathmlarg_point:nnn
5401       { pt \int_use:N \l__spintent_sparc_pt_idx_int }
5402       { sp #1 }
5403       { ##1 }
5404     }
5405   }
5406 }
5407 }

```

(End of definition for `_\l__spintent_sparc_mathml:n`.)

`_\l__spintent_sparc_exec:nnn`

The function `_\l__spintent_sparc_exec:nnn` receives the *short name* of the command passed in ‘#1’, the optional *<keys>* passed in ‘#2’ and the *<argument>* passed in ‘#3’, delegating the rendering to `_\l__spintent_sparc_process:nnn`.

```

5408 \cs_new_protected:Nn \l__spintent_sparc_exec:nnn
5409 {
5410   \keys_set:nn { spintent / sp #1 } { #2 }
5411   \l__spintent_sparc_process:nn { #1 } { #3 }
5412 }

```

(End of definition for `_\l__spintent_sparc_exec:nnn`.)

`\sparc` Both public commands `\sparc` and `\spmarc` enforces *math mode*, opens a local group, and delegates execution to `_\l__spintent_sparc_exec:nnn`.

```

5413 \NewDocumentCommand \sparc { O{} m }
5414 {
5415   \mode_if_math:TF
5416   {

```

```

5417     \group_begin:
5418         \attribute \l__spintent_scope_attr_int = 1
5419         \__spintent_sparc_exec:nnn { arc } { #1 } { #2 }
5420     \group_end:
5421 }
5422 { \msg_error:nnn { spintent } { need-math-mode } { \sparc } }
5423 }
5424 \NewDocumentCommand \spmarc { 0{} m }
5425 {
5426     \mode_if_math:TF
5427     {
5428         \group_begin:
5429             \attribute \l__spintent_scope_attr_int = 1
5430             \__spintent_sparc_exec:nnn { marc } { #1 } { #2 }
5431         \group_end:
5432     }
5433     { \msg_error:nnn { spintent } { need-math-mode } { \spmarc } }
5434 }

```

(End of definition for `\sparc` and `\spmarc`. These functions are documented on page 25.)

12.47 The command `\spPoly`

The user command `\spPoly` typesets a polygon defined by its vertices. The figure type and its visual symbol are inferred from the vertex count: three points produce a triangle, four a square, five or more a generic polygon with no symbol. It never sends a boolean to Lua: `\l__spintent_geo_luaset_intent_str` (the full `intent`, with the figure name).

12.47.1 Definition of variables for `\spPoly`

The variable `\l__spintent_spPoly_mode_str` stores the value set by the key `read-cmd`; the variable `\l__spintent_spPoly_prefix_tl` stores the value set by the key `prefix`.

```

5435 \str_new:N \l__spintent_spPoly_mode_str
5436 \tl_new:N \l__spintent_spPoly_prefix_tl

```

(End of definition for `\l__spintent_spPoly_mode_str` and `\l__spintent_spPoly_prefix_tl`.)

12.47.2 Definition of error messages for `\spPoly`

Raised when the argument is not a comma-separated list of at least three uppercase letters.

```

5437 \msg_new:nnnn { spintent } { spPoly-invalid-points }
5438 {
5439     Invalid~point~format~in~\token_to_str:N \spPoly:~'#1'~\msg_line_context:.
5440 }
5441 {
5442     See~the~documentation~of~\token_to_str:N \spPoly~for~the~accepted~point~formats.
5443 }

```

(End of definition for `spPoly-invalid-points`.)

12.47.3 Definition of keys for `\spPoly`

We define the *(keys)* `read-cmd`, `prefix`, `print-sym`, `silent` and `concept`.

```

read-cmd 5444 \keys_define:nn { spintent / spPoly }
prefix   5445 {
print-sym 5446     read-cmd             .choices:nn = { school , mathcat }
silent    5447     {
concept    5448         \int_case:nn { \l_keys_choice_int }
unknown   5449         {
5450             {1} { \str_set:Nn \l__spintent_spPoly_mode_str { school } }
5451             {2} { \str_set:Nn \l__spintent_spPoly_mode_str { mathcat } }
5452         }
5453     },
5454     read-cmd             .initial:n = school,
5455     read-cmd             .value_required:n = true,
5456     read-cmd / unknown   .code:n =
5457         \msg_error:nneee { spintent } { unknown-choice }
5458         { read-cmd } { school,~mathcat } { \exp_not:n {#1} },
5459     prefix               .code:n =
5460         { \__spintent_sanitize_affix:Nn \l__spintent_spPoly_prefix_tl {#1} },
5461     prefix               .initial:n = ,
5462     prefix               .value_required:n = true,
5463     print-sym            .bool_set:N = \l__spintent_spPoly_print_sym_bool,
5464     print-sym            .initial:n = true,
5465     print-sym            .value_required:n = true,

```

```

5466     silent                .bool_set:N = \l__spintent_spPoly_silent_bool,
5467     silent                .initial:n = false,
5468     silent                .value_required:n = true,
5469     concept               .bool_set:N = \l__spintent_spPoly_concept_bool,
5470     concept               .initial:n = true,
5471     concept               .value_required:n = true,
5472     unknown               .code:n =
5473     {
5474         \str_set:Nn \l__spintent_command_name_str { spPoly }
5475         \__spintent_unknown_keys_cmd:n {#1}
5476     },
5477 }

```

(End of definition for read-cmd and others.)

12.47.4 Internal functions for \spPoly

\l__spintent_spPoly_pt_seq Scratch variables to split \l__spintent_geo_luaset_points_clist and build the \$pt1,\$pt2,... reference list, and to hold the final concept intent once built.

```

\l__spintent_spPoly_refs_tl
\l__spintent_spPoly_intent_arg_tl
\l__spintent_spPoly_pt_idx_int
5478 \seq_new:N \l__spintent_spPoly_pt_seq
5479 \tl_new:N \l__spintent_spPoly_refs_tl
5480 \tl_new:N \l__spintent_spPoly_intent_arg_tl
5481 \int_new:N \l__spintent_spPoly_pt_idx_int

```

(End of definition for \l__spintent_spPoly_pt_seq and others.)

__spintent_spPoly_sym: Unchanged: reads \l__spintent_geo_luaset_poly_sym_str and typesets the corresponding symbol.

```

5482 \cs_new_protected:Nn \__spintent_spPoly_sym:
5483 {
5484     \str_case:Vn \l__spintent_geo_luaset_poly_sym_str
5485     {
5486         { triangle } { \triangle }
5487         { square } { \mdlgwhtsquare }
5488     }
5489 }

```

(End of definition for __spintent_spPoly_sym:.)

__spintent_spPoly_build_refs: Splits \l__spintent_geo_luaset_points_clist into \l__spintent_spPoly_pt_seq and walks it once to build \l__spintent_spPoly_refs_tl, the comma-separated \$pt1,\$pt2,... reference list, naming each vertex by position.

```

5490 \cs_new_protected:Nn \__spintent_spPoly_build_refs:
5491 {
5492     \seq_set_from_clist:NN \l__spintent_spPoly_pt_seq \l__spintent_geo_luaset_points_clist
5493     \tl_clear:N \l__spintent_spPoly_refs_tl
5494     \int_zero:N \l__spintent_spPoly_pt_idx_int
5495     \seq_map_inline:Nn \l__spintent_spPoly_pt_seq
5496     {
5497         \int_incr:N \l__spintent_spPoly_pt_idx_int
5498         \tl_if_empty:NF \l__spintent_spPoly_refs_tl
5499         { \tl_put_right:Nn \l__spintent_spPoly_refs_tl { , } }
5500         \tl_put_right:Ne \l__spintent_spPoly_refs_tl { $ pt \int_use:N \l__spintent_spPoly_pt_idx.
5501     }
5502 }

```

(End of definition for __spintent_spPoly_build_refs:.)

__spintent_spPoly_build_concept: Builds \l__spintent_spPoly_intent_arg_tl, independently of print-sym: _- when silent is true, _ (refs) when concept is false; otherwise \l__spintent_geo_luaset_concept_str:prefix(refs), with prefix placed after the leading _ when given. Must be called after __spintent_spPoly_build_refs:.

```

5503 \cs_new_protected:Nn \__spintent_spPoly_build_concept:
5504 {
5505     \bool_if:NTF \l__spintent_spPoly_silent_bool
5506     {
5507         \tl_set:Nn \l__spintent_spPoly_intent_arg_tl { _- }
5508     }
5509     {
5510         \bool_if:NTF \l__spintent_spPoly_concept_bool
5511         {
5512             \tl_set:Ne \l__spintent_spPoly_intent_arg_tl
5513             {
5514                 -
5515                 \tl_if_empty:NF \l__spintent_spPoly_prefix_tl

```

```

5516         { \tl_use:N \l__spintent_spPoly_prefix_tl - }
5517         \l__spintent_geo_luaset_concept_str :prefix( \l__spintent_spPoly_refs_tl )
5518     }
5519 }
5520 {
5521     \tl_set:Nx \l__spintent_spPoly_intent_arg_tl { _ ( \l__spintent_spPoly_refs_tl ) }
5522 }
5523 }
5524 }

```

(End of definition for `\__spintent_spPoly_build_concept:`)

`\__spintent_spPoly_points_loop:`

Walks `\l__spintent_spPoly_pt_seq` a second time, calling `\__spintent_mathmlarg_point:nnn` once per vertex with its `ptn` reference name, looked up under `spPoly`: it alone decides `arg`/literal `intent` per vertex — replacing the single combined `\MathMLarg{pts}{...}` that lost `arg` on any vertex with a subscript or prime (the same `annotate_attribute`/sub-sup limitation already found and worked around for `\spPoint`).

```

5525 \cs_new_protected:Nn \__spintent_spPoly_points_loop:
5526 {
5527     \int_zero:N \l__spintent_spPoly_pt_idx_int
5528     \seq_map_inline:Nn \l__spintent_spPoly_pt_seq
5529     {
5530         \int_incr:N \l__spintent_spPoly_pt_idx_int
5531         \__spintent_mathmlarg_point:nnn
5532         { pt \int_use:N \l__spintent_spPoly_pt_idx_int }
5533         { spPoly }
5534         { ##1 }
5535     }
5536 }

```

(End of definition for `\__spintent_spPoly_points_loop:`)

`\__spintent_spPoly_process:n`

Calls `\__spintent_luafun_geo_poly_parse_and_set:n`, reports `spPoly-invalid-points` on failure, then delegates to `\__spintent_spPoly_mathml:`. No `intent` is built here: it needs the reference list, which `\__spintent_spPoly_mathml:` builds after splitting the vertices.

```

5537 \cs_new_protected:Nn \__spintent_spPoly_process:n
5538 {
5539     \__spintent_luafun_geo_poly_parse_and_set:n { \exp_not:n {#1} }
5540     \str_if_eq:VnT \l__spintent_geo_luaset_error_str { true }
5541     {
5542         \msg_error:nne { spintent } { spPoly-invalid-points }
5543         { \exp_not:n {#1} }
5544     }
5545     \__spintent_spPoly_mathml:
5546 }

```

(End of definition for `\__spintent_spPoly_process:n`)

`\__spintent_spPoly_mathml:`

Builds the reference list and the concept `intent` (`\__spintent_spPoly_build_refs:`, `\__spintent_spPoly_build_concept:`) then wraps everything in a single native `\MathMLintent`, preceded by `\__spintent_inner_sep:` (a second sibling so the sole `<mrow>` does not collapse into `<math>`, `luamml` issue #26).

The keys `print-sym` and `concept/silent` are independent: the former only decides whether the glyph (`\__spintent_spPoly_sym:`, silenced with `\__spintent_luafun_mrow_intent_arg:nn`) is attempted — it is skipped anyway when `\l__spintent_geo_luaset_poly_sym_str` is empty (no symbol defined for that vertex count, e.g. pentagon and above) — and never affects whether the concept word itself is spoken.

```

5547 \cs_new_protected:Nn \__spintent_spPoly_mathml:
5548 {
5549     \__spintent_invisible_sep: % (luamml issues #26)
5550     \__spintent_spPoly_build_refs:
5551     \__spintent_spPoly_build_concept:
5552     \MathMLintent { \l__spintent_spPoly_intent_arg_tl }
5553     {
5554         \__spintent_inner_sep:
5555         \bool_if:NT \l__spintent_spPoly_print_sym_bool
5556         {
5557             \str_if_empty:NF \l__spintent_geo_luaset_poly_sym_str
5558             {
5559                 \__spintent_luafun_mrow_intent_arg:nn
5560                 { } { intent = ":silent" }
5561                 { \__spintent_spPoly_sym: }
5562             }
5563         }
5564     }

```

```

5564     \__spintent_spPoly_points_loop:
5565     }
5566 }

```

(End of definition for `\__spintent_spPoly_mathml:`.)

`\__spintent_spPoly_exec:nn`

The function `\__spintent_spPoly_exec:nn` receives the optional keys in `#1` and the argument in `#2`, delegating the rendering to `\__spintent_spPoly_process:n`.

```

5567 \cs_new_protected:Nn \__spintent_spPoly_exec:nn
5568 {
5569     \keys_set:nn { spintent / spPoly } { #1 }
5570     \__spintent_spPoly_process:n { #2 }
5571 }

```

(End of definition for `\__spintent_spPoly_exec:nn`.)

`\spPoly`

The public command `\spPoly` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_spPoly_exec:nn`.

```

5572 \NewDocumentCommand \spPoly { 0{} m }
5573 {
5574     \mode_if_math:TF
5575     {
5576         \group_begin:
5577         \attribute \l__spintent_scope_attr_int = 1
5578         \__spintent_spPoly_exec:nn { #1 } { #2 }
5579         \group_end:
5580     }
5581     { \msg_error:nnn { spintent } { need-math-mode } { \spPoly } }
5582 }

```

(End of definition for `\spPoly`. This function is documented on page 26.)

12.48 The commands `\spAfig` and `\spPfig`

The user commands `\spAfig` and `\spPfig` typeset the area and the perimeter of a polygonal figure, using the delimited-optional argument syntax `d()`. The key `print-sym` is never sent to Lua: the symbol type is returned unconditionally in `\l__spintent_geo_luaset_poly_sym_str`, and `expl3` decides whether to wrap it in `\MathMLintenc{<:silent>}`.

12.48.1 Definition of variables for `\spAfig` and `\spPfig`

`\l__spintent_geo_final_print_tl`

Scratch variables used by `\__spintent_spfig_process:nn` to assemble the visual and the `intent`. The variable `\l__spintent_geo_final_print_tl` holds the final visual placed under `\mathcal`'s subscript. The final literal `intent` and the concept forwarded when `read-cmd` is `mathcat`.

```

5583 \tl_new:N \l__spintent_geo_final_print_tl

```

(End of definition for `\l__spintent_geo_final_print_tl`.)

`\l__spintent_spAfig_mode_str`

These variables store the value set by each corresponding key: `read-cmd`, `print-sym`, `prefix` and `read-arg`, respectively. The value set by `read-arg` overrides the figure name inferred from the vertex count, independently of `print-sym`.

`\l__spintent_spPfig_mode_str`

```

5584 \clist_map_inline:nn { Afig , Pfig }
5585 {
5586     \str_new:c { \l__spintent_sp #1 _mode_str }
5587     \str_new:c { \l__spintent_sp #1 _print_sym_str }
5588     \tl_new:c { \l__spintent_sp #1 _prefix_tl }
5589     \tl_new:c { \l__spintent_sp #1 _read_arg_tl }
5590 }

```

(End of definition for `\l__spintent_spAfig_mode_str` and others.)

12.48.2 Definition of error messages for `\spAfig` and `\spPfig`

All eight messages are generated by the same map, using the same `##1/##2` doubling convention as `\spside/\spmside`. `sp #1-invalid-points` is currently unused by `\__spintent_spfig_process:nn`, which raises only `sp #1-invalid-arg` for any parse failure, regardless of whether a comma was present. `sp #1-sym-letra-conflict` and `sp #1-sym-cardinality-conflict` are raised when `print-sym` is forced to a value incompatible with the actual argument.

```

5591 \clist_map_inline:nn { Afig , Pfig }
5592 {
5593     \msg_new:nnnn { spintent } { sp #1 -invalid-points }
5594     {
5595         Invalid~point~format~in~##1:~'##2'~\msg_line_context:.
5596     }

```

```

5597     {
5598         See~the~documentation~of~##1~for~the~accepted~point~formats.
5599     }
5600     \msg_new:nnnn { spintent } { sp #1 -invalid-arg }
5601     {
5602         Invalid~argument~format~in~##1:~'##2'\msg_line_context:.
5603     }
5604     {
5605         See~the~documentation~of~##1~for~the~accepted~argument~forms.
5606     }
5607     \msg_new:nnnn { spintent } { sp #1 -sym-lettra-conflict }
5608     {
5609         print-sym~cannot~be~combined~with~a~lettered~argument~in~'##1'\msg_line_context:.
5610     }
5611     {
5612         See~the~documentation~of~##1~for~details~on~print-sym.
5613     }
5614     \msg_new:nnnn { spintent } { sp #1 -sym-cardinality-conflict }
5615     {
5616         print-sym~does~not~match~the~vertex~count~in~'##1'\msg_line_context:.
5617     }
5618     {
5619         See~the~documentation~of~##1~for~details~on~print-sym.
5620     }
5621     \msg_new:nnnn { spintent } { sp #1 -read-arg-key }
5622     {
5623         read-arg~has~no~effect~with~a~single~argument~in~'##1'\msg_line_context:.
5624     }
5625     {
5626         See~the~documentation~of~##1~for~details~on~read-arg.
5627     }
5628 }

```

(End of definition for `spAfig-invalid-points` and others.)

12.48.3 Definition of keys for `\spAfig` and `\spPfig`

The key `print-sym` chooses the visual symbol: `auto` (default) infers it from the vertex count (three points give `\triangle`, four give `\mdlgwhtsquare`, five or more give none); `triangle`/square force one, requiring the vertex count to match; `none` omits it, even with vertices. With a bare number/letter argument, `auto` and `none` behave the same (no symbol); `triangle`/square nest the argument under the forced symbol instead, and cannot be combined with a table letter (F/P/B/L). The key `read-arg` overrides the spoken figure name for real vertices, independently of `print-sym`. The key `read-sub` controls whether the bare number/letter argument is read with the word «sub» or as a plain number.

```

5629 \clist_map_inline:nn { Afig , Pfig }
5630 {
5631     \keys_define:nn { spintent / sp #1 }
5632     {
5633         read-cmd .choices:nn = { school , mathcat }
5634         {
5635             \int_case:nn { \l_keys_choice_int }
5636             {
5637                 {1} { \str_set:cn { l__spintent_sp #1 _mode_str } { school } }
5638                 {2} { \str_set:cn { l__spintent_sp #1 _mode_str } { mathcat } }
5639             }
5640         },
5641         read-cmd .initial:n = school,
5642         read-cmd .value_required:n = true,
5643         read-cmd / unknown .code:n =
5644             \msg_error:nneee { spintent } { unknown-choice }
5645             { read-cmd } { school,~mathcat } { \exp_not:n {##1} },
5646         prefix .code:n =
5647             { \__spintent_sanitize_affix:cn { l__spintent_sp #1 _prefix_tl } {##1} },
5648         prefix .initial:n = ,
5649         prefix .value_required:n = true,
5650         print-sym .choices:nn = { auto , triangle , square , none }
5651         {
5652             \int_case:nn { \l_keys_choice_int }
5653             {
5654                 {1} { \str_set:cn { l__spintent_sp #1 _print_sym_str } { auto } }
5655                 {2} { \str_set:cn { l__spintent_sp #1 _print_sym_str } { triangle } }
5656                 {3} { \str_set:cn { l__spintent_sp #1 _print_sym_str } { square } }

```

```

5657         {4} { \str_set:cn { \__spintent_sp #1 _print_sym_str } { none } }
5658     }
5659 },
5660 print-sym          .initial:n = auto,
5661 print-sym          .value_required:n = true,
5662 print-sym / unknown .code:n =
5663     \msg_error:nneee { spintent } { unknown-choice }
5664     { print-sym } { auto,~triangle,~square,~none } { \exp_not:n {##1} },
5665 read-arg           .code:n =
5666     { \__spintent_sanitizize_affix:cn { \__spintent_sp #1 _read_arg_tl } {##1} },
5667 read-arg           .initial:n = ,
5668 read-arg           .value_required:n = true,
5669 read-sub           .bool_set:c = { \__spintent_sp #1 _read_sub_bool },
5670 read-sub           .initial:n = false,
5671 read-sub           .value_required:n = true,
5672 silent             .bool_set:c = { \__spintent_sp #1 _silent_bool },
5673 silent             .initial:n = false,
5674 silent             .value_required:n = true,
5675 concept            .bool_set:c = { \__spintent_sp #1 _concept_bool },
5676 concept            .initial:n = true,
5677 concept            .value_required:n = true,
5678 concept*           .meta:n =
5679     {
5680         concept = false,
5681         read-arg = { \str_if_eq:eeTF {#1} { Afig } { _área } { _perímetro } },
5682     },
5683 concept*           .value_forbidden:n = true,
5684 unknown            .code:n =
5685     {
5686         \str_set:Nn \__spintent_command_name_str { sp #1 }
5687         \__spintent_unknown_keys_cmd:n {##1}
5688     },
5689 }
5690 }

```

(End of definition for read-cmd and others.)

12.48.4 Internal functions for \spAfig and \spPfig

`\__spintent_spfig_sym:` The function `\__spintent_spfig_sym:` reads `\__spintent_geo_print_sym_str` and typesets the corresponding symbol: `\triangle` for the value `triangle`, `\mdlgwhtsquare` for the value `square`.

```

5691 \cs_new_protected:Nn \__spintent_spfig_sym:
5692 {
5693     \str_case:Vn \__spintent_geo_print_sym_str
5694     {
5695         { triangle } { \triangle }
5696         { square } { \mdlgwhtsquare }
5697     }
5698 }

```

(End of definition for `\__spintent_spfig_sym:`)

`\__spintent_spfig_op_tl` The variable `\__spintent_spfig_op_tl` holds `área` or `perímetro`. The variable `\__spintent_spfig_intent_tl` holds the concept word for the vertices case (`read-arg` or the auto-detected figure name). The rest are scratch variables to split `\__spintent_geo_luaset_points_clist` and build the `$pt1`, `$pt2`, ... reference list.

```

\__spintent_spfig_op_tl
\__spintent_spfig_intent_tl
\__spintent_spfig_symbol_tl
\__spintent_spfig_pt_seq
\__spintent_spfig_refs_tl
\__spintent_spfig_intent_arg_tl
\__spintent_spfig_pt_idx_int
\__spintent_geo_print_sym_str
\__spintent_geo_temp_tl
5699 \tl_new:N \__spintent_spfig_op_tl
5700 \tl_new:N \__spintent_spfig_intent_tl
5701 \tl_new:N \__spintent_spfig_desc_tl
5702 \tl_new:N \__spintent_spfig_symbol_tl
5703 \seq_new:N \__spintent_spfig_pt_seq
5704 \tl_new:N \__spintent_spfig_refs_tl
5705 \tl_new:N \__spintent_spfig_intent_arg_tl
5706 \int_new:N \__spintent_spfig_pt_idx_int
5707 \str_new:N \__spintent_geo_print_sym_str
5708 \tl_new:N \__spintent_geo_temp_tl

```

(End of definition for `\__spintent_spfig_op_tl` and others.)

`\__spintent_spfig_process:nn` Receives the short command name (`Afig` or `Pfig`) in ‘#1’ and the `{\argument}` in ‘#2’. Calls `\__spintent_luafun_geo` or `\__spintent_luafun_geo_pfig_parse_and_set:n`, reports `sp#1-invalid-arg` on failure, and `sp#1-read-arg` when `read-arg` is given for a bare number/letter argument (no vertices), where it has no effect. Sets `\__spintent_spfig_op_tl` to `área`/`perímetro`, then delegates to `\__spintent_spfig_mathml:n`.

```

5709 \cs_new_protected:Nn \__spintent_spfig_process:nn

```

```

5710 {
5711   \str_if_eq:eeTF {#1} { Afig }
5712   { \__spintent_luafun_geo_afig_parse_and_set:n { \exp_not:n {#2} } }
5713   { \__spintent_luafun_geo_pfig_parse_and_set:n { \exp_not:n {#2} } }
5714   \str_if_eq:VnT \l__spintent_geo_luaset_error_str { true }
5715   {
5716     \msg_error:nnee { spintent } { sp #1 -invalid-arg }
5717     { \c_backslash_str sp #1 } { \exp_not:n {#2} }
5718   }
5719   \clist_if_empty:NT \l__spintent_geo_luaset_points_clist
5720   {
5721     \tl_if_empty:cF { \l__spintent_sp #1 _read_arg_tl }
5722     {
5723       \msg_error:nne { spintent } { sp #1 -read-arg-key }
5724       { \c_backslash_str sp #1 }
5725     }
5726   }
5727   \str_if_eq:eeTF {#1} { Afig }
5728   { \tl_set:Nn \l__spintent_spfig_op_tl { área } }
5729   { \tl_set:Nn \l__spintent_spfig_op_tl { perímetro } }
5730   \str_set:Ne \l__spintent_geo_print_sym_str { \str_use:c { \l__spintent_sp #1 _print_sym_str } }
5731   \__spintent_spfig_mathml:n {#1}
5732 }

```

(End of definition for __spintent_spfig_process:nn.)

__spintent_spfig_build_simple:n Handles the argument-less, table-letter and bare number/letter cases (no vertices). Builds \l__spintent_geo_final_print_tl (the visual placed under \mathcal's subscript) and \l__spintent_spfig_desc_tl (the descriptive part, without área/perímetro). Receives the short command name in '#1'.
With an empty argument, both are cleared for print-sym auto/ none; a forced print-sym instead places the symbol alone, with its own name as the description.

```

5733 \cs_new_protected:Nn \__spintent_spfig_build_simple:n
5734 {
5735   \tl_if_empty:nTF { \l__spintent_geo_luaset_print_tl }
5736   {
5737     \str_case:VnF \l__spintent_geo_print_sym_str
5738     {
5739       { auto }
5740       {
5741         \tl_clear:N \l__spintent_geo_final_print_tl
5742         \tl_clear:N \l__spintent_spfig_desc_tl
5743       }
5744       { none }
5745       {
5746         \tl_clear:N \l__spintent_geo_final_print_tl
5747         \tl_clear:N \l__spintent_spfig_desc_tl
5748       }
5749     }
5750     {
5751       \tl_set:Nn \l__spintent_geo_final_print_tl { \__spintent_spfig_sym: }
5752       \tl_set:Ne \l__spintent_spfig_desc_tl
5753       {
5754         \str_if_eq:VnTF \l__spintent_geo_print_sym_str { triangle }
5755         { triángulo } { cuadrado }
5756       }
5757     }
5758   }

```

With a bare number/letter (empty \l__spintent_geo_luaset_letra_word_str), the value itself becomes the visual for print-sym auto/none, or its own subscript under a forced symbol otherwise; the description is the spoken value, prefixed with «sub-» when read-sub is true (further prefixed with the symbol's own name for a forced print-sym).

```

5759 {
5760   \str_if_empty:NTF \l__spintent_geo_luaset_letra_word_str
5761   {
5762     \str_case:VnF \l__spintent_geo_print_sym_str
5763     {
5764       { auto } { \tl_set:Nn \l__spintent_geo_final_print_tl { \l__spintent_geo_luaset_letra_word_str } }
5765       { none } { \tl_set:Nn \l__spintent_geo_final_print_tl { \l__spintent_geo_luaset_letra_word_str } }
5766     }
5767   }

```

```

5768         \tl_set:Nn \l__spintent_geo_final_print_tl
5769         {
5770             \__spintent_spfig_sym:
5771             \c_math_subscript_token { \l__spintent_geo_luaset_print_tl }
5772         }
5773     }
5774     \str_case:VnF \l__spintent_geo_print_sym_str
5775     {
5776         { auto }
5777         {
5778             \tl_set:Ne \l__spintent_spfig_desc_tl
5779             {
5780                 \bool_if:cT { \l__spintent_sp #1 _read_sub_bool } { sub - }
5781                 \l__spintent_geo_luaset_sub_spoken_str
5782             }
5783         }
5784         { none }
5785         {
5786             \tl_set:Ne \l__spintent_spfig_desc_tl
5787             {
5788                 \bool_if:cT { \l__spintent_sp #1 _read_sub_bool } { sub - }
5789                 \l__spintent_geo_luaset_sub_spoken_str
5790             }
5791         }
5792     }
5793     {
5794         \tl_set:Ne \l__spintent_spfig_desc_tl
5795         {
5796             del-
5797             \str_if_eq:VnTF \l__spintent_geo_print_sym_str { triangle }
5798             { triángulo } { cuadrado }
5799             -
5800             \bool_if:cT { \l__spintent_sp #1 _read_sub_bool } { sub - }
5801             \l__spintent_geo_luaset_sub_spoken_str
5802         }
5803     }
5804 }

```

With a table letter (F/P/B/L/T, non-empty `\l__spintent_geo_luaset_letra_word_str`), a forced `print-sym` raises `sp#1-sym-letra-conflict` — a table letter is never nested under a symbol. The visual is the value as-is; the description is its own word, connected as `\l__spintent_geo_luaset_letra_conector_str` requires, with the same spoken subscript as above when present.

```

5805     {
5806         \str_case:VnF \l__spintent_geo_print_sym_str
5807         {
5808             { auto } { }
5809             { none } { }
5810         }
5811         {
5812             \msg_error:nne { spintent } { sp #1 -sym-letra-conflict }
5813             { \c_backslash_str sp #1 }
5814         }
5815     }
5816     \tl_set:Nn \l__spintent_geo_final_print_tl { \l__spintent_geo_luaset_print_tl }
5817     \tl_set:Ne \l__spintent_spfig_desc_tl
5818     {
5819         \str_if_empty:NF \l__spintent_geo_luaset_letra_conector_str
5820         { \l__spintent_geo_luaset_letra_conector_str - }
5821         \l__spintent_geo_luaset_letra_word_str
5822         \str_if_empty:NF \l__spintent_geo_luaset_sub_spoken_str
5823         {
5824             -
5825             \bool_if:cT { \l__spintent_sp #1 _read_sub_bool } { sub - }
5826             \l__spintent_geo_luaset_sub_spoken_str
5827         }
5828     }
5829 }

```

The variable `\l__spintent_spfig_intent_tl` is always the word- shape, without the leading `_`: `\l__spintent_spfig_intent_tl` alone when `\l__spintent_spfig_desc_tl` is empty, both joined by a dash otherwise. `silent` overrides `\l__spintent_spfig_intent_arg_tl` with `_`; with `silent=false` and `concept false`, it becomes `\l__spintent_spfig_desc_tl-` (the descriptive part alone, without área/perímetro nor, for

a table letter, its connector de-la/del); with `concept true` it is `_`, then `prefix` when given, and then `\l__spintent_spfig_intent_tl`.

```

5830 \tl_set:Nc \l__spintent_spfig_intent_tl
5831 {
5832   \tl_if_empty:NTF \l__spintent_spfig_desc_tl
5833   { \l__spintent_spfig_op_tl - }
5834   { \l__spintent_spfig_op_tl - \l__spintent_spfig_desc_tl - }
5835 }
5836 \bool_if:cTF { \l__spintent_sp #1 _silent_bool }
5837 { \tl_set:Nn \l__spintent_spfig_intent_arg_tl { _- } }
5838 {
5839   \bool_if:cTF { \l__spintent_sp #1 _concept_bool }
5840   {
5841     \tl_set:Nc \l__spintent_spfig_intent_arg_tl
5842     {
5843       -
5844       \tl_if_empty:cF { \l__spintent_sp #1 _prefix_tl }
5845       { \tl_use:c { \l__spintent_sp #1 _prefix_tl } - }
5846       \l__spintent_spfig_intent_tl
5847     }
5848   }
5849   {
5850     \tl_set_eq:NN \l__spintent_geo_temp_tl \l__spintent_spfig_desc_tl
5851     \str_if_empty:NF \l__spintent_geo_luaset_letra_conector_str
5852     {
5853       \tl_replace_once:Nee \l__spintent_geo_temp_tl
5854       { \l__spintent_geo_luaset_letra_conector_str - } { }
5855     }
5856     \tl_set:Nc \l__spintent_spfig_intent_arg_tl { _ \l__spintent_geo_temp_tl - }
5857   }
5858 }
5859 }

```

(End of definition for `\l__spintent_spfig_build_simple:n`.)

`\l__spintent_spfig_build_refs:`

Splits `\l__spintent_geo_luaset_points_clist` into `\l__spintent_spfig_pt_seq` and walks it once to build `\l__spintent_spfig_refs_tl`, the comma-separated `$pt1, $pt2, ...` reference list, naming each vertex by position.

```

5860 \cs_new_protected:Nn \l__spintent_spfig_build_refs:
5861 {
5862   \seq_set_from_clist:NN \l__spintent_spfig_pt_seq \l__spintent_geo_luaset_points_clist
5863   \tl_clear:N \l__spintent_spfig_refs_tl
5864   \int_zero:N \l__spintent_spfig_pt_idx_int
5865   \seq_map_inline:Nn \l__spintent_spfig_pt_seq
5866   {
5867     \int_incr:N \l__spintent_spfig_pt_idx_int
5868     \tl_if_empty:NF \l__spintent_spfig_refs_tl
5869     { \tl_put_right:Nn \l__spintent_spfig_refs_tl { , } }
5870     \tl_put_right:Nc \l__spintent_spfig_refs_tl { $ \int_use:N \l__spintent_spfig_pt_idx_int
5871   }
5872 }

```

(End of definition for `\l__spintent_spfig_build_refs:`.)

`\l__spintent_spfig_build_concept_vertices:n`

Handles the vertices case. Receives the short command name in ‘#1’. Must be called after `\l__spintent_spfig_build_refs:`. The key `read-arg`, when given, overrides the auto-detected figure name in `\l__spintent_spfig_intent_tl`: with `concept true` it is still prefixed with `área-del-/perímetro-del-`, with `concept false` it is used verbatim. With `read-arg` empty, `\l__spintent_spfig_intent_tl` is simply `\l__spintent_geo_luaset_concept_`.

```

5873 \cs_new_protected:Nn \l__spintent_spfig_build_concept_vertices:n
5874 {
5875   \tl_if_empty:cTF { \l__spintent_sp #1 _read_arg_tl }
5876   { \tl_set:Nc \l__spintent_spfig_intent_tl { \l__spintent_geo_luaset_concept_str } }
5877   {
5878     \bool_if:cTF { \l__spintent_sp #1 _concept_bool }
5879     {
5880       \tl_set:Nc \l__spintent_spfig_intent_tl
5881       {
5882         \l__spintent_spfig_op_tl -del-
5883         \tl_use:c { \l__spintent_sp #1 _read_arg_tl }
5884       }
5885     }

```

```

5886     {
5887         \tl_set:Nc \l__spinttent_spfig_intent_tl { \tl_use:c { l__spinttent_sp #1 _read_arg_tl } }
5888     }
5889 }

```

Determines `\l__spinttent_spfig_symbol_tl` (empty when none applies): for `print-sym=auto`, the symbol matching `\l__spinttent_geo_luaset_poly_sym_str(\triangle/\mdlgwhtsquare)`, or none when that is empty; `print-sym=none` always clears it; a forced `print-sym` requires it to match the actual vertex count, reporting `sp#1-sym-cardinality-conflict` otherwise.

```

5890 \str_case:VnF \l__spinttent_geo_print_sym_str
5891 {
5892     { auto }
5893     {
5894         \str_case:Vn \l__spinttent_geo_luaset_poly_sym_str
5895         {
5896             { triangle } { \tl_set:Nn \l__spinttent_spfig_symbol_tl { \triangle } }
5897             { square } { \tl_set:Nn \l__spinttent_spfig_symbol_tl { \mdlgwhtsquare } }
5898         }
5899         \str_if_empty:NT \l__spinttent_geo_luaset_poly_sym_str
5900         { \tl_clear:N \l__spinttent_spfig_symbol_tl }
5901     }
5902     { none }
5903     { \tl_clear:N \l__spinttent_spfig_symbol_tl }
5904 }
5905 {
5906     \str_if_eq:VVTf \l__spinttent_geo_print_sym_str \l__spinttent_geo_luaset_poly_sym_str
5907     { \tl_set:Nn \l__spinttent_spfig_symbol_tl { \__spinttent_spfig_sym: } }
5908     {
5909         \msg_error:nne { spinttent } { sp #1 -sym-cardinality-conflict }
5910         { \c_backslash_str sp #1 }
5911     }
5912 }

```

The key `silent` sets `\l__spinttent_spfig_intent_arg_tl` to `_-`. Otherwise, with `read-arg` empty, `concept` chooses between `\l__spinttent_spfig_intent_tl:prefix(refs)` (with `prefix` placed after the leading `_` when given) and `_ (refs)` alone; with `read-arg` given, it is always `\l__spinttent_spfig_intent_tl: - read-arg` is never silenced by `concept` alone, only by `silent`.

```

5913 \bool_if:cTF { l__spinttent_sp #1 _silent_bool }
5914 {
5915     \tl_set:Nn \l__spinttent_spfig_intent_arg_tl { _- }
5916 }
5917 {
5918     \tl_if_empty:cTF { l__spinttent_sp #1 _read_arg_tl }
5919     {
5920         \bool_if:cTF { l__spinttent_sp #1 _concept_bool }
5921         {
5922             \tl_set:Nc \l__spinttent_spfig_intent_arg_tl
5923             {
5924                 -
5925                 \tl_if_empty:cF { l__spinttent_sp #1 _prefix_tl }
5926                 { \tl_use:c { l__spinttent_sp #1 _prefix_tl } - }
5927                 \l__spinttent_spfig_intent_tl :prefix( \l__spinttent_spfig_refs_tl )
5928             }
5929         }
5930         {
5931             \tl_set:Nc \l__spinttent_spfig_intent_arg_tl { _ ( \l__spinttent_spfig_refs_tl ) }
5932         }
5933     }
5934     {
5935         \tl_set:Nc \l__spinttent_spfig_intent_arg_tl
5936         {
5937             -
5938             \tl_if_empty:cF { l__spinttent_sp #1 _prefix_tl }
5939             { \tl_use:c { l__spinttent_sp #1 _prefix_tl } - }
5940             \l__spinttent_spfig_intent_tl :prefix( \l__spinttent_spfig_refs_tl )
5941         }
5942     }
5943 }
5944 }

```

(End of definition for `\__spinttent_spfig_build_concept_vertices:n`)

`\__spintent_sfig_points_loop:n`

Walks `\__spintent_sfig_pt_seq` a second time, calling `\__spintent_mathmlarg_point:nnn` once per vertex with its `ptn` reference name, looked up under `sp#1`. Receives the short command name in ‘`#1`’.

```

5945 \cs_new_protected:Nn \__spintent_sfig_points_loop:n
5946 {
5947   \int_zero:N \__spintent_sfig_pt_idx_int
5948   \seq_map_inline:Nn \__spintent_sfig_pt_seq
5949   {
5950     \int_incr:N \__spintent_sfig_pt_idx_int
5951     \__spintent_mathmlarg_point:nnn
5952     { pt \int_use:N \__spintent_sfig_pt_idx_int }
5953     { sp #1 }
5954     { ##1 }
5955   }
5956 }
```

(End of definition for `\__spintent_sfig_points_loop:n`)

`\__spintent_sfig_mathml:n`

Receives the short command name in ‘`#1`’. Branches on whether `\__spintent_geo_luaset_points_clist` is empty.

Without vertices, `\__spintent_sfig_build_simple:n` builds the content; a `\MathMLintent` wraps it only when `concept` is `true` — with `concept=false` the content is typeset directly, with no `intent` at all.

With vertices, `\__spintent_sfig_build_refs:` then `\__spintent_sfig_build_concept_vertices:n` build the reference list and the concept `intent`, applied with a single native `\MathMLintent`. Inside, `\__spintent_mathcal:n{⟨A | P⟩}` is silenced with `\__spintent_luafun_mrow_intent_arg:nn` (it would otherwise be claimed by the queue meant for the symbol/vertices, the same ordering issue already found and fixed for `\spangle`’s own symbol); `\__spintent_sfig_symbol_tl` is silenced the same way when non-empty; the sequence is then walked by `\__spintent_sfig_points_loop:n`. All of this sits nested inside a subscript (`\c_math_subscript_token`), reached by the shared `mathml_filter` callback’s recursive lookup through a noad’s sub field, not just its nucleus.

The function `\__spintent_invisible_sep:` precedes both branches (a second sibling so the sole `<mrow>` does not collapse into `<math>`).

```

5957 \cs_new_protected:Nn \__spintent_sfig_mathml:n
5958 {
5959   \__spintent_invisible_sep: % (luamml issues #26)
5960   \clist_if_empty:NTF \__spintent_geo_luaset_points_clist
5961   {
5962     \__spintent_sfig_build_simple:n {#1}
5963     \MathMLintent { \__spintent_sfig_intent_arg_tl }
5964     {
5965       \__spintent_inner_sep:
5966       \str_if_eq:eeTF {#1} { Afig } { \__spintent_mathcal:n { A } } { \__spintent_mathcal:n
5967       \tl_if_empty:NF \__spintent_geo_final_print_tl
5968       {
5969         \c_math_subscript_token { \__spintent_geo_final_print_tl }
5970       }
5971     }
5972   }
5973   {
5974     \__spintent_sfig_build_refs:
5975     \__spintent_sfig_build_concept_vertices:n {#1}
5976     \MathMLintent { \__spintent_sfig_intent_arg_tl }
5977     {
5978       \__spintent_inner_sep:
5979       \__spintent_luafun_mrow_intent_arg:nn
5980       { } { intent = ":silent" }
5981       {
5982         \str_if_eq:eeTF {#1} { Afig } { \__spintent_mathcal:n { A } } { \__spintent_mathcal:n
5983       }
5984       \c_math_subscript_token
5985       {
5986         \tl_if_empty:NF \__spintent_sfig_symbol_tl
5987         {
5988           \__spintent_luafun_mrow_intent_arg:nn
5989           { } { intent = ":silent" }
5990           { \tl_use:N \__spintent_sfig_symbol_tl }
5991         }
5992         \__spintent_sfig_points_loop:n {#1}
5993       }
5994     }
5995   }
```

```
5996 }
```

(End of definition for `\__spintent_spfig_mathml:n`.)

```
\__spintent_spfig_exec:nnn
```

The function `\__spintent_spfig_exec:nnn` receives the short name of the command in #1, the optional keys in #2, and the argument in #3, delegating the rendering to `\__spintent_spfig_process:nn`.

```
5997 \cs_new_protected:Nn \__spintent_spfig_exec:nnn
5998 {
5999   \keys_set:nn { spintent / sp #1 } { #2 }
6000   \__spintent_spfig_process:nn { #1 } { #3 }
6001 }
```

(End of definition for `\__spintent_spfig_exec:nnn`.)

```
\spAfig
```

```
\spPfig
```

Both public commands `\spAfig` and `\spPfig` enforce math mode, open a local group, and delegate execution to `\__spintent_spfig_exec:nnn`.

```
6002 \NewDocumentCommand \spAfig { 0{} d() }
6003 {
6004   \mode_if_math:TF
6005   {
6006     \group_begin:
6007     \attribute \__spintent_scope_attr_int = 1
6008     \tl_if_novalue:nTF {#2}
6009     { \__spintent_spfig_exec:nnn { Afig } { #1 } { } }
6010     { \__spintent_spfig_exec:nnn { Afig } { #1 } { #2 } }
6011     \group_end:
6012   }
6013   { \msg_error:nnn { spintent } { need-math-mode } { \spAfig } }
6014 }
6015 \NewDocumentCommand \spPfig { 0{} d() }
6016 {
6017   \mode_if_math:TF
6018   {
6019     \group_begin:
6020     \attribute \__spintent_scope_attr_int = 1
6021     \tl_if_novalue:nTF {#2}
6022     { \__spintent_spfig_exec:nnn { Pfig } { #1 } { } }
6023     { \__spintent_spfig_exec:nnn { Pfig } { #1 } { #2 } }
6024     \group_end:
6025   }
6026   { \msg_error:nnn { spintent } { need-math-mode } { \spPfig } }
6027 }
```

(End of definition for `\spAfig` and `\spPfig`. These functions are documented on page 26.)

12.49 The commands `\sprad` and `\spdiam`

The user-level commands `\sprad` and `\spdiam` provide a semantic interface for the radius and diameter of a circle.

12.49.1 Definition of variables for `\sprad` and `\spdiam`

```
\__spintent_sp_rad_diam_letra_tl
\__spintent_sp_rad_diam_word_tl
\__spintent_sp_rad_diam_read_arg_full_tl
```

The variables `\__spintent_sp_rad_diam_letra_tl` and `\__spintent_sp_rad_diam_word_tl` are scratch variables used by `\__spintent_sp_rad_diam_print:n`. `\__spintent_sp_rad_diam_read_arg_full_tl` is used by `\__spintent_sp_rad_diam_read_arg_print:n` to hold the text to voice when `read-arg` is given.

```
6028 \tl_new:N \__spintent_sp_rad_diam_letra_tl
6029 \tl_new:N \__spintent_sp_rad_diam_word_tl
6030 \tl_new:N \__spintent_sp_rad_diam_read_arg_full_tl
```

(End of definition for `\__spintent_sp_rad_diam_letra_tl`, `\__spintent_sp_rad_diam_word_tl`, and `\__spintent_sp_rad_diam_read_arg_full_tl`.)

12.49.2 Definition of keys for `\sprad` and `\spdiam`

```
read-cmd
upper-case
sub-index
read-sub
read-arg
concept
silent
print-sym
unknown
```

We define the shared `<keys>` `read-cmd`, `upper-case`, `sub-index`, `read-sub`, `read-arg`, `concept`, `silent` and `print-sym`.

```
6031 \clist_map_inline:nn { rad , diam }
6032 {
6033   \keys_define:nn { spintent / sp #1 }
6034   {
6035     read-cmd .choices:nn = { school , mathcat }
6036     {
6037       \int_case:nn { \l_keys_choice_int }
```

```

6038         {
6039             {1} { \str_set:cn { \__spinttent_sp #1 _mode_str } { school } }
6040             {2} { \str_set:cn { \__spinttent_sp #1 _mode_str } { mathcat } }
6041         }
6042     },
6043     read-cmd      .initial:n = school,
6044     read-cmd      .value_required:n = true,
6045     read-cmd / unknown .code:n =
6046         \msg_error:nneee { spinttent } { unknown-choice }
6047         { read-cmd } { school,~mathcat } { \exp_not:n {##1} },
6048     upper-case    .bool_set:c = { \__spinttent_sp #1 _upper_case_bool },
6049     upper-case    .initial:n = false,
6050     upper-case    .value_required:n = true,
6051     sub-index     .code:n =
6052         { \str_set:cn { \__spinttent_sp #1 _sub_index_str } {##1} },
6053     sub-index     .initial:n = ,
6054     sub-index     .value_required:n = true,
6055     read-sub      .bool_set:c = { \__spinttent_sp #1 _read_sub_bool },
6056     read-sub      .initial:n = false,
6057     read-sub      .value_required:n = true,
6058     read-arg      .code:n =
6059         { \__spinttent_sanitizize_affix:cn { \__spinttent_sp #1 _read_arg_tl } {##1} },
6060     read-arg      .initial:n = ,
6061     read-arg      .value_required:n = true,
6062     concept       .bool_set:c = { \__spinttent_sp #1 _concept_bool },
6063     concept       .initial:n = true,
6064     concept       .value_required:n = true,
6065     silent        .bool_set:c = { \__spinttent_sp #1 _silent_bool },
6066     silent        .initial:n = false,
6067     silent        .value_required:n = true,
6068     print-sym     .bool_set:c = { \__spinttent_sp #1 _print_sym_bool },
6069     print-sym     .initial:n = false,
6070     print-sym     .value_required:n = true,
6071     unknown       .code:n =
6072         {
6073             \str_set:Nn \__spinttent_command_name_str { sp #1 }
6074             \__spinttent_unknown_keys_cmd:n {##1}
6075         },
6076     }
6077 }

```

🔗 The keys `print-sym` are defined for both commands for simplicity, but has no effect on `\sprad`.

(End of definition for `read-cmd` and others.)

12.49.3 Internal functions for `\sprad` and `\spdiam`

`\__spinttent_sp_rad_diam_print:n`

The function `\__spinttent_sp_rad_diam_print:n` sets `\__spinttent_sp_rad_diam_letra_tl` to the bare letter — `r/R` for `\sprad`, `d/D` for `\spdiam` — per `upper-case`, and, for `\spdiam` with `print-sym` true, silently typesets `\diameter` first. Receives the short command name in ‘`#1`’ (rad/diam).

```

6078 \cs_new_protected:Nn \__spinttent_sp_rad_diam_print:n
6079 {
6080     \bool_if:cTF { \__spinttent_sp #1 _upper_case_bool }
6081     {
6082         \str_if_eq:eeTF {#1} { rad }
6083         { \tl_set:Nn \__spinttent_sp_rad_diam_letra_tl { R } }
6084         { \tl_set:Nn \__spinttent_sp_rad_diam_letra_tl { D } }
6085     }
6086     {
6087         \str_if_eq:eeTF {#1} { rad }
6088         { \tl_set:Nn \__spinttent_sp_rad_diam_letra_tl { r } }
6089         { \tl_set:Nn \__spinttent_sp_rad_diam_letra_tl { d } }
6090     }
6091     \__spinttent_invisible_sep:
6092     \str_if_eq:eeT {#1} { diam }
6093     {
6094         \bool_if:NT \__spinttent_spdiam_print_sym_bool
6095         { \MathMLintent { :silent } { \diameter } }
6096     }

```

With `silent` true, the intent is `_` — the letter alone through the shared queue when there is no `sub-index`, or letter and subscript together in one native `\MathMLintent` (subscript silenced) when there is.

```

6097     \bool_if:cTF { \__spinttent_sp #1 _silent_bool }

```

```

6098 {
6099   \str_if_empty:cTF { \__spintent_sp #1 _sub_index_str }
6100   {
6101     \__spintent_luafun_mrow_intent_arg:nn
6102     { } { intent = "_-" }
6103     { \__spintent_sp_rad_diam_letra_tl }
6104   }
6105   {
6106     \MathMLintent { _- }
6107     {
6108       \__spintent_inner_sep:
6109       \__spintent_sp_rad_diam_letra_tl
6110       \c_math_subscript_token
6111       {
6112         \MathMLintent { :silent }
6113         { \str_use:c { \__spintent_sp #1 _sub_index_str } }
6114       }
6115     }
6116   }
6117 }

```

Otherwise, with `concept true` (the default), it verbalizes radio/diámetro (`read-arg` replaces this word when given), with `sub-index`, if present, appended right after (`read-sub` choosing `sub-⟨value⟩` or `⟨value⟩` alone) — the letter itself is never read here. With no `sub-index`, the shared queue tags the letter alone; with one, letter and subscript are wrapped together in one native `\MathMLintent` for `read-cmd=school` (subscript silenced, already spoken), or the shared queue tags the letter alone with the subscript left bare for `read-cmd=mathcat`.

```

6118 {
6119   \tl_if_empty:cTF { \__spintent_sp #1 _read_arg_tl }
6120   {
6121     \bool_if:cTF { \__spintent_sp #1 _concept_bool }
6122     {
6123       \str_if_eq:eeTF {#1} { rad }
6124       { \tl_set:Nn \__spintent_sp_rad_diam_word_tl { radio } }
6125       { \tl_set:Nn \__spintent_sp_rad_diam_word_tl { diámetro } }
6126       \str_if_empty:cTF { \__spintent_sp #1 _sub_index_str }
6127       {
6128         \__spintent_luafun_mrow_intent_arg:nn
6129         { } { intent = "_ \__spintent_sp_rad_diam_word_tl :nofix" }
6130         { \__spintent_sp_rad_diam_letra_tl }
6131       }
6132     }
6133     \str_if_eq:eeTF
6134     { \str_use:c { \__spintent_sp #1 _mode_str } } { school }
6135     {
6136       \MathMLintent
6137       {
6138         _ \__spintent_sp_rad_diam_word_tl -
6139         \bool_if:cT { \__spintent_sp #1 _read_sub_bool } { sub- }
6140         \str_use:c { \__spintent_sp #1 _sub_index_str }
6141       }
6142       {
6143         \__spintent_inner_sep:
6144         \__spintent_sp_rad_diam_letra_tl
6145         \c_math_subscript_token
6146         {
6147           \MathMLintent { :silent }
6148           { \str_use:c { \__spintent_sp #1 _sub_index_str } }
6149         }
6150       }
6151     }
6152     {
6153       \__spintent_luafun_mrow_intent_arg:nn
6154       { } { intent = "_ \__spintent_sp_rad_diam_word_tl :nofix" }
6155       { \__spintent_sp_rad_diam_letra_tl }
6156       \c_math_subscript_token
6157       { \str_use:c { \__spintent_sp #1 _sub_index_str } }
6158     }
6159   }
6160 }

```

With `concept=false`, the letter itself is read instead — but only when `read-cmd` is `school`, built the same

way from the letter (with `sub-index`, letter and subscript wrapped together, subscript silenced). With `read-cmd mathcat`, the intent is left empty: MathCAT applies its own rules to the letter and its subscript, unmarked by us.

```

6161         {
6162             \str_if_empty:cTF { \__spintent_sp #1 _sub_index_str }
6163             {
6164                 \str_if_eq:eeTF
6165                 { \str_use:c { \__spintent_sp #1 _mode_str } } { school }
6166                 {
6167                     \__spintent_luafun_mrow_intent_arg:nn
6168                     { } { intent = "_ \__spintent_sp_rad_diam_letra_tl -" }
6169                     { \__spintent_sp_rad_diam_letra_tl }
6170                 }
6171                 { \__spintent_sp_rad_diam_letra_tl }
6172             }
6173             {
6174                 \str_if_eq:eeTF
6175                 { \str_use:c { \__spintent_sp #1 _mode_str } } { school }
6176                 {
6177                     \MathMLintent
6178                     {
6179                         _ \__spintent_sp_rad_diam_letra_tl -
6180                         \bool_if:cT { \__spintent_sp #1 _read_sub_bool } { sub- }
6181                         \str_use:c { \__spintent_sp #1 _sub_index_str }
6182                     }
6183                     {
6184                         \__spintent_inner_sep:
6185                         \__spintent_sp_rad_diam_letra_tl
6186                         \c_math_subscript_token
6187                         {
6188                             \MathMLintent { :silent }
6189                             { \str_use:c { \__spintent_sp #1 _sub_index_str } }
6190                         }
6191                     }
6192                 }
6193             }
6194             \__spintent_sp_rad_diam_letra_tl
6195             \c_math_subscript_token
6196             { \str_use:c { \__spintent_sp #1 _sub_index_str } }
6197         }
6198     }
6199 }
6200 { \__spintent_sp_rad_diam_read_arg_print:n {#1} }
6201 }
6202 }
6203 }

```

(End of definition for `\__spintent_sp_rad_diam_print:n`.)

`\__spintent_sp_rad_diam_read_arg_print:n`

With `read-arg` given, `sub-index/read-sub` are ignored for the *spoken* content (the subscript, if any, still displays, silenced). With `concept true` (the default), `\__spintent_sp_rad_diam_read_arg_full_tl` becomes `radio/diámetro` followed by `read-arg`'s text; with `false`, `read-arg`'s text alone. Combined natively, regardless of `read-cmd`: with no `sub-index`, the shared queue tags the letter alone; with one, letter and subscript are wrapped together in one native `\MathMLintent`.

```

6204 \cs_new_protected:Nn \__spintent_sp_rad_diam_read_arg_print:n
6205 {
6206     \bool_if:cTF { \__spintent_sp #1 _concept_bool }
6207     {
6208         \str_if_eq:eeTF {#1} { rad }
6209         {
6210             \tl_set:Nc \__spintent_sp_rad_diam_read_arg_full_tl
6211             { radio- \tl_use:c { \__spintent_sp #1 _read_arg_tl } }
6212         }
6213         {
6214             \tl_set:Nc \__spintent_sp_rad_diam_read_arg_full_tl
6215             { diámetro- \tl_use:c { \__spintent_sp #1 _read_arg_tl } }
6216         }
6217     }
6218     {
6219         \tl_set_eq:Nc \__spintent_sp_rad_diam_read_arg_full_tl
6220         { \__spintent_sp #1 _read_arg_tl }

```

```

6221     }
6222     \str_if_empty:cTF { l__spintent_sp #1 _sub_index_str }
6223     {
6224         \__spintent_luafun_mrow_intent_arg:nn
6225         { } { intent = "_ \l__spintent_sp_rad_diam_read_arg_full_tl :nofix" }
6226         { \l__spintent_sp_rad_diam_letra_tl }
6227     }
6228     {
6229         \MathMLintent
6230         { _ \l__spintent_sp_rad_diam_read_arg_full_tl :nofix }
6231         {
6232             \__spintent_inner_sep:
6233             \l__spintent_sp_rad_diam_letra_tl
6234             \c_math_subscript_token
6235             {
6236                 \MathMLintent { :silent }
6237                 { \str_use:c { l__spintent_sp #1 _sub_index_str } }
6238             }
6239         }
6240     }
6241 }

```

(End of definition for `\__spintent_sp_rad_diam_read_arg_print:n`.)

`\__spintent_sp_rad_diam_exec:nn`

The function `\__spintent_sp_rad_diam_exec:nn` receives the *short name* of the command passed in ‘`#1`’ and the optional (*keys*) passed in ‘`#2`’ then delegating the rendering to `\__spintent_sp_rad_diam_print:n`.

```

6242 \cs_new_protected:Nn \__spintent_sp_rad_diam_exec:nn
6243 {
6244     \keys_set:nn { spintent / sp #1 } {#2}
6245     \__spintent_sp_rad_diam_print:n {#1}
6246 }

```

(End of definition for `\__spintent_sp_rad_diam_exec:nn`.)

`\sprad`
`\spdiam`

Both public commands `\sprad` and `\spdiam` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_sp_rad_diam_exec:nn`.

```

6247 \NewDocumentCommand \sprad { 0{} }
6248 {
6249     \mode_if_math:TF
6250     {
6251         \group_begin:
6252         \attribute \l__spintent_scope_attr_int = 1
6253         \__spintent_sp_rad_diam_exec:nn { rad } {#1}
6254         \group_end:
6255     }
6256     { \msg_error:nnn { spintent } { need-math-mode } { \sprad } }
6257 }
6258 \NewDocumentCommand \spdiam { 0{} }
6259 {
6260     \mode_if_math:TF
6261     {
6262         \group_begin:
6263         \attribute \l__spintent_scope_attr_int = 1
6264         \__spintent_sp_rad_diam_exec:nn { diam } {#1}
6265         \group_end:
6266     }
6267     { \msg_error:nnn { spintent } { need-math-mode } { \spdiam } }
6268 }

```

(End of definition for `\sprad` and `\spdiam`. These functions are documented on page 29.)

12.50 The command `\spcirc`

The user command `\spcirc` provides a semantic interface for a circle or a circumference, using the delimited-optional argument syntax `d()`. The argument holds a required center point and an optional radius, separated by a comma. The radius is never required to declare its own kind via a key: its shape alone decides whether it is a pure number, a measure (number + unit), a bare letter, or a two-letter segment — reusing, respectively, `\spnum`, `\spunit`, and `\spside`/`\spmside` directly as sub-commands, rather than reimplementing any of their parsing.

12.50.1 Definition of variables for \spcirc

__spinttent_geo_luaset_circ_radio_type_str
 __spinttent_geo_luaset_circ_radio_raw_str

Shared outputs of `\__spinttent_luafun_geo_circ_parse_and_set:n`. The variable `\__spinttent_geo_luaset_circ_radio_type_str` holds one of `number`, `measure`, `label`, `segment`, or an empty string when no radius was given. `\__spinttent_geo_luaset_circ_radio_raw_str` holds the corresponding raw text, ready to be forwarded to the sub-command that renders it.

```
6269 \str_new:N \__spinttent_geo_luaset_circ_radio_type_str
6270 \str_new:N \__spinttent_geo_luaset_circ_radio_raw_str
```

(End of definition for `\__spinttent_geo_luaset_circ_radio_type_str` and `\__spinttent_geo_luaset_circ_radio_raw_str`.)

__spinttent_spcirc_mode_str
 __spinttent_spcirc_prefix_tl
 __spinttent_spcirc_print_sym_str
 __spinttent_spcirc_read_sty_str
 __spinttent_spcirc_radio_sty_str
 __spinttent_spcirc_sub_index_str
 __spinttent_spcirc_articulo_str

The variable `\__spinttent_spcirc_mode_str` stores the value set by the key `read-cmd`; the variable `\__spinttent_spcirc_prefix_tl` stores the value set by the key `prefix`. The variable `\__spinttent_spcirc_print_sym_str` stores the value set by the key `print-sym` (`usual` or `odot`). The variable `\__spinttent_spcirc_read_sty_str` stores the value set by the key `read-sty` (`circun` or `circle`). The variable `\__spinttent_spcirc_radio_sty_str` stores the value set by the key `radio-sty`, only meaningful when the radius is a segment. The variable `\__spinttent_spcirc_sub_index_str` stores the value set by the key `sub-index`. The variable `\__spinttent_spcirc_articulo_str` is set internally by `\__spinttent_spcirc_word_and_article:` to `la` or `el`, matching the grammatical gender of the chosen word.

```
6271 \str_new:N \__spinttent_spcirc_mode_str
6272 \tl_new:N \__spinttent_spcirc_prefix_tl
6273 \str_new:N \__spinttent_spcirc_print_sym_str
6274 \str_new:N \__spinttent_spcirc_read_sty_str
6275 \str_new:N \__spinttent_spcirc_radio_sty_str
6276 \str_new:N \__spinttent_spcirc_sub_index_str
6277 \str_new:N \__spinttent_spcirc_articulo_str
6278 \tl_new:N \__spinttent_spcirc_read_arg_tl
```

(End of definition for `\__spinttent_spcirc_mode_str` and others.)

__spinttent_spcirc_word_tl

Scratch variables used by `\__spinttent_spcirc_process:n` to assemble the chosen word (`circunferencia/círculo` or the `read-arg` override) and the final literal `intent` string.

```
6279 \tl_new:N \__spinttent_spcirc_word_tl
```

(End of definition for `\__spinttent_spcirc_word_tl`.)

12.50.2 Definition of error messages for \spcirc

spcirc-invalid-arg

Raised when the argument does not match a single center point, optionally followed by a comma and a valid radius shape.

```
6280 \msg_new:nnnn { spinttent } { spcirc-invalid-arg }
6281 {
6282   Invalid~argument~format~in~\token_to_str:N \spcirc:~'#1'~\msg_line_context:.
6283 }
6284 {
6285   See~the~documentation~of~\token_to_str:N \spcirc~for~the~accepted~argument~forms.
6286 }
```

(End of definition for `spcirc-invalid-arg`.)

12.50.3 Definition of keys for \spcirc

read-cmd
 prefix
 print-sym
 read-sty
 diameter
 radio-sty
 sub-index
 read-sub
 read-arg
 silent
 concept
 spside
 spmside
 unknown

The key `print-sym` chooses the visual symbol: `usual` for a plain C, `odot` (default) for `\odot`. The key `read-sty` chooses the verbalized word and its article: `circun` (default) for «*la circunferencia*», `circle` for «*el círculo*». The key `radio-sty` chooses, only when the radius is a two-letter segment, whether it is rendered as the *object* (`side`, via `\spside`) or its *measure* (`m-side`, default, via `\spmside`). The key `sub-index` sets a subscript on the symbol; the key `read-sub` controls whether it is read with the word «sub» or as a plain number, matching the same convention as `\spAfig`.

```
6287 \keys_define:nn { spinttent / spcirc }
6288 {
6289   read-cmd      .choices:nn = { school , mathcat }
6290   {
6291     \int_case:nn { \l_keys_choice_int }
6292     {
6293       {1}
6294       {
6295         \str_set:Nn \__spinttent_spcirc_mode_str { school }
6296         \keys_set:nn { spinttent / spside } { read-cmd = school }
6297         \keys_set:nn { spinttent / spmside } { read-cmd = school }
6298       }
6299       {2}
6300       {
6301         \str_set:Nn \__spinttent_spcirc_mode_str { mathcat }
```

```

6302         \keys_set:nn { spintent / spside } { read-cmd = mathcat }
6303         \keys_set:nn { spintent / spmside } { read-cmd = mathcat }
6304     }
6305 }
6306 },
6307 read-cmd .initial:n = school,
6308 read-cmd .value_required:n = true,
6309 prefix .code:n =
6310 { \__spintent_sanitizet_affix:Nn \l__spintent_spcirc_prefix_tl {#1} },
6311 prefix .initial:n = ,
6312 prefix .value_required:n = true,
6313 print-sym .choices:nn = { usual , odot }
6314 {
6315     \int_case:nn { \l_keys_choice_int }
6316     {
6317         {1} { \str_set:Nn \l__spintent_spcirc_print_sym_str { usual } }
6318         {2} { \str_set:Nn \l__spintent_spcirc_print_sym_str { odot } }
6319     }
6320 },
6321 print-sym .initial:n = odot,
6322 print-sym .value_required:n = true,
6323 read-sty .choices:nn = { circun , circle }
6324 {
6325     \int_case:nn { \l_keys_choice_int }
6326     {
6327         {1} { \str_set:Nn \l__spintent_spcirc_read_sty_str { circun } }
6328         {2} { \str_set:Nn \l__spintent_spcirc_read_sty_str { circle } }
6329     }
6330 },
6331 read-sty .initial:n = circun,
6332 read-sty .value_required:n = true,
6333 diameter .bool_set:N = \l__spintent_spcirc_diameter_bool,
6334 diameter .initial:n = false,
6335 diameter .value_required:n = true,
6336 radio-sty .choices:nn = { side , mside }
6337 {
6338     \int_case:nn { \l_keys_choice_int }
6339     {
6340         {1} { \str_set:Nn \l__spintent_spcirc_radio_sty_str { side } }
6341         {2} { \str_set:Nn \l__spintent_spcirc_radio_sty_str { mside } }
6342     }
6343 },
6344 radio-sty .initial:n = mside,
6345 radio-sty .value_required:n = true,
6346 sub-index .code:n =
6347 { \str_set:Nn \l__spintent_spcirc_sub_index_str {#1} },
6348 sub-index .initial:n = ,
6349 sub-index .value_required:n = true,
6350 read-sub .bool_set:N = \l__spintent_spcirc_read_sub_bool,
6351 read-sub .initial:n = false,
6352 read-sub .value_required:n = true,
6353 read-arg .code:n =
6354 { \__spintent_sanitizet_affix:Nn \l__spintent_spcirc_read_arg_tl {#1} },
6355 read-arg .initial:n = ,
6356 read-arg .value_required:n = true,
6357 silent .bool_set:N = \l__spintent_spcirc_silent_bool,
6358 silent .initial:n = false,
6359 silent .value_required:n = true,
6360 concept .bool_set:N = \l__spintent_spcirc_concept_bool,
6361 concept .initial:n = true,
6362 concept .value_required:n = true,
6363 spside .meta:nn = { spintent / spside } { #1 },
6364 spside .value_required:n = true,
6365 spmside .meta:nn = { spintent / spmside } { #1 },
6366 spmside .value_required:n = true,
6367 unknown .code:n =
6368 {
6369     \str_set:Nn \l__spintent_command_name_str { spcirc }
6370     \__spintent_unknown_keys_cmd:n {#1}
6371 },
6372 }

```

(End of definition for `read-cmd` and others.)

12.50.4 Internal functions for `\spcirc`

```
\l_spintrent_spcirc_has_arg_bool
\l_spintrent_spcirc_intent_tl
\l_spintrent_spcirc_intent_arg_tl
\l_spintrent_spcirc_connector_tl
\l_spintrent_spcirc_connector_arg_tl
\l_spintrent_spcirc_radio_arg_tl
```

The variable `\l__spintrent_spcirc_has_arg_bool` records whether the `d()` argument was given at all (`\spcirc` with no `(...)` at all has neither a center nor a radius). `\l__spintrent_spcirc_intent_tl` holds the word (`circunferencia` or `círculo`, with an optional `sub-index` suffix) paired with the symbol. The variable `\l__spintrent_spcirc_connector_tl` holds `de-centro-en`, paired with the `(center)` group; the center is read through `\__spintrent_mathmlarg_point:nnn`, referenced as `$pt`, not concatenated into either word — it can be subscripted or primed (e.g. `O_{1}`), as in the Euler circle) without breaking anything. The `_arg_tl` variables hold each word’s final wrapped form; a third, `\l__spintrent_spcirc_radio_arg_tl`, is spoken directly on the `,` delimiter after the center, replacing its own reading rather than pairing with a group.

```
6373 \bool_new:N \l__spintrent_spcirc_has_arg_bool
6374 \tl_new:N \l__spintrent_spcirc_intent_tl
6375 \tl_new:N \l__spintrent_spcirc_intent_arg_tl
6376 \tl_new:N \l__spintrent_spcirc_connector_tl
6377 \tl_new:N \l__spintrent_spcirc_connector_arg_tl
6378 \tl_new:N \l__spintrent_spcirc_radio_arg_tl
```

(End of definition for `\l__spintrent_spcirc_has_arg_bool` and others.)

```
\__spintrent_spcirc_sym:
```

Typesets `\bigodot` or the plain letter C, according to `print-sym`.

```
6379 \cs_new_protected:Nn \__spintrent_spcirc_sym:
6380 {
6381   \str_if_eq:NnTF \l__spintrent_spcirc_print_sym_str { odot }
6382   {
6383     \group_begin:
6384     \textstyle\bigodot
6385     \group_end:
6386   }
6387   { C }
6388 }
```

(End of definition for `\__spintrent_spcirc_sym:`.)

```
\__spintrent_spcirc_word_and_article:
```

Sets `\l__spintrent_spcirc_word_tl` to `read-arg` or `circunferencia/círculo` from `read-sty`, and `\l__spintrent_spcirc_articulo_str` to the matching article, which always follows `read-sty` even when `read-arg` overrides the word.

```
6389 \cs_new_protected:Nn \__spintrent_spcirc_word_and_article:
6390 {
6391   \tl_if_empty:NTF \l__spintrent_spcirc_read_arg_tl
6392   {
6393     \str_if_eq:NnTF \l__spintrent_spcirc_read_sty_str { circun }
6394     {
6395       \tl_set:Nn \l__spintrent_spcirc_word_tl { circunferencia }
6396       \str_set:Nn \l__spintrent_spcirc_articulo_str { la }
6397     }
6398     {
6399       \tl_set:Nn \l__spintrent_spcirc_word_tl { círculo }
6400       \str_set:Nn \l__spintrent_spcirc_articulo_str { el }
6401     }
6402   }
6403   {
6404     \tl_set:Nn \l__spintrent_spcirc_word_tl { \l__spintrent_spcirc_read_arg_tl }
6405     \str_if_eq:NnTF \l__spintrent_spcirc_read_sty_str { circun }
6406     { \str_set:Nn \l__spintrent_spcirc_articulo_str { la } }
6407     { \str_set:Nn \l__spintrent_spcirc_articulo_str { el } }
6408   }
6409 }
```

(End of definition for `\__spintrent_spcirc_word_and_article:`.)

```
\__spintrent_spcirc_radio_print:
```

Dispatches on `\l__spintrent_geo_luaset_circ_radio_type_str`, reusing `\spnum/\spunit/\spside/\spmside` directly rather than reimplementing any parsing.

```
6410 \cs_new_protected:Nn \__spintrent_spcirc_radio_print:
6411 {
6412   \str_case:Nn \l__spintrent_geo_luaset_circ_radio_type_str
6413   {
6414     { number } { \exp_args:Ne \spnum { \l__spintrent_geo_luaset_circ_radio_raw_str } }
6415     { measure } { \exp_args:Ne \spunit { \l__spintrent_geo_luaset_circ_radio_raw_str } }
6416     { label } { \l__spintrent_geo_luaset_circ_radio_raw_str }
6417     { segment }
```

```

6418     {
6419         \str_if_eq:VnTF \l__spintent_spcirc_radio_sty_str { side }
6420         { \exp_args:Ne \spside { \l__spintent_geo_luaset_circ_radio_raw_str } }
6421         { \exp_args:Ne \spmside { \l__spintent_geo_luaset_circ_radio_raw_str } }
6422     }
6423 }
6424 }

```

(End of definition for `\__spintent_spcirc_radio_print:`.)

`\__spintent_spcirc_build_concept:`

Builds two words: `\l__spintent_spcirc_intent_tl` (via `\__spintent_spcirc_word_and_article:`, plus an optional `sub-index` suffix), paired with the symbol, and `\l__spintent_spcirc_connector_tl` (de-centro-en), paired with the (center) group — built even when `\l__spintent_spcirc_has_arg_bool` is `false`, though `\__spintent_spcirc_mathml:` never uses it in that case. Neither carries the center's own text: it is read separately, by reference, through `\__spintent_mathmlarg_point:nnn` — so a subscripted or primed center (e.g. $O_{\{1\}}$, as in the Euler circle) is never an issue.

```

6425 \cs_new_protected:Nn \__spintent_spcirc_build_concept:
6426 {
6427     \__spintent_spcirc_word_and_article:
6428     \tl_set:Nn \l__spintent_spcirc_intent_tl { \l__spintent_spcirc_word_tl }
6429     \str_if_empty:NF \l__spintent_spcirc_sub_index_str
6430     {
6431         \tl_put_right:Nn \l__spintent_spcirc_intent_tl
6432         {
6433             -
6434             \bool_if:NT \l__spintent_spcirc_read_sub_bool { sub - }
6435             \l__spintent_spcirc_sub_index_str
6436         }
6437     }
6438     \tl_set:Nn \l__spintent_spcirc_connector_tl { de-centro-en }

```

The key `silent`, when `true`, overrides `\l__spintent_spcirc_intent_arg_tl`, `\l__spintent_spcirc_connector_arg_tl` and `\l__spintent_spcirc_radio_arg_tl` with `_`, regardless of `read-arg`: a composite center (one with a subscript or a prime) is not reliably silenced by the per-point `:silent` alone.

```

6439     \bool_if:NTF \l__spintent_spcirc_silent_bool
6440     {
6441         \tl_set:Nn \l__spintent_spcirc_intent_arg_tl { _- }
6442         \tl_set:Nn \l__spintent_spcirc_connector_arg_tl { _- }
6443         \tl_set:Nn \l__spintent_spcirc_radio_arg_tl { _- }
6444     }

```

With `concept true`, `\l__spintent_spcirc_intent_arg_tl` is `\l__spintent_spcirc_intent_tl` alone (bare — the group it wraps has nothing to reference, only a silenced symbol), with `prefix` placed after the leading `_` when given; `\l__spintent_spcirc_connector_arg_tl` is `\l__spintent_spcirc_connector_tl:pr` and `\l__spintent_spcirc_radio_arg_tl` — spoken directly on the `,` delimiter, replacing its own reading rather than pairing with any group — is the literal `_y-radio-`, or `_y-diámetro-` when `diameter` is `true`, read as its own phrase right before `\__spintent_spcirc_radio_print:`'s independent reading follows as the next sibling.

```

6445     {
6446         \bool_if:NTF \l__spintent_spcirc_concept_bool
6447         {
6448             \tl_set:Nn \l__spintent_spcirc_intent_arg_tl
6449             {
6450                 -
6451                 \tl_if_empty:NF \l__spintent_spcirc_prefix_tl
6452                 { \tl_use:N \l__spintent_spcirc_prefix_tl - }
6453                 \l__spintent_spcirc_intent_tl
6454             }
6455             \tl_set:Nn \l__spintent_spcirc_connector_arg_tl
6456             { _ \l__spintent_spcirc_connector_tl :prefix( $ pt ) }
6457             \bool_if:NTF \l__spintent_spcirc_diameter_bool
6458             { \tl_set:Nn \l__spintent_spcirc_radio_arg_tl { _y-diámetro- } }
6459             { \tl_set:Nn \l__spintent_spcirc_radio_arg_tl { _y-radio- } }
6460         }

```

With `concept false` and `read-arg` empty, the same `_/_($pt)` silencing applies as `silent`; with `read-arg` given, none of the three is silenced instead — the connector and `y-radio/y-diámetro` words are fixed structural pieces, not customized by `read-arg`, but once a real word is being spoken there is nothing left to silence around it (same convention as `\__spintent_spside_process:nn`).

```

6461     {
6462         \tl_if_empty:NTF \l__spintent_spcirc_read_arg_tl

```

```

6463         {
6464             \tl_set:Nn \__spintent_spcirc_intent_arg_tl { _- }
6465             \tl_set:Ne \__spintent_spcirc_connector_arg_tl { _ ( $ pt ) }
6466             \tl_set:Nn \__spintent_spcirc_radio_arg_tl { _- }
6467         }
6468         {
6469             \tl_set:Ne \__spintent_spcirc_intent_arg_tl { _ \__spintent_spcirc_intent_tl }
6470             \tl_set:Ne \__spintent_spcirc_connector_arg_tl
6471             { _ \__spintent_spcirc_connector_tl :prefix( $ pt ) }
6472             \bool_if:NTF \__spintent_spcirc_diameter_bool
6473             { \tl_set:Nn \__spintent_spcirc_radio_arg_tl { _y-diámetro- } }
6474             { \tl_set:Nn \__spintent_spcirc_radio_arg_tl { _y-radio- } }
6475         }
6476     }
6477 }
6478 }

```

(End of definition for `\__spintent_spcirc_build_concept:`)

`\__spintent_spcirc_process:n` Receives the `d()` argument in ‘#1’, an explicit empty group when omitted (`\spcirc` itself converts `-NoValue-` to `{ }` before calling `\__spintent_spcirc_exec:nn`) — `\__spintent_spcirc_has_arg_bool` set to `false` in that case, and `\__spintent_luafun_geo_circ_parse_and_set:n` never called at all — there is nothing to parse. Otherwise calls it, reports `spcirc-invalid-arg` on failure, then delegates to `\__spintent_spcirc_mathml:`, which calls `\__spintent_spcirc_build_concept:` itself.

```

6479 \cs_new_protected:Nn \__spintent_spcirc_process:n
6480 {
6481     \tl_if_empty:NTF {#1}
6482     {
6483         \bool_set_false:N \__spintent_spcirc_has_arg_bool
6484     }
6485     {
6486         \bool_set_true:N \__spintent_spcirc_has_arg_bool
6487         \__spintent_luafun_geo_circ_parse_and_set:n { \exp_not:n {#1} }
6488         \str_if_eq:VnT \__spintent_geo_luaset_error_str { true }
6489         {
6490             \msg_error:nne { spintent } { spcirc-invalid-arg } { \exp_not:n {#1} }
6491         }
6492     }
6493     \__spintent_spcirc_mathml:
6494 }

```

(End of definition for `\__spintent_spcirc_process:n`)

`\__spintent_spcirc_mathml:` Calls `\__spintent_spcirc_build_concept:`, then wraps the symbol (`\__spintent_spcirc_intent_arg_tl`) in its own native `\MathMLintent` — this alone is the whole result when `\__spintent_spcirc_has_arg_bool` is `false` (`\spcirc` with no `(...)` at all: no center, no radius).

Otherwise, the `(center)` group (`\__spintent_spcirc_connector_arg_tl`) follows as a separate sibling, never nested inside the symbol’s own `\MathMLintent` — or MathCAT falls back to its own structural reading of the underlying `<mover>/<msub>` instead, the same problem already found for `\spangle`’s symbol and `\sparc`’s accent. When a radius is present, the `,` delimiter after the center carries `\__spintent_spcirc_radio_arg_tl` directly (replacing its own reading, not paired with any group), and `\__spintent_spcirc_radio_print:` follows as an ordinary sibling right after — except with `silent=true`, when it is wrapped in a literal `_-` `\MathMLintent`, which silences whatever it produces. Otherwise, wrapping it in another `\MathMLintent` would lose the reading of whatever it produces (itself a native `\MathMLintent`, from `\spnum/\spunit`, nested inside another one does not compose).

The symbol and, when `sub-index` is given, its subscript number are each silenced with `\__spintent_luafun_mrow_inseparately` — one queue entry per leaf. The two are reached differently by the shared `mathml_filter` call-back’s recursive lookup (the symbol via `nucleus`, the subscript via `sub`), so a single entry covering both leaves nothing registered for the second one, which the recursion then assigns whatever comes next in the queue instead — shifting every claim after it by one. The `(` delimiter is silenced the same way, but *without* the usual `{...}` around the character: `\uamml` only keeps a delimiter’s own math class (giving `<mo>` instead of `<mi>`, needed for MathCAT to read the group at all) when it is not itself sitting inside a brace group.

The final `)` — following `\__spintent_spcirc_radio_print:`, whose own output the shared queue does not reliably skip over — is typeset as `\rparen` inside its own native `\MathMLintent{<silent>}` instead of a bare `)`: `\rparen` keeps its own math class through the wrapping, unlike a bare delimiter character, and this sits outside the shared queue entirely, so nothing in between can claim an entry meant for it.

The center itself is read through `\__spintent_mathmlarg_point:nnn`, referenced as `$pt`.

```

6495 \cs_new_protected:Nn \__spintent_spcirc_mathml:
6496 {

```

```

6497 \__spintent_invisible_sep: % (luamml issues #26)
6498 \__spintent_spcirc_build_concept:
6499 \MathMLintent { \__spintent_spcirc_intent_arg_tl }
6500 {
6501   \__spintent_inner_sep:
6502   \__spintent_luafun_mrow_intent_arg:nn
6503   { } { intent = ":silent" }
6504   { \__spintent_spcirc_sym: }
6505   \str_if_empty:NF \__spintent_spcirc_sub_index_str
6506   {
6507     \c_math_subscript_token
6508     {
6509       \__spintent_luafun_mrow_intent_arg:nn
6510       { } { intent = ":silent" }
6511       { \__spintent_spcirc_sub_index_str }
6512     }
6513   }
6514 }
6515 \bool_if:NT \__spintent_spcirc_has_arg_bool
6516 {
6517   \str_if_empty:NTF \__spintent_geo_luaset_circ_radio_type_str
6518   {
6519     \MathMLintent { \__spintent_spcirc_connector_arg_tl }
6520     {
6521       \__spintent_inner_sep:
6522       \__spintent_luafun_mrow_intent_arg:nn { } { intent = ":silent" } (
6523       \__spintent_mathmlarg_point:nnV { pt } { spcirc } \__spintent_geo_luaset_points_
6524       \__spintent_luafun_mrow_intent_arg:nn { } { intent = ":silent" } )
6525     }
6526   }
6527   {
6528     \MathMLintent { \__spintent_spcirc_connector_arg_tl }
6529     {
6530       \__spintent_inner_sep:
6531       \__spintent_luafun_mrow_intent_arg:nn { } { intent = ":silent" } (
6532       \__spintent_mathmlarg_point:nnV { pt } { spcirc } \__spintent_geo_luaset_points_
6533     }
6534     \MathMLintent { \__spintent_spcirc_radio_arg_tl } { , }
6535     \bool_if:NTF \__spintent_spcirc_silent_bool
6536     {
6537       \MathMLintent { _- }
6538       {
6539         \__spintent_inner_sep:
6540         \__spintent_spcirc_radio_print:
6541       }
6542     }
6543     { \__spintent_spcirc_radio_print: }
6544     \MathMLintent { :silent } { \rparen }
6545   }
6546 }
6547 }

```

(End of definition for __spintent_spcirc_mathml:.)

__spintent_spcirc_exec:nn

The function `\__spintent_spcirc_exec:nn` receives the optional keys in #1 and the argument in #2, delegating the rendering to `\__spintent_spcirc_print:n`.

```

6548 \cs_new_protected:Nn \__spintent_spcirc_exec:nn
6549 {
6550   \keys_set:nn { spintent / spcirc } { #1 }
6551   \__spintent_spcirc_process:n { #2 }
6552 }

```

(End of definition for __spintent_spcirc_exec:nn.)

\spcirc

The public command `\spcirc` enforces math mode, opens a local group, and delegates execution to `\__spintent_spcirc_exec:nn`. It follows the same `d()`-argument pattern as `\spmcdd/\spmcmm` and `\spAfig/\spPfig`; an omitted radius forwards an explicit empty group rather than the `-NoValue-` marker.

```

6553 \NewDocumentCommand \spcirc { 0{} } d() {
6554   {
6555     \mode_if_math:TF
6556     {

```

```

6557     \group_begin:
6558     \attribute \l__spintent_scope_attr_int = 1
6559     \tl_if_novalue:nTF {#2}
6560     { \__spintent_spcirc_exec:nn { #1 } { } }
6561     { \__spintent_spcirc_exec:nn { #1 } { #2 } }
6562     \group_end:
6563   }
6564   { \msg_error:nnn { spintent } { need-math-mode } { \spcirc } }
6565 }

```

(End of definition for `\spcirc`. This function is documented on page 30.)

12.51 The commands `\spAcirc` and `\spPcirc`

The user commands `\spAcirc` and `\spPcirc` typeset the area and the perimeter of a circle or a circumference. The argument follows `\spcirc`'s own point-and-radius grammar for a real center (with an optional radius); when no comma is present and no valid center is found, it falls back to a bare number/letter shortcut, reusing the same Lua entry point as `\spAfig/\spPfig`. A table letter matched by that fallback is rejected, since it has no meaning for a circle.

12.51.1 Definition of variables for `\spAcirc` and `\spPcirc`

These variables store the value set by each corresponding key: `read-cmd`, `print-sym`, `read-sty`, `radio-sty`, `sub-index`, `prefix` and `read-arg`, respectively.

```

\__spintent_spAcirc_mode_str
\__spintent_spPcirc_mode_str
\__spintent_spAcirc_print_sym_str
\__spintent_spPcirc_print_sym_str
\__spintent_spAcirc_read_sty_str
\__spintent_spPcirc_read_sty_str
\__spintent_spAcirc_radio_sty_str
\__spintent_spPcirc_radio_sty_str
\__spintent_spAcirc_sub_index_str
\__spintent_spPcirc_sub_index_str
\__spintent_spAcirc_prefix_tl
\__spintent_spPcirc_prefix_tl
\__spintent_spAcirc_read_arg_tl
\__spintent_spPcirc_read_arg_tl
\__spintent_geo_circfig_mode_str

```

```

6566 \clist_map_inline:nn { Acirc , Pcirc }
6567 {
6568   \str_new:c { \__spintent_sp #1 _mode_str }
6569   \str_new:c { \__spintent_sp #1 _print_sym_str }
6570   \str_new:c { \__spintent_sp #1 _read_sty_str }
6571   \str_new:c { \__spintent_sp #1 _radio_sty_str }
6572   \str_new:c { \__spintent_sp #1 _sub_index_str }
6573   \tl_new:c { \__spintent_sp #1 _prefix_tl }
6574   \tl_new:c { \__spintent_sp #1 _read_arg_tl }
6575 }

```

(End of definition for `\__spintent_spAcirc_mode_str` and others.)

The variable `\__spintent_geo_circfig_mode_str` stores whether the argument was resolved as a real circle (`circle`) or as the bare number/letter shortcut (bare).

```

6576 \str_new:N \__spintent_geo_circfig_mode_str

```

(End of definition for `\__spintent_geo_circfig_mode_str`.)

12.51.2 Definition of error messages for `\spAcirc` and `\spPcirc`

```

spAcirc-invalid-arg
spPcirc-invalid-arg

```

The error message shown when the argument does not match any of the accepted forms: empty, a center point (optionally followed by a comma and a radius), or a bare number/letter.

```

6577 \clist_map_inline:nn { Acirc , Pcirc }
6578 {
6579   \msg_new:nnnn { spintent } { sp #1 -invalid-arg }
6580   {
6581     Invalid~argument~in~##1:~'##2'~\msg_line_context:.
6582   }
6583   {
6584     See~the~documentation~of~##1~for~the~accepted~argument.
6585   }

```

(End of definition for `spAcirc-invalid-arg` and `spPcirc-invalid-arg`.)

12.51.3 Definition of keys for `\spAcirc` and `\spPcirc`

We define the shared (*keys*) `read-cmd`, `prefix`, `print-sym`, `read-sty`, `concept`, `sub-index`, `read-sub`, `read-arg` and `silent`.

```

6586 \keys_define:nn { spintent / sp #1 }
6587 {
6588   read-cmd .choices:nn = { school , mathcat }
6589   {
6590     \int_case:nn { \l_keys_choice_int }
6591     {
6592       {1} { \str_set:cn { \__spintent_sp #1 _mode_str } { school } }
6593       {2} { \str_set:cn { \__spintent_sp #1 _mode_str } { mathcat } }
6594     }
6595   },
6596   read-cmd .initial:n = school,
6597   read-cmd .value_required:n = true,

```

```

6598     read-cmd / unknown .code:n =
6599         \msg_error:nneee { spintent } { unknown-choice }
6600         { read-cmd } { school,~mathcat } { \exp_not:n {##1} },
6601     prefix .code:n =
6602         { \__spintent_sanitizet_affix:cn { l__spintent_sp #1 _prefix_tl } {##1} },
6603     prefix .initial:n = ,
6604     prefix .value_required:n = true,
6605     print-sym .choices:nn = { usual , odot }
6606     {
6607         \int_case:nn { \l_keys_choice_int }
6608         {
6609             {1} { \str_set:cn { l__spintent_sp #1 _print_sym_str } { usual } }
6610             {2} { \str_set:cn { l__spintent_sp #1 _print_sym_str } { odot } }
6611         }
6612     },
6613     print-sym .initial:n = odot,
6614     print-sym .value_required:n = true,
6615     print-sym / unknown .code:n =
6616         \msg_error:nneee { spintent } { unknown-choice }
6617         { print-sym } { usual,~odot } { \exp_not:n {##1} },
6618     read-sty .choices:nn = { circun , circle }
6619     {
6620         \int_case:nn { \l_keys_choice_int }
6621         {
6622             {1} { \str_set:cn { l__spintent_sp #1 _read_sty_str } { circun } }
6623             {2} { \str_set:cn { l__spintent_sp #1 _read_sty_str } { circle } }
6624         }
6625     },
6626     read-sty .initial:n = circun,
6627     read-sty .value_required:n = true,
6628     concept .bool_set:c = { l__spintent_sp #1 _concept_bool },
6629     concept .initial:n = true,
6630     concept .value_required:n = true,
6631     sub-index .code:n =
6632         { \str_set:cn { l__spintent_sp #1 _sub_index_str } {##1} },
6633     sub-index .initial:n = ,
6634     sub-index .value_required:n = true,
6635     read-sub .bool_set:c = { l__spintent_sp #1 _read_sub_bool },
6636     read-sub .initial:n = false,
6637     read-sub .value_required:n = true,
6638     read-arg .code:n =
6639         { \__spintent_sanitizet_affix:cn { l__spintent_sp #1 _read_arg_tl } {##1} },
6640     read-arg .initial:n = ,
6641     read-arg .value_required:n = true,
6642     silent .bool_set:c = { l__spintent_sp #1 _silent_bool },
6643     silent .initial:n = false,
6644     silent .value_required:n = true,
6645     unknown .code:n =
6646         {
6647             \str_set:Nn \l__spintent_command_name_str { sp #1 }
6648             \__spintent_unknown_keys_cmd:n {##1}
6649         },
6650     }
6651 }

```

(End of definition for read-cmd and others.)

```

\l__spintent_spcircfig_short_bool
\l__spintent_spcircfig_op_tl
\l__spintent_spcircfig_intent_tl
\l__spintent_spcircfig_intent_arg_tl

```

The variable `\l__spintent_spcircfig_short_bool` records whether the argument resolved as the short number/letter form (via `\__spintent_luafun_geo_afig_parse_and_set:n/\__spintent_luafun_geo_pfig_p` the same entry point `\spAfig/\spPfig` use) rather than a real circle. `\l__spintent_spcircfig_op_tl` holds *área* or *perímetro*. The variable `\l__spintent_spcircfig_intent_tl` holds the full spoken phrase: `\l__spintent_spcircfig_del_word_tl` (via `\__spintent_spcircfig_del_word:n`: the word, contracted with `del-/de-la-,` or `read-arg`), an optional `sub-index` suffix, and, when a center is given, `-con-centro-en` — the center itself is never concatenated into it: it is read separately, by reference, through `\__spintent_mathmlarg_point:nnn`, so a subscripted or primed center (e.g. `O_{1}`), as in the Euler circle) is never an issue.

```

6652 \bool_new:N \l__spintent_spcircfig_short_bool
6653 \tl_new:N \l__spintent_spcircfig_op_tl
6654 \tl_new:N \l__spintent_spcircfig_del_word_tl
6655 \tl_new:N \l__spintent_spcircfig_intent_tl
6656 \tl_new:N \l__spintent_spcircfig_intent_arg_tl

```

(End of definition for `\__spintent_spcircfig_short_bool` and others.)

`\__spintent_spcircfig_sym:n` Typesets `\bigodot` or the plain letter C, according to `print-sym`. Receives the short command name in ‘`#1`’.

```

6657 \cs_new_protected:Nn \__spintent_spcircfig_sym:n
6658 {
6659   \str_if_eq:vnTF { \__spintent_sp #1 _print_sym_str } { odot }
6660   { \bigodot }
6661   { C }
6662 }
```

(End of definition for `\__spintent_spcircfig_sym:n`.)

`\__spintent_spcircfig_word_and_article:n` Sets `\__spintent_spcirc_word_tl` to `read-arg` or `circunferencia/círculo` from `read-sty`, and `\__spintent_spcirc_articulo_str` to the matching article, which always follows `read-sty` even when `read-arg` overrides the word. Receives the short command name in ‘`#1`’.

```

6663 \cs_new_protected:Nn \__spintent_spcircfig_word_and_article:n
6664 {
6665   \str_if_eq:vnTF { \__spintent_sp #1 _read_sty_str } { circun }
6666   { \str_set:Nn \__spintent_spcirc_articulo_str { la } }
6667   { \str_set:Nn \__spintent_spcirc_articulo_str { el } }
6668   \tl_if_empty:cTF { \__spintent_sp #1 _read_arg_tl }
6669   {
6670     \str_if_eq:vnTF { \__spintent_sp #1 _read_sty_str } { circun }
6671     { \tl_set:Nn \__spintent_spcirc_word_tl { circunferencia } }
6672     { \tl_set:Nn \__spintent_spcirc_word_tl { círculo } }
6673   }
6674   {
6675     \tl_set:Ne \__spintent_spcirc_word_tl
6676     {
6677       \tl_use:c { \__spintent_sp #1 _read_arg_tl }
6678     }
6679   }
6680 }
```

(End of definition for `\__spintent_spcircfig_word_and_article:n`.)

`\__spintent_spcircfig_del_word:n` Sets `\__spintent_spcircfig_del_word_tl` to `\__spintent_spcircfig_word_and_article:n`’s word, prefixed with `del-` or `de-la-` — contracted, not `de-el-`, since a screen reader does not contract that on its own. Receives the short command name in ‘`#1`’.

```

6681 \cs_new_protected:Nn \__spintent_spcircfig_del_word:n
6682 {
6683   \__spintent_spcircfig_word_and_article:n {#1}
6684   \str_if_eq:VnTF \__spintent_spcirc_articulo_str { el }
6685   {
6686     \tl_set:Ne \__spintent_spcircfig_del_word_tl { del- \__spintent_spcirc_word_tl }
6687   }
6688   {
6689     \tl_set:Ne \__spintent_spcircfig_del_word_tl { de-la- \__spintent_spcirc_word_tl }
6690   }
6691 }
```

(End of definition for `\__spintent_spcircfig_del_word:n`.)

`\__spintent_spcircfig_build_short:n` Handles the short number/letter case: builds `\__spintent_spcircfig_intent_arg_tl` as `\__spintent_spcircfig_intent_arg_tl` (e.g. `ber word`), with `prefix` placed after the leading `_` when given, with no `prefix` (no reference — there is nothing to point to), or `_-` when `silent` is `true` or `concept` is `false`. Receives the short command name in ‘`#1`’.

```

6692 \cs_new_protected:Nn \__spintent_spcircfig_build_short:n
6693 {
6694   \bool_if:cTF { \__spintent_sp #1 _silent_bool }
6695   {
6696     \tl_set:Nn \__spintent_spcircfig_intent_arg_tl { _- }
6697   }
6698   {
6699     \bool_if:cTF { \__spintent_sp #1 _concept_bool }
6700     {
6701       \__spintent_spcircfig_del_word:n {#1}
6702       \tl_set:Ne \__spintent_spcircfig_intent_arg_tl
6703       {
6704         -
6705         \tl_if_empty:cF { \__spintent_sp #1 _prefix_tl }

```

```

6706         { \tl_use:c { l__spintent_sp #1 _prefix_tl } - }
6707         \l__spintent_spcircfig_op_tl - \l__spintent_spcircfig_del_word_tl -
6708         \bool_if:cT { l__spintent_sp #1 _read_sub_bool } { sub- }
6709         \l__spintent_geo_luaset_sub_spoken_str
6710     }
6711 }
6712 {
6713     \tl_set:Nn \l__spintent_spcircfig_intent_arg_tl { _- }
6714 }
6715 }
6716 }

```

(End of definition for `\l__spintent_spcircfig_build_short:n`.)

`\l__spintent_spcircfig_finish_intent:nn`

Appends an optional `sub-index` suffix and, when `#2` is non-empty, `-con-centro-en` to `\l__spintent_spcircfig_intent_arg_tl` — shared by both branches of `\l__spintent_spcircfig_build_concept:nn`, after either one has set the word itself. Receives the short command name in ‘`#1`’ and the argument in ‘`#2`’.

```

6717 \cs_new_protected:Nn \l__spintent_spcircfig_finish_intent:nn
6718 {
6719     \str_if_empty:cF { l__spintent_sp #1 _sub_index_str }
6720     {
6721         \tl_put_right:Ne \l__spintent_spcircfig_intent_tl
6722         {
6723             -
6724             \bool_if:cT { l__spintent_sp #1 _read_sub_bool } { sub- }
6725             \str_use:c { l__spintent_sp #1 _sub_index_str }
6726         }
6727     }
6728     \tl_if_empty:nF {#2}
6729     {
6730         \tl_put_right:Nn \l__spintent_spcircfig_intent_tl { -con-centro-en }
6731     }
6732 }

```

(End of definition for `\l__spintent_spcircfig_finish_intent:nn`.)

`\l__spintent_spcircfig_build_concept:nn`

Builds `\l__spintent_spcircfig_intent_arg_tl`. `_-` when `silent` is `true`, whatever `#2` is, or when `false` and `concept` is `false` with `read-arg` empty (`_( $\$$ pt)`) instead when `#2` is non-empty — the center is still read). Otherwise `\l__spintent_spcircfig_intent_tl` — with `:prefix( $\$$ pt)` appended only when `#2` is non-empty — with `prefix` placed after the leading `_` when given, where `\l__spintent_spcircfig_intent_tl` is: with `concept true`, `\l__spintent_spcircfig_op_tl` - `\l__spintent_spcircfig_del_word_tl` (via `\l__spintent_spcircfig_del_word:n`: the word, contracted with `del-/de-la-`, or `read-arg` the same way); with `false` and `read-arg` given, `read-arg` verbatim instead, with `\l__spintent_spcircfig_op_tl` dropped entirely — `área/ perímetro` is itself the concept `concept` toggles, not the `-con-centro-en` that follows, which always stays (it identifies which figure, not the concept). No automatic article either — the word (and any article it needs) is the user’s to write, same convention as `\l__spintent_spcirc_process:n`. Either way, an optional `sub-index` suffix and, when `#2` is non-empty, `-con-centro-en` follow. Receives the short command name in ‘`#1`’ and the argument in ‘`#2`’.

```

6733 \cs_new_protected:Nn \l__spintent_spcircfig_build_concept:nn
6734 {
6735     \bool_if:cTF { l__spintent_sp #1 _silent_bool }
6736     {
6737         \tl_set:Nn \l__spintent_spcircfig_intent_arg_tl { _- }
6738     }
6739     {
6740         \bool_if:cTF { l__spintent_sp #1 _concept_bool }
6741         {
6742             \l__spintent_spcircfig_del_word:n {#1}
6743             \tl_set:Ne \l__spintent_spcircfig_intent_tl
6744             {
6745                 \l__spintent_spcircfig_op_tl - \l__spintent_spcircfig_del_word_tl
6746             }
6747             \l__spintent_spcircfig_finish_intent:nn {#1} {#2}
6748             \tl_set:Ne \l__spintent_spcircfig_intent_arg_tl
6749             {
6750                 -
6751                 \tl_if_empty:cF { l__spintent_sp #1 _prefix_tl }
6752                 {
6753                     \tl_use:c { l__spintent_sp #1 _prefix_tl } -
6754                 }
6755                 \l__spintent_spcircfig_intent_tl

```

```

6756         \tl_if_empty:nF {#2} { :prefix ( $ pt ) }
6757     }
6758 }
6759 {
6760     \tl_if_empty:cTF { \__spintent_sp #1 _read_arg_tl }
6761     {
6762         \tl_if_empty:nTF {#2}
6763         { \tl_set:Nn \__spintent_spcircfig_intent_arg_tl { _- } }
6764         { \tl_set:Ne \__spintent_spcircfig_intent_arg_tl { _ ( $ pt ) } }
6765     }
6766     {
6767         \tl_set:Ne \__spintent_spcircfig_intent_tl
6768         { \tl_use:c { \__spintent_sp #1 _read_arg_tl } }
6769         \__spintent_spcircfig_finish_intent:nn {#1} {#2}
6770         \tl_set:Ne \__spintent_spcircfig_intent_arg_tl
6771         {
6772             -
6773             \__spintent_spcircfig_intent_tl
6774             \tl_if_empty:nF {#2} { :prefix ( $ pt ) }
6775         }
6776     }
6777 }
6778 }
6779 }

```

(End of definition for `\__spintent_spcircfig_build_concept:nn`.)

`\__spintent_spcircfig_process:nn` Receives the short command name in ‘#1’ and the argument in ‘#2’. Sets `\__spintent_spcircfig_op_tl` to `área/perímetro`. With #2 empty, `\__spintent_spcircfig_short_bool` is set `false` and nothing else is parsed. With a comma in #2, reports `sp#1-invalid-arg` immediately — `\@@_luafun_geo_circ_parse_and_set:n` (shared with `\spcirc`) still accepts a second, comma-separated part as a generic `label`-type radius, matching almost any text, so it would never itself report an error for a comma-containing argument here, where a comma is never meaningful at all. Without a comma, calls `\__spintent_luafun_geo_circ_parse_and_set:n`; on error, retries through `\__spintent_luafun_geo_afig_parse_and_set:n/\__spintent_luafun_geo_pfig_parse_and_set:n` (the same entry point `\spAfig/\spPfig` use) for the short number/letter form, rejecting a table-letter match (not meaningful for a circle) back to an error. Reports `sp#1-invalid-arg` on failure either way, then delegates to `\__spintent_spcircfig_build_short:n/\__spintent_spcircfig_build_concept:nn` and `\__spintent_spcircfig_mathml:nn`.

- Unlike `\spcirc`, there is no radius/diameter here at all: a formula’s left-hand side (`A_0`) names the figure, it does not carry the values needed to compute it — those belong on the right-hand side, elsewhere in the document.

```

6780 \cs_new_protected:Nn \__spintent_spcircfig_process:nn
6781 {
6782     \str_if_eq:eeTF {#1} { Acirc }
6783     { \tl_set:Nn \__spintent_spcircfig_op_tl { área } }
6784     { \tl_set:Nn \__spintent_spcircfig_op_tl { perímetro } }
6785     \tl_if_empty:nTF {#2}
6786     {
6787         \bool_set_false:N \__spintent_spcircfig_short_bool
6788     }
6789     {
6790         \bool_set_false:N \__spintent_spcircfig_short_bool
6791         \tl_if_in:nnTF {#2} { , }
6792         {
6793             \msg_error:nnee { spintent } { sp #1 -invalid-arg }
6794             { \c_backslash_str sp #1 } { \exp_not:n {#2} }
6795         }
6796         {
6797             \__spintent_luafun_geo_circ_parse_and_set:n { \exp_not:n {#2} }
6798             \str_if_eq:VnT \__spintent_geo_luaset_error_str { true }
6799             {
6800                 \str_if_eq:eeTF {#1} { Acirc }
6801                 { \__spintent_luafun_geo_afig_parse_and_set:n { \exp_not:n {#2} } }
6802                 { \__spintent_luafun_geo_pfig_parse_and_set:n { \exp_not:n {#2} } }
6803                 \str_if_eq:VnF \__spintent_geo_luaset_error_str { true }
6804                 {
6805                     \str_if_empty:NTF \__spintent_geo_luaset_letra_word_str
6806                     { \bool_set_true:N \__spintent_spcircfig_short_bool }
6807                     { \str_set:Nn \__spintent_geo_luaset_error_str { true } }
6808                 }
6809             }

```

```

6810         \bool_if:NF \__spintent_spcircfig_short_bool
6811         {
6812             \str_if_eq:VnT \__spintent_geo_luaset_error_str { true }
6813             {
6814                 \msg_error:nnee { spintent } { sp #1 -invalid-arg }
6815                 { \c_backslash_str sp #1 } { \exp_not:n {#2} }
6816             }
6817         }
6818     }
6819 }
6820 \bool_if:NTF \__spintent_spcircfig_short_bool
6821 { \__spintent_spcircfig_build_short:n {#1} }
6822 { \__spintent_spcircfig_build_concept:nn {#1} {#2} }
6823 \__spintent_spcircfig_mathml:nn { #1 } { #2 }
6824 }

```

(End of definition for `\__spintent_spcircfig_process:nn`.)

`\__spintent_spcircfig_mathml:nn`

Renders `\__spintent_spcircfig_intent_arg_tl`, already built by `\__spintent_spcircfig_process:nn`. Three cases: the short case prints the symbol with the short value as its own subscript; an empty `#2` prints the symbol alone; a real center prints the symbol (plus optional `sub-index` subscript) and the center between parentheses. In every case a single native `\MathMLintent` wraps `\__spintent_mathcal:n{⟨A|P⟩}` and the whole subscript together — splitting the symbol and the center into separate `\MathMLintent` siblings inside the same subscript was tried and reads as `<msub>`'s own native structural narration instead. The center is read through `\__spintent_mathmlarg_point:nnn`, referenced as `$pt` (same convention as `\spcirc`). Receives the short command name in `'#1'` and the argument in `'#2'`.

```

6825 \cs_new_protected:Nn \__spintent_spcircfig_mathml:nn
6826 {
6827     \__spintent_invisible_sep: % (luamml issues #26)
6828     \MathMLintent { \__spintent_spcircfig_intent_arg_tl }
6829     {
6830         \__spintent_inner_sep:
6831         \__spintent_luafun_mrow_intent_arg:nn
6832         { } { intent = ":silent" }
6833         {
6834             \str_if_eq:eeTF {#1} { Acirc } { \__spintent_mathcal:n { A } } { \__spintent_mathcal:n
6835         }
6836         \bool_if:NTF \__spintent_spcircfig_short_bool
6837         {
6838             \c_math_subscript_token
6839             {
6840                 \__spintent_spcircfig_sym:n {#1}
6841                 \c_math_subscript_token { \__spintent_geo_luaset_print_tl }
6842             }
6843         }
6844         {
6845             \c_math_subscript_token
6846             {
6847                 \MathMLintent { :silent } { \__spintent_spcircfig_sym:n {#1} }
6848                 \str_if_empty:cF { \__spintent_sp #1 _sub_index_str }
6849                 {
6850                     \c_math_subscript_token
6851                     {
6852                         \MathMLintent { :silent }
6853                         { \str_use:c { \__spintent_sp #1 _sub_index_str } }
6854                     }
6855                 }
6856                 \tl_if_empty:nF {#2}
6857                 {
6858                     \MathMLintent { :silent } { \lparen }
6859                     \__spintent_mathmlarg_point:nnV { pt } { sp #1 } \__spintent_geo_luaset_poin
6860                     \MathMLintent { :silent } { \rparen }
6861                 }
6862             }
6863         }
6864     }
6865 }

```

(End of definition for `\__spintent_spcircfig_mathml:nn`.)

```

\__spintent_spcircfig_exec:nnn
6866 \cs_new_protected:Nn \__spintent_spcircfig_exec:nnn
6867 {
6868   \keys_set:nn { spintent / sp #1 } { #2 }
6869   \__spintent_spcircfig_process:nn { #1 } { #3 }
6870 }

```

(End of definition for `\__spintent_spcircfig_exec:nnn`.)

`\spAcirc` Both public commands `\spAcirc` and `\spPcirc` enforce math mode, open a local group, and delegate execution to `\__spintent_spcircfig_exec:nnn`.

`\spPcirc`

```

6871 \NewDocumentCommand \spAcirc { 0{} d() }
6872 {
6873   \mode_if_math:TF
6874   {
6875     \group_begin:
6876     \attribute \l__spintent_scope_attr_int = 1
6877     \tl_if_novalue:nTF {#2}
6878     { \__spintent_spcircfig_exec:nnn { Acirc } { #1 } { } }
6879     { \__spintent_spcircfig_exec:nnn { Acirc } { #1 } { #2 } }
6880     \group_end:
6881   }
6882   { \msg_error:nnn { spintent } { need-math-mode } { \spAcirc } }
6883 }
6884 \NewDocumentCommand \spPcirc { 0{} d() }
6885 {
6886   \mode_if_math:TF
6887   {
6888     \group_begin:
6889     \attribute \l__spintent_scope_attr_int = 1
6890     \tl_if_novalue:nTF {#2}
6891     { \__spintent_spcircfig_exec:nnn { Pcirc } { #1 } { } }
6892     { \__spintent_spcircfig_exec:nnn { Pcirc } { #1 } { #2 } }
6893     \group_end:
6894   }
6895   { \msg_error:nnn { spintent } { need-math-mode } { \spPcirc } }
6896 }

```

(End of definition for `\spAcirc` and `\spPcirc`. These functions are documented on page 31.)

12.52 The command `\spVol`

The user-level command `\spVol` provides a semantic interface for the volume of a solid, always typesetting `\mathcal{V}`.

12.52.1 Definition of variables for `\spVol`

The variable `\l__spintent_spVol_mode_str` holds the `read-cmd` choice (`school` or `mathcat`); `\l__spintent_spVol_read_arg_tl` the raw `sub-index` text; `\l__spintent_spVol_read_arg_tl` the sanitized `read-arg` text. `\l__spintent_spVol_read_arg_tl` and `\l__spintent_spVol_suffix_tl` are scratch variables holding the word to voice (volumen) and the text to attach after it; `\l__spintent_spVol_native_bool` records whether that text should be combined natively with the subscript (silenced) or left for MathCAT to read on its own. The variable `\l__spintent_spVol_read_arg_full_tl` holds the text to voice when `read-arg` is given.

```

6897 \str_new:N \l__spintent_spVol_mode_str
6898 \str_new:N \l__spintent_spVol_sub_index_str
6899 \tl_new:N \l__spintent_spVol_read_arg_tl
6900 \tl_new:N \l__spintent_spVol_word_tl
6901 \tl_new:N \l__spintent_spVol_suffix_tl
6902 \bool_new:N \l__spintent_spVol_native_bool
6903 \tl_new:N \l__spintent_spVol_read_arg_full_tl

```

(End of definition for `\l__spintent_spVol_mode_str` and others.)

12.52.2 Definition of keys for `\spVol`

We define the `(keys)` `read-cmd`, `sub-index`, `read-sub`, `read-arg`, `concept` and `silent`.

```

6904 \keys_define:nn { spintent / spVol }
6905 {
6906   read-cmd .choices:nn = { school , mathcat }
6907   {
6908     \int_case:nn { \l_keys_choice_int }
6909     {
6910       {1} { \str_set:Nn \l__spintent_spVol_mode_str { school } }

```

```

6911         {2} { \str_set:Nn \l__spintent_spVol_mode_str { mathcat } }
6912     }
6913 },
6914 read-cmd .initial:n = school,
6915 read-cmd .value_required:n = true,
6916 read-cmd / unknown .code:n =
6917     \msg_error:nneee { spintent } { unknown-choice }
6918     { read-cmd } { school,~mathcat } { \exp_not:n {#1} },
6919 sub-index .code:n =
6920     { \str_set:Nn \l__spintent_spVol_sub_index_str {#1} },
6921 sub-index .initial:n = ,
6922 sub-index .value_required:n = true,
6923 read-sub .bool_set:N = \l__spintent_spVol_read_sub_bool,
6924 read-sub .initial:n = false,
6925 read-sub .value_required:n = true,
6926 read-arg .code:n =
6927     { \__spintent_sanitizize_affix:Nn \l__spintent_spVol_read_arg_tl {#1} },
6928 read-arg .initial:n = ,
6929 read-arg .value_required:n = true,
6930 concept .bool_set:N = \l__spintent_spVol_concept_bool,
6931 concept .initial:n = true,
6932 concept .value_required:n = true,
6933 silent .bool_set:N = \l__spintent_spVol_silent_bool,
6934 silent .initial:n = false,
6935 silent .value_required:n = true,
6936 unknown .code:n =
6937     {
6938         \str_set:Nn \l__spintent_command_name_str { spVol }
6939         \__spintent_unknown_keys_cmd:n {#1}
6940     },
6941 }

```

(End of definition for read-cmd and others.)

12.52.3 Internal functions for \spVol

`\__spintent_spVol_print:` The function `\__spintent_spVol_print:` dispatches to `\__spintent_spVol_silent:` when `silent` is `true`, or to `\__spintent_spVol_verbal:` otherwise.

```

6942 \cs_new_protected:Nn \__spintent_spVol_print:
6943 {
6944     \__spintent_invisible_sep:
6945     \bool_if:NTF \l__spintent_spVol_silent_bool
6946     { \__spintent_spVol_silent: }
6947     { \__spintent_spVol_verbal: }
6948 }

```

(End of definition for `\__spintent_spVol_print:`.)

`\__spintent_spVol_silent:` The function `\__spintent_spVol_silent:` typesets `\mathcal{V}` with the intent `_` — the symbol alone through the shared queue when there is no `sub-index`, or symbol and subscript together in one native `\MathMLintent` (subscript silenced) when there is.

```

6949 \cs_new_protected:Nn \__spintent_spVol_silent:
6950 {
6951     \str_if_empty:NTF \l__spintent_spVol_sub_index_str
6952     {
6953         \__spintent_luafun_mrow_intent_arg:nn
6954         { } { intent = "_" }
6955         { \mathcal{V} }
6956     }
6957     {
6958         \MathMLintent { _- }
6959         {
6960             \__spintent_inner_sep:
6961             \mathcal{V}
6962             \c_math_subscript_token
6963             {
6964                 \MathMLintent { :silent }
6965                 { \l__spintent_spVol_sub_index_str }
6966             }
6967         }
6968     }
6969 }

```

(End of definition for `\__spintent_spVol_silent:`.)

`\__spintent_spVol_verbal:` The function `\__spintent_spVol_verbal:` delegates to `\__spintent_spVol_read_arg_print:` when `read-arg` is given, and otherwise, with `concept false`, to `\__spintent_spVol_symbol_print:`. With `concept true` it sets `\l__spintent_spVol_word_tl` to `volumen` and `\l__spintent_spVol_suffix_tl` to `sub-(value)/(value)` (per `read-sub`), combined natively only under `read-cmd=school` — under `read-cmd=mathcat` the suffix is left for MathCAT to read on its own. Delegates the actual typesetting to `\__spintent_spVol_attach_subscript:`

```

6970 \cs_new_protected:Nn \__spintent_spVol_verbal:
6971 {
6972   \tl_if_empty:NTF \l__spintent_spVol_read_arg_tl
6973   {
6974     \bool_if:NTF \l__spintent_spVol_concept_bool
6975     {
6976       \tl_set:Nn \l__spintent_spVol_word_tl { volumen }
6977       \str_if_eq:VnTF \l__spintent_spVol_mode_str { school }
6978       { \bool_set_true:N \l__spintent_spVol_native_bool }
6979       { \bool_set_false:N \l__spintent_spVol_native_bool }
6980       \tl_set:Ne \l__spintent_spVol_suffix_tl
6981       {
6982         \bool_if:NT \l__spintent_spVol_read_sub_bool { sub- }
6983         \l__spintent_spVol_sub_index_str
6984       }
6985       \__spintent_spVol_attach_subscript:
6986     }
6987     { \__spintent_spVol_symbol_print: }
6988   }
6989   { \__spintent_spVol_read_arg_print: }
6990 }

```

(End of definition for `\__spintent_spVol_verbal:`.)

`\__spintent_spVol_symbol_print:` With `concept false`, the symbol itself is read instead — but only when `read-cmd` is `school`: with no `sub-index`, the shared queue tags the symbol with the intent `_V-`; with one, symbol and subscript are wrapped together in one native `\MathMLintent` (subscript silenced). With `read-cmd mathcat`, the intent is left empty: MathCAT applies its own rules to the symbol and its subscript, unmarked by us.

```

6991 \cs_new_protected:Nn \__spintent_spVol_symbol_print:
6992 {
6993   \str_if_empty:NTF \l__spintent_spVol_sub_index_str
6994   {
6995     \str_if_eq:VnTF \l__spintent_spVol_mode_str { school }
6996     {
6997       \__spintent_luafun_mrow_intent_arg:nn
6998       { } { intent = "_V-" }
6999       { \mathcal{V} } }
7000     }
7001     { \mathcal{V} } }
7002   }
7003   {
7004     \str_if_eq:VnTF \l__spintent_spVol_mode_str { school }
7005     {
7006       \MathMLintent
7007       {
7008         _V-
7009         \bool_if:NT \l__spintent_spVol_read_sub_bool { sub- }
7010         \l__spintent_spVol_sub_index_str
7011       }
7012       {
7013         \__spintent_inner_sep:
7014         \mathcal{V}
7015         \c_math_subscript_token
7016         {
7017           \MathMLintent { :silent }
7018           { \l__spintent_spVol_sub_index_str }
7019         }
7020       }
7021     }
7022     {
7023       \mathcal{V}
7024       \c_math_subscript_token
7025       { \l__spintent_spVol_sub_index_str }
7026     }

```

```

7027     }
7028 }

```

(End of definition for `\__spintent_spVol_symbol_print:`.)

`\__spintent_spVol_attach_subscript:`

The function `\__spintent_spVol_attach_subscript:` typesets `\mathcal{V}`. An empty `sub-index` tags the symbol alone through the shared queue (`\l__spintent_spVol_suffix_tl` is discarded, since there's no subscript to attach it to). A given one combines symbol, subscript, and `\l__spintent_spVol_suffix_tl` in one native `\MathMLintent` (subscript silenced) when `\l__spintent_spVol_native_bool` is `true`, or tags the symbol alone through the shared queue with the subscript left bare when `false`.

```

7029 \cs_new_protected:Nn \__spintent_spVol_attach_subscript:
7030 {
7031   \str_if_empty:NTF \l__spintent_spVol_sub_index_str
7032   {
7033     \__spintent_luafun_mrow_intent_arg:nn
7034     { } { intent = "_ \l__spintent_spVol_word_tl :nofix" }
7035     { \mathcal{V} }
7036   }
7037   {
7038     \bool_if:NTF \l__spintent_spVol_native_bool
7039     {
7040       \MathMLintent
7041       { _ \l__spintent_spVol_word_tl - \l__spintent_spVol_suffix_tl :nofix }
7042       {
7043         \__spintent_inner_sep:
7044         \mathcal{V}
7045         \c_math_subscript_token
7046         {
7047           \MathMLintent { :silent }
7048           { \l__spintent_spVol_sub_index_str }
7049         }
7050       }
7051     }
7052     {
7053       \__spintent_luafun_mrow_intent_arg:nn
7054       { } { intent = "_ \l__spintent_spVol_word_tl :nofix" }
7055       { \mathcal{V} }
7056       \c_math_subscript_token
7057       { \l__spintent_spVol_sub_index_str }
7058     }
7059   }
7060 }

```

(End of definition for `\__spintent_spVol_attach_subscript:`.)

`\__spintent_spVol_read_arg_print:`

With `read-arg` given, `sub-index/read-sub` are ignored for the *spoken* content (the subscript, if any, still displays, silenced). With `concept true`, `\l__spintent_spVol_read_arg_full_tl` becomes `volumen` followed by `read-arg`'s text; with `false`, `read-arg`'s text alone. Combined natively regardless of `read-cmd:` with no `sub-index`, the shared queue tags the symbol alone; with one, symbol and subscript are wrapped together in one native `\MathMLintent`.

```

7061 \cs_new_protected:Nn \__spintent_spVol_read_arg_print:
7062 {
7063   \bool_if:NTF \l__spintent_spVol_concept_bool
7064   {
7065     \tl_set:Ne \l__spintent_spVol_read_arg_full_tl
7066     { volumen- \tl_use:N \l__spintent_spVol_read_arg_tl }
7067   }
7068   { \tl_set_eq:NN \l__spintent_spVol_read_arg_full_tl \l__spintent_spVol_read_arg_tl }
7069   \str_if_empty:NTF \l__spintent_spVol_sub_index_str
7070   {
7071     \__spintent_luafun_mrow_intent_arg:nn
7072     { } { intent = "_ \l__spintent_spVol_read_arg_full_tl :nofix" }
7073     { \mathcal{V} }
7074   }
7075   {
7076     \MathMLintent
7077     { _ \l__spintent_spVol_read_arg_full_tl :nofix }
7078     {
7079       \__spintent_inner_sep:
7080       \mathcal{V}
7081       \c_math_subscript_token

```

```

7082         {
7083             \MathMLintent { :silent }
7084             { \l__spinttent_spVol_sub_index_str }
7085         }
7086     }
7087 }
7088 }

```

(End of definition for `\__spinttent_spVol_read_arg_print:`)

`\spVol` The public command `\spVol` enforces *math mode*, opens a local group, and delegates execution to `\__spinttent_spVol_print:`.

```

7089 \NewDocumentCommand \spVol { 0{} }
7090 {
7091     \mode_if_math:TF
7092     {
7093         \group_begin:
7094         \attribute \l__spinttent_scope_attr_int = 1
7095         \keys_set:nn { spinttent / spVol } {#1}
7096         \__spinttent_spVol_print:
7097         \group_end:
7098     }
7099     { \msg_error:nnn { spinttent } { need-math-mode } { \spVol } }
7100 }

```

(End of definition for `\spVol`. This function is documented on page 34.)

12.53 Finish package

Finish package implementation.

```

7101 \file_input_stop:
7102 \endpackage

```

13 Índice de la Implementación

The italic numbers denote the pages where the corresponding entry is described, the numbers underlined and all others indicate the line on which they are implemented in the package code.

A			
\angle	146	silent	146
autofit	4174	Keys for \sparc provide by spintent :	
		concept	151
		prefix	151
		read-cmd	151
		silent	151
C		Keys for \spcirc provide by spintent :	
cifras-sep	289, 574, 861, 1900, 1971, 2594	prefix	170
Document class:		print-sym	170, 172
l3doc	43	radio-sty	170
cmd-key-unknown	208	read-arg	170
cmd-key-value-unknown	208	read-cmd	170
Commands provide by spintent :		read-sty	170
\spmoney	61	read-sub	170
\spnum	51	sub-index	170
\spsiglo	71, 73	Keys for \spcoord provide by spintent :	
\spunit	56	silent	105
concept	2062, 2274, 3082, 3438, 4174, 4794, 5003, 5290, 5444, 5629, 6031, 6287, 6586, 6904	Keys for \spdiam provide by spintent :	
concept*	5629	concept	165
currency	861	read-arg	165
currency-no-subunits	830	read-cmd	165
		read-sub	165
D		silent	165
diameter	6287	sub-index	165
		upper-case	165
I		Keys for \spdiv provide by spintent :	
inline	2456	cifras-sep	84
inline-numeric-denominator	552	notation-sym	84
invalid-angle-format	2544	read-parens	84
invalid-century-format	1256	read-period-sep	84
invalid-currency	830	read-sign	84
invalid-date-format	1486	read-sym	84
invalid-time-format	1407	set-sym	84
		wrap-parens	84
J		Keys for \spdom provide by spintent :	
join-word	2672	prefix	123, 124
		silent	124
K		Keys for \spdsym provide by spintent :	
Keys for \spAcirc provide by spintent :		prefix	81
concept	176	read-sym	81
prefix	176	set-sym	81
print-sym	176, 178	suffix	81
radio-sty	176	Keys for \spfuncomp provide by spintent :	
read-arg	176	silent	120
read-cmd	176	Keys for \spfun provide by spintent :	
read-sty	176	concept	115
read-sub	176	prefix	115
silent	176	read-cmd	115
sub-index	176	silent	115
Keys for \spAfig provide by spintent :		Keys for \spinterval provide by spintent :	
prefix	157	concept	109, 112
print-sym	157, 158	prefix	108, 109, 112
read-arg	157–159	read-arg	108, 109, 112
read-cmd	157	read-cmd	108, 109, 113
read-sub	158	silent	109, 112, 114
Keys for \spangle provide by spintent :		size	108–110
concept	146	Keys for \spmangle provide by spintent :	
prefix	145, 146	concept	146
print-m	146	prefix	145, 146
read-cmd	145, 146	print-m	145, 146
recto	145, 146		

read-cmd	145, 146	prefix	91
recto	145, 146	result-num	91
silent	146	result	91
Keys for \spmarc provide by spintent :		silent	91
concept	151	Keys for \spnum provide by spintent :	
prefix	151	cifras-sep	50–52
read-cmd	151	notation-sym	51
silent	151	read-period-sep	51, 53
Keys for \spmcld provide by spintent :		read-sign	51
concept	87	Keys for \spnZ provide by spintent :	
prefix	87	cifras-sep	97
read-cmd	87	read-parens	97, 98
result	87	read-sign	97
silent	87	wrap-parens	97, 98
upper-case	87	Keys for \spPcirc provide by spintent :	
Keys for \spmcm provide by spintent :		concept	176
concept	87	prefix	176
prefix	87	print-sym	176, 178
read-cmd	87	radio-sty	176
result	87	read-arg	176
silent	87	read-cmd	176
upper-case	87	read-sty	176
Keys for \spmoney provide by spintent :		read-sub	176
cifras-sep	63	silent	176
currency	63	sub-index	176
money-math	63	Keys for \spPfig provide by spintent :	
sub-units	63	prefix	157
Keys for \spmQ provide by spintent :		print-sym	157, 158
join-word	99, 101	read-arg	157–159
read-parens	99, 101	read-cmd	157
use-dfrac	99–103	read-sub	158
use-spnZ	99, 102, 103	Keys for \spPoint provide by spintent :	
wrap-parens	99, 101	concept	128
Keys for \spmside provide by spintent :		prefix	127, 128
concept	142	read-cmd	127, 128
prefix	141, 142	silent	128
print-m	141, 142	Keys for \spPoly provide by spintent :	
print-sym	165	concept	154
read-arg	141, 142	prefix	154
read-cmd	141, 142, 144	print-sym	154
read-sty	141, 142	read-cmd	154
silent	142, 144	silent	154
Keys for \spmsym provide by spintent :		Keys for \sprad provide by spintent :	
not-print	82	concept	165
prefix	82	read-arg	165
read-sym	82	read-cmd	165
set-sym	82	read-sub	165
suffix	82	silent	165
Keys for \spmul provide by spintent :		sub-index	165
cifras-sep	85	upper-case	165
notation-sym	85	Keys for \sprayo provide by spintent :	
read-parens	85	concept	139
read-period-sep	85	prefix	138, 139
read-sign	85	read-cmd	138–140
read-sym	85	read-sty	138, 139
set-sym	85	silent	139, 140
wrap-parens	85	Keys for \sprazon provide by spintent :	
Keys for \spnD provide by spintent :		inline	94
concept	91	silent	94
prefix	91	use-dfrac	94
result-num	91	Keys for \spread provide by spintent :	
result	91	read-arg	79
silent	91	silent	79
Keys for \spnM provide by spintent :		Keys for \sprecta provide by spintent :	
concept	91	prefix	134

read-cmd	134	R	
silent	134	radio-sty	6287
Keys for \spshort provide by spintent :		read-arg	1134, 1674, 3082, 4794, 5629, 6031, 6287, 6586, 6904
read-arg	68	read-cdot	574
small-caps	68	read-cmd	2062, 3082, 3438, 4062, 4174, 4417, 4654, 4794, 5003, 5290, 5444, 5629, 6031, 6287, 6586, 6904
Keys for \spside provide by spintent :		read-parens	1900, 1971, 2594, 2672
concept	142	read-period-sep	289, 574, 1900, 1971
prefix	141, 142	read-sign	289, 1900, 1971, 2594
read-arg	141, 142	read-slash	574
read-cmd	141, 142, 144	read-sty	4654, 4794, 6287, 6586
read-sty	141, 142	read-sub	5629, 6031, 6287, 6586, 6904
silent	142, 144	read-sym	1754, 1825, 1900, 1971
Keys for \spsiglo provide by spintent :		recto	5003
print-era	71, 73	result	2062, 2274
Keys for \spunit provide by spintent :		result-num	2274
cifras-sep	57	right-angle-sqr-fallback	80
notation-sym	57	\rightangle	146
print-cdot	57		
read-cdot	57	S	
read-period-sep	57	seq commands:	
read-slash	57	\seq_set_from_clist:Nn	62
Keys for \spusep provide by spintent :		set-sym	1754, 1825, 1900, 1971
silent	77	\setspintent	3, 47, 196
size	77	silent	1534, 1674, 2062, 2274, 2456, 2887, 3082, 3438, 3677, 3854, 4062, 4174, 4417, 4654, 4794, 5003, 5290, 5444, 5629, 6031, 6287, 6586, 6904
Keys for \spvec provide by spintent :		size	1534, 3082
autofit	130	small-caps	1134
concept	130	\spAcirc	31, 176, 6871
prefix	130	spAcirc-invalid-arg	6577
read-cmd	130	\spAfig	26, 157, 6002
silent	130	spAfig-invalid-arg	5591
Keys for \spVol provide by spintent :		spAfig-invalid-points	5591
concept	182	spAfig-read-arg-key	5591
read-arg	182	spAfig-sym-cardinality-conflict	5591
read-cmd	182	spAfig-sym-letra-conflict	5591
read-sub	182	\spang	24, 96, 2584
silent	182	\spangle	23, 145, 5253
sub-index	182	spangle-invalid-arg	4993
		\sparc	25, 151, 5413
M		sparc-invalid-points	5280
mathcal-fallback	80	\spcirc	30, 169, 6553
\MathMLintent	149	spcirc-invalid-arg	6280
mc-invalid-arguments	2055	\spcod	36, 123, 3921
mc-too-few-arguments	2300	\spcoord	19, 104, 3029
md-invalid-argument	2300	spcoord-invalid-arg	2883
Lua module provide by spintent :		\spdate	11, 75, 1510
spintent.lua	43, 44, 46, 49-51, 56, 60-63, 68, 69, 71-75, 87, 91, 96, 99, 104, 127	\spdiam	29, 165, 6247
money-math	861	\spdiv	13, 83, 1961
		spdiv-empty-argument	1920
N		\spdom	36, 123, 3921
not-print	1825	spdom-invalid-arg	3850
notation-sym	289, 574, 1900, 1971	\spdsym	12, 81, 1801
		\spfun	35, 115, 3652
P		spfun-invalid-arg	3434
Packages:		\spfuncomp	36, 120, 3832
expl3	49, 51	spfuncomp-invalid-arg	3673
lua-unicode-math	43	spintent internal commands:	
luamml	46, 99, 101	__spintent_collapse_hyphens:N	128
tagpdf	67, 68, 72	\l__spintent_command_name_str	208
unicode-math	43, 78		
prefix	1754, 1825, 2062, 2274, 3082, 3438, 3854, 4062, 4174, 4417, 4654, 4794, 5003, 5290, 5444, 5629, 6287, 6586		
print-era	1264		
print-m	4794, 5003		
print-sym	5444, 5629, 6031, 6287, 6586		

<code>\l__spintent_geo_circfig_mode_str</code>	6576	<code>\l__spintent_luaset_only_part_period_-</code>	
<code>\l__spintent_geo_final_print_tl</code>	5583	<code>str</code>	253
<code>\l__spintent_geo_luaset_angulo_case_-</code>		<code>\l__spintent_luaset_part_dec_tl</code>	246
<code>str</code>	2855	<code>\l__spintent_luaset_part_int_tl</code> ...	101 , 246
<code>\l__spintent_geo_luaset_circ_radio_raw_-</code>		<code>\l__spintent_luaset_part_period_tl</code> ...	246
<code>str</code>	6269	<code>\l__spintent_luaset_scope_attr_str</code>	67
<code>\l__spintent_geo_luaset_circ_radio_type_-</code>		<code>\l__spintent_luaset_sign_tl</code>	102 , 246
<code>str</code>	6269	<code>\l__spintent_luaset_status_str</code>	265
<code>\l__spintent_geo_luaset_concept_str</code> ...	2855	<code>\l__spintent_luaset_sym_fam_ok_str</code>	67
<code>\l__spintent_geo_luaset_error_str</code>	2855	<code>__spintent_mathcal:n</code>	94
<code>\l__spintent_geo_luaset_intent_str</code> ...	2855	<code>__spintent_mathmlarg_point:nnn</code>	3955
<code>\l__spintent_geo_luaset_letra_str</code>	2855	<code>\l__spintent_mathmlarg_point_kv_tl</code> ...	3951
<code>\l__spintent_geo_luaset_letra_word_str</code>	2855	<code>\l__spintent_mathmlarg_point_mode_str</code> .	3951
<code>\l__spintent_geo_luaset_nombre_es_num_-</code>		<code>\l__spintent_mathmlarg_point_ref_tl</code> ...	3951
<code>str</code>	2855	<code>\l__spintent_mathmlarg_point_silent_-</code>	
<code>\l__spintent_geo_luaset_points_-</code>		<code>bool</code>	3951
<code>clist_{uuuu}\l__spintent_geo_luaset_is_-</code>		<code>\l__spintent_mc_args_seq</code>	2042
<code>name_str</code>	2855	<code>\l__spintent_mc_luaset_all_numeric_str</code>	2042
<code>\l__spintent_geo_luaset_poly_sym_str</code> ..	2855	<code>\l__spintent_mc_luaset_types_str</code>	2042
<code>\l__spintent_geo_luaset_print_tl</code>	2855	<code>__spintent_mc_process_args:nn</code>	2103
<code>\l__spintent_geo_luaset_sub_spoken_str</code>	2855	<code>\l__spintent_mc_raw_args_seq</code>	2042
<code>\l__spintent_geo_luaset_sub_str</code>	2855	<code>\l__spintent_mc_types_seq_{uuu}\l__spintent_-</code>	
<code>\l__spintent_geo_luaset_tipo_str</code>	2855	<code>spmcd_print_result_bool</code>	2042
<code>\l__spintent_geo_print_sym_str</code>	5699	<code>\l__spintent_md_out_seq</code>	2265
<code>\l__spintent_geo_temp_tl</code>	5699	<code>__spintent_print_spunit_hierarchy:</code> ...	675
<code>__spintent_inner_sep:</code>	120	<code>__spintent_right_angle_sqr:</code>	107
<code>__spintent_invisible_sep:</code>	116	<code>__spintent_sanitize_affix:Nn</code>	136
<code>\c__spintent_invisible_sep_int</code>	124	<code>\l__spintent_saved_luamml_flag_int</code> 101 , 2653	
<code>\c__spintent_invisible_sep_tl</code>	101	<code>\l__spintent_scope_attr_int</code>	74
<code>__spintent_luafun_clean_split_and_set:n</code>	76 , 246	<code>\l__spintent_setspintent_input_arg_cmd_-</code>	
<code>__spintent_luafun_mrow_intent_arg:nn</code> .	116	<code>tl</code>	159
<code>__spintent_luafun_set_debug_ghost:n</code> ...	71	<code>__spintent_setspintent_parse_args:nn</code> .	179
<code>__spintent_luafun_spang_parse:n</code>	96	<code>__spintent_setspintent_set_keys:nn</code> ...	168
<code>__spintent_luafun_sptime_parse:n</code>	76	<code>\l__spintent_smparc_prefix_tl</code>	5275
<code>__spintent_luafun_spmoney_lookup_data:n</code>	76	<code>__spintent_sp_rad_diam_exec:nn</code>	6242
<code>__spintent_luafun_spmQ_scale_int:nn</code> ...	76	<code>\l__spintent_sp_rad_diam_letra_tl</code>	6028
<code>__spintent_luafun_sptime_parse:n</code>	75	<code>__spintent_sp_rad_diam_print:n</code>	6078
<code>__spintent_luafun_spunit_lookup_alias:n</code>	76	<code>\l__spintent_sp_rad_diam_read_arg_full_-</code>	
<code>\l__spintent_luaset_arg_above_tl</code>	265	<code>tl</code>	6028
<code>\l__spintent_luaset_arg_below_tl</code>	265	<code>__spintent_sp_rad_diam_read_arg_-</code>	
<code>\l__spintent_luaset_cal_fam_ok_str</code>	67	<code>print:n</code>	6204
<code>\l__spintent_luaset_dec_is_all_zeros_-</code>		<code>\l__spintent_sp_rad_diam_word_tl</code>	6028
<code>str</code>	258	<code>\l__spintent_spAcirc_mode_str</code>	6566
<code>\l__spintent_luaset_dec_sep_tl</code>	246	<code>\l__spintent_spAcirc_prefix_tl</code>	6566
<code>\l__spintent_luaset_decimal_period_str</code>	258	<code>\l__spintent_spAcirc_print_sym_str</code> ...	6566
<code>\l__spintent_luaset_denom_has_number_-</code>		<code>\l__spintent_spAcirc_radio_sty_str</code> ...	6566
<code>str</code>	265	<code>\l__spintent_spAcirc_read_arg_tl</code>	6566
<code>\l__spintent_luaset_exponent_tl</code>	258	<code>\l__spintent_spAcirc_read_sty_str</code>	6566
<code>\l__spintent_luaset_format_part_dec_str</code>	253	<code>\l__spintent_spAcirc_sub_index_str</code> ...	6566
<code>\l__spintent_luaset_format_part_int_str</code>	253	<code>\l__spintent_spAfig_mode_str</code>	5584
<code>\l__spintent_luaset_has_exponent_str</code> ..	258	<code>\l__spintent_spAfig_prefix_tl</code>	5584
<code>\l__spintent_luaset_has_integer_str</code> 102 , 103 ,	246	<code>\l__spintent_spAfig_print_sym_str</code>	5584
		<code>\l__spintent_spAfig_read_arg_tl</code>	5584
		<code>\l__spintent_spang_luaset_error_str</code> ...	2540
<code>\l__spintent_luaset_has_natural_str</code> ...	246	<code>\l__spintent_spang_luaset_grado_str</code> ...	2540
<code>\l__spintent_luaset_has_units_str</code>	265	<code>\l__spintent_spang_luaset_minuto_str</code> ..	2540
<code>\l__spintent_luaset_millions_str</code>	258	<code>\l__spintent_spang_luaset_segundo_str</code> .	2540
<code>\l__spintent_luaset_not_decimal_not_-</code>		<code>__spintent_spang_process:n</code>	2549
<code>period_str</code>	258	<code>__spintent_spangle_exec:nnn</code>	5248
<code>\l__spintent_luaset_not_decimal_period_-</code>		<code>\l__spintent_spangle_intent_arg_tl</code> ...	5062
<code>str</code>	258	<code>\l__spintent_spangle_intent_tl</code>	5062
<code>\l__spintent_luaset_only_part_dec_str</code> .	253	<code>__spintent_spangle_mathml:n</code>	5104
<code>\l__spintent_luaset_only_part_int_str</code> .	253		

<code>\l__spintent_spangle_mode_str</code>	4988	<code>\l__spintent_spcoord_luaset_sep_str</code>	2869
<code>\l__spintent_spangle_prefix_tl</code>	4988	<code>\l__spintent_spcoord_luaset_x_has_frac_-</code> <code>str</code>	2869
<code>\l__spintent_spangle_print_m_now_bool</code>	5062	<code>\l__spintent_spcoord_luaset_x_sign_str</code>	2869
<code>__spintent_spangle_process:nn</code>	5069	<code>\l__spintent_spcoord_luaset_x_type_str</code>	2869
<code>\l__spintent_spangle_pt_idx_int</code>	5062	<code>\l__spintent_spcoord_luaset_x_value_-</code> <code>str</code>	2869
<code>\l__spintent_spangle_pt_seq</code>	5062	<code>\l__spintent_spcoord_luaset_y_has_frac_-</code> <code>str</code>	2869
<code>\l__spintent_spangle_refs_tl</code>	5062	<code>\l__spintent_spcoord_luaset_y_sign_str</code>	2869
<code>__spintent_spangle_sym:n</code>	5043	<code>\l__spintent_spcoord_luaset_y_type_str</code>	2869
<code>__spintent_sparc_exec:nnn</code>	5408	<code>\l__spintent_spcoord_luaset_y_value_-</code> <code>str</code>	2869
<code>\l__spintent_sparc_intent_arg_tl</code>	5334	<code>__spintent_spcoord_mathml:</code>	2995
<code>\l__spintent_sparc_intent_tl</code>	5334	<code>__spintent_spcoord_print:n</code>	2954
<code>__spintent_sparc_mathml:n</code>	5352	<code>\l__spintent_spcoord_raw_arg_tl</code>	2879
<code>\l__spintent_sparc_mode_str</code>	5275	<code>__spintent_spcoord_set_wrap_parens_-</code> <code>size:</code>	2898
<code>\l__spintent_sparc_prefix_tl</code>	5275	<code>__spintent_spcoord_value:n</code>	2945
<code>__spintent_sparc_print_arc:n</code>	5328	<code>\l__spintent_spcoord_wrap_parens_l_tl</code>	2879
<code>__spintent_sparc_process:nn</code>	5339	<code>\l__spintent_spcoord_wrap_parens_r_tl</code>	2879
<code>\l__spintent_sparc_pt_idx_int</code>	5334	<code>\l__spintent_spdate_luaset_error_str</code>	1484
<code>\l__spintent_sparc_pt_seq</code>	5334	<code>\l__spintent_spdate_luaset_output_str</code>	1484
<code>\l__spintent_sparc_refs_tl</code>	5334	<code>__spintent_spdate_process:n</code>	76, 1495
<code>__spintent_sparc_wide_arc:n</code>	5324	<code>__spintent_spdiv_process:nnn</code>	1946
<code>\l__spintent_spcirc_articulo_str</code>	6271	<code>__spintent_spdiv_render_op:nn</code>	1925
<code>__spintent_spcirc_build_concept:</code>	6425	<code>__spintent_spdiv_render_parens:nn</code>	1934
<code>\l__spintent_spcirc_connector_arg_tl</code>	6373	<code>\l__spintent_spdom_cmd_str</code>	3843
<code>\l__spintent_spcirc_connector_tl</code>	6373	<code>__spintent_spdom_exec:nnnnn</code>	3907
<code>__spintent_spcirc_exec:nn</code>	6548	<code>\l__spintent_spdom_intent_arg_tl</code>	3843
<code>\l__spintent_spcirc_has_arg_bool</code>	6373	<code>\l__spintent_spdom_luaset_error_str</code>	3843
<code>\l__spintent_spcirc_intent_arg_tl</code>	6373	<code>__spintent_spdom_mathml:</code>	3878
<code>\l__spintent_spcirc_intent_tl</code>	6373	<code>__spintent_spdom_name:</code>	3869
<code>__spintent_spcirc_mathml:</code>	6495	<code>\l__spintent_spdom_op_tl</code>	3843
<code>\l__spintent_spcirc_mode_str</code>	6271	<code>\l__spintent_spdom_prefix_tl</code>	3843
<code>\l__spintent_spcirc_prefix_tl</code>	6271	<code>\l__spintent_spdom_word_tl</code>	3843
<code>\l__spintent_spcirc_print_sym_str</code>	6271	<code>\l__spintent_spdsym_prefix_tl</code>	1750
<code>__spintent_spcirc_process:n</code>	6479	<code>\l__spintent_spdsym_read_sym_tl</code>	1750
<code>\l__spintent_spcirc_radio_arg_tl</code>	6373	<code>\l__spintent_spdsym_suffix_tl</code>	1750
<code>__spintent_spcirc_radio_print:</code>	6410	<code>\l__spintent_spdsym_symbol_tl</code>	1750
<code>\l__spintent_spcirc_radio_sty_str</code>	6271	<code>__spintent_spfig_build_concept_-</code> <code>vertices:n</code>	5873
<code>\l__spintent_spcirc_read_sty_str</code>	6271	<code>__spintent_spfig_build_refs:</code>	5860
<code>\l__spintent_spcirc_sub_index_str</code>	6271	<code>__spintent_spfig_build_simple:n</code>	5733
<code>__spintent_spcirc_sym:</code>	6379	<code>__spintent_spfig_exec:nnn</code>	5997
<code>__spintent_spcirc_word_and_article:</code>	6389	<code>\l__spintent_spfig_intent_arg_tl</code>	5699
<code>\l__spintent_spcirc_word_tl</code>	6279	<code>\l__spintent_spfig_intent_tl</code>	5699
<code>__spintent_spcircfig_build_concept:nn</code>	6733	<code>__spintent_spfig_mathml:n</code>	5957
<code>__spintent_spcircfig_build_short:n</code>	6692	<code>\l__spintent_spfig_op_tl</code>	5699
<code>__spintent_spcircfig_del_word:n</code>	6681	<code>__spintent_spfig_points_loop:n</code>	5945
<code>__spintent_spcircfig_exec:nnn</code>	6866	<code>__spintent_spfig_process:nn</code>	5709
<code>__spintent_spcircfig_finish_intent:nn</code>	6717	<code>\l__spintent_spfig_pt_idx_int</code>	5699
<code>\l__spintent_spcircfig_intent_arg_tl</code>	6652	<code>\l__spintent_spfig_pt_seq</code>	5699
<code>\l__spintent_spcircfig_intent_tl</code>	6652	<code>\l__spintent_spfig_refs_tl</code>	5699
<code>__spintent_spcircfig_mathml:nn</code>	6825	<code>__spintent_spfig_sym:</code>	5691
<code>\l__spintent_spcircfig_op_tl</code>	6652	<code>\l__spintent_spfig_symbol_tl</code>	5699
<code>__spintent_spcircfig_process:nn</code>	6780	<code>__spintent_spfun_aname:n</code>	3469
<code>\l__spintent_spcircfig_short_bool</code>	6652	<code>__spintent_spfun_arg:n</code>	3525
<code>__spintent_spcircfig_sym:n</code>	6657	<code>__spintent_spfun_args:</code>	3546
<code>__spintent_spcircfig_word_and_-</code> <code>article:n</code>	6663	<code>__spintent_spfun_args_plain:</code>	3555
<code>__spintent_spcoord_exec:nn</code>	3012	<code>__spintent_spfun_body:</code>	3593
<code>__spintent_spcoord_inner_sep:n</code>	2977		
<code>__spintent_spcoord_intent_infix:</code>	2983		
<code>\l__spintent_spcoord_intent_str</code>	2879		
<code>\l__spintent_spcoord_luaset_error_str</code>	2869		

<code>__spintent_spfun_exec:nn</code>	3638	<code>\l__spintent_spinterval_luaset_sep_str</code>	3040
<code>__spintent_spfun_fname:</code>	3469	<code>\l__spintent_spinterval_luaset_tall_-</code>	3040
<code>\l__spintent_spfun_intent_arg_tl</code>	3420	<code>str</code>	3040
<code>\l__spintent_spfun_kv_tl</code>	3420	<code>\l__spintent_spinterval_luaset_type_-</code>	3040
<code>\l__spintent_spfun_luaset_error_str</code> ...	3420	<code>str</code>	3040
<code>\l__spintent_spfun_luaset_nargs_str</code> ...	3420	<code>__spintent_spinterval_mathml:</code>	3371
<code>__spintent_spfun_mathml:</code>	3588	<code>\l__spintent_spinterval_mode_str</code>	3058
<code>\l__spintent_spfun_mode_str</code>	3420	<code>__spintent_spinterval_plain_number:n</code> .	3314
<code>__spintent_spfun_name_mathcat:</code>	3517	<code>\l__spintent_spinterval_prefix_tl</code>	3058
<code>__spintent_spfun_name_plain:</code>	3482	<code>__spintent_spinterval_print:n</code>	3339
<code>__spintent_spfun_name_school:</code>	3494	<code>\l__spintent_spinterval_raw_arg_tl</code> ...	3058
<code>\l__spintent_spfun_nargs_int</code>	3420	<code>\l__spintent_spinterval_read_arg_tl</code> ...	3058
<code>__spintent_spfun_nested:n</code>	3627	<code>\l__spintent_spinterval_reason_str</code> ...	3058
<code>\l__spintent_spfun_prefix_tl</code>	3420	<code>__spintent_spinterval_set_delims:</code> ...	3170
<code>\l__spintent_spfun_refs_tl</code>	3420	<code>__spintent_spinterval_set_intents:</code> ...	3258
<code>__spintent_spfun_set_intent:</code>	3563	<code>__spintent_spinterval_set_reason:</code> ...	3134
<code>\l__spintent_spfun_tmp_str</code>	3420	<code>\l__spintent_spinterval_size_l_tl</code>	3058
<code>__spintent_spfuncomp_exec:nn</code>	3818	<code>\l__spintent_spinterval_size_r_tl</code>	3058
<code>__spintent_spfuncomp_fun:n</code>	3688	<code>\l__spintent_spinterval_size_str</code>	3058
<code>__spintent_spfuncomp_group:</code>	3724	<code>\l__spintent_spinterval_sym_l_tl</code>	3058
<code>\l__spintent_spfuncomp_intent_tl</code>	3663	<code>\l__spintent_spinterval_sym_r_tl</code>	3058
<code>\l__spintent_spfuncomp_luaset_apply_-</code>		<code>\l__spintent_spinterval_tmp_tl</code>	3058
<code>str</code>	3663	<code>__spintent_spinterval_value:n</code>	3322
<code>\l__spintent_spfuncomp_luaset_error_-</code>		<code>\l__spintent_spinterval_word_g_tl</code>	3058
<code>str</code>	3663	<code>\l__spintent_spinterval_word_w_tl</code>	3058
<code>\l__spintent_spfuncomp_luaset_n_str</code> ...	3663	<code>\l__spintent_spmangle_mode_str</code>	4988
<code>\l__spintent_spfuncomp_luaset_paren_-</code>		<code>\l__spintent_spmangle_prefix_tl</code>	4988
<code>str</code>	3663	<code>\l__spintent_spmarc_mode_str</code>	5275
<code>__spintent_spfuncomp_mathml:</code>	3762	<code>__spintent_spmc_process:nnnn</code>	2159
<code>\l__spintent_spfuncomp_n_int</code>	3663	<code>\l__spintent_spmcd_prefix_tl</code>	2042
<code>__spintent_spfuncomp_plain:</code>	3748	<code>\l__spintent_spmcd_print_upper_bool</code> ...	2042
<code>\l__spintent_spfuncomp_refs_tl</code>	3663	<code>\l__spintent_spmcd_read_terse_bool</code> ...	2042
<code>\l__spintent_spinterval_delim_l_tl</code> ...	3058	<code>\l__spintent_spmcd_suffix_tl</code>	2042
<code>\l__spintent_spinterval_delim_r_tl</code> ...	3058	<code>\l__spintent_spmcm_prefix_tl</code>	2042
<code>__spintent_spinterval_exec:nn</code>	3391	<code>\l__spintent_spmcm_print_result_bool</code> ..	2042
<code>__spintent_spinterval_inner_sep:n</code> ...	3308	<code>\l__spintent_spmcm_print_upper_bool</code> ...	2042
<code>\l__spintent_spinterval_intent_arg_tl</code> .	3058	<code>\l__spintent_spmcm_read_terse_bool</code> ...	2042
<code>\l__spintent_spinterval_intent_group_-</code>		<code>\l__spintent_spmcm_spmcd_luaset_error_-</code>	
<code>tl</code>	3058	<code>str</code>	2042
<code>\l__spintent_spinterval_luaset_#1_-</code>		<code>\l__spintent_spmcm_suffix_tl</code>	2042
<code>composite_str</code>	3049	<code>__spintent_spmoney_build_tree:</code>	1046
<code>\l__spintent_spinterval_luaset_#1_-inf_-</code>		<code>__spintent_spmoney_check_sub_units:</code> ..	997
<code>str</code>	3049	<code>\l__spintent_spmoney_extracted_curr_str</code>	822
<code>\l__spintent_spinterval_luaset_#1_-sign_-</code>		<code>\l__spintent_spmoney_extracted_num_tl</code> .	822
<code>str</code>	3049	<code>__spintent_spmoney_intent_int:</code>	962
<code>\l__spintent_spinterval_luaset_#1_-tall_-</code>		<code>\l__spintent_spmoney_intent_money_str</code> .	822
<code>str</code>	3049	<code>\l__spintent_spmoney_intent_sign_str</code> ..	822
<code>\l__spintent_spinterval_luaset_#1_-type_-</code>		<code>\l__spintent_spmoney_intent_sub_unit_-</code>	
<code>str</code>	3049	<code>str</code>	822
<code>\l__spintent_spinterval_luaset_#1_-</code>		<code>\l__spintent_spmoney_luaset_conde_str</code> .	812
<code>value_str</code>	3049	<code>\l__spintent_spmoney_luaset_currency_arg_-</code>	
<code>\l__spintent_spinterval_luaset_error_-</code>		<code>str</code>	812
<code>code_str</code>	3040	<code>\l__spintent_spmoney_luaset_plur_str</code> ..	812
<code>\l__spintent_spinterval_luaset_error_-</code>		<code>\l__spintent_spmoney_luaset_position_-</code>	
<code>str</code>	3040	<code>str</code>	812
<code>\l__spintent_spinterval_luaset_left_open_-</code>		<code>\l__spintent_spmoney_luaset_print_iso_-</code>	
<code>str</code>	3040	<code>str</code>	812
<code>\l__spintent_spinterval_luaset_left_sym_-</code>		<code>\l__spintent_spmoney_luaset_sing_str</code> ..	812
<code>str</code>	3040	<code>\l__spintent_spmoney_luaset_status_str</code>	812
<code>\l__spintent_spinterval_luaset_right_-</code>		<code>\l__spintent_spmoney_luaset_subplur_str</code>	812
<code>open_str</code>	3040	<code>\l__spintent_spmoney_luaset_subsing_str</code>	812
<code>\l__spintent_spinterval_luaset_right_sym_-</code>		<code>\l__spintent_spmoney_luaset_symbol_str</code>	812
<code>str</code>	3040	<code>__spintent_spmoney_normalize_key:n</code> ...	847

<code>__spintrent_spmoney_parse_and_route:n</code> .	1087	<code>__spintrent_spnM_spnD_silent:nn</code>	2328
<code>__spintrent_spmoney_print_dec_sub_-</code> <code>unit:</code>	1046	<code>\l__spintrent_spnmd_luaset_is_number_-</code> <code>str</code>	2265
<code>__spintrent_spmoney_print_int:</code>	933	<code>\l__spintrent_spnnum_notation_sym_tl</code>	51
<code>__spintrent_spmoney_print_sep_con:</code> . . .	1046	<code>\l__spintrent_spnnum_intent_read_dec_sep_-</code> <code>str</code>	51, 53, 284
<code>__spintrent_spmoney_print_symbol:</code>	895	<code>\l__spintrent_spnnum_intent_read_sign_str</code> .	51, 284
<code>__spintrent_spmoney_render_layout:</code> . . .	912	<code>\l__spintrent_spnnum_notation_sym_tl</code> . . .	284
<code>__spintrent_spmoney_render_number_node:</code>	924	<code>__spintrent_spnnum_print_integer:</code>	451
<code>__spintrent_spmoney_render_only_integer_-</code> <code>node:</code>	924	<code>__spintrent_spnnum_print_num_start:</code> . . .	451
<code>\l__spintrent_spmoney_res_main_voice_str</code>	822	<code>__spintrent_spnnum_print_number_core:</code> . .	451
<code>__spintrent_spmoney_set_and_print_sub_-</code> <code>units:</code>	997	<code>__spintrent_spnnum_print_part_dec:</code>	430
<code>__spintrent_spmoney_set_currency:n</code> . . .	847	<code>__spintrent_spnnum_print_part_period:</code> . .	430
<code>\l__spintrent_spmoney_sub_units_bool</code> . . .	822	<code>__spintrent_spnnum_print_period_bar:</code> . . .	359
<code>\l__spintrent_spmoney_subnum_int</code>	822	<code>\l__spintrent_spnnum_read_period_sep_str</code> .	51, 284
<code>__spintrent_spmoney_validate_money:</code> . . .	879	<code>\l__spintrent_spnnum_read_terse_bool</code> . .	51, 284
<code>__spintrent_spmQ_exec:nnnn</code>	2833	<code>__spintrent_spnnum_render_dec:</code>	342
<code>\l__spintrent_spmQ_intent_read_sign_str</code>	2653	<code>__spintrent_spnnum_render_decimal_sep:</code> . .	396
<code>\l__spintrent_spmQ_join_word_str</code>	101, 2653	<code>__spintrent_spnnum_render_int:</code>	342
<code>__spintrent_spmQ_print:nnn</code>	2825	<code>__spintrent_spnnum_render_only_period:</code> . .	359
<code>__spintrent_spmQ_print_scale_int:</code>	2771	<code>__spintrent_spnnum_render_part:n</code>	342
<code>__spintrent_spmQ_print_scale_int:n</code> . . .	2780	<code>__spintrent_spnnum_render_period_sep:</code> . .	359
<code>__spintrent_spmQ_print_spnZ_dfrac:nn</code> . .	2808	<code>__spintrent_spnz_process:nn</code>	2627
<code>\l__spintrent_spmQ_read_parens_bool</code> . . .	101	<code>__spintrent_spnz_render_parens:</code>	2615
<code>__spintrent_spmQ_set_join_word:</code>	2762	<code>\l__spintrent_spPcirc_mode_str</code>	6566
<code>__spintrent_spmQ_set_wrap_parens_size:</code>	2707	<code>\l__spintrent_spPcirc_prefix_tl</code>	6566
<code>__spintrent_spmQ_start_wrap_parens:</code> . . .	2742	<code>\l__spintrent_spPcirc_print_sym_str</code> . . .	6566
<code>__spintrent_spmQ_stop_wrap_parens:</code> . . .	2742	<code>\l__spintrent_spPcirc_radio_sty_str</code> . . .	6566
<code>\l__spintrent_spmQ_use_dfrac_bool</code>	102	<code>\l__spintrent_spPcirc_read_arg_tl</code>	6566
<code>\l__spintrent_spmQ_use_spnZ_bool</code>	102	<code>\l__spintrent_spPcirc_read_sty_str</code>	6566
<code>\l__spintrent_spmQ_wrap_parens_l_tl</code> . . .	100, 101, 2653	<code>\l__spintrent_spPcirc_sub_index_str</code> . . .	6566
<code>\l__spintrent_spmQ_wrap_parens_r_tl</code> . . .	100, 101, 2653	<code>\l__spintrent_spPfig_mode_str</code>	5584
<code>\l__spintrent_spm_side_mode_str</code>	4777	<code>\l__spintrent_spPfig_prefix_tl</code>	5584
<code>\l__spintrent_spm_side_prefix_tl</code>	4777	<code>\l__spintrent_spPfig_print_sym_str</code>	5584
<code>\l__spintrent_spm_side_read_arg_tl</code>	4777	<code>\l__spintrent_spPfig_read_arg_tl</code>	5584
<code>\l__spintrent_spm_side_read_sty_str</code>	4777	<code>__spintrent_spPoint_exec:nnn</code>	4135
<code>\l__spintrent_spm_sym_prefix_tl</code>	1821	<code>\l__spintrent_spPoint_intent_arg_tl</code> . . .	4051
<code>\l__spintrent_spm_sym_read_sym_tl</code>	1821	<code>__spintrent_spPoint_mathml:n</code>	4126
<code>\l__spintrent_spm_sym_suffix_tl</code>	1821	<code>\l__spintrent_spPoint_mode_str</code>	4051
<code>\l__spintrent_spm_sym_symbol_tl</code>	1821	<code>\l__spintrent_spPoint_prefix_tl</code>	4051
<code>__spintrent_spmul_process:nnn</code>	2017	<code>__spintrent_spPoint_print:n</code>	4093
<code>__spintrent_spmul_render_op:nn</code>	1996	<code>__spintrent_spPoly_build_concept:</code>	5503
<code>__spintrent_spmul_render_parens:nn</code> . . .	2005	<code>__spintrent_spPoly_build_refs:</code>	5490
<code>\l__spintrent_spnD_luaset_is_truncated_-</code> <code>str</code>	2265	<code>__spintrent_spPoly_exec:nn</code>	5567
<code>\l__spintrent_spnD_luaset_result_tl</code> . . .	2265	<code>\l__spintrent_spPoly_intent_arg_tl</code>	5478
<code>\l__spintrent_spnD_prefix_tl</code>	2265	<code>__spintrent_spPoly_mathml:</code>	5547
<code>\l__spintrent_spnD_print_result_num_tl</code> . .	2265	<code>\l__spintrent_spPoly_mode_str</code>	5435
<code>\l__spintrent_spnM_luaset_is_truncated_-</code> <code>str</code>	2265	<code>__spintrent_spPoly_points_loop:</code>	5525
<code>\l__spintrent_spnM_luaset_result_tl</code> . . .	2265	<code>\l__spintrent_spPoly_prefix_tl</code>	5435
<code>\l__spintrent_spnM_prefix_tl</code>	2265	<code>__spintrent_spPoly_process:n</code>	5537
<code>\l__spintrent_spnM_print_result_num_tl</code> . .	2265	<code>\l__spintrent_spPoly_pt_idx_int</code>	5478
<code>__spintrent_spnM_spnD_exec:nnnn</code>	2371	<code>\l__spintrent_spPoly_pt_seq</code>	5478
<code>__spintrent_spnM_spnD_intent:nn</code>	2319	<code>\l__spintrent_spPoly_refs_tl</code>	5478
<code>\l__spintrent_spnM_spnD_luaset_error_-</code> <code>str</code>	2265	<code>__spintrent_spPoly_sym:</code>	5482
<code>__spintrent_spnM_spnD_process:nnnn</code> . . .	2314	<code>__spintrent_sprayo_exec:nn</code>	4761
<code>__spintrent_spnM_spnD_result:nn</code>	2336	<code>\l__spintrent_sprayo_intent_arg_tl</code>	4698
		<code>__spintrent_sprayo_mathml:</code>	4711
		<code>\l__spintrent_sprayo_mode_str</code>	4644
		<code>\l__spintrent_sprayo_prefix_tl</code>	4644

<code>__spintent_sprayo_print:n</code>	4702	<code>\l__spintent_sptime_luaset_base_str</code> ...	1402
<code>\l__spintent_sprayo_pt_idx_int</code>	4698	<code>\l__spintent_sptime_luaset_error_str</code> ..	1402
<code>\l__spintent_sprayo_pt_seq</code>	4698	<code>\l__spintent_sptime_luaset_has_seconds_-</code> <code>str</code>	1402
<code>\l__spintent_sprayo_read_sty_str</code>	4644	<code>\l__spintent_sptime_luaset_is_fraction_-</code> <code>str</code>	1402
<code>\l__spintent_sprayo_refs_tl</code>	4698	<code>\l__spintent_sptime_luaset_seconds_str</code>	1402
<code>__spintent_sprazon_exec:nnn</code>	2473	<code>\l__spintent_spunit_exp_seq</code>	545
<code>__spintent_spread_exec:nn</code>	1691	<code>\l__spintent_spunit_exponent_tl</code>	545
<code>\l__spintent_spread_luaset_reading_str</code>	1670	<code>__spintent_spunit_format_canonical:</code> ..	608
<code>\l__spintent_spread_luaset_status_str</code> .	1670	<code>\l__spintent_spunit_is_mult_bool</code>	545
<code>\l__spintent_spread_read_arg_tl</code>	1670	<code>\l__spintent_spunit_luaset_canonical_-</code> <code>str</code>	538
<code>\l__spintent_spread_read_tmp_tl</code>	1670	<code>\l__spintent_spunit_luaset_compact_exp_-</code> <code>str</code>	538
<code>__spintent_sprecta_exec:nn</code>	4628	<code>\l__spintent_spunit_luaset_is_compact_-</code> <code>str</code>	538
<code>\l__spintent_sprecta_intent_arg_tl</code> ...	4403	<code>\l__spintent_spunit_luaset_is_-</code> <code>sexagesimal_str</code>	538
<code>\l__spintent_sprecta_kv_tl</code>	4403	<code>\l__spintent_spunit_luaset_lookup_status_-</code> <code>str</code>	538
<code>__spintent_sprecta_mathml:n</code>	4457	<code>\l__spintent_spunit_luaset_read_str</code> ...	538
<code>\l__spintent_sprecta_mode_str</code>	4403	<code>\l__spintent_spunit_luaset_status_str</code> .	538
<code>\l__spintent_sprecta_prefix_tl</code>	4403	<code>\l__spintent_spunit_mult_seq</code>	545
<code>__spintent_sprecta_print:n</code>	4448	<code>__spintent_spunit_process_base_exp:n</code> .	616
<code>\l__spintent_sprecta_pt_idx_int</code>	4403	<code>__spintent_spunit_process_mult:n</code>	668
<code>\l__spintent_sprecta_pt_seq</code>	4403	<code>\l__spintent_spunit_read_cdot_bool</code> ...	545
<code>\l__spintent_sprecta_refs_tl</code>	4403	<code>\l__spintent_spunit_read_slash_str</code> ...	545
<code>__spintent_spshort_execute_layout:</code> ...	1195	<code>__spintent_spunit_safe_exec:n</code>	687
<code>\l__spintent_spshort_luaset_base_str</code> ..	1112	<code>\l__spintent_spunit_unit_name_tl</code>	545
<code>\l__spintent_spshort_luaset_expand_str</code>	1112	<code>\l__spintent_spusep_arg_tl</code>	77, 78, 1531
<code>\l__spintent_spshort_luaset_layout_str</code>	1112	<code>__spintent_spusep_convert:n</code>	1593
<code>\l__spintent_spshort_luaset_output_str</code>	1112	<code>\l__spintent_spusep_left_tl</code>	77, 78, 1531
<code>\l__spintent_spshort_luaset_status_str</code>	1112	<code>\l__spintent_spusep_right_tl</code>	77, 78, 1531
<code>\l__spintent_spshort_luaset_suffix_str</code>	1112	<code>__spintent_spusep_set_parens:nn</code>	1582
<code>__spintent_spshort_print:nn</code>	1148	<code>\l__spintent_spusep_silent_bool</code>	78
<code>__spintent_spshort_process_and_-</code> <code>render:n</code>	1160	<code>__spintent_spvec_exec:nnn</code>	4372
<code>__spintent_spshort_render_bypass:n</code> ...	1171	<code>\l__spintent_spvec_intent_arg_tl</code>	4163
<code>__spintent_spshort_render_error:n</code> ...	1171	<code>\l__spintent_spvec_kv_tl</code>	4163
<code>__spintent_spshort_render_fallback:</code> ..	1171	<code>__spintent_spvec_mathml:n</code>	4225
<code>__spintent_spshort_render_success:</code> ...	1171	<code>\l__spintent_spvec_mode_str</code>	4163
<code>__spintent_spshort_tag_span:nn</code>	1121	<code>\l__spintent_spvec_prefix_tl</code>	4163
<code>__spintent_spside_exec:nnn</code>	4961	<code>__spintent_spvec_print:n</code>	4208
<code>\l__spintent_spside_intent_arg_tl</code>	4851	<code>\l__spintent_spvec_pt_idx_int</code>	4163
<code>\l__spintent_spside_intent_tl</code>	4851	<code>\l__spintent_spvec_pt_seq</code>	4163
<code>__spintent_spside_mathml:n</code>	4884	<code>\l__spintent_spvec_refs_tl</code>	4163
<code>\l__spintent_spside_mode_str</code>	4777	<code>__spintent_spVol_attach_subscript:</code> ...	7029
<code>__spintent_spside_points_loop:n</code>	4949	<code>\l__spintent_spVol_mode_str</code>	6897
<code>\l__spintent_spside_prefix_tl</code>	4777	<code>\l__spintent_spVol_native_bool</code>	6897
<code>__spintent_spside_process:nn</code>	4856	<code>__spintent_spVol_print:</code>	6942
<code>\l__spintent_spside_pt_idx_int</code>	4851	<code>\l__spintent_spVol_read_arg_full_tl</code> ...	6897
<code>\l__spintent_spside_pt_seq</code>	4851	<code>__spintent_spVol_read_arg_print:</code>	7061
<code>\l__spintent_spside_read_arg_tl</code>	4777	<code>\l__spintent_spVol_read_arg_tl</code>	6897
<code>\l__spintent_spside_read_sty_str</code>	4777	<code>__spintent_spVol_silent:</code>	6949
<code>\l__spintent_spside_refs_tl</code>	4851	<code>\l__spintent_spVol_sub_index_str</code>	6897
<code>\l__spintent_spsiglo_luaset_arabic_str</code>	1252	<code>\l__spintent_spVol_suffix_tl</code>	6897
<code>\l__spintent_spsiglo_luaset_error_str</code> .	1252	<code>__spintent_spVol_symbol_print:</code>	6991
<code>\l__spintent_spsiglo_luaset_roman_min_-</code> <code>str</code>	1252	<code>__spintent_spVol_verbal:</code>	6970
<code>\l__spintent_spsiglo_print_era_str</code> ...	1252	<code>\l__spintent_spVol_word_tl</code>	6897
<code>__spintent_spsiglo_print_math:n</code>	1336	<code>__spintent_spVol_unknown_keys:n</code>	235
<code>__spintent_spsiglo_print_text:n</code>	1336	<code>__spintent_spVol_unknown_keys:nn</code>	235
<code>__spintent_spsiglo_render_simple:nn</code> ..	1287	<code>spintent.lua</code>	70
<code>__spintent_spsiglo_render_with_-</code> <code>era:nnn</code>	1287		
<code>__spintent_spsiglo_tag_span:nn</code>	1323		
<code>__spintent_sptime_build_seconds:nn</code> ...	1416		
<code>__spintent_sptime_build_tree:</code>	75, 1434		

[\spinterval](#) [34](#), [108](#), [3409](#)
[spinterval-invalid-arg](#) [3075](#)
[\spmangle](#) [24](#), [145](#), [5253](#)
[spmangle-invalid-arg](#) [4993](#)
[\spmarc](#) [25](#), [151](#), [5413](#)
[spmarc-invalid-points](#) [5280](#)
[\spmc](#) [15](#), [87](#), [2245](#)
[\spmc](#) [16](#), [87](#), [2245](#)
[\spmoney](#) [6](#), [61](#), [1101](#)
[\spmQ](#) [17](#), [99](#), [2845](#)
[spmQ-not-integer](#) [2658](#)
[spmQ-zero-denominator](#) [2658](#)
[\spmside](#) [22](#), [141](#), [4966](#)
[spmside](#) [6287](#)
[spmside-invalid-points](#) [4784](#)
[\spmsym](#) [13](#), [82](#), [1874](#)
[\spmul](#) [14](#), [85](#), [2032](#)
[spmul-empty-argument](#) [1991](#)
[\spnD](#) [14](#), [91](#), [2436](#)
[\spnewread](#) [11](#), [1730](#)
[spnewread-duplicate](#) [1642](#)
[spnewread-empty-args](#) [1642](#)
[\spnewunit](#) [6](#), [60](#), [780](#)
[spnewunit-duplicate](#) [759](#)
[\spnM](#) [15](#), [91](#), [2436](#)
[\spnum](#) [3](#), [51](#), [52](#), [520](#)
[spnum-has-units](#) [506](#)
[spnum-no-digits](#) [506](#)
[\spnZ](#) [17](#), [97](#), [2643](#)
[spnz-not-integer](#) [2610](#)
[\spPcirc](#) [32](#), [176](#), [6871](#)
[spPcirc-invalid-arg](#) [6577](#)
[\spPfig](#) [28](#), [157](#), [6002](#)
[spPfig-invalid-arg](#) [5591](#)
[spPfig-invalid-points](#) [5591](#)
[spPfig-read-arg-key](#) [5591](#)
[spPfig-sym-cardinality-conflict](#) [5591](#)
[spPfig-sym-letra-conflict](#) [5591](#)
[\spPoint](#) [19](#), [127](#), [4152](#)
[spPoint-invalid-point](#) [4054](#)
[\spPoly](#) [26](#), [154](#), [5572](#)
[spPoly-invalid-points](#) [5437](#)
[\spProp](#) [18](#), [96](#), [2524](#)
[\sprad](#) [29](#), [165](#), [6247](#)
[\sprayo](#) [21](#), [138](#), [4766](#)

[sprayo-invalid-points](#) [4647](#)
[\sprazon](#) [18](#), [94](#), [2514](#)
[\spread](#) [12](#), [47](#), [79](#), [1720](#)
[spread-unknown-sym](#) [1642](#)
[\sprec](#) [36](#), [123](#), [3921](#)
[\sprecta](#) [20](#), [134](#), [4633](#)
[sprecta-invalid-points](#) [4410](#)
[\spshort](#) [8](#), [67](#), [1239](#)
[\spside](#) [21](#), [141](#), [4966](#)
[spside](#) [6287](#)
[spside-invalid-points](#) [4784](#)
[\spsiglo](#) [10](#), [71](#), [1389](#)
[\sptime](#) [11](#), [74](#), [1466](#)
[\spunit](#) [4](#), [56](#), [57](#), [60](#), [747](#)
[spunit-invalid-unit](#) [552](#)
[spunit-no-units](#) [552](#)
[\spunitalias](#) [6](#), [60](#), [794](#)
[spunitalias-duplicate](#) [759](#)
[spunitalias-invalid-expr](#) [759](#)
[\spusep](#) [12](#), [77](#), [1631](#)
[spusep-empty-arg](#) [1520](#)
[spusep-invalid-arg](#) [1520](#)
[\spvec](#) [20](#), [129](#), [4392](#)
[spvec-invalid-arg](#) [4170](#)
[\spVol](#) [34](#), [182](#), [7089](#)
str commands:
 [\str_if_eq:nnTF](#) [62](#)
 [\str_lowercase:n](#) [62](#)
 [\str_uppercase:n](#) [62](#)
sub-index [6031](#), [6287](#), [6586](#), [6904](#)
sub-units [861](#)
suffix [1754](#), [1825](#)

U

unknown [1534](#), [1674](#), [1754](#), [1825](#), [2062](#), [2274](#), [2456](#), [2672](#), [3082](#),
 [3438](#), [3677](#), [3854](#), [4062](#), [4174](#), [4417](#), [4654](#), [4794](#), [5003](#),
 [5290](#), [5444](#), [5629](#), [6031](#), [6287](#), [6586](#), [6904](#)
unknown-abbreviation [1118](#)
unknown-choice [208](#)
unknown-command-setup [159](#)
upper-case [2062](#), [6031](#)
use-dfrac [2456](#), [2672](#)
use-spnZ [2672](#)

W

wrap-parens [1900](#), [1971](#), [2594](#), [2672](#)